Taming the Curse of Reality: Towards Robust Physical Agents from Perception to Action

Chonghao Sima

司马崇昊

School of Computing and Data Science

The University of Hong Kong

This dissertation is submitted for

Doctor of Philosophy

September, 2026

To my family.

Declaration

I hereby declare that except where specific reference is made to the work of others, the contents of this dissertation are original and have not been submitted in whole or in part for consideration for any other degree or qualification in this, or any other University. This dissertation is the result of my own work and includes nothing which is the outcome of work done in collaboration, except where specifically indicated in the text.

Chonghao Sima

September, 2026

Acknowledgements

First and foremost, I am deeply grateful to my supervisors, Prof. Ping Luo and Prof. Hongyang Li. Prof. Luo gave me the freedom to pursue the questions I cared about and the perspective to see how they fit together. Prof. Li introduced me to autonomous driving research at OpenDriveLab and taught me, by example, that a good benchmark can move an entire community. This thesis would not exist without their guidance, patience and trust.

I thank Dr. Kashyap Chitta and Prof. Andreas Geiger, with whom I worked closely on DriveLM and Centaur; their rigour in evaluation and clarity in writing have shaped how I do research. I thank Dr. José M. Álvarez and Dr. Zhiding Yu for mentoring me during my internship at NVIDIA Autonomous Vehicle Applied Research, where the ideas behind Centaur took shape. I am also grateful to Prof. Yexiang Xue, who advised me during my years at Purdue University, and to Prof. Kun He, who first guided me into research at Huazhong University of Science and Technology.

Much of the work in this thesis was done together with colleagues at OpenDriveLab, Shanghai AI Laboratory, and at MMLab@HKU. I thank all of my co-authors, in particular Li Chen, Jiazhi Yang, Yihan Hu, Zhiqi Li, Huijie Wang, Tianyu Li, Wenwen Tong, Katrin Renz, Checheng Yu, Modi Shi, Lirui Zhao and Qingwen Bu, for countless discussions, late-night experiments and deadlines shared. I also thank the organisers, participants and sponsors of the challenges we hosted, whose engagement turned our benchmarks into community resources.

Parts of the research in this thesis were supported by the General Research Fund of Hong Kong (Nos. 17200622 and 17209324) and the National Key R&D Program of China (No. 2022ZD0161000), as acknowledged in the corresponding publications. During my doctoral study, I was supported by a Senior Research Assistant position.

Finally, I thank my family for their unconditional love and support throughout this long journey.

Abstract of thesis entitled

"Taming the Curse of Reality: Towards Robust Physical Agents from Perception to Action"

Submitted by

Chonghao Sima

for the degree of Doctor of Philosophy

at The University of Hong Kong

in September, 2026

Learning-based vehicles and manipulators must operate in conditions that no finite training set can anticipate. This dissertation calls that persistent mismatch between training and deployment the *curse of reality* and studies how physical agents can remain dependable under it across perception, reasoning, and action.

For perception, Chapter 2 develops a volumetric alternative to box-centred scene descriptions. OccNet reconstructs 3D semantic occupancy from surround-view cameras through a coarse-to-fine cascade of voxel decoders, and OpenOcc provides dense voxel and motion-flow supervision on nuScenes. On geometrically irregular instances of known vehicle classes, predicted voxels match occupancy labels more closely than voxelised detections, and the learned representation transfers to 3D detection and BEV segmentation. In single-run open-loop nuScenes evaluations, occupancy-based planning reduced collision rates by 0.06–0.24 percentage points relative to boxes predicted by the same network, a small difference without a significance test. The representation remains bounded by a closed 16-class vocabulary and was not tested under distribution shift. OpenScene extends volumetric supervision to more than 120 hours of nuPlan logs and later supplied the sensor data for end-to-end driving and world-modelling challenges.

For reasoning, Chapter 3 introduces Graph Visual Question Answering (GVQA), which organises perception, prediction, and planning questions into a directed acyclic graph, together with the DriveLM datasets and the DriveLM-Agent baseline. Under zero-shot transfer from nuScenes to a different Waymo camera configuration, the baseline achieved lower trajectory error than UniAD-Single, although most of that margin was already present in the pre-trained vision-language backbone and the models differ in scale and camera inputs. Conditioning on graph context improved speed classification under this sensor shift but degraded in-domain behaviour accuracy and open-loop planning; on withheld CARLA pedestrians, recovery required a class-specific prompt.

At the action stage, Chapter 4 proposes Centaur, which adapts an end-to-end planner at deployment by minimising Cluster Entropy without test-time rules, while distilling rule-based scores during training. In non-reactive NAVSIM replay, where each frame resets to the logged state, Centaur increased Hydra-MDP from 90.3 to 92.6 PDMS in one v1.0 run and reached 92.10 ± 0.33 over three v1.1 seeds, alongside a diagnostic stress split (`navsafe`). Chapter 5 presents the co-led manipulation framework χ_0, which aligns demonstrations, policy learning, and hardware execution. After post-training $\pi_{0.5}$ with about 20 hours of base demonstrations per task (22.5–33.6 hours including targeted recovery rollouts), real-robot T-shirt folding completion rose from 8/30 (27%) to 29/30 (97%). Because targeted recovery data produced the largest single gain and data volume was not varied at a matched budget, alignment and scale remain complementary hypotheses.

Finally, Chapter 6 synthesises these findings into a graded research agenda for self-correcting physical agents combining queryable 3D representations, fast and slow decision pathways, competence monitoring, test-time adaptation, and continual learning loops.

(An abstract of 446 words)

Contents

Contents **xi**

List of Figures **xvii**

List of Tables **xxv**

1 Introduction **1**

1.1 Motivation 1

1.2 Contributions 6

1.3 Thesis Outline 10

1.4 Related Publications 11

2 Perceive: Scene as Occupancy **17**

2.1 Motivation 19

2.2 Related Work 22

2.3 OccNet 22

2.3.1 Cascade Voxel Decoder 24

2.3.2 Exploiting Occupancy for Driving Tasks 25

2.4 The OpenOcc Benchmark 25

2.4.1 Benchmark Overview 26

2.4.2 Generating High-Quality Annotations 26

2.5 Experiments 27

2.5.1 Experimental Setup 27

2.5.2 Main Results 28
2.5.3 Analysis 35
2.6 Discussion and Limitations 37
2.6.1 What the Experiments Show 37
2.6.2 Limitations and Threats to Validity 37
2.6.3 Subsequent Developments (2023–2026) 38
2.7 Scaling Up: The OpenScene Benchmark 38
2.8 Implementation Details and Additional Ablations 40
2.8.1 Evaluation Metrics and Architecture Details 40
2.8.2 OpenOcc Pipeline and Dataset Statistics 41
2.8.3 Supplementary Ablations and Visualisations 42
2.9 Chapter Summary 47

3 Reason: Driving with Graph Visual Question Answering 49
3.1 Motivation 51
3.2 Related Work 52
3.3 Graph Visual Question Answering 54
3.3.1 Task Formulation 54
3.3.2 Evaluation Metrics 55
3.4 DriveLM-Data 55
3.4.1 DriveLM-nuScenes 56
3.4.2 PDM-Lite: An Expert Driver for CARLA Leaderboard 2.0 . . . 57
3.4.3 DriveLM-CARLA 58
3.5 DriveLM-Agent: A GVQA Baseline 58
3.5.1 Prompting with Context 59
3.5.2 Trajectory Tokenisation for Motion 59
3.6 Experiments 59
3.6.1 VLMs for End-to-End Driving 60
3.6.2 Generalisation Across Sensor Configurations 61
3.6.3 Generalisation to Unseen Objects 62

3.6.4 Question-wise Analysis on DriveLM-nuScenes 64
3.6.5 Performance for P_{1-3} via GVQA 65
3.7 Discussion and Limitations 66
3.8 From Benchmark to Community 68
3.8.1 Subsequent Developments (2023–2026) 68
3.9 Dataset Statistics, Implementation Details, and Additional Results . . . 69
3.9.1 DriveLM-nuScenes Schema, Workflow, and Statistics 69
3.9.2 DriveLM-CARLA Topology, PDM-Lite, and Collection 71
3.9.3 DriveLM-Metrics, Tokenisation, and Implementation 73
3.10 Chapter Summary 78

4 Act I: Robust End-to-End Driving via Test-Time Training 81
4.1 Motivation 83
4.2 Related Work 84
4.3 Method 85
4.3.1 Preliminaries 85
4.3.2 Cluster Entropy 87
4.3.3 Test-Time Training 88
4.4 `navsafe`: A Safety-Critical Scenario Set 89
4.5 Experiments 91
4.5.1 Experimental Setup 91
4.5.2 Impact of Test-Time Training on `navtest` 92
4.5.3 Failure Identification 93
4.5.4 `navtest` Leaderboard 94
4.5.5 Scenario-Level Evaluation on `navsafe` 94
4.6 Analysis 95
4.6.1 Qualitative Results 95
4.6.2 Generality and Design Choices 95
4.6.3 Computational Cost 96
4.7 Discussion and Limitations 96

4.8 Implementation Details and Supplementary Evaluations 98
4.8.1 Related Literature and Methodological Formulations 98
4.8.2 Hyper-Parameters, Baselines, and Extended Ablations 101
4.8.3 Arithmetic Consistency Audits 102
4.9 Chapter Summary 105

5 Act II: Resource-Aware Robust Manipulation 107
5.1 Motivation 109
5.2 Related Work 112
5.2.1 Imitation Learning and Real-World Policy Deployment 112
5.2.2 Model Merging and Weight Interpolation 113
5.2.3 Advantage Estimation for Long-Horizon Tasks 113
5.3 Distributional Inconsistencies in Robot Learning 114
5.3.1 Preliminaries 114
5.3.2 Three Distributions of the Robot Learning Cycle 114
5.3.3 Three Systematic Inconsistencies 115
5.4 Method 116
5.4.1 Overview of the Framework 116
5.4.2 Model Arithmetic 116
5.4.3 Stage Advantage 118
5.4.4 Train–Deploy Alignment 119
5.5 System and Data Setup 122
5.5.1 Robot Platform 122
5.5.2 Evaluation Tasks 122
5.5.3 Metrics 123
5.5.4 Evaluation Protocol and Uncertainty 124
5.5.5 Data Collection and Training 125
5.6 Experiments 125
5.6.1 Baselines and Ablation Design 126
5.6.2 System Efficacy Breakdown 127

5.6.3 Quantitative Summary . . . 127
5.7 Ablation Studies . . . 128
5.7.1 Model Arithmetic . . . 129
5.7.2 Stage Advantage . . . 131
5.7.3 Train–Deploy Alignment . . . 132
5.8 Discussion and Limitations . . . 133
5.9 Implementation Details and Supplementary Ablations . . . 135
5.9.1 Module Implementation, Hardware, and Evaluation Rubric . . . 135
5.9.2 Extended Module Ablations and Failure Analysis . . . 137
5.10 Chapter Summary . . . 146

6 Conclusion **147**
6.1 Summary of Contributions . . . 147
6.2 Unified Perspective: Towards Self-Correcting Physical Agents . . . 150
6.3 Limitations and Future Directions . . . 154

A Supplementary Material **157**
A.1 LLM Prompts and Question Templates for DriveLM . . . 157
A.1.1 Graph VQA Question Templates Across Reasoning Stages . . . 157
A.1.2 GPT Score Evaluation Prompt and Autoregressive Context Formatting . . . 160

References **163**

List of Figures

1.1 **Overview of the dissertation.** The core chapters follow the perceive–reason–act loop of a physical agent confronting the curse of reality: Chapter 2 addresses 3D occupancy perception (Stage 1, RQ1), Chapter 3 addresses graph-structured vision-language reasoning (Stage 2, RQ2), and Chapters 4 and 5 address reliable action via test-time planner adaptation and resource-aware manipulation alignment (Stage 3, RQ3). Chapter 6 synthesises these findings into an evidence-graded architecture for self-correcting physical agents (RQ4). Works marked * are co-authored rather than led by the author. 4

1.2 **Research lineage of the publications to which the author contributed, organised by year and perceive–reason–act pillar.** Solid arrows denote direct technical or data inheritance; dashed arrows denote motivation or contrast; dark bold outlines mark the five author-led core-chapter works; squares mark dataset and benchmark releases. Two capsules (PersFormer + OpenLane and OccNet + OpenOcc) pair a method and benchmark from the same paper, so the 18 capsules represent the 20 outputs in Section 1.4. 6

2.1 **Scene as occupancy.** Industrial camera systems estimate pseudo-LiDAR points via Mobileye's VIDAR [42] (a) or 3D voxel occupancy as in Tesla's stack [43] (b) (frames adapted from [42, 43]). Rigid 3D boxes miss overhanging cargo (a) or crane booms (c), whereas 3D occupancy (d) captures arbitrary contours as a shared descriptor (e) for perception and planning. 20

2.2 **The OccNet pipeline. I. Occupancy reconstruction.** Surround-view images pass through a spatiotemporal BEV encoder, after which a cascade voxel decoder progressively expands vertical resolution, fusing historical voxel features $V'_{t-1,i}$ and image features via 3D deformable attention (skip connections omitted). **II. Occupancy exploitation.** Task heads decode semantic scene completion and 3D boxes from the shared descriptor; vertical projection yields a BEV obstacle map for trajectory planning [66]. 23

2.3 **Comparison of 3D occupancy ground truth.** Compared with (a) single-sweep sparse LiDAR voxels [30] and (b) OccData [72], OpenOcc yields (c) denser surface coverage and (d) per-voxel foreground motion flow. 27

2.4 **Qualitative occupancy predictions.** OccNet recovers finer geometry and small foreground actors (pedestrian in dashed box) than TPVFormer [30]. 28

2.5 **Detection transfer across training fractions** (co-author-led, single runs): (a) mAP and (b) NDS when initialising from sparse/dense OpenOcc OccNet vs. monocular FCOS3D [51]. 32

2.6 **Planning visualisation** (co-author-led): planned trajectory (blue) over rasterised boxes (left) vs. occupancy (right). 34

2.7 **Predicted 3D boxes (top) vs. semantic occupancy (bottom)** on articulated vehicles and overhead structures (dashed ellipses). 36

2.8 **Overview of OpenScene and its downstream benchmarks.** Downsampling nuPlan to 2 Hz and augmenting it with 3D occupancy and flow annotations, OpenScene underpins large-scale occupancy prediction as well as the end-to-end driving and world-modelling tracks at the CVPR 2024 and CVPR 2025 (NAVSIM-v2) Autonomous Grand Challenges. Both NAVSIM and the end-to-end driving tracks consume the redistributed sensor logs rather than the voxel occupancy targets. . 40

2.9 **Four-stage OpenOcc pipeline:** (a) voxelisation (unlabelled sweeps in black), (b) propagation, (c) denoising, (d) hole-filling (red box). . . . 41

2.10 **OpenOcc class frequencies.** Static background exceeds 94% of occupied voxels. 42

2.11 **Moving-voxel share ($v > 0.2\,\mathbf{m/s}$)** across the eight movable foreground classes. 42

2.12 **Planning input comparison** (co-author-led): predicted 3D boxes (left) vs. occupancy (right). 46

2.13 **Validation occupancy predictions** across six camera views and BEV voxel grids. 46

3.1 **Overview of DriveLM.** Following [98], Graph Visual Question Answering (GVQA) links QA pairs via object-level and stage-level edges across perception → prediction → planning → behaviour → motion, supported by DriveLM-Data, DriveLM-Metrics and DriveLM-Agent. . . 52

3.2 **Annotation pipelines.** DriveLM-nuScenes combines rule-extracted perception QAs from nuScenes and OpenLane-V2 with human prediction/planning annotations and multi-round quality checks; DriveLM-CARLA uses an automated rule pipeline driven by PDM-Lite [31]. . . . 56

3.3 **Pipeline of DriveLM-Agent.** From the front-camera image, a fine-tuned VLM forwards parent QAs as "Context:" prefixes along graph edges across all five stages. 58

3.4 **Qualitative examples from DriveLM-Agent** on DriveLM-nuScenes `val` (left) and Waymo `val` (right), showing questions (Q), context (C) and predicted answers (A). 62
3.5 **Schema of DriveLM-nuScenes.** Clip summaries pair with *perception*, *planning* and *prediction* QAs grounded via *c tags* with 2D box centres. 70
3.6 **Left: three-stage human annotation workflow. Right: verification and translation pipeline** over 8-video batches. 70
3.7 **Breakdown of QAs across perception, prediction and planning.** 71
3.8 **Fine-grained question subcategories within each stage of DriveLM-nuScenes.** 72
3.9 **Left: class breakdown of key objects in DriveLM-nuScenes. Right: question-type distribution by object group.** 72
3.10 **Topology of the DriveLM-CARLA question graph** across perception, prediction and planning. 75
3.11 **End-to-end pipeline of DriveLM-Agent** quantising waypoints into `blip-2-flan-t5-xl` tokens. 77

4.1 **Minimising directional score entropy via test-time training (TTT).** The trajectory-scoring planner Hydra-MDP [203] scores candidate trajectories on `navtest` [23] across five directional clusters (★: top-scoring plan). **Left:** Entering a roundabout, the unadapted model picks an isolated 'slight left' outlier despite low cluster support. **Right:** Minimising Cluster Entropy via TTT sharpens the score distribution toward 'slight right', avoiding oncoming traffic. 83

4.2 **Overview of test-time training (TTT) in Centaur. Top:** At each deployment frame, a fixed subset of M candidate trajectories is grouped into five directional clusters to compute Cluster Entropy and its gradient via backpropagation. **Bottom:** Gradients from the F preceding evaluation steps are buffered and averaged for a single gradient step from base decoder weights θ (Algorithm 1). 86

4.3 **Curation workflow of `navsafe`.** `navtest` clips are ranked by Hydra-MDP PDMS (Step 0), inspected by human annotators (Step 1), and grouped into ten scenario classes (Steps 2–4) guided by the NHTSA pre-crash typology [202] (top-left inset), with a qualitative 2D OpenCLIP [236] projection of `navtest` (blue) and `navsafe` (red) frames at top right. 90

4.4 **Qualitative results** (★: top-scoring plan ordered by lateral endpoint). **Top:** TTT shifts selection from 'Slight Left' to 'Forward', improving TTC. **Bottom:** TTT fails to overturn an overconfident choice. 95

4.5 **Score distributions on failed (top) vs. successful (bottom) pairs.** 102

5.1 **Top: platform overview.** Two ALOHA-style bimanual stations collaborate on multi-stage garment flattening, folding, and rack hanging. **Bottom: distributional alignment and Task A summary.** Across the learning loop (P_{train}, Q_{model}, P_{test}), MA reconciles Q_{model} with P_{train}, TDA bridges P_{train} and P_{test}, and SA steers Q_{model} toward P_{test}. On Task A (Fig. 5.7), χ_0 lifts $\pi_{0.5}$ [6] completion from roughly 27% (8/30) to roughly 97% (29/30) over 30 rollouts (Table 5.2). 111

5.2 **Architecture of χ_0. Left** (P_{train}): heuristic DAgger recovery collection, spatiotemporal augmentation, and stage annotations. **Middle** (Q_{model}): Model Arithmetic weight interpolation and Stage Advantage conditioning. **Right** (P_{test}): temporal chunk-wise smoothing and on-policy DAgger feedback. 116

5.3 **Weight-merging schemes in Model Arithmetic.** Checkpoints trained on disjoint subsets are merged via convex weight interpolation using inverse-loss weighting (**top**) or uniform averaging, gradient optimisation, and greedy selection (**bottom**). 117

5.4 **Cumulative progress curves from Stage Advantage.** Green and red shading indicate positive and negative predicted advantage on Task A (cloth slip and recovery, **top**), Task B (retrieval and placement adjustment, **middle**), and Task C (rack pull-over under occlusion, **bottom**). 120

5.5 **Train–Deploy Alignment mechanisms (left) and qualitative t-SNE trajectory projections (right)** illustrating closer overlap among distributions as each mechanism is added. 121

5.6 **Physical layout** of the dual-station bimanual manipulation platform. . 123

5.7 **Factorial module breakdown of χ_0 on Task A** (30-trial means per configuration; Table 5.2). Both TDA and SA boost completion rates across all pairings, whereas MA provides gains only when paired with TDA or SA. Whiskers reflect the plotted evaluation summaries (Section 5.5.4). 128

5.8 **Model Arithmetic comparison on Task C** (30 rollouts per bar). All weight-merged policies exceed both unmerged baselines in mean completion rate and throughput, with OOD-validated greedy search achieving the highest completion rate (Section 5.7.1). Whiskers reflect the plotted evaluation summaries (Section 5.5.4). 130

5.9 **Stage Advantage evaluation on Tasks A and B** (developed by a co-author; 30 trials per bar). **Left**: temporal smoothness via SFR and MSTD. **Right**: closed-loop rollout metrics. 131

5.10 **Comparison of heuristic and standard DAgger on Task A** (30 rollouts per bar). Both recovery-collection schemes boost completion, throughput, and rubric score, with heuristic DAgger incurring more retries and standard DAgger achieving higher throughput and fewer retries on $\pi_{0.5}$. 132

5.11 **Comparison of heuristic and standard DAgger on Task C** (30 rollouts per bar). Both recovery schemes raise completion rates while heuristic DAgger increases retry frequency. 133

5.12 **Execution controllers and spatiotemporal augmentation on Task A** (30 rollouts per setting; T-scale: frame-skipping speed augmentation; S-mirror: horizontal mirroring with arm swap). 134

5.13 **Loss comparison between SA (by co-author Lirui Zhao; Chapter 5) and the $\pi^*_{0.6}$-style baseline.** SA converges to a lower terminal loss. 139

5.14 **Model Arithmetic on Tasks A and B** (30 rollouts per bar; Section 5.5.4). Merged policies beat full-data and single-best baselines on most metrics and lower all Task B retries (see Section 5.9 on panel b score bars). 140

5.15 **Stage Advantage on Task C** (co-author module, Chapter 5; 30 rollouts). SA improves MSTD, SFR, and retries ("Recover Cost"), whereas Direct+Stage trails in completion, throughput, and score. 141

5.16 **Task A control and augmentation across joint spaces** (30 rollouts per bar). Chunk-wise smoothing leads in nearly all regimes for absolute (a) and delta (b) joints; RTC helps in 3 of 4 absolute regimes versus 1 delta regime. 142

5.17 **Task B control and augmentation** (30 rollouts per bar). Delta joints trail absolute joints, completion stays near or below 30%, and chunk-wise smoothing gives no consistent margin. 143

5.18 **Task C control and augmentation** (30 rollouts per bar). Tight hanger-insertion tolerances keep completion modest and sensitive to joint parameterisation. 144

5.19 **Task A failure breakdown.** Distribution of 1026 failure events pooled across tested policies; flattening errors dominate (1010 instances), led by prolonged re-grasping (874). 145

6.1 Proposed anatomy of a self-correcting physical agent synthesising the perceive–reason–act contributions of this dissertation (* denotes co-authored works; BEV refers to BEVFormer [1]). Evidence for each architectural element comes from L0/L1 driving evaluations or offline-aligned L3 robot trials and is graded in Table 6.1. 152

List of Tables

2.1 **Comparison of scene representations.** Volumetric 3D occupancy models dynamic foreground actors and static background structure in one voxel grid without test-time LiDAR (training labels may still use LiDAR, as in OpenOcc). 21

2.2 **Comparison of OpenOcc with earlier occupancy datasets.** *Multi-view*: 360-degree cameras. *Flow*: per-voxel foreground velocity vectors. *Density*: mean occupied voxels per frame relative to OpenOcc (= 1). Concurrent dense nuScenes benchmarks [62, 70, 71] appear in Table 2.10. 26

2.3 **Semantic scene completion on OpenOcc** (single runs). BEV baselines attach a completion head to BEV features and appear under-tuned (Section 2.5.2). *: retrained on OpenOcc. R50/R101: ResNet50/ResNet101. 29

2.4 **LiDAR segmentation on nuScenes validation** (ResNet101). Camera methods are queried at ground-truth LiDAR points. *: trained on OpenOcc. †: published validation numbers from [30] (one decimal; Cylinder3D's 76.10 mIoU is reproduced as printed in [30], whereas the unweighted mean of its 16 one-decimal class cells is 76.21). 31

2.5 **Joint occupancy and 3D detection training** on nuScenes validation (ResNet50, single runs). Shifts of +0.007 to +0.013 NDS and 0 to +0.012 mAP lie within single-run noise. 32

2.6 **Pre-training transfer to BEV segmentation** (IoU; co-author-led, single runs; Mean IoU: 4-class mean). 33
2.7 **Open-loop planning across scene representations** on nuScenes under ST-P3 [66] (co-author-led, single runs; *full*: all 16 occupancy classes vs. vehicle/pedestrian only; †published ST-P3 [66]). 33
2.8 **Efficiency and accuracy of the three decoders** on OpenOcc SSC (ResNet50, single runs, V100 GPU). 35
2.9 **Voxelised 3D boxes vs. occupancy on irregular classes** (single runs; mIoU_{ir}: truck, trailer, construction vehicle mean; no regular-class control). 35
2.10 **Comparison of 3D occupancy dataset scale**, adapted from the OpenScene release notes [29] (selected entries shown). Duration values indicate source-corpus sensor hours. †Although omitted from the flow column in the public OpenScene table (under the name OccNet), OpenOcc includes foreground flow vectors (Section 2.4.1). 39
2.11 **History frame sweep on SSC** for OccNet with ResNet50 (single runs; #Num: past sweeps; *: two-frame default). 43
2.12 **History frame sweep on LiDAR segmentation** for OccNet with ResNet50 (single runs; #Num: past sweeps). 43
2.13 **Open-loop planning under voxel collisions** (co-author-led, single runs). 43
2.14 **Pre-training comparison for trajectory fine-tuning** (co-author-led, single runs; cumulative L2 and collisions). 43
2.15 **Per-category decoder comparison on OpenOcc** (single runs; R50/R101: ResNet50/ResNet101). 44
2.16 **Voxel size Δs sweep on LiDAR segmentation** using OccNet with ResNet50 (single runs). 45

3.1 **Comparison of DriveLM-Data with prior driving-language benchmarks.** DriveLM-Data provides dense frame-level QAs across **perception, prediction and planning** linked in a **graph**. Perc., Pred., Plan.: QAs per stage (✗: absent; 144k/153k/146k are rounded to the nearest thousand whereas 91.4 QA/frame is computed from the exact unrounded QA count 445,210/4,871); †: full dataset; ‡: keyframe subset; *: hybrid rule/human; **: rule-based; -: unreported. 57

3.2 **Open-loop planning on DriveLM-nuScenes `val` and zero-shot transfer to 1k Waymo `val` frames.** Conditioning motion (M) on behaviour (B) lets DriveLM-Agent ('None') beat UniAD-Single in-domain, where Chain/Graph context hurts; on Waymo (where UniAD-Single loses its rear camera while VLMs use only the front view), Graph context achieves the best behaviour accuracy and ADE. Italicised UniAD uses video and CAN-bus ego status. Single-run results. 61

3.3 **Transfer to unseen pedestrians on DriveLM-CARLA** (by co-authors; single runs). Adding a hand-crafted pedestrian prompt lifts DriveLM-Agent accuracy on DriveLM-CARLA-ped from 4.59 to 27.04 while lowering standard-scene accuracy from 59.63 to 52.17. Italicised 'GT' rows supply ground-truth context at test time, directly disclosing pedestrian presence under '+ Pedestrian QA'. All methods score 100.00 on steering on DriveLM-CARLA-ped because every target is `straight`. 63

3.4 **Question-level context ablation on DriveLM-nuScenes `val`** (single runs). Adding prediction and planning prompts (P_2, P_3) raises behaviour accuracy by 1.75–5.01 points over perception-only prompts (P_1), whereas intra-stage variations stay within 1.75 points. 65

3.5 **Stage P_{1-3} VQA baselines** (CARLA by co-authors; single runs). 'Graph' context raises GPT Score across all four model–dataset pairs and SPICE in three, though BLIP-2 SPICE on CARLA (10.46 → 10.30) and 'GT' GPT Score on nuScenes (42.97 → 41.10) dip slightly. 65

3.6 **Model size and inference cost.** DriveLM-Agent updates fewer trainable weights than UniAD-Single but incurs higher total parameter count, FLOPs and latency. 66

3.7 **Validation performance of PDM-Lite on the 20 official CARLA Leaderboard 2.0 routes** (by co-authors): Driving Score (DS), Route Completion (RC), Infraction Score (IS), Normalised Driving Score (NDS) and Normalised Infraction Score (NIS). Route cells show mean ± SD over 3 seeds; Avg. pools all 60 runs (36.3 ± 28.1 is the 60-run population mean and SD: 36.35, rounding to 36.4 from the one-decimal route cells, with sample route-mean SD 23.9). Routes 10–19 repeat routes 0–9 under altered weather/traffic; simulator crashes on routes 3 and 13 received floor scores. 74

3.8 **Full VQA metric suite on DriveLM-nuScenes `val`** (Comp.: SPICE > 0.5 percentage; single runs). 77

3.9 **Stage-wise context ablation for zero-shot transfer to Waymo** on 1k Waymo `val` frames (B^*: ground-truth behaviour QA context; single runs). 78

3.10 **Alternative VLM backbones on DriveLM-nuScenes** (*: multi-frame input to LLaMA-Adapter V2). 78

4.1 **Impact of TTT on `navtest` (NAVSIM v1.0, single seed).** TTT lifts PDMS under all uncertainty measures, led by Cluster Entropy. Prior fallback rows are audited against Table 4.2 in Section 4.5.2; three-seed v1.1 results appear in Table 4.3. 92

4.2 **Failure identification on `navtest`** (audited in Section 4.5.3). 93

4.3 **`navtest` leaderboard** (NAVSIM v1.1, March 2025, 3 seeds). *Hydra-MDP is a single run of the 2024 3-model ensemble. 93

4.4 **`navsafe` PDM scores** across **RDBT** (Roundabout), **YLLT** (Yellow light), **EXR** (Exit ramp), **UNPL** (Unprotected left), **ENR** (Enter ramp), **UNTS** (Uncommon traffic sign), **OTLC** (Overtaking with lane change), **NLA** (No lane area), **BWTH** (Bad weather), and **YLD** (Yielding). Preprint Overall [17] (74.14 vs. 56.47, 17.67 gap) differs from the frame-weighted (49.36 vs. 68.86, or 49.4 vs. 68.9) and simple (48.33 vs. 71.96, or 48.3 vs. 72.0) means in Table 4.10 (Section 4.4). . . 94

4.5 **Summary of mathematical symbols and definitions in Chapter 4.** 99

4.6 **Single-seed NAVSIM v1.0 results** (2024 Challenge protocol; cf. Table 4.3). 101

4.7 **NAVSIM v1.0 ablations and runtime.** Fallback is audited in Section 4.8; TTT* uses SVD; Hydra-SE$^{\mathbf{f}}$ buffers f steps; Hydra-SE$^{\mathbf{4}}$ gates TTT by uncertainty. 103

4.8 **Failure classification across models and cut-offs** (cf. Table 4.2; audited in Section 4.8). 104

4.9 **Implied fallback trigger proportions at the cut-offs of Table 4.2**, derived from Tables 4.1 and 4.2 at $p = 15.6\%$ (in %). 105

4.10 **Comparison of `navsafe` summary aggregations** from Table 4.4 (frame-weighted accurate to ± 0.05). 105

5.1 **Characteristics of the three cloth manipulation benchmarks.** Semantic sub-goals (S) match Stage Advantage (Section 5.4.3); episode counts and mean durations cover both base demonstrations and shorter DAgger recovery clips, with no fixed cutoff logged for Task C. 124

5.2 **Factorial component results on Task A (T-shirt flattening and folding).** Values report 30-trial evaluation summaries (10 rollouts per garment across three garments) from Fig. 5.7, paired with reconstructed success counts and 95% Wilson intervals (Section 5.5.4); throughput, retry cost, and 100-point rubric score are point means. 129

5.3 Fine-tuning configuration for $\pi_{0.5}$ post-training (not a controlled from-scratch budget; Section 5.8). 137

5.4 **Milestone point allocation for the average score metric (normalised to 100).** 138

6.1 **Evidence grading for the five elements of a self-correcting physical agent (Fig. 6.1),** distinguishing author-led chapter contributions from co-authored works. 151

Chapter 1

Introduction

A vehicle drives through an unfamiliar city and encounters a flatbed truck carrying an overhanging crane boom. A dual-arm robot trained to fold garments from human teleoperation is handed a T-shirt crumpled in an unseen configuration. Neither situation is exotic, yet each lies outside the agent's training distribution. This dissertation studies such conditions and how physical agents can be built and evaluated to cope with them across the perceive–reason–act loop.

1.1 Motivation

A *physical agent* perceives its surroundings through onboard sensors, reasons about scene semantics and multi-agent interactions, and acts through physical actuators in a closed control loop. This dissertation focuses on two representative embodiments: autonomous vehicles and robotic manipulators. Over the past decade, both fields have shifted from hand-engineered pipelines toward learning from data. Perception modules are trained on annotated multi-sensor logs, motion planners and visuomotor policies imitate human demonstrations, and large foundation models pretrained on web-scale vision and language corpora are adapted to driving and manipulation.

Between 2021 and 2026, when the research in this dissertation was conducted, three developments shaped the architecture of data-driven physical agents. Multi-camera

BEV models such as BEVFormer replaced isolated per-camera detections with shared spatiotemporal features for detection, mapping, and tracking [1, 2]. UniAD then trained perception, forecasting, and planning jointly, reducing open-loop planning error by optimising upstream features for the driving objective [3]. Most recently, multimodal foundation models brought web-scale visual and linguistic priors to driving and paired those priors with large manipulation datasets and generalist robot policies [4–6].

Learning from finite datasets, however, confronts a fundamental obstacle when deployed in the physical world. Formally, let s denote the underlying world state, o the sensor observation, and a the agent's action. Offline imitation learning draws training pairs from

$$p_{\text{train}}(o, a) \;=\; \sum_{s} p_{\text{train}}(s)\, p_{\text{train}}(o \mid s)\, \pi^{*}(a \mid o), \tag{1.1}$$

where π^{*} denotes the human demonstrator or expert controller; for continuous states, the sum in Eq. (1.1) becomes an integral. The equation is a one-step observation–action model and should not be confused with the closed-loop visitation distribution. At deployment, a learned policy π encounters a different state distribution $p_{\text{deploy}}(s)$ and sensor model $p_{\text{deploy}}(o \mid s)$, induces its own visitation distribution d_{π} rather than $d_{\pi^{*}}$, and acts through imperfect hardware dynamics $a_{\text{exec}} \sim p(a_{\text{exec}} \mid a)$. These differences allow stepwise errors to compound [7, 8]. They also interact with the *curse of rarity*: individually uncommon hazards occur frequently in aggregate [9]. This dissertation uses *curse of reality* for this combined mismatch across $p(s)$, $p(o \mid s)$, d_{π}, and $p(a_{\text{exec}} \mid a)$: whatever finite dataset an agent is trained on, physical deployment keeps presenting conditions absent from training.

Used here as an organising analytical lens, the curse of reality can be decomposed into four overlapping distribution shifts alongside two structural failure sources:

1. **Perceptual shift and representational limits.** Changes in the sensing distribution $p(o \mid s)$ arise from day-to-night illumination changes [2], camera–LiDAR feature-fusion limitations [10], or cross-dataset camera-rig changes (such as trans-

ferring from nuScenes [11] to Waymo [12]). Distinct from sensing shift is *representational limitation*: fixed 3D bounding boxes presuppose rigid cuboids and cannot faithfully represent articulated trailers, construction vehicles with protruding booms, or arbitrary roadside debris even for known categories [13].

2. **Semantic shift.** Changes in the world state distribution $p(s)$ expose the agent to unseen object categories or unusual multi-agent interactions; separately, 3D lane and topology benchmarks represent variation in road structure [14, 15]. While pretrained VLMs offer broad semantic priors, naive language conditioning can rely on ungrounded linguistic shortcuts without attending to visual evidence [16].

3. **Behavioural shift and rare scenarios.** In sequential control, closed-loop execution causes d_π to drift away from expert demonstrations d_{π^*}, while rare tail scenarios expose miscalibrated trajectory scoring in end-to-end planners [17].

4. **Deployment shift and demonstrator inconsistency.** In physical robot manipulation, discrepancies arise across recorded human demonstrations P_{train}, the learned policy distribution Q_{model}, and hardware-executed trajectories P_{test} due to state-space coverage gaps, action-chunk boundary discontinuities, and multimodal or suboptimal human demonstrators π^* [18].

The curse of reality is especially acute for physical agents for four reasons: control mistakes carry physical safety consequences; collecting and annotating dense 3D driving logs or long-horizon robot trajectories is expensive; closed-loop execution compounds small prediction errors; and onboard latency, memory, and compute budgets preclude achieving reliability solely by scaling model parameters at inference time.

To address these challenges systematically, this dissertation asks: *how can physical agents be made robust to conditions that the physical world presents but their training data do not contain?* Following the perceive–reason–act loop illustrated in Fig. 1.1, we investigate four research questions across a progressive three-stage research programme:

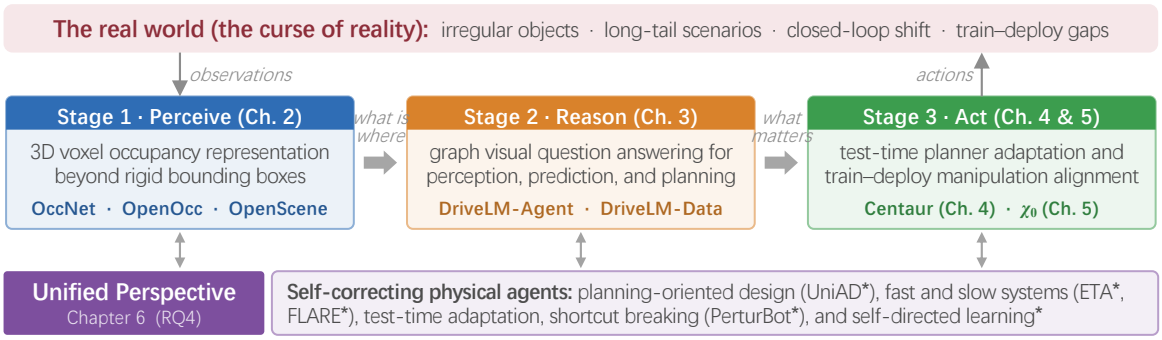


Fig. 1.1 **Overview of the dissertation.** The core chapters follow the perceive–reason–act loop of a physical agent confronting the curse of reality: Chapter 2 addresses 3D occupancy perception (Stage 1, RQ1), Chapter 3 addresses graph-structured vision-language reasoning (Stage 2, RQ2), and Chapters 4 and 5 address reliable action via test-time planner adaptation and resource-aware manipulation alignment (Stage 3, RQ3). Chapter 6 synthesises these findings into an evidence-graded architecture for self-correcting physical agents (RQ4). Works marked * are co-authored rather than led by the author.

RQ1 **Perceive (Stage 1).** How should a physical agent represent its 3D surroundings so that the representation remains geometrically valid for arbitrary obstacle shapes while remaining learnable from multi-view cameras and queryable by downstream driving tasks (Chapter 2)?

RQ2 **Reason (Stage 2).** How can the world knowledge of pretrained vision-language models be grounded in driving scenes through structured, verifiable perception-to-planning reasoning, and when does such reasoning improve generalisation under sensor and semantic shifts (Chapter 3)?

RQ3 **Act (Stage 3).** How can a learned policy remain reliable when deployment departs from training: (a) for end-to-end autonomous driving, can a planner adapt its per-frame decisions at test time without ground-truth labels or hand-crafted fallback rules (Chapter 4); and (b) for real-world robot manipulation, how can distributional inconsistencies across demonstrations, policy learning, and hardware execution be tamed under bounded data and compute budgets (Chapter 5)?

RQ4 Architecture (Synthesis). What design principles and architectural requirements for self-correcting physical agents follow from Stages 1–3, and what is the strength of the empirical evidence supporting each element (Chapter 6)?

Methodologically, the works in this dissertation follow a design-science paradigm [19, 20] built around a benchmark-driven research cycle: observing a deployment failure mode, formalising a task representation and metric, curating a public benchmark, establishing a scalable baseline, organising open community challenges (including the CVPR 2023 3D Occupancy Challenge and the 152-team CVPR 2024 "Driving with Language" Challenge), and analysing residual failures to motivate subsequent iterations. Fig. 1.2 traces the technical and dataset lineage across the twenty publications to which the author contributed between 2021 and 2026.

To prevent claims from being overstated across simulation regimes, every experimental result in this dissertation is reported at its explicit evaluation fidelity along a four-level ladder: **L0** open-loop evaluation on logged sensor frames (Chapters 2 and 3, where ego-status shortcuts must be accounted for on nuScenes [21, 22]); **L1** non-reactive short-horizon rollout simulation from logged states in NAVSIM [23] (Chapter 4, which evaluates 4-second trajectories with kinematic simulation but resets to logged states at each frame so closed-loop errors cannot compound); **L2** reactive closed-loop simulation in CARLA [24] (used by the rule-based PDM-Lite expert to generate DriveLM-CARLA in Chapter 3); and **L3** closed-loop deployment on physical robots (Chapter 5). Furthermore, following statistical best practices for empirical AI and robotics [25–27], single-seed training runs, benchmark construction biases (such as non-held-out stress-split selection in `navsafe` and answer-distribution skew in DriveLM-Data [16]), co-author-led experiments, and Wilson 95% confidence intervals for finite-trial robot evaluations are stated explicitly throughout the dissertation.

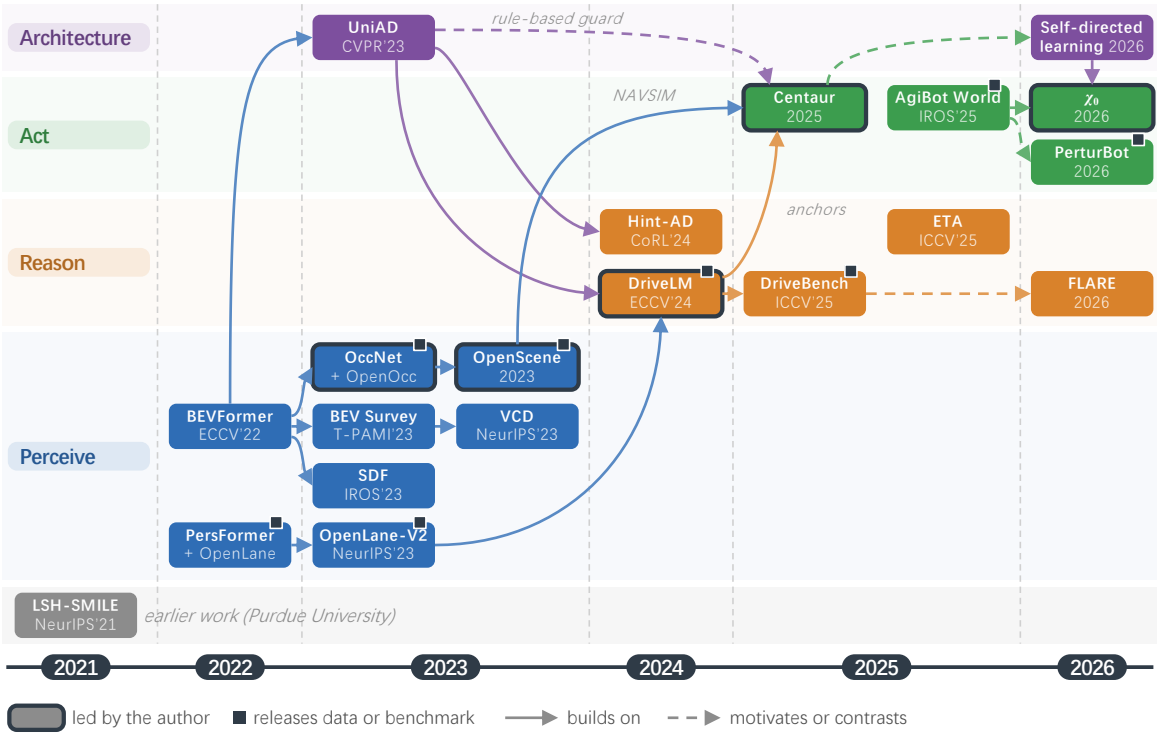


Fig. 1.2 **Research lineage of the publications to which the author contributed, organised by year and perceive–reason–act pillar.** Solid arrows denote direct technical or data inheritance; dashed arrows denote motivation or contrast; dark bold outlines mark the five author-led core-chapter works; squares mark dataset and benchmark releases. Two capsules (PersFormer + OpenLane and OccNet + OpenOcc) pair a method and benchmark from the same paper, so the 18 capsules represent the 20 outputs in Section 1.4.

1.2 Contributions

The main contributions of this dissertation are organised along the three stages of the perceive–reason–act loop, followed by related co-authored contributions.

Stage 1: Perceiving Open-World 3D Geometry.

OccNet, OpenOcc, and OpenScene (Chapter 2). Stage 1 contributes OccNet, OpenOcc, and OpenScene. OccNet extends a BEVFormer encoder with a coarse-to-fine 3D voxel decoder (contemporary with VoxFormer [28]); OpenOcc supplies 34,149 nuScenes keyframes with more than 1.4 billion labelled voxels across 16 classes and flow for the eight movable foreground classes; and OpenScene scales related supervision to

more than 120 hours of nuPlan logs [13, 29]. On OpenOcc, OccNet reaches 26.98 mIoU and 41.08 IoU_{geo}, compared with 23.67 and 37.47 for our retrained TPVFormer [30], and reaches 60.46 mIoU when queried at nuScenes LiDAR points. Occupancy also fits irregular vehicle geometry more closely than voxelised boxes (by 2.21 to 3.51 mIoU as voxel size tightens from 0.50 m to 0.25 m) and supports low-data detection pre-training. These findings are bounded: the vocabulary is closed at 16 classes, distribution shift was not tested, the 0.06–0.24 percentage-point planning advantage comes from single runs, and occupancy-based mechanisms did not improve the two NAVSIM planners examined in Chapter 4.

Stage 2: Structured Reasoning with Vision-Language Models.

DriveLM (Chapter 3). Stage 2 introduces Graph Visual Question Answering (GVQA), DriveLM-nuScenes, DriveLM-CARLA, and the DriveLM-Agent baseline [4]. GVQA connects perception (P_1), prediction (P_2), planning (P_3), behaviour (B), and motion (M) questions in a directed acyclic graph. The two datasets contain 4,871 nuScenes keyframes with 91.4 QA pairs per frame and about 1.6 million rule-generated CARLA QA pairs collected with PDM-Lite [31], respectively, and supported the 152-team CVPR 2024 "Driving with Language" Challenge. Under zero-shot transfer to 1,000 Waymo frames, full graph context yields 2.63 m ADE (versus 4.16 m for UniAD-Single) and raises speed accuracy from 43.90% to 54.29%. Most of the trajectory advantage over UniAD-Single is already present without graph context, however, and the comparison is confounded by a roughly thirtyfold parameter difference and asymmetric camera inputs. In-domain graph context worsens behaviour accuracy, ADE, and collision rate; on withheld CARLA pedestrians, recovery requires a class-specific prompt.

Stage 3: Self-Correction and Alignment in Action.

Centaur (Chapter 4). To adapt end-to-end driving planners at deployment without ground-truth labels or hand-crafted fallback rules, we propose Centaur [17], which updates planner decoder parameters via a single gradient step of test-time training (TTT) minimising Cluster Entropy, a label-free uncertainty measure computed over

five lateral driving-direction clusters inspired by DriveLM. We also curate `navsafe`, a 229-frame, ten-category safety-critical diagnostic subset of `navtest`. In non-reactive L1 NAVSIM v1 simulation on logged frames (where states reset every frame so multi-step errors cannot compound), Centaur improves Hydra-MDP from 90.3 to 92.6 PDMS in a single-seed NAVSIM v1.0 run and achieves 92.10 ± 0.33 PDMS across three seeds on the official NAVSIM v1.1 leaderboard (ranking first among three-seed entries in March 2025, vs. 94.8 for human trajectories). Across ablation studies, all four TTT objectives on Hydra-MDP gain 1.2–2.3 PDMS (0.8–2.3 PDMS across base planners, or 0.4 with SVD gradient projection) whereas uncertainty-triggered fallback to prior trajectories degrades all tested planners to 59.9–67.4 PDMS (63.1–65.9 PDMS on Hydra-MDP). Key qualifications are stated explicitly: the three-seed v1.1 margin over standard semantic entropy is small (92.10 ± 0.33 vs. 91.87 ± 0.19, 0.23 PDMS) and lacks a matched three-seed unadapted Hydra-MDP entry; Cluster Entropy's standalone failure classification accuracy at a fixed cutoff (73.6%, TPR 62.8%) trails a trivial never-flag baseline (84.4% at a 15.6% failure rate); and on `navsafe` (which was selected using base-planner scores rather than held out), Centaur improves frame-weighted mean PDMS from 49.4 to 68.9 but regresses on yielding (59.3 vs. 71.6) because lateral clustering omits longitudinal stop-or-go ambiguity.

χ_0 (Chapter 5). Extending reliable action to closed-loop physical robot manipulation (L3 fidelity), we formulate long-horizon deformable manipulation failures as distributional inconsistencies across human demonstrations P_{train}, policy-induced trajectory distribution Q_{model}, and hardware execution P_{test}, and introduce χ_0 [18] (co-led with L. Chen). χ_0 combines three modules under a bounded post-training budget ($\sim$20 hours of base demonstrations per task, or 22.5 to 33.6 hours including targeted DAgger recovery rollouts, on eight A100 GPUs): Model Arithmetic (merging checkpoints trained on diverse data subsets in weight space; author contribution), Stage Advantage (stage-aware progress estimation for advantage-weighted policy training; co-author component by L. Zhao), and Train–Deploy Alignment (combining targeted DAgger failure-recovery trajectories, spatiotemporal observation augmentation, heuristic tem-

poral chunk smoothing, and real-time chunking; author contribution). When post-training $\pi_{0.5}$ [6] on dual-arm bimanual garment manipulation, χ_0 raises 30-trial Task A (T-shirt flattening and folding) completion rate from $\sim$27% to $\sim$97% (8/30 vs. 29/30 successes, chart-read from the factorial progression; Wilson 95% confidence intervals 14–44% vs. 83–99%) while increasing hourly throughput roughly elevenfold ($\approx$ 6 to $\approx$ 66 tasks/h) across long-horizon cyclic rollouts. We note that targeted DAgger recovery data accounts for the largest single-step improvement and that total dataset size was not varied against alignment at a matched resource budget.

Additional Contributions. Beyond the four core chapters, fifteen additional publications (including first-authored and co-first-authored works) and three US patents trace the broader perceive–reason–act programme (Fig. 1.2). In 3D perception and architecture, PersFormer and the OpenLane benchmark [14] (co-first author) established single-camera 3D lane detection; BEVFormer [1] and our T-PAMI BEV survey [2] (co-first author, winning 1st place in the 2022 Waymo 3D Camera-Only Detection Challenge) established multi-camera BEV perception recipes; SDF [10] and VCD [32] addressed camera–LiDAR feature fusion and LiDAR-guided distillation, respectively; OpenLane-V2 [15] introduced road-topology reasoning; and UniAD [3] (CVPR 2023 Best Paper Award) unified perception, prediction, and planning in a single end-to-end network. In language-grounded reasoning, Hint-AD [33] aligned natural-language explanations with end-to-end driving features; DriveBench [16] audited VLM driving reliability, exposing visual-grounding shortcuts and dataset answer skew in DriveLM-Data; ETA [34] mitigated VLM inference latency via an asynchronous dual-speed fast–slow architecture; and FLARE [35] distilled future-aware VLM latent priors without test-time text generation. In embodied manipulation and simulation, LSH-SMILE [36] (first author) accelerated physics simulation and learning via locality-sensitive hashing; AgiBot World Colosseo [5] (IROS 2025 Best Paper Finalist) established a 1-million-trajectory, 100-robot manipulation platform and the GO-1 policy; PerturBot [37] diagnosed and mitigated visual-language-motor shortcuts in VLA policies; and our

CVPRW 2026 perspective paper [38] formulated self-directed robot learning beyond human imitation.

1.3 Thesis Outline

The remainder of this dissertation is organised into five chapters following the perceive–reason–act loop, supplemented by Appendix A.

Chapter 2 (*Perceive: Scene as Occupancy*) introduces 3D semantic occupancy prediction, the OccNet cascade voxel-decoder architecture, the OpenOcc benchmark on nuScenes, and the large-scale OpenScene dataset on nuPlan, evaluating multi-task 3D perception and open-loop motion planning at L0 fidelity.

Chapter 3 (*Reason: Driving with Graph Visual Question Answering*) formulates Graph Visual Question Answering (GVQA), presents the DriveLM-nuScenes and DriveLM-CARLA datasets and evaluation metrics, and evaluates the DriveLM-Agent baseline on in-domain VQA as well as zero-shot cross-sensor (Waymo) and unseen-object (CARLA) transfer at L0 fidelity.

Chapter 4 (*Act I: Robust End-to-End Driving via Test-Time Training*) presents Centaur, which adapts end-to-end trajectory-scoring planners online via single-step test-time training on Cluster Entropy, and evaluates per-frame adaptation and failure detection on NAVSIM `navtest` and the 229-frame `navsafe` diagnostic split at L1 fidelity.

Chapter 5 (*Act II: Resource-Aware Robust Manipulation*) analyses distributional inconsistencies across demonstration collection, policy training, and hardware execution in real-world robot manipulation, and evaluates χ_0 (Model Arithmetic, Stage Advantage, and Train–Deploy Alignment) on physical dual-arm garment manipulation tasks at L3 fidelity.

Chapter 6 (*Conclusion*) summarises the stage-by-stage contributions, synthesises a unified perspective and evidence-graded architecture for self-correcting physical agents, and discusses thesis-wide limitations and future research directions.

1.4 Related Publications

* denotes equal contribution; † denotes project lead. The author's role is stated for every entry: "first author", "co-first author" (equal contribution), "project lead" or "co-project lead", or "co-author" where the author is neither first author nor project lead.

Core Publications Included in This Dissertation

[1] W. Tong*, **C. Sima***†, T. Wang, L. Chen, S. Wu, H. Deng, Y. Gu, L. Lu, P. Luo, D. Lin, and H. Li†. "Scene as Occupancy" [13]. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 8406–8415, 2023. *Role:* co-first author and co-project lead (ICCV proceedings list W. Tong first; arXiv:2306.02851 lists C. Sima first). Lead organiser of the CVPR 2023 3D Occupancy Prediction Challenge (co-hosted with Occ3D on Occ3D-nuScenes) and contributor of OpenOcc v2 to the CVPR 2024 Autonomous Grand Challenge. (Chapter 2)

[2] OpenScene Contributors. "OpenScene: The Largest Up-to-Date 3D Occupancy Prediction Benchmark in Autonomous Driving" [29]. Dataset and toolkit release, `https://github.com/OpenDriveLab/OpenScene`, 2023. A 2 Hz redistribution of nuPlan with occupancy and flow labels. *Role:* project lead (with the OpenScene Contributors). (Chapter 2)

[3] **C. Sima***, K. Renz*, K. Chitta, L. Chen, H. Zhang, C. Xie, J. Beißwenger, P. Luo, A. Geiger†, and H. Li†. "DriveLM: Driving with Graph Visual Question Answering" [4]. In *European Conference on Computer Vision (ECCV)*, pp. 256–274, 2024. **Oral presentation.** *Role:* co-first author, and lead organiser of the "Driving with Language" track of the CVPR 2024 Autonomous Grand Challenge (152 teams). (Chapter 3)

[4] **C. Sima***, K. Chitta*, Z. Yu, S. Lan, P. Luo, A. Geiger, H. Li, and J. M. Alvarez. "Centaur: Robust End-to-End Autonomous Driving with Test-Time

Training" [17]. *arXiv preprint arXiv:2503.11650*, 2025. *Role:* co-first author. (Chapter 4)

[5] C. Yu, **C. Sima**†, G. Jiang, H. Zhang, H. Mai, H. Li, H. Wang, J. Chen, K. Wu, L. Chen†, L. Zhao, M. Shi, P. Luo, Q. Bu, S. Peng, T. Li, and Y. Yuan. "χ_0: Resource-Aware Robust Manipulation via Taming Distributional Inconsistencies" [18]. *arXiv preprint arXiv:2602.09021*, 2026. *Role:* co-project lead, with L. Chen (authors listed alphabetically by first name). (Chapter 5)

Other Related Publications

[6] Y. Hu, J. Yang, L. Chen, K. Li, **C. Sima**, X. Zhu, S. Chai, S. Du, T. Lin, W. Wang, L. Lu, X. Jia, Q. Liu, J. Dai, Y. Qiao, and H. Li. "Planning-oriented Autonomous Driving" (UniAD) [3]. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 17853–17862, 2023. **Best Paper Award.** *Role:* co-author.

[7] AgiBot-World-Contributors: Q. Bu, J. Cai, L. Chen, X. Cui, Y. Ding, S. Feng, X. He, X. Huang, *et al.* (including **C. Sima**). "AgiBot World Colosseo: A Large-Scale Manipulation Platform for Scalable and Intelligent Embodied Systems" [5]. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025. **Best Paper Finalist.** *Role:* co-author; the cited IROS release lists contributors alphabetically by surname, and the public contributor roster is version-dependent (co-led manuscript preparation).

[8] M. Liu, **C. Sima**, T. Feng, H. Wang, C. Chen, H. Chen, and C. Shen. "PerturBot: Breaking Shortcut Priors in Vision-Language-Action Models with Perturbative Training" [37]. *arXiv preprint arXiv:2610.04616*, 2026. *Role:* co-author.

[9] C. Xie, **C. Sima**, T. Li, B. Sun, J. Wu, Z. Hao, and H. Li. "FLARE: Learning Future-Aware Latent Representations from Vision-Language Models for Au-

tonomous Driving" [35]. *arXiv preprint arXiv:2601.05611*, 2026. *Role:* co-author.

[10] L. Chen, **C. Sima**, K. Chitta, A. Loquercio, P. Luo, Y. Ma, and H. Li. "Intelligent Robot Manipulation Requires Self-Directed Learning" [38]. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pp. 842–853, 2026. *Role:* co-author.

[11] S. Hamdan, **C. Sima**, Z. Yang, H. Li, and F. Güney. "ETA: Efficiency Through Thinking Ahead, A Dual Approach to Self-Driving with Large Models" [34]. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 26529–26538, 2025. *Role:* co-author.

[12] S. Xie, L. Kong, Y. Dong, **C. Sima**, W. Zhang, Q. A. Chen, Z. Liu, and L. Pan. "Are VLMs Ready for Autonomous Driving? An Empirical Study from the Reliability, Data, and Metric Perspectives" (DriveBench) [16]. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 6585–6597, 2025. *Role:* co-author.

[13] K. Ding, B. Chen, Y. Su, H.-a. Gao, B. Jin, **C. Sima**, W. Zhang, X. Li, P. Barsch, H. Li, and H. Zhao. "Hint-AD: Holistically Aligned Interpretability in End-to-End Autonomous Driving" [33]. In *Proceedings of the 8th Conference on Robot Learning (CoRL 2024)*, PMLR 270, pp. 3742–3765, published 2025. *Role:* co-author.

[14] H. Wang, T. Li, Y. Li, L. Chen, **C. Sima**, Z. Liu, B. Wang, P. Jia, Y. Wang, S. Jiang, F. Wen, H. Xu, P. Luo, J. Yan, W. Zhang, and H. Li. "OpenLane-V2: A Topology Reasoning Benchmark for Unified 3D HD Mapping" [15]. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 36, pp. 18873–18884, 2023. *Role:* co-author.

[15] L. Huang, Z. Li, **C. Sima**, W. Wang, J. Wang, Y. Qiao, and H. Li. "Leveraging Vision-Centric Multi-Modal Expertise for 3D Object Detection" (VCD) [32].

In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, pp. 38504–38519, 2023. *Role:* co-author.

[16] Y. Gao, **C. Sima**, S. Shi, S. Di, S. Liu, and H. Li. "Sparse Dense Fusion for 3D Object Detection" (SDF) [10]. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 10939–10946, 2023. *Role:* co-author.

[17] H. Li*, **C. Sima***, J. Dai*, W. Wang*, L. Lu*, H. Wang*, J. Zeng*, Z. Li*, J. Yang*, H. Deng*, *et al.* "Delving into the Devils of Bird's-Eye-View Perception: A Review, Evaluation and Recipe" [2]. *IEEE Transactions on Pattern Analysis and Machine Intelligence (T-PAMI)*, vol. 46, no. 4, pp. 2151–2170, 2024 (published online in 2023). *Role:* co-first author.

[18] L. Chen*, **C. Sima***, Y. Li*, Z. Zheng, J. Xu, X. Geng, H. Li, C. He, J. Shi, Y. Qiao, and J. Yan. "PersFormer: 3D Lane Detection via Perspective Transformer and the OpenLane Benchmark" [14]. In *European Conference on Computer Vision (ECCV)*, LNCS 13698, pp. 550–567, 2022. **Oral presentation.** *Role:* co-first author.

[19] Z. Li, W. Wang, H. Li, E. Xie, **C. Sima**, T. Lu, Y. Qiao, and J. Dai. "BEVFormer: Learning Bird's-Eye-View Representation from Multi-Camera Images via Spatiotemporal Transformers" [1]. In *European Conference on Computer Vision (ECCV)*, pp. 1–18, 2022. *Role:* co-author.

[20] **C. Sima** and Y. Xue. "LSH-SMILE: Locality Sensitive Hashing Accelerated Simulation and Learning" [36]. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, pp. 7484–7496, 2021. *Role:* first author.

Patents

The author is a co-inventor of the following US patents and patent applications:

[P1] H. Li, Z. Li, W. Wang, **C. Sima**, L. Chen, Y. Li, Y. Qiao, and J. Dai. "Image Processing Method, Apparatus and Device, and Computer-Readable Storage Medium." US Patent 12,469,277, 2025.

[P2] H. Li, L. Chen, S. Gao, J. Yang, Y. Qiu, **C. Sima**, T. Li, J. Zeng, Y. Li, H. Wang, J. Yan, P. Luo, and Y. Qiao. "Method for Training Autonomous Driving Model, Electronic Device, and Storage Medium." US Patent Application 18/936,908, 2025.

[P3] H. Li, L. Chen, S. Gao, J. Yang, Y. Qiu, **C. Sima**, T. Li, J. Zeng, Y. Li, H. Wang, J. Yan, P. Luo, and Y. Qiao. "Method for Training Autonomous Driving Model, Method for Predicting Autonomous Driving Video, Electronic Device, and Storage Medium." US Patent Application 18/888,671, 2025.

Chapter 2

Perceive: Scene as Occupancy

Statement of contribution

This chapter draws on a conference paper, a benchmark release, and two international challenges:

- *Scene as Occupancy* (ICCV 2023) [13]. Chonghao Sima and Wenwen Tong are co-first authors (Tong listed first in ICCV proceedings, Sima first on arXiv), with Hongyang Li as co-corresponding lead. Sima formulated the occupancy scene representation, designed the cascade voxel decoder and 3D deformable attention, built the OpenOcc occupancy-and-flow pipeline, ran the semantic scene completion and LiDAR segmentation experiments, and drafted the paper. Tong co-developed the codebase, ran the 3D detection, BEV segmentation, joint-training, and irregular-obstacle experiments (Tables 2.5 and 2.9), and co-wrote the manuscript. Tai Wang and Li Chen conducted the pre-training and planning evaluations (Fig. 2.5, Tables 2.6 and 2.7, Fig. 2.6, Tables 2.13 and 2.14, and Fig. 2.12), credited in their captions.
- *OpenScene* (dataset release, 2023) [29], led by the author under "OpenScene Contributors". The author curated the nuPlan logs, generated the 3D occupancy and flow labels, built the devkit, and integrated the benchmark into the CVPR 2024 and CVPR 2025 challenges.
- *Challenges.* The author led the CVPR 2023 3D Occupancy Prediction Challenge (evaluated on Occ3D-nuScenes) and contributed OpenOcc v2 to the CVPR 2024 Autonomous Grand Challenge Occupancy and Flow track (Section 2.6.3).

© 2023 IEEE. Reprinted, with permission, from W. Tong, C. Sima, T. Wang, L. Chen, S. Wu, H. Deng, Y. Gu, L. Lu, P. Luo, D. Lin and H. Li, "Scene as Occupancy", in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2023. In reference to IEEE copyrighted material which is used with permis-

sion in this thesis, the IEEE does not endorse any of The University of Hong Kong's products or services. Internal or personal use of this material is permitted. If interested in reprinting/republishing IEEE copyrighted material for advertising or promotional purposes or for creating new collective works for resale or redistribution, please go to `http://www.ieee.org/publications_standards/publications/rights/rights_link.html` to learn how to obtain a License from RightsLink.

An embodied agent can only act on what its internal representation resolves. Before braking, changing lanes, or yielding, an automated vehicle must determine which 3D regions contain physical matter and what occupies them. Classical driving perception splits this task into two approximations: dynamic actors are enclosed in 3D cuboids from a fixed taxonomy, while static road layout is projected onto a planar bird's-eye-view (BEV) grid. This split assumes that every hazard belongs to a catalogued class, fits a rigid box, and lacks relevant vertical structure. Urban streets violate all three assumptions: truck-mounted cranes swing past their chassis, flatbeds carry overhanging pipes, gantries and bridges restrict vertical clearance, and irregular barriers border the road. Cuboids clip protruding geometry, whereas planar BEV maps discard elevation.

This chapter addresses the first research question (RQ1): whether an embodied driving system can represent arbitrary 3D obstacle geometry from surround-view cameras in a form that downstream planners can query. We use *3D occupancy*: a regular ego-centric voxel grid in which occupied cells receive semantic labels and dynamic cells receive motion vectors. The grid imposes no cuboid shape prior and represents foreground objects and background structure in one coordinate frame, although its 16-class vocabulary remains closed. We evaluate occupancy as a shared scene descriptor on nuScenes under single-run perception and open-loop planning experiments. The study covers unusual geometries within known classes; it does not test open-set or cross-domain generalisation.

This chapter makes four contributions. First, OccNet reconstructs dense 3D occupancy from a spatiotemporal BEV representation through a cascade voxel decoder

with temporal attention and 3D deformable attention[1]. Second, OpenOcc provides dense foreground and background semantics together with foreground velocity vectors on nuScenes [11, 40]. Third, experiments cover semantic scene completion (SSC), point-queried LiDAR segmentation, 3D detection, BEV segmentation, and open-loop planning against planar BEV baselines and retrained TPVFormer [30]; the small open-loop planning differences (0.06–0.24 percentage points) are reported as single-run observations rather than established safety gains. Fourth, OpenScene extends occupancy and flow labels from 5.5 hours of nuScenes to more than 120 hours of multi-city nuPlan driving [29], whose redistributed sensor streams underpin NAVSIM and the CVPR 2024 and CVPR 2025 Autonomous Grand Challenge tracks (which use the sensor streams rather than the voxel labels). Methodologically, OccNet shares BEVFormer [1] and 3D deformable-attention [41] foundations with contemporaneous VoxFormer [28], while contributing the coarse-to-fine height cascade, voxel temporal self-attention, OpenOcc pipeline, and cross-task evaluation.

Sections 2.1 and 2.2 cover motivation and related work; Section 2.3 presents OccNet; Section 2.4 details OpenOcc; Section 2.5 reports main experiments; Section 2.6 discusses limitations, NAVSIM comparisons (Chapter 4), and 2023–2026 work; Section 2.7 describes OpenScene; Section 2.8 presents implementation details and additional ablations; and Section 2.9 summarises the chapter.

2.1 Motivation

Human drivers judge 3D obstacle geometry before identifying fine-grained categories, whether passing a saloon car or trailing a flatbed lorry with an overhanging pipe. For automated vehicles, determining whether a 3D volume is physically occupied precedes category recognition, yet inferring 3D structure from cameras alone is challenging across cluttered urban streets.

[1]Distinct from the continuous single-object Occupancy Networks of Mescheder *et al.* [39].

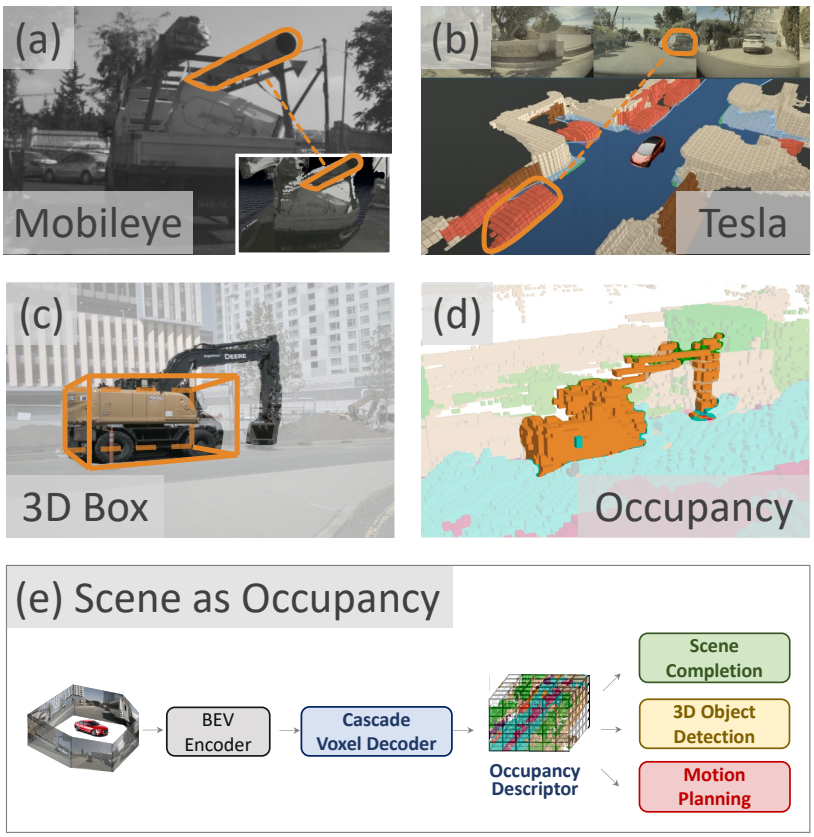


Fig. 2.1 **Scene as occupancy.** Industrial camera systems estimate pseudo-LiDAR points via Mobileye's VIDAR [42] (a) or 3D voxel occupancy as in Tesla's stack [43] (b) (frames adapted from [42, 43]). Rigid 3D boxes miss overhanging cargo (a) or crane booms (c), whereas 3D occupancy (d) captures arbitrary contours as a shared descriptor (e) for perception and planning.

Partitioning space into semantically labelled voxels (3D occupancy) directly captures arbitrary 3D geometry. Industrial stacks have adopted similar designs: Mobileye estimates pseudo-LiDAR via VIDAR [42], and Tesla predicts 3D occupancy grids [43] (Fig. 2.1(a)–(b)).[2] Whereas a rigid cuboid encloses only a truck's cab, voxels trace its extended crane boom (Fig. 2.1(c)–(d)).

Table 2.1 contrasts 3D occupancy with standard representations: 3D boxes cover only enumerated foreground categories; LiDAR segmentation resolves $\sim$0.02 m surfaces but requires range sensors at deployment; and BEV segmentation collapses height. Volumetric occupancy unifies foreground and background in 3D at 0.25–

[2]Mobileye's VIDAR differs from ViDAR [44], the point-cloud forecasting model in Section 2.7.

Table 2.1 **Comparison of scene representations.** Volumetric 3D occupancy models dynamic foreground actors and static background structure in one voxel grid without test-time LiDAR (training labels may still use LiDAR, as in OpenOcc).

Representation	Output space	Foreground object	Background & mapping	Description granularity	Requires point cloud input
3D box	3D	✓	–	0.4–12 m	–
BEV seg.	BEV	✓	✓	0.5–1 m	–
Point cloud	3D	✓	–	∼0.02 m	✓
3D occupancy	3D	✓	✓	0.25–0.5 m	–

0.5 m resolution from cameras alone. Unlike classical binary occupancy grids [45] and early SSC models [28, 30, 46–48], which mostly processed single frames (except VoxFormer-T [28]) and were disconnected from planning, OccNet lifts temporal BEV features [1, 14, 49] into a shared 3D occupancy descriptor for SSC, 3D detection, BEV segmentation, and open-loop planning; planning-oriented end-to-end design is represented separately by UniAD [3]. Trained on OpenOcc (34,149 nuScenes frames [11, 40], >1.4B voxels across 16 classes plus unbenchmarked foreground flow), our experiments show three results (qualified in Sections 2.5 and 2.6):

- *Perception.* OccNet reaches 26.98 mIoU on OpenOcc SSC (+14% relative over retrained TPVFormer [30] at 23.67 mIoU) and trails LiDAR RangeNet++ [50] by ∼5 mIoU on point-queried LiDAR segmentation. Occupancy pre-training lifts low-data 3D detection (1/256–1/64 splits) over FCOS3D [51] initialisation by up to ∼10 mAP at 1/128.

- *Open-loop planning.* Rasterised occupancy lowers open-loop nuScenes collision rates by 0.06–0.24 percentage points relative to boxes from OccNet's detection head (single runs), without uniform L2 gains.

- *Supervision density.* Dense OpenOcc pre-training beats sparse LiDAR-voxel pre-training at the 1/128 detection split while performing similarly at 1/256 and 1/64.

2.2 Related Work

We situate 3D occupancy against standard driving perception outputs and contemporaneous SSC models; broader multi-camera 3D perception is surveyed in Chapter 1 and [2], while 2023–2026 occupancy work appears in Section 2.6.3.

3D detection, LiDAR segmentation, and BEV segmentation. 3D detectors [1, 51–53] fit oriented cuboids to foreground actors, omitting background hazards and distorting articulated geometry. Point-cloud segmenters [50, 54] label 3D laser returns directly but require LiDAR hardware and suffer from distance-dependent sparsity and occlusion [46]. Planar BEV segmentation [55, 56] jointly maps road layout and actors but discards vertical clearance (gantries, overhangs, bridges). Volumetric occupancy retains 3D height and arbitrary contours from cameras alone.

3D reconstruction and semantic scene completion. Multi-view 3D reconstruction [57, 58] and neural implicit fields [39, 59–61] typically optimise per-scene geometry or rendering rather than real-time feed-forward semantic perception. Semantic scene completion was established for depth input by SSCNet [46]; camera-only SSC was pioneered by MonoScene [47] and expanded concurrently by depth-seeded VoxFormer [28], stereo-distilled OccDepth [48], tri-perspective TPVFormer [30], and OpenOccupancy [62]. Earlier studies targeted front-view SemanticKITTI [63] or indoor NYUv2 [64], and apart from VoxFormer-T [28], BEVDet4D [65], and unpublished industry systems [43], contemporaneous SSC models were single-frame and disconnected from motion planning.

2.3 OccNet

As shown in Fig. 2.2, OccNet maps multi-camera images to a shared 3D *occupancy descriptor* in two phases: *occupancy reconstruction* and *occupancy exploitation*.

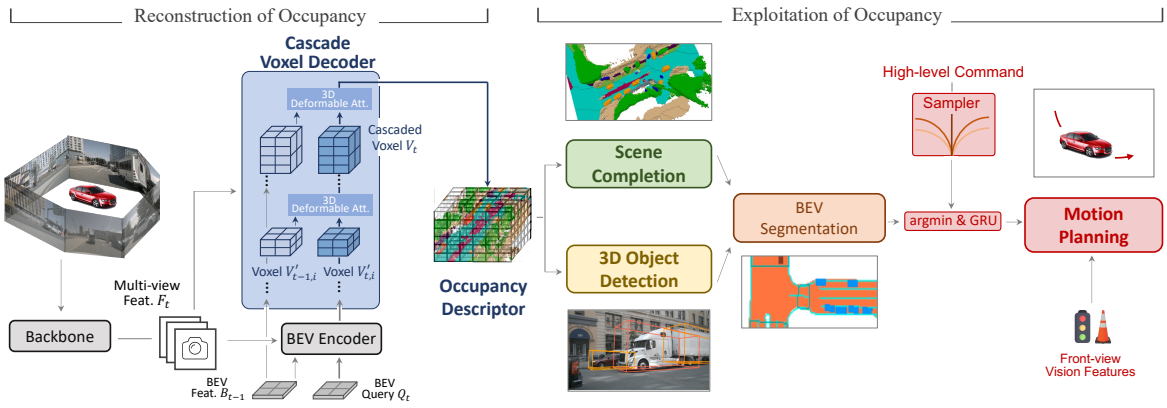


Fig. 2.2 **The OccNet pipeline. I. Occupancy reconstruction.** Surround-view images pass through a spatiotemporal BEV encoder, after which a cascade voxel decoder progressively expands vertical resolution, fusing historical voxel features $V'_{t-1,i}$ and image features via 3D deformable attention (skip connections omitted). **II. Occupancy exploitation.** Task heads decode semantic scene completion and 3D boxes from the shared descriptor; vertical projection yields a BEV obstacle map for trajectory planning [66].

Reconstruction and exploitation. To lift 2D camera features [1, 14, 67] into 3D voxel space, we contrast two baseline extremes: *BEVNet*, which feeds a 2D BEV grid directly into task heads at minimal cost but lacks height resolution, and *VoxelNet*, which maintains full-resolution 3D voxel queries throughout at high memory and latency cost.[3] OccNet bridges both extremes, matching or exceeding VoxelNet's completion accuracy with fewer parameters and less memory while outperforming BEVNet (Section 2.5.3). A BEVFormer encoder [1] combines multi-camera features F_t, previous BEV features B_{t-1}, and queries Q_t into B_t, which the cascade voxel decoder (Section 2.3.1) lifts alongside B_{t-1} and F_t into the 3D occupancy descriptor. Following UniAD [3], modular heads decode 3D SSC [46] and 3D boxes, whose vertical projections form a binary BEV obstacle map for a command-conditioned sampling and GRU-refinement planner (Section 2.3.2).

[3]BEVNet and VoxelNet denote the ablation baselines in this chapter; our camera-based VoxelNet differs from the LiDAR detector of Zhou and Tuzel [68].

2.3.1 Cascade Voxel Decoder

From BEV to cascaded voxels. Decoding 3D voxels directly from 2D BEV loses vertical fidelity, whereas full-grid 3D transformers incur heavy memory and latency costs (Table 2.8). We factor the mapping from $B_t \in \mathbb{R}^{H\times W\times C_{\text{BEV}}}$ to $V_t \in \mathbb{R}^{Z\times H\times W\times C_{\text{Voxel}}}$ across N stages, where $H \times W$ is horizontal resolution, Z is final voxel height, and $C_{\text{BEV}}, C_{\text{Voxel}}$ are channel widths. Stage i produces $V'_{t,i} \in \mathbb{R}^{Z_i\times H\times W\times C_i}$, doubling height Z_i towards Z while halving channels C_i from C_{BEV} to C_{Voxel} (Section 2.5.1). Linear projections lift B_{t-1} and B_t to $V'_{t-1,1}$ and $V'_{t,1}$, and inter-stage MLPs map $Z_i \times C_i$ to $Z_{i+1} \times C_{i+1}$ (Section 2.8.1). Each stage applies voxel temporal self-attention between $V'_{t,i}$ and ego-motion-aligned $V'_{t-1,i}$, followed by voxel spatial cross-attention over image features F_t. On OpenOcc, adding two history frames improves single-run mIoU by 0.27 points over single-frame input, and four frames by 1.60 points (Table 2.11).

3D deformable attention and spatial cross-attention. Because dense 3D self-attention is quadratic in the voxel token count ZHW, we extend 2D deformable attention [41] to 3D (*3D-DA*). Given volume $V'_{t,i} \in \mathbb{R}^{Z_i\times H\times W\times C_i}$, query $\boldsymbol{q} \in \mathbb{R}^{C_i}$, and normalised 3D reference point $\boldsymbol{p} \in [0,1]^3$, 3D-DA computes

$$\text{3D-DA}(\boldsymbol{q}, \boldsymbol{p}, V'_{t,i}) = \sum_{m=1}^{M} W_m \sum_{k=1}^{K} A_{mk}(\boldsymbol{q})\, W'_m\, \phi\Big(V'_{t,i};\, \boldsymbol{p} + \Delta\boldsymbol{p}_{mk}(\boldsymbol{q})\Big), \tag{2.1}$$

with $M = 8$ heads, $K = 4 \ll Z_iHW$ sampling points per head, projections $W_m \in \mathbb{R}^{C_i\times(C_i/M)}$ and $W'_m \in \mathbb{R}^{(C_i/M)\times C_i}$, softmax weights $A_{mk}(\boldsymbol{q}) \geq 0$ ($\sum_k A_{mk}(\boldsymbol{q}) = 1$), and signed learned 3D offsets $\Delta\boldsymbol{p}_{mk}(\boldsymbol{q}) \in \mathbb{R}^3$ evaluated via zero-padded trilinear sampling $\phi(V'_{t,i}; \boldsymbol{p} + \Delta\boldsymbol{p}_{mk}(\boldsymbol{q}))$. For spatial cross-attention, each voxel in $V'_{t,i}$ projects $N_{\text{ref},i}$ 3D reference heights onto the camera planes and aggregates multi-scale features F_t via 2D deformable attention [41], preserving per-voxel height resolution rather than collapsing pillars as in BEVFormer [1].

2.3.2 Exploiting Occupancy for Driving Tasks

From the shared descriptor V_t, lightweight heads decode four tasks. For *semantic scene completion*, an MLP trained with focal loss [69] classifies each voxel into a semantic class or empty space, while an L_1-supervised branch regresses horizontal velocity vectors for occupied foreground voxels. For *3D object detection*, V_t is pooled vertically into a BEV map for a BEVFormer-style Deformable DETR head [1, 41]. For *BEV segmentation*, four ST-P3 branches [66] decode drivable area, lanes, vehicles, and pedestrians from pooled BEV features. For *motion planning*, predicted 3D occupancy or 3D boxes are projected vertically into a binary BEV obstacle mask (1 occupied, 0 free; Fig. 2.2). Sampled velocity-acceleration-curvature trajectories are ranked by safety, comfort, and progress costs; because dense occupancy populates more background cells than sparse boxes, the safety cost weight is scaled between the two rasters (without a sensitivity sweep), and voxel flow is not used. Conditioned on the driving command (forward, left, or right), the lowest-cost trajectory is refined by a front-camera GRU following ST-P3 [66].

2.4 The OpenOcc Benchmark

To train and evaluate dense camera-based 3D occupancy, we build OpenOcc on nuScenes [11, 40]. Developed concurrently with Occ3D [70], OpenOccupancy [62], and SurroundOcc [71], OpenOcc pairs 360-degree six-camera views with surround 3D voxel labels (unlike forward-frustum SemanticKITTI [63]) and includes per-voxel foreground motion vectors (Table 2.10). Because nuScenes occupancy benchmarks use different bounds, voxel sizes, taxonomies, and visibility masks, scores are not directly comparable across suites (*e.g.* the CVPR 2023 challenge used Occ3D-nuScenes; Section 2.6.3).

Table 2.2 **Comparison of OpenOcc with earlier occupancy datasets.** *Multi-view*: 360-degree cameras. *Flow*: per-voxel foreground velocity vectors. *Density*: mean occupied voxels per frame relative to OpenOcc (= 1). Concurrent dense nuScenes benchmarks [62, 70, 71] appear in Table 2.10.

Dataset	Multi-view	Scenes	Flow	Density
SemanticKITTI [63]	–	22	–	–
TPVFormer (sparse labels) [30]	✓	850	–	∼0.11
OccData [72]	✓	850	–	∼0.76
OpenOcc (ours)	✓	850	✓	1

2.4.1 Benchmark Overview

OpenOcc accumulates multi-sweep LiDAR returns and 3D box tracks across all 700 training and 150 validation scenes of nuScenes (34,149 frames, >1.4B occupied voxels). Occupied voxels take 16 classes: 10 foreground (barrier, bicycle, bus, car, construction vehicle, motorcycle, pedestrian, traffic cone, trailer, truck), of which the 8 movable vehicle, pedestrian, and cyclist classes carry non-zero 2D horizontal velocity vectors (barriers and traffic cones are static), and 6 background (driveable surface, other flat, sidewalk, terrain, manmade, vegetation). The ego LiDAR volume $\mathcal{V} = [-50\,\text{m}, 50\,\text{m}] \times [-50\,\text{m}, 50\,\text{m}] \times [-5\,\text{m}, 3\,\text{m}]$ is quantised at $\Delta s = 0.5\,\text{m}$ into a $200 \times 200 \times 16$ grid.

As shown in Table 2.2 and Fig. 2.3, single-sweep LiDAR voxelisation [30] is sparse (∼0.11 relative density), whereas OpenOcc recovers dense surfaces and foreground flow vectors (distributions in Section 2.8.2).

2.4.2 Generating High-Quality Annotations

To aggregate nuScenes' 2 Hz annotated key frames with 20 Hz unannotated intermediate sweeps [11, 40] without motion streaking, we separate static background points (registered in the world frame) from points inside 3D boxes (registered in each object's canonical frame using linearly interpolated box centres and yaw angles across intermediate sweeps; Section 2.8.2). Transformed back into the target key frame and quantised into 0.5 m voxels, each cell takes the majority panoptic LiDAR label [40]

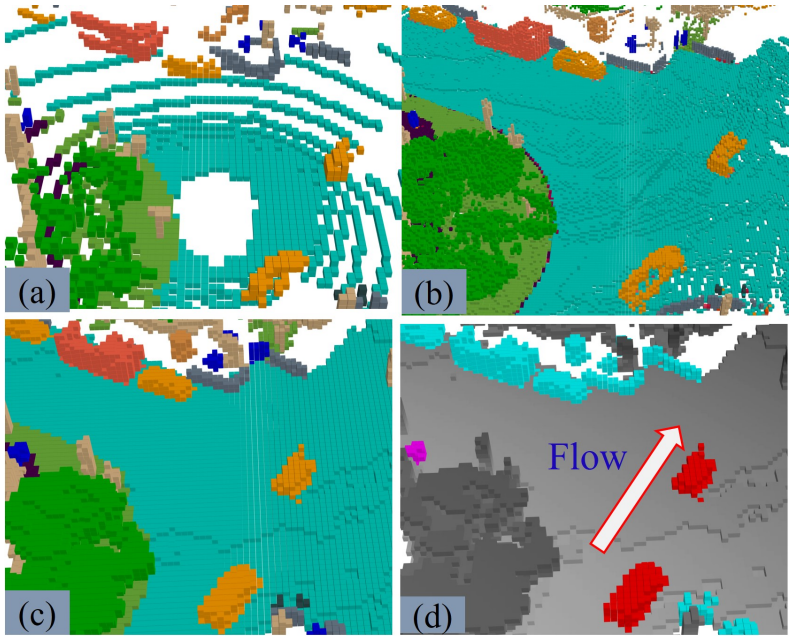


Fig. 2.3 **Comparison of 3D occupancy ground truth.** Compared with (a) single-sweep sparse LiDAR voxels [30] and (b) OccData [72], OpenOcc yields (c) denser surface coverage and (d) per-voxel foreground motion flow.

of its key-frame points, while foreground cells inherit finite-difference box velocities. Voxels containing only unlabelled intermediate points inherit the majority class within a $3\times3\times3$ neighbourhood; morphological hole-filling closes z-axis odometry gaps in the drivable surface; and ray casting from the camera centres masks out occluded voxels (Fig. 2.9, Section 2.8.2). Heuristics were tuned via qualitative spot checks without exhaustive frame-by-frame manual verification.

2.5 Experiments

2.5.1 Experimental Setup

Implementation. Following BEVFormer [1], we use ImageNet-pretrained ResNet50 [73, 74] and FCOS3D-pretrained ResNet101-DCN [51, 73] backbones. The BEV grid B_t has $H = W = 200$ and $C_{\text{BEV}} = 256$. Across $N = 4$ cascade stages, $V'_{t,i} \in \mathbb{R}^{Z_i \times H \times W \times C_i}$ uses heights $Z_i = 2^i$, widths $C_1 = C_2 = 128$ and $C_3 = C_4 = 64$,

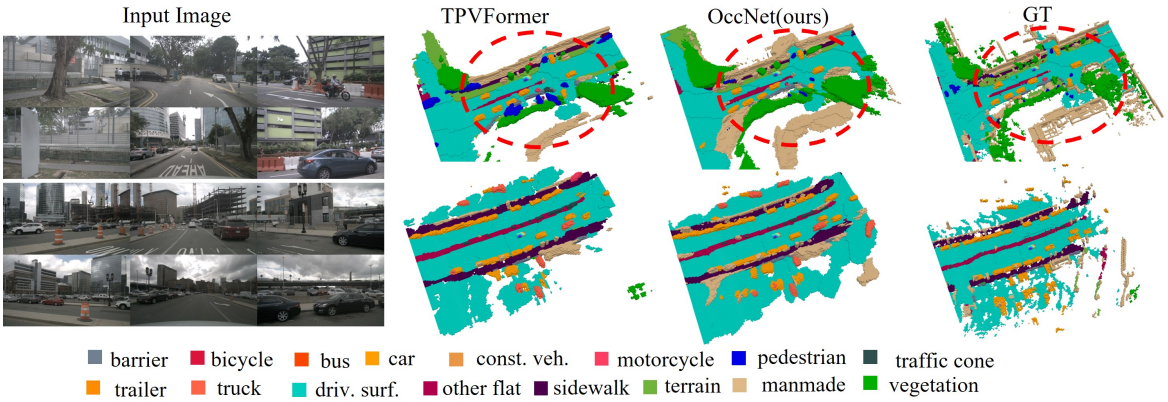


Fig. 2.4 **Qualitative occupancy predictions.** OccNet recovers finer geometry and small foreground actors (pedestrian in dashed box) than TPVFormer [30].

and $N_{\text{ref},i} = 4$ reference heights per voxel. Models train for 24 epochs at learning rate 2×10^{-4} with two history frames (matching BEVFormer-small for memory efficiency, though four frames reach 20.81 vs. 19.48 mIoU in Table 2.11). Details for BEVNet and VoxelNet appear in Section 2.8.1.

Tasks and metrics. OpenOcc SSC is evaluated by 16-class voxel mIoU and binary geometric IoU_{geo} (RayIoU [75] was introduced later; Section 2.6.3). LiDAR segmentation and 3D detection use standard nuScenes validation metrics, BEV segmentation uses class IoU, and open-loop planning follows ST-P3 [66] (L2 error and box collision rate on recorded logs, which ego kinematics alone can largely satisfy [21, 22]; Chapter 1). Metric definitions appear in Section 2.8.1; all tables report single runs.

2.5.2 Main Results

Semantic scene completion. Table 2.3 and Fig. 2.4 compare camera models on OpenOcc. We adapt BEVDet4D [65], BEVDepth [76], and BEVDet [77] by replacing their box heads with a volumetric completion head under the 24-epoch schedule. With ResNet50, OccNet reaches 19.48 mIoU and 37.69 IoU_{geo}, exceeding BEVDet (12.49 mIoU, 27.46 IoU_{geo}). Notably, multi-frame BEVDet4D trails single-frame BEVDet by over 9 IoU_{geo} points (18.27 *vs.* 27.46), showing that naively lifting detector BEV

Table 2.3 **Semantic scene completion on OpenOcc** (single runs). BEV baselines attach a completion head to BEV features and appear under-tuned (Section 2.5.2). *: retrained on OpenOcc. R50/R101: ResNet50/ResNet101.

Method	Backbone	IoU_{geo}	mIoU	driv. surf.	other flat	sidewalk	terrain	manmade	vegetation	barrier	bicycle	bus	car	const. veh.	motorcycle	pedestrian	traffic cone	trailer	truck
BEVDet4D [65]	R50	18.27	9.85	27.23	11.09	13.64	12.04	6.42	9.56	13.56	0.00	13.04	26.98	0.61	1.20	6.76	0.93	1.93	12.63
BEVDepth [76]	R50	23.45	11.88	31.92	7.88	17.08	16.27	8.76	14.75	15.15	0.02	20.75	27.05	1.10	2.01	9.69	1.45	1.91	14.31
BEVDet [77]	R50	27.46	12.49	33.89	10.84	17.55	22.03	11.72	18.15	16.06	0.11	18.27	21.09	2.62	1.42	7.78	1.08	3.40	13.76
OccNet (ours)	R50	**37.69**	**19.48**	**46.13**	**20.60**	**26.75**	**29.37**	**16.90**	**27.21**	**20.63**	**5.52**	**24.16**	**27.72**	**9.79**	**7.73**	**13.38**	**7.18**	**10.68**	**18.00**
TPVFormer* [30]	R101	37.47	23.67	47.88	25.43	30.62	30.18	15.51	23.12	27.95	12.75	33.24	**38.70**	12.41	17.84	11.65	8.49	**16.42**	**26.47**
OccNet (ours)	R101	**41.08**	**26.98**	**50.74**	**27.93**	**31.98**	**33.24**	**20.80**	**30.68**	**29.77**	**16.89**	**34.16**	37.35	**15.58**	**21.92**	**21.29**	**16.75**	16.37	26.23

grids leaves them under-optimised; our controlled BEVNet baseline sharing OccNet's encoder and recipe (Table 2.8) gives the fairer 2D baseline (19.48 *vs.* 17.37 mIoU). With ResNet101, OccNet beats retrained TPVFormer [30] by 3.31 mIoU (26.98 *vs.* 23.67): TPVFormer edges out large vehicles (car, truck, trailer), whereas OccNet gains 8–10 IoU points on compact actors such as pedestrians (21.29 *vs.* 11.65) and traffic cones (16.75 *vs.* 8.49). Because evaluation uses OpenOcc rather than Occ3D-nuScenes [70], SurroundOcc [71], or VoxFormer [28] (Section 2.6.3), comparisons hold strictly within the OpenOcc protocol.

Occupancy for LiDAR segmentation. As $\Delta s \to 0$, voxel completion approaches point segmentation. Assigning each ground-truth LiDAR point the class of its enclosing camera-predicted voxel (so LiDAR still provides query locations), OccNet reaches 60.46 mIoU on nuScenes validation (Table 2.4), trailing LiDAR RangeNet++ [50] (65.50, though beating it on bicycle: 32.58 *vs.* 21.30) and Cylinder3D [54] (76.10; listed as Cylinder3D++ in [30] with RangeNet++ truck at 72.3). OccNet beats OpenOcc-trained TPVFormer by 2.01 mIoU (60.46 *vs.* 58.45), whereas point-supervised TPVFormer-Base [30] reaches 68.9 mIoU on validation (69.4 test), outperforming RangeNet++.

Occupancy for 3D detection. Co-training occupancy and 3D detection with ResNet50 (Table 2.5) shifts single-run NDS by +0.013 (BEVNet), +0.007 (VoxelNet), and +0.008 (OccNet), and mAP by +0.012, +0.006, and 0.000. Joint supervision reduces orientation (mAOE), velocity (mAVE), and attribute (mAAE) errors but slightly worsens scale error (mASE) and, on BEVNet/OccNet, translation error (mATE), as $\Delta s = 0.5\,\text{m}$ is too coarse for sub-voxel box regression. These small single-seed shifts simply show that auxiliary occupancy does not harm detection in this lightweight (0.377–0.390 NDS) setup.

Pre-trained occupancy for 3D detection and BEV segmentation. OpenOcc pre-training improves low-data multi-camera detection relative to monocular

Table 2.4 **LiDAR segmentation on nuScenes validation** (ResNet101). Camera methods are queried at ground-truth LiDAR points. *: trained on OpenOcc. †: published validation numbers from [30] (one decimal; Cylinder3D's 76.10 mIoU is reproduced as printed in [30], whereas the unweighted mean of its 16 one-decimal class cells is 76.21).

Method	Input	mIoU	driv. surf.	other flat	sidewalk	terrain	manmade	vegetation	barrier	bicycle	bus	car	const. veh.	motorcycle	pedestrian	traffic cone	trailer	truck
RangeNet++† [50]	LiDAR	65.50	94.10	66.60	63.50	70.10	83.10	79.80	66.00	21.30	77.20	80.90	30.20	66.80	69.60	52.10	54.20	72.20
Cylinder3D† [54]	LiDAR	**76.10**	96.80	71.60	76.40	75.40	90.50	87.40	76.40	40.30	91.20	93.80	51.30	78.00	78.90	64.90	62.10	84.40
TPVFormer-Base† (point labels) [30]	Camera	68.90	93.30	64.40	64.30	64.50	81.60	79.30	70.00	40.90	93.70	85.60	49.80	68.40	59.70	38.20	65.30	83.00
TPVFormer* [30]	Camera	58.45	88.55	61.63	59.46	63.15	75.76	74.17	65.99	24.50	80.88	74.28	47.04	47.09	33.42	14.52	53.96	70.79
OccNet (ours)	Camera	60.46	88.72	57.99	58.04	63.06	78.91	76.97	66.95	32.58	77.37	73.88	37.62	50.87	51.45	33.69	52.20	67.08

Table 2.5 **Joint occupancy and 3D detection training** on nuScenes validation (ResNet50, single runs). Shifts of +0.007 to +0.013 NDS and 0 to +0.012 mAP lie within single-run noise.

Model	Joint	mAP↑	NDS↑	mAOE↓	mAVE↓	mAAE↓	mATE↓	mASE↓
BEVNet	–	0.259	0.377	0.600	0.592	0.216	**0.828**	**0.290**
	✓	**0.271**	**0.390**	**0.578**	**0.541**	**0.211**	0.835	0.293
VoxelNet	–	0.271	0.380	0.603	0.616	0.219	0.832	**0.284**
	✓	**0.277**	**0.387**	**0.586**	**0.614**	**0.203**	**0.828**	0.285
OccNet	–	0.276	0.382	0.655	0.588	0.209	**0.817**	**0.290**
	✓	**0.276**	**0.390**	**0.585**	**0.570**	**0.190**	0.842	0.292

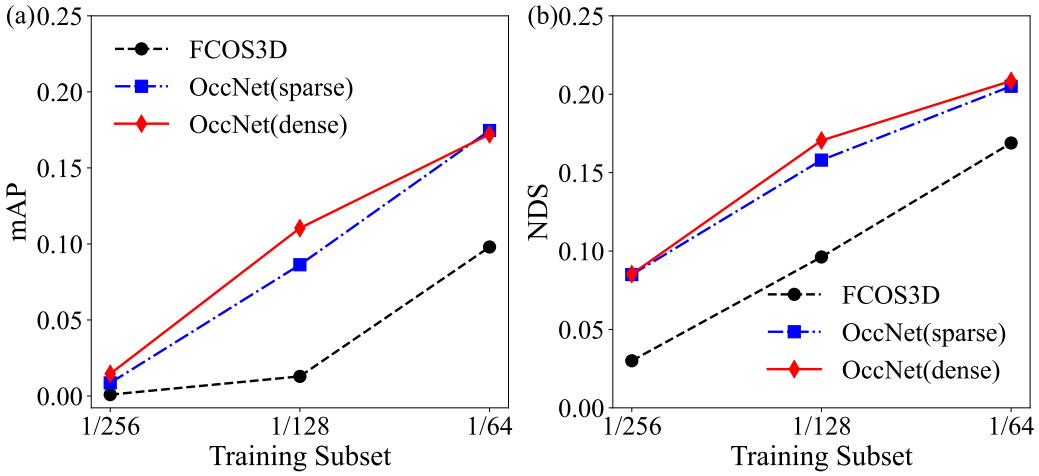


Fig. 2.5 **Detection transfer across training fractions** (co-author-led, single runs): (a) mAP and (b) NDS when initialising from sparse/dense OpenOcc OccNet vs. monocular FCOS3D [51].

FCOS3D [51] initialisation across the 1/256, 1/128, and 1/64 nuScenes splits, with the largest observed difference of about 10 mAP at the 1/128 split (Fig. 2.5). This comparison measures fine-tuning efficiency, not total label efficiency: OccNet initialises the BEV and voxel modules as well as the image backbone (and ResNet101 OccNet itself starts from FCOS3D image weights), the smallest FCOS3D runs collapse to near-zero mAP, and OpenOcc itself uses the full nuScenes LiDAR and box annotations. A stricter control would pre-train the same multi-view architecture and include larger data fractions. In the separate BEV-segmentation experiment, occupancy initialisa-

Table 2.6 **Pre-training transfer to BEV segmentation** (IoU; co-author-led, single runs; Mean IoU: 4-class mean).

Pre-training	Mean IoU↑	Drivable area	Lane	Vehicle	Pedestrian
Det	18.18	44.59	13.62	12.29	2.21
Occ	**19.17**	**47.21**	**13.83**	**12.91**	**2.74**

Table 2.7 **Open-loop planning across scene representations** on nuScenes under ST-P3 [66] (co-author-led, single runs; *full*: all 16 occupancy classes vs. vehicle/pedestrian only; †published ST-P3 [66]).

Input	Collision (%)↓			L2 (m)↓		
	1 s	2 s	3 s	1 s	2 s	3 s
Bbox GT	0.23	0.66	1.50	1.32	2.16	3.00
Occupancy GT	**0.20**	**0.56**	**1.30**	**1.29**	**2.13**	**2.98**
Segmentation pred. (ST-P3, re-run)	0.50	0.88	1.49	1.39	2.21	3.02
Bbox pred. (OccNet detection head)	0.27	0.68	1.59	1.32	2.17	3.03
Occupancy pred. (OccNet)	**0.21**	**0.55**	**1.35**	1.31	2.18	3.07
Occupancy pred. (OccNet, full)	**0.21**	0.59	1.37	**1.29**	**2.13**	**2.99**
ST-P3, published†	0.23	0.62	1.27	1.33	2.11	2.90

tion improves all four targets in single runs, including mean IoU from 18.18 to 19.17 and drivable area from 44.59 to 47.21 (Table 2.6).

Occupancy for planning. Projecting OccNet's jointly trained 3D boxes (Table 2.5) or semantic voxels into BEV rasters for the ST-P3 cost filter and GRU refiner [66], Table 2.7 compares two-class (vehicle/pedestrian) and 16-class (*full*) occupancy against OccNet boxes, a re-run ST-P3 segmentation baseline, and ground-truth oracles under non-reactive open-loop nuScenes replay. Ground-truth occupancy gives the lowest oracle collisions. Among predicted inputs, two-class OccNet occupancy lowers collision rates at 1 s, 2 s, and 3 s from 0.27%, 0.68%, and 1.59% (predicted boxes) to 0.21%, 0.55%, and 1.35%, a single-run reduction of 0.06–0.24 percentage points (15%–22% relative, or ∼4–14 of ∼6,000 validation frames) without paired significance tests or confidence intervals.

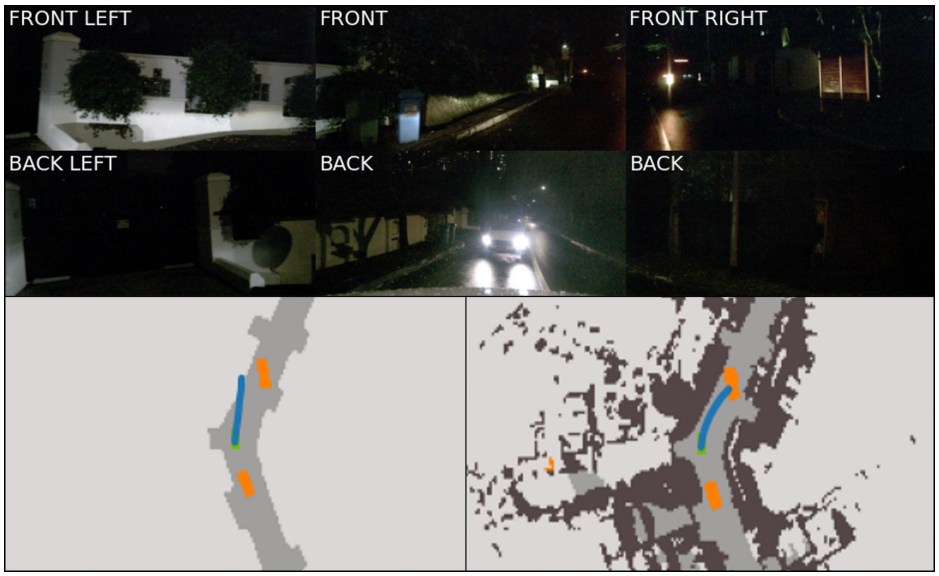


Fig. 2.6 **Planning visualisation** (co-author-led): planned trajectory (blue) over rasterised boxes (left) vs. occupancy (right).

How strong is the planning evidence? Table 2.7 provides limited evidence for a planning benefit. Two-class occupancy lowers collision rate relative to boxes but has slightly higher 3 s L2 error (3.07 m *vs.* 3.03 m), whereas full occupancy gives the lowest L2 values (1.29 m, 2.13 m, 2.99 m) with somewhat higher 2 s and 3 s collisions (0.59%, 1.37%), and neither ego-progress speeds nor ego-status baselines [21, 22] were tested. Separate-batch runs in Table 2.13 move 3 s L2 on identical inputs by as much as 0.05 m, so differences of that order should not be over-interpreted. The comparison also uses OccNet's 0.276-mAP auxiliary box head (Table 2.5) rather than a dedicated detector, and the ground-truth collision gap is only 0.03–0.20 percentage points. Finally, the safety-cost weights differ between box and occupancy rasters (Section 2.3.2) and were not swept. The result therefore supports a same-model representational comparison rather than a general claim that occupancy improves planning.

Comparison with the published ST-P3. Published ST-P3 [66] reports 0.23%, 0.62%, and 1.27% collisions and 1.33 m, 2.11 m, and 2.90 m L2 at 1 s, 2 s, and 3 s, beating our ST-P3 re-run (0.50%, 0.88%, 1.49% collisions; 1.39 m, 2.21 m, 3.02 m L2) and even beating our ground-truth oracles on 3 s collisions (1.27% *vs.* 1.30%/1.50%).

Table 2.8 **Efficiency and accuracy of the three decoders** on OpenOcc SSC (ResNet50, single runs, V100 GPU).

Method	mIoU↑	IoU_{geo}↑	Params↓	Memory↓	FPS↑
BEVNet	17.37	36.11	**39M**	**8 GB**	**4.5**
VoxelNet	19.06	37.59	72M	23 GB	1.9
OccNet	**19.48**	**37.69**	40M	18 GB	2.6

Table 2.9 **Voxelised 3D boxes vs. occupancy on irregular classes** (single runs; mIoU_{ir}: truck, trailer, construction vehicle mean; no regular-class control).

Task	Δs	mIoU_{ir}	Truck	Trailer	Cons. veh.
Det	0.5 m	15.92	23.19	9.57	15.00
Occ	0.5 m	**18.13** (+2.21)	**25.59**	**12.29**	**16.51**
Det	0.25 m	9.90	14.84	5.10	9.75
Occ	0.25 m	**13.41** (+3.51)	**18.85**	**7.14**	**14.25**

Against published ST-P3, OccNet occupancy has lower 1 s/2 s collisions (0.21%, 0.55%) but higher 3 s collisions (1.35%) and 2 s/3 s L2 (2.18 m, 3.07 m). Thus, the controlled comparison between boxes and occupancy from the same OccNet model is the primary internal reference; qualitatively, full-class occupancy avoids unboxed background obstacles in Fig. 2.6.

2.5.3 Analysis

Model efficiency. In Table 2.8, OccNet outperforms planar BEVNet by 2.11 mIoU (19.48 *vs.* 17.37; 8 GB, 4.5 FPS) and exceeds dense VoxelNet by 0.42 mIoU on ResNet50 (19.48 *vs.* 19.06) and 0.92 mIoU on ResNet101 (26.98 *vs.* 26.06 in Table 2.15, comparable to single-frame temporal noise in Table 2.11), while cutting VoxelNet's parameters from 72M to 40M, memory from 23 GB to 18 GB, and raising speed from 1.9 to 2.6 FPS. 3D-DA was not ablated separately.

Irregular objects. Comparing predicted occupancy against voxelised 3D box detections across truck, trailer, and construction vehicle (mIoU_{ir} in Table 2.9; Fig. 2.7),

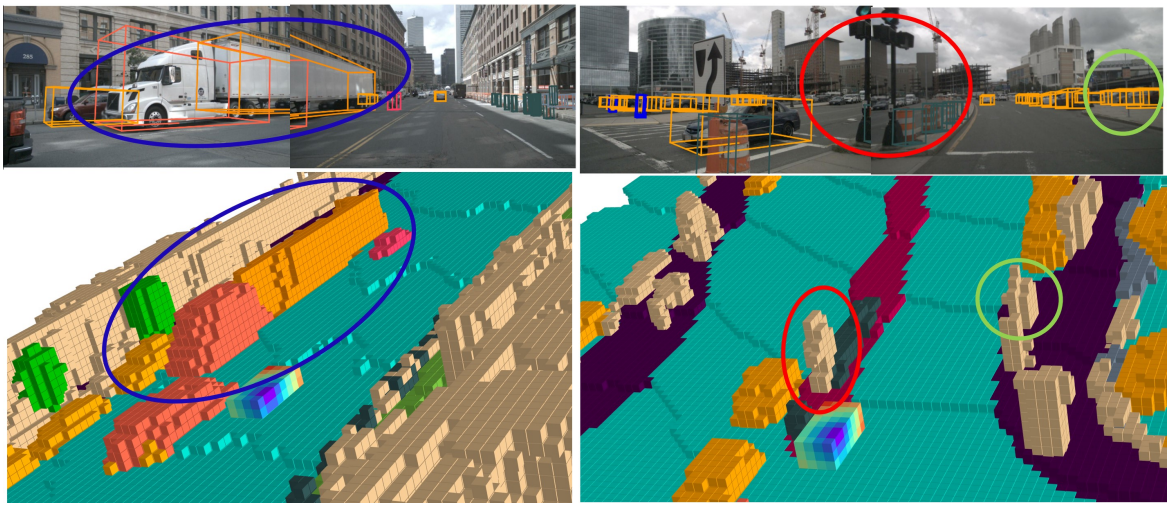


Fig. 2.7 **Predicted 3D boxes (top) vs. semantic occupancy (bottom)** on articulated vehicles and overhead structures (dashed ellipses).

occupancy leads by +2.21 points at $\Delta s = 0.5\,\mathrm{m}$ (18.13 *vs.* 15.92) and +3.51 points at $\Delta s = 0.25\,\mathrm{m}$ (13.41 *vs.* 9.90). Two artefacts qualify this gap: first, OpenOcc collects foreground LiDAR points inside interpolated 3D boxes (missing out-of-box protrusions like Fig. 2.1(d)), so ground-truth object voxels form exterior surface shells whereas voxelised predicted boxes fill the solid interior; second, regular classes (car, bus) were not included as a control.

Dense versus sparse occupancy. In Fig. 2.5, pre-training on dense multi-sweep OpenOcc labels (Fig. 2.3) beats single-sweep sparse LiDAR voxels at the 1/128 detection split (∼0.11 *vs.* 0.09 mAP), but both perform similarly at 1/256 and 1/64 (where sparse is slightly higher); planning was not compared under sparse vs. dense pre-training.

2.6 Discussion and Limitations

2.6.1 What the Experiments Show

What occupancy provides. Our experiments address RQ1 across three dimensions. First, voxel occupancy imposes no cuboid shape prior, capturing articulated vehicles better than voxelised detections (Table 2.9, subject to the surface-vs.-volume caveat in Section 2.5.3) while representing unboxed background obstacles in the binarised planning grid (Fig. 2.6); its 16-class taxonomy remains semantically closed. Second, camera-only OccNet infers 3D semantics from surround video and trails LiDAR RangeNet++ by ∼5 mIoU when queried at laser returns (Table 2.4). Third, in single-run open-loop nuScenes planning, predicted occupancy trims collision rates by 0.06–0.24 percentage points relative to OccNet boxes without a uniform L2 reduction (Section 2.5.2).

Relation to the occupancy results of Chapter 4. In non-reactive NAVSIM rollouts (Chapter 4, Table 4.6), UniAD's post-hoc occupancy cost lowers PDMS from 83.4 to 75.2 (lowering NC, DAC, TTC, and EP), and adding an auxiliary occupancy head to PARA-Drive shifts PDMS from 84.0 to 83.6. Those single-seed runs use different backbones and coupling mechanisms (post-hoc optimisation or auxiliary regularisation rather than sampling-based raster filtering), showing that downstream gains depend on how the planner consumes the 3D grid.

2.6.2 Limitations and Threats to Validity

Key boundaries include: (i) *LiDAR and box supervision*: OpenOcc relies on calibrated nuScenes LiDAR sweeps and interpolated 3D boxes without manual voxel audits; (ii) *single home benchmark*: all 3D perception runs use OpenOcc against baselines retrained in our codebase without cross-evaluating on Occ3D-nuScenes, SurroundOcc, or OpenOccupancy (Section 1.1); (iii) *voxel metrics and resolution*: evaluation uses voxel mIoU/IoU_{geo} rather than RayIoU, queries LiDAR segmentation at known laser

returns, and uses a 0.5 m grid that is too coarse for sub-voxel box regression (Table 2.5) compared with 0.25 m (Tables 2.9 and 2.16); (iv) *task-dependent transfers and unvalidated flow*: occupancy pre-training does not beat detection pre-training for planning (Table 2.14), voxel-collision ranking flips at 2 s (Table 2.13), TPVFormer leads on large vehicles, Cylinder3D leads LiDAR segmentation (76.10 vs. 60.46 mIoU), and foreground flow vectors are not benchmarked; and (v) *single-seed open-loop within nuScenes*: all tables report single runs without confidence intervals, reactive closed-loop rollouts, or cross-domain/open-set tests.

2.6.3 Subsequent Developments (2023–2026)

Challenges and metrics. The CVPR 2023 3D Occupancy Prediction challenge used Occ3D-nuScenes [70] (0.4 m, ±40 m, voxel mIoU; won by FB-OCC [78]), whereas the CVPR 2024 Autonomous Grand Challenge Occupancy and Flow track adopted OpenOcc v2 (0.4 m, Occ3D grid with foreground flow) ranked by RayIoU [75] and mAVE.[4]

Efficiency, supervision, and world models. Later architectures reduced OccNet's dense 3D cost (18G, 2.6 FPS in Table 2.8) via channel-to-height reshaping in FlashOcc [79], coarse-to-fine panoptic queries in PanoOcc [80], sparse grids/tokens/-points [75, 81, 82], and 3D Gaussians [83, 84]. Neural-rendering supervision [85, 86] and open-vocabulary distillation [87, 88] relaxed LiDAR and closed-set constraints, while 4D forecasting, generative world models [89–92], and 3D pre-training [44, 93, 94] extended volumetric representations to temporal prediction.

2.7 Scaling Up: The OpenScene Benchmark

To scale beyond nuScenes' 5.5 hours (Table 2.10), OpenScene [29] curates over 120 hours of 2 Hz multi-sensor logs (eight cameras + merged LiDAR) and 3D occupan-

[4]Challenge repositories: `https://github.com/CVPR2023-3D-Occupancy-Prediction/CVPR2023-3D-Occupancy-Prediction` and `https://github.com/OpenDriveLab/OccNet`.

Table 2.10 **Comparison of 3D occupancy dataset scale**, adapted from the OpenScene release notes [29] (selected entries shown). Duration values indicate source-corpus sensor hours. †Although omitted from the flow column in the public OpenScene table (under the name OccNet), OpenOcc includes foreground flow vectors (Section 2.4.1).

Dataset	Source database	Sensor data (h)	Flow
MonoScene [47]	NYUv2 / SemanticKITTI	5 / 6	✗
Occ3D [70]	nuScenes / Waymo	5.5 / 5.7	✗
Occupancy-for-nuScenes [72]	nuScenes	5.5	✗
SurroundOcc [71]	nuScenes	5.5	✗
OpenOccupancy [62]	nuScenes	5.5	✗
OpenOcc (Section 2.4)	nuScenes	5.5	✓†
OpenScene	nuPlan	120	✓

cy/flow labels across 1,521 nuPlan [95] logs in Las Vegas (64%), Singapore (15%), Pittsburgh (12%), and Boston (9

Labels, tasks, and downstream use. Within $[-50\,\mathrm{m}, 50\,\mathrm{m}] \times [-50\,\mathrm{m}, 50\,\mathrm{m}] \times [-4\,\mathrm{m}, 4\,\mathrm{m}]$ at 0.5 m resolution, 20 s LiDAR accumulation provides per-voxel semantics (seven foreground classes plus merged background) and 3D velocity vectors alongside ego states, traffic lights, route commands, and tracked 3D boxes. OpenScene defines a large-scale occupancy track (Eq. (2.2), ResNet50 OccNet baseline) and a nine-task OpenScene Metric track. Its redistributed 2 Hz sensor logs and v1.1 splits underpin NAVSIM [23] (Chapter 4) and the End-to-End Driving and Predictive World Model (ViDAR [44]) tracks at the CVPR 2024 and CVPR 2025 Autonomous Grand Challenges (Fig. 2.8). Public documentation omits a manual label-accuracy audit, leaves a few v1.1 frames without voxel files, and inherits non-commercial nuPlan/CC BY-NC-SA terms.

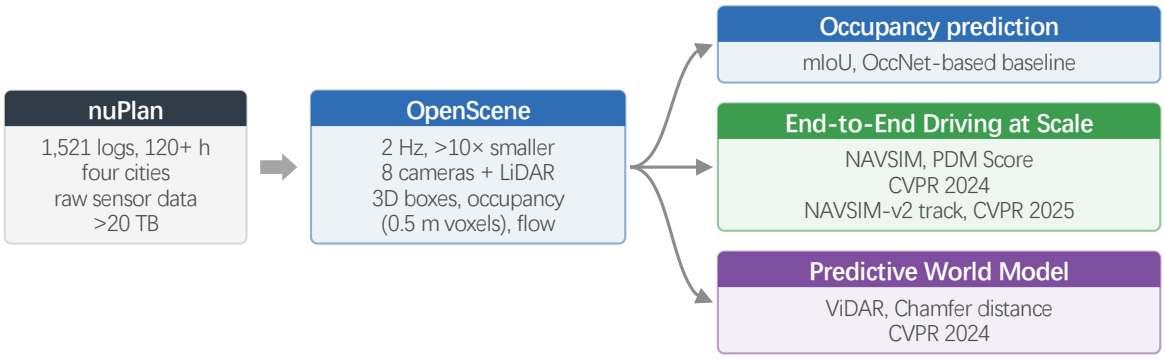


Fig. 2.8 **Overview of OpenScene and its downstream benchmarks.** Downsampling nuPlan to 2 Hz and augmenting it with 3D occupancy and flow annotations, OpenScene underpins large-scale occupancy prediction as well as the end-to-end driving and world-modelling tracks at the CVPR 2024 and CVPR 2025 (NAVSIM-v2) Autonomous Grand Challenges. Both NAVSIM and the end-to-end driving tracks consume the redistributed sensor logs rather than the voxel occupancy targets.

2.8 Implementation Details and Additional Ablations

2.8.1 Evaluation Metrics and Architecture Details

Completion, detection, and planning metrics. For $C = 16$ classes with voxel counts $\mathrm{TP}_c, \mathrm{FP}_c, \mathrm{FN}_c$ accumulated globally across validation frames within the camera visibility (ray-casting) mask and reported as percentages ($\times 100$), semantic mIoU and binary geometric $\mathrm{IoU}_{\mathrm{geo}}$ are

$$\mathrm{mIoU} = \frac{1}{C} \sum_{c=1}^{C} \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}, \tag{2.2}$$

$$\mathrm{IoU}_{\mathrm{geo}} = \frac{\mathrm{TP}_{\mathrm{occ}}}{\mathrm{TP}_{\mathrm{occ}} + \mathrm{FP}_{\mathrm{occ}} + \mathrm{FN}_{\mathrm{occ}}}. \tag{2.3}$$

Detection on nuScenes [11] combines mAP and five TP errors into NDS; open-loop ST-P3 [66] reports 1 s, 2 s, and 3 s L2 error and box collision rate (%).

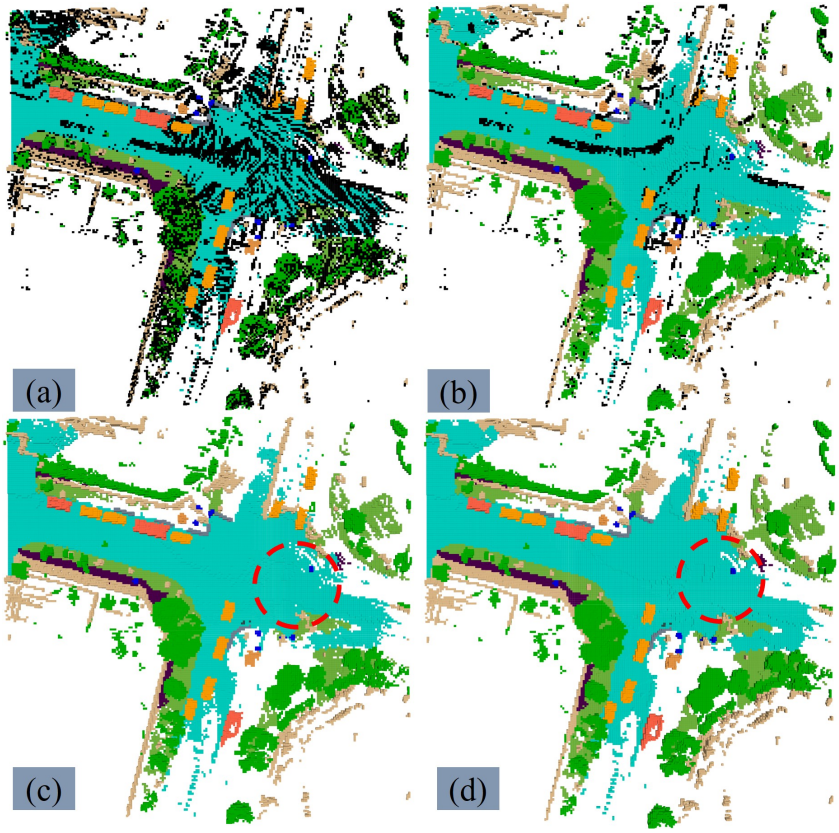


Fig. 2.9 **Four-stage OpenOcc pipeline:** (a) voxelisation (unlabelled sweeps in black), (b) propagation, (c) denoising, (d) hole-filling (red box).

Encoder, cascade, and baselines. ResNet101 [73] with FPN [96] fuses S_3, S_4, S_5 into three 256-channel levels (1/16, 1/32, 1/64) fed to two BEVFormer [1] blocks on queries Q_t. Stage $i \to i+1$ uses an MLP at (h, w) mapping $Z_i C_i \to Z_{i+1} C_{i+1}$ and a linear layer mapping FPN features to C_i, trained with AdamW [97] for 24 epochs (lr 2×10^{-4}, weight decay 10^{-2}, batch size 1 per GPU). Single-scale VoxelNet lifts $4 \times H \times W$ queries into F_{voxel} ($4 \times H \times W \times C_1$) via 3D-DA (Eq. (2.1)) before expanding to $16 \times H \times W \times C_2$; BEVNet reshapes planar F_{bev} ($H \times W \times C_1$) into $16 \times H \times W \times C_2$.

2.8.2 OpenOcc Pipeline and Dataset Statistics

Interpolating 2 Hz nuScenes [11] box centres and yaw across 20 Hz sweeps accumulates instance points for the four stages in Fig. 2.9. Across 700 training and 150 validation

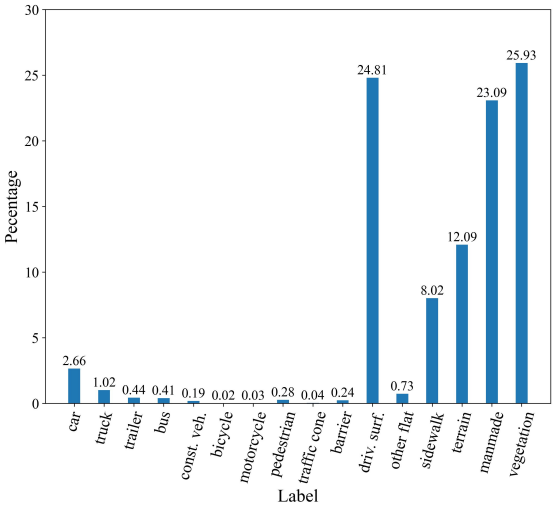


Fig. 2.10 **OpenOcc class frequencies.** Static background exceeds 94% of occupied voxels.

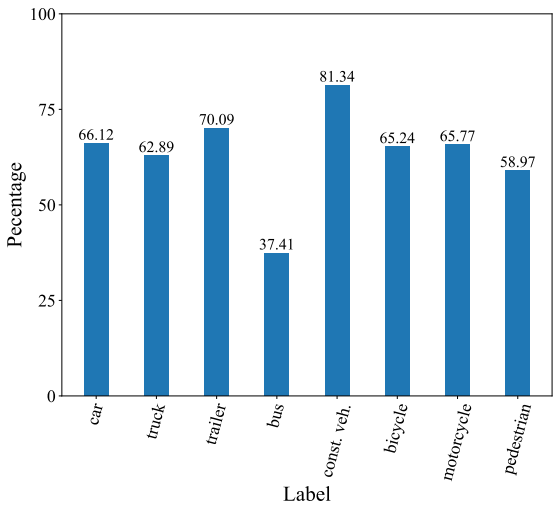


Fig. 2.11 **Moving-voxel share ($v >$ 0.2 m/s)** across the eight movable foreground classes.

logs (34,149 key frames, > 1.4 billion occupied voxels in 16 classes; Fig. 2.10), the ten foreground classes form 5.33% of voxels (bicycles 0.02%, motorcycles 0.03%); across the eight movable foreground classes (excluding static barriers and traffic cones), over half of voxels move above $v_{\text{th}} = 0.2\,\text{m/s}$ (Fig. 2.11).

2.8.3 Supplementary Ablations and Visualisations

Temporal window, voxel collisions, and pre-training. In single-run history sweeps (Tables 2.11 and 2.12), SSC mIoU rises from 19.21 (0 frames) to 20.81 (4 frames), though 1 history frame dips below 0 (18.35 vs. 19.21 SSC; 48.41 vs. 53.82 LiDAR segmentation, ∼1 mIoU single-seed noise); Table 2.12 (2 frames) logs 52.33 mIoU from multi-task training vs. 47.29 in the single-task resolution sweep of Table 2.16. Scoring collisions against occupied voxels of 14 classes (Table 2.13), predicted occupancy lowers collisions vs. boxes at 1 s and 3 s (1.68% vs. 1.75%; 4.32% vs. 4.37

Table 2.11 **History frame sweep on SSC** for OccNet with ResNet50 (single runs; #Num: past sweeps; *: two-frame default).

#Num	IoU_{geo}↑	mIoU↑	Barrier↑	Bicycle↑	Bus↑
0	37.49	19.21	20.07	4.70	24.11
1	36.89	18.35	18.77	4.51	21.66
2*	37.69	19.48	20.63	5.52	24.16
3	38.36	20.30	21.39	6.47	24.65
4	39.21	**20.81**	**22.30**	5.66	**25.13**
9	**39.36**	20.68	20.75	**7.83**	24.79

Table 2.12 **History frame sweep on LiDAR segmentation** for OccNet with ResNet50 (single runs; #Num: past sweeps).

#Num	mIoU↑	Barrier↑	Bicycle↑	Bus↑	Car↑	Truck↑
0	53.82	59.02	24.05	67.61	69.59	59.38
1	48.41	57.45	21.52	57.71	67.26	44.60
2	52.33	61.41	26.07	73.97	70.56	52.64
3	53.49	60.98	24.45	70.20	69.37	56.84
4	**54.59**	**62.09**	21.06	75.05	70.20	59.40
9	54.35	60.04	**30.83**	**75.49**	**71.02**	**61.63**

Table 2.13 **Open-loop planning under voxel collisions** (co-author-led, single runs).

Input	Collision (%)↓			L2 (m)↓		
	1 s	2 s	3 s	1 s	2 s	3 s
Bbox GT	1.66	2.88	4.37	1.33	2.18	3.03
Occupancy GT	**1.63**	**2.85**	**4.29**	**1.29**	**2.13**	**2.99**
Bbox pred. (OccNet)	1.75	**2.85**	4.37	1.33	2.17	3.04
Occupancy pred. (OccNet)	**1.68**	2.94	**4.32**	**1.30**	**2.15**	**3.02**

Table 2.14 **Pre-training comparison for trajectory fine-tuning** (co-author-led, single runs; cumulative L2 and collisions).

Pre-training	Collision (%)↓			L2 (m)↓		
	1 s	2 s	3 s	1 s	2 s	3 s
Det	**0.38**	**0.40**	**0.82**	**0.85**	**1.18**	**1.57**
Occ	0.47	0.68	1.03	0.93	1.26	1.70

Table 2.15 **Per-category decoder comparison on OpenOcc** (single runs; R50/R101: ResNet50/ResNet101).

Method	Backbone	IoU_{geo}	mIoU	driv. surf.	other flat	sidewalk	terrain	manmade	vegetation	barrier	bicycle	bus	car	const. veh.	motorcycle	pedestrian	traffic cone	trailer	truck
BEVNet	R50	36.11	17.37	44.41	14.42	23.87	27.76	13.73	24.49	14.02	5.07	20.85	24.94	8.64	7.75	12.80	8.93	10.21	16.02
VoxelNet	R50	37.59	19.06	46.70	23.31	26.04	29.08	16.52	26.46	19.31	6.25	22.16	26.89	9.96	6.91	12.70	6.27	9.43	16.96
OccNet	R50	37.69	19.48	46.13	20.60	26.75	29.37	16.90	27.21	20.63	5.52	24.16	27.72	9.79	7.73	13.38	7.18	10.68	18.00
BEVNet	R101	40.15	24.62	49.16	21.52	30.57	31.39	18.99	28.71	26.39	15.79	32.07	35.83	11.93	19.72	19.75	15.38	12.82	23.90
VoxelNet	R101	40.73	26.06	49.07	27.82	31.04	32.43	20.45	29.99	27.98	15.95	32.31	36.15	14.88	20.55	20.72	16.52	15.13	25.94
OccNet	R101	**41.08**	**26.98**	**50.74**	**27.93**	**31.98**	**33.24**	**20.80**	**30.68**	**29.77**	**16.89**	**34.16**	**37.35**	**15.58**	**21.92**	**21.29**	**16.75**	**16.37**	**26.23**

Table 2.16 **Voxel size Δs sweep on LiDAR segmentation** using OccNet with ResNet50 (single runs).

Method	Δs (m)	mIoU	driv. surf.	other flat	sidewalk	terrain	manmade	vegetation	barrier	bicycle	bus	car	const. veh.	motorcycle	pedestrian	traffic cone	trailer	truck
OccNet	1.00	46.60	83.93	48.84	50.52	57.89	69.49	68.29	52.78	21.04	65.94	62.45	18.31	15.49	30.71	15.82	33.94	50.22
OccNet	0.50	47.29	86.46	53.12	52.03	59.14	71.55	73.68	59.06	20.63	48.32	63.05	24.12	20.24	41.82	18.84	23.38	41.12
OccNet	0.25	53.00	89.05	57.41	58.06	64.30	75.09	73.92	65.93	22.84	64.09	72.69	32.73	28.73	52.21	17.64	22.05	51.26

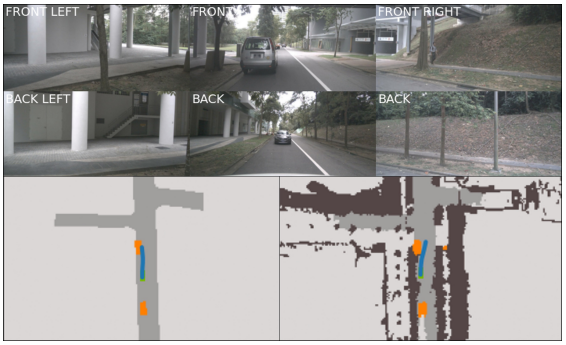


Fig. 2.12 **Planning input comparison** (co-author-led): predicted 3D boxes (left) vs. occupancy (right).

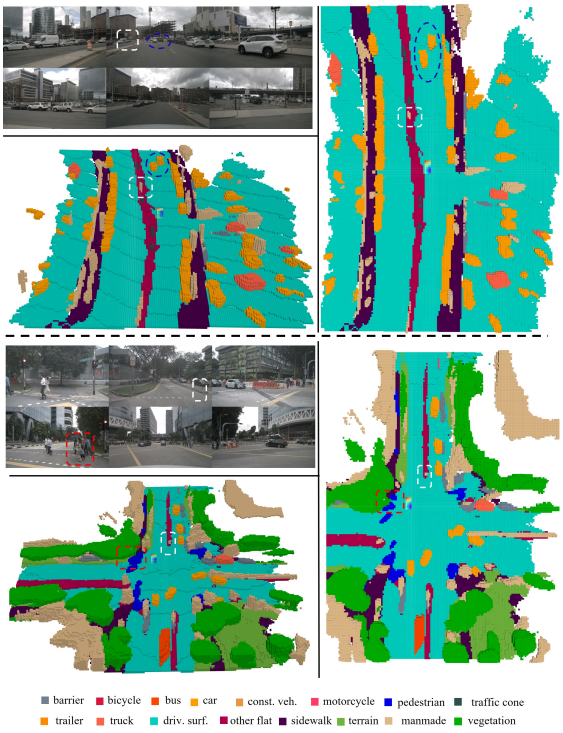


Fig. 2.13 **Validation occupancy predictions** across six camera views and BEV voxel grids.

2.9 Chapter Summary

To address RQ1, this chapter formulated 3D semantic occupancy as a geometry-open spatial representation for autonomous driving. Rather than fitting cuboids to a fixed list of foreground categories and projecting static infrastructure onto a 2D plane, volumetric occupancy discretises the surrounding 3D environment into semantic voxels with motion vectors, capturing arbitrarily shaped obstacles and overhead structures within a unified grid. Around this representation we developed three contributions: OccNet, which lifts surround-view camera features into 3D volumes through a coarse-to-fine cascade voxel decoder with voxel temporal self-attention and 3D deformable cross-attention; OpenOcc, a dense nuScenes benchmark comprising 34,149 frames and over 1.4 billion labelled voxels across 16 semantic categories with foreground flow vectors; and OpenScene, which expands occupancy and flow annotations to over 120 hours of nuPlan logs and subsequently supplied the sensor corpus for NAVSIM as well as the end-to-end driving and predictive world-modelling tracks at the CVPR 2024 and CVPR 2025 (NAVSIM-v2) Autonomous Grand Challenges.

Our experimental evaluation provides graded evidence across the three requirements of RQ1. First, volumetric occupancy can be inferred from surround-view cameras alone: on OpenOcc, OccNet attains 26.98 mIoU and 41.08 IoU_{geo}, outperforming our retrained TPVFormer baseline (23.67 mIoU and 37.47 IoU_{geo}), and when evaluated at ground-truth LiDAR returns it reaches 60.46 mIoU, trailing the LiDAR segmenter RangeNet++ (65.50) by roughly 5 points and the point-supervised camera model TPVFormer-Base (68.9). Second, volumetric grids capture non-cuboid vehicles more accurately than voxelised 3D detections, with the mIoU_{ir} margin expanding from 2.21 to 3.51 points as grid resolution tightens from 0.5 m to 0.25 m, though part of this gap reflects comparing surface-shell labels against solid cuboids. Third, in single-run open-loop nuScenes planning, rasterised occupancy predictions lower collision percentages by 0.06–0.24 points relative to box predictions from the same network without a consistent improvement in L2 error. Architecturally, with a ResNet50 backbone, the cascade decoder improves upon both planar BEVNet and dense VoxelNet while

trimming VoxelNet's parameter count from 72M to 40M. At the same time, clear boundaries apply: supervision still depends on curated LiDAR sweeps and interpolated 3D boxes; semantic predictions remain confined to 16 fixed classes; occupancy flow is annotated but not experimentally evaluated; occupancy pre-training does not aid the trajectory planner; all perception benchmarks rely on our own OpenOcc split; and every table reports single-seed runs without closed-loop simulation or domain-shift testing. Combined with the neutral or negative occupancy results observed on NAVSIM in Chapter 4, this thesis does not establish that 3D occupancy universally improves downstream planning.

Ultimately, a 3D occupancy grid specifies *what occupies each location*, but not *why a traffic configuration matters* or *how the ego vehicle should respond*. Identical occupied voxels represent a kerbside pedestrian whether waiting or stepping into the roadway, and a stationary lead vehicle whether parked or held by a red signal, even though each scenario demands a distinct driving manoeuvre. Resolving such ambiguities requires world knowledge of traffic conventions together with explicit reasoning that links scene perception to agent intent prediction and ego trajectory planning. Chapter 3 addresses this gap by grounding vision-language models in driving scenes through structured, verifiable graph reasoning spanning perception, prediction, and planning; note that Chapters 2, 3, and 4 investigate perception, reasoning, and test-time adaptation on their respective standard task stacks rather than passing intermediate tensors inside one monolithic software binary, and Chapter 3 adopts a 2D front-camera VLM (BLIP-2) to evaluate language-model priors without confounding the representation with this chapter's 3D voxel decoder.

Chapter 3

Reason: Driving with Graph Visual Question Answering

Statement of contribution

This chapter adapts the ECCV 2024 paper "DriveLM: Driving with Graph Visual Question Answering" [4], co-led by equal-contribution first authors Chonghao Sima and Katrin Renz under Andreas Geiger and Hongyang Li. Chonghao Sima defined the Graph VQA task and stage-wise topology, built DriveLM-nuScenes (templates, annotation workflow and quality control) and DriveLM-Agent, ran all nuScenes and Waymo experiments (Tables 3.2, 3.4, 3.8, 3.9 and 3.10, plus the DriveLM-nuScenes columns of Table 3.5), designed the metrics, drafted the paper, and led the CVPR 2024 "Driving with Language" challenge track. Katrin Renz, Kashyap Chitta and Jens Beißwenger developed DriveLM-CARLA, PDM-Lite [31] and the CARLA experiments (Tables 3.3 and 3.7, plus the DriveLM-CARLA columns of Table 3.5). Adapted by permission of Springer Nature (ECCV 2024, LNCS, vol. 15110, pp. 256–274).

Chapter 2 established a geometry-first 3D occupancy representation, yet occupancy alone does not dictate ego control. A planner must still identify salient road users, anticipate their trajectories and choose a safe manoeuvre. When sensor layouts or obstacle classes depart from the training distribution, memorised input–output mappings falter and structured multi-step reasoning becomes essential, forming the second pillar of this thesis.

Web-pretrained vision-language models (VLMs) bring broad internet priors, human-readable text rationales and sequential prompting, but lack physical grounding in vehicle control. Moreover, early driving QA datasets reduced reasoning to isolated scene summaries or single-hazard chains rather than modelling multi-agent interactions from perception through prediction to planning. This gap motivates our second research question:

> **RQ2 (Reason).** *How can the general knowledge of vision-language models be grounded in driving in a structured and verifiable way that mirrors how humans reason from perception through prediction to planning, and does such reasoning help a driving agent generalise to unseen situations?*

To investigate RQ2, DriveLM comprises the Graph Visual Question Answering (GVQA) task, two datasets (DriveLM-Data), an evaluation suite, and the DriveLM-Agent baseline. GVQA represents dependencies among perception, prediction, planning, behaviour, and motion questions as a directed acyclic graph. DriveLM-nuScenes contains 4,871 keyframes with an average of 91.4 mostly human-authored QA pairs per frame, while DriveLM-CARLA contains about 1.6 million rule-generated QA pairs collected with the PDM-Lite expert [31] on CARLA Leaderboard 2.0. DriveLM-Agent combines BLIP-2 parent-answer context conditioning with RT-2-style waypoint tokens. Graph context harms in-domain nuScenes planning (ADE 1.39 $\rightarrow$ 1.74, collision rate 1.67 $\rightarrow$ 1.89, behaviour accuracy 61.45 $\rightarrow$ 57.49) but improves speed classification (43.90% to 54.29%) after zero-shot transfer to Waymo (a perceptual sensing shift in Chapter 1); most of the Waymo trajectory advantage over single-frame UniAD (2.63 m versus 4.16 m ADE) is already present in the pre-trained backbone (2.78 m), and the comparison is confounded by model size (3.955 billion versus 131.9 million parameters) and camera input. On withheld CARLA pedestrians, a class-specific prompt improves stopping accuracy (4.59% to 27.04%) while lowering regular-scene accuracy (59.63% to 52.17%). All reported model comparisons come from single-seed runs.

Sections 3.1–3.5 present motivation, related work, the GVQA formalism, DriveLM-Data and DriveLM-Agent; Sections 3.6–3.10 cover experiments, limitations, community adoption, dataset and implementation details (Section 3.9), and conclusions.

3.1 Motivation

Autonomous driving stacks face persistent challenges in generalising to altered sensor rigs or rare hazards and in explaining decisions to human occupants or regulators [98–100]. Whereas most end-to-end planners map dense bird's-eye-view (BEV) grids directly to trajectories [2, 3, 101], we model driving through an object-centric progression P_{1-3}: spotting key road users, forecasting their future motions and selecting a safe ego manoeuvre. Inspired conceptually by object-centred vision [102] and core object cognition [103], P_{1-3} is treated here strictly as a structured task-design hypothesis; Chapters 2, 3, and 4 investigate perception, reasoning, and test-time adaptation on their respective standard task stacks rather than passing intermediate tensors inside one monolithic software binary, and this chapter adopts a 2D front-camera VLM (BLIP-2 [104]) to evaluate language-model priors without confounding the representation with Chapter 2's 3D voxel decoder.

Pre-trained VLMs [104–107] support this formulation through three properties: web-scale priors that aid zero-shot robotic transfer [108, 109]; interpretable language outputs in place of opaque boxes or waypoints [1, 66, 110, 111]; and multi-hop autoregressive reasoning [108, 112–116]. Passing discrete text between stages, however, blocks end-to-end gradient flow [98], and DriveLM-Agent trains under teacher forcing with separate motion adapter weights (Section 3.5).

Earlier driving VQA benchmarks used either scene-level rationales, *e.g.*, "The car is moving into the right lane because it is safe to do so" [117, 118], or single-object chains, *e.g.*, "The ego vehicle stops because there is a pedestrian in a white shirt crossing the intersection in front of the ego vehicle and it does not want to crash into the pedestrian" [119–121]. Neither captures multi-object dependencies across P_{1-3}.

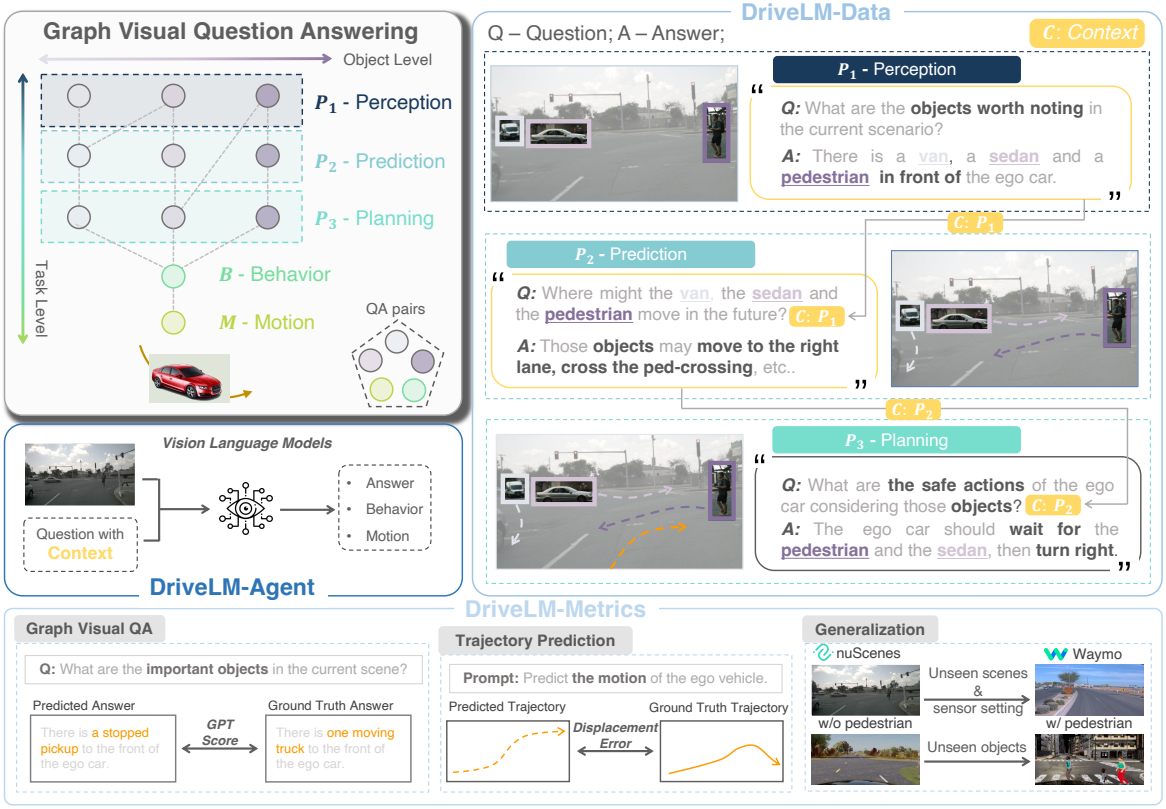


Fig. 3.1 **Overview of DriveLM.** Following [98], Graph Visual Question Answering (GVQA) links QA pairs via object-level and stage-level edges across perception → prediction → planning → behaviour → motion, supported by DriveLM-Data, DriveLM-Metrics and DriveLM-Agent.

DriveLM instead organises frame QAs into a directed acyclic graph terminating in ego behaviour and motion waypoints (Fig. 3.1), and pairs open datasets, metrics and evaluation servers with zero-shot tests across camera rigs (nuScenes [11] to Waymo [12]) and withheld classes (pedestrians in DriveLM-CARLA), following Chapter 1.

3.2 Related Work

We summarise prior and concurrent 2024 work here; Chapter 1 gives broader background and Section 3.8.1 reviews later advances.

Generalisation in autonomous driving. Long-tail traffic events remain a core safety bottleneck [98, 122, 123]. Beyond data augmentation and critical-scenario simulation such as TrafficSim [124–128], recent methods leverage semantic embeddings to detect out-of-distribution obstacles [129–132]; we test whether structured GVQA reasoning aids zero-shot transfer.

Language-grounded driving. Concurrent works couple LLMs with driving [130–145]: GPT-Driver [134] and Driving-with-LLMs [135] convert scene detections into text prompts for trajectory generation, while DriveGPT4 [137] predicts control signals and rationales from visual tokens. Unlike those studies, DriveLM evaluates zero-shot transfer across camera rigs and withheld object categories (Section 3.6).

Vision-language benchmarks for driving. Prior driving-language datasets [117–121, 139, 146–150] target isolated subtasks: perception queries in NuScenes-QA [121] and NuPrompt [149], post-hoc rationales in BDD-X [117], or single-hazard chains in DRAMA [120] and Rank2Tell [119]. DriveLM instead provides dense multi-object graphs spanning perception, prediction and planning (Table 3.1).

Embodied planning with LLMs. Robotics pipelines use LLMs [151–153] for embodied planning [108, 109, 154–158] via multimodal token fusion in PaLM-E [109], policy code synthesis in Code as Policies [156], and discrete action tokenisation in RT-2 [108], which we adapt for vehicle waypoints.

Reasoning over graph structures. Explicit graph topologies aid multi-step inference [115, 159–161], including scene-graph reasoning in Explainable Neural Modules (XNMs) [160] and directed prompt graphs in Graph of Thoughts [115]. DriveLM casts driving QA as a DAG across P_{1-3}, B and M.

3.3 Graph Visual Question Answering

Human driving progresses from perceiving salient road users to predicting their interactions and planning an ego response [162, 163]. We formalise this progression as GVQA (Section 3.3.1) and define stage-wise metrics (Section 3.3.2).

3.3.1 Task Formulation

For each frame, GVQA organises question-answer pairs into a directed acyclic graph $G = (V, E)$, where each vertex $v = (q, a) \in V$ queries one or more traffic participants and each directed edge $e = (v_p, v_c) \in E \subseteq V \times V$ passes the parent answer text a_p as context to child question q_c following the topological stage order $P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow B \rightarrow M$. Edges encode both cross-object interactions (Fig. 3.1, centre) and five sequential stages:

- **Perception** (P_1): identifying, describing and localising salient road users in the current frame.
- **Prediction** (P_2): forecasting future motions and interactions conditioned on P_1.
- **Planning** (P_3): enumerating safe candidate ego manoeuvres.
- **Behaviour** (B): classifying the high-level ego manoeuvre in natural language.
- **Motion** (M): generating continuous BEV future trajectory waypoints.

Stages P_{1-3} mirror modular driving stacks [98], whereas B and M are derived from the ground-truth future trajectory $M = \{(x_0, y_0), (x_1, y_1), \ldots, (x_N, y_N)\}$ sampled at uniform intervals, with step displacements

$$\{x, y\}_{\text{dist}} = \{(\delta_{x,1}, \delta_{y,1}), \ldots, (\delta_{x,N}, \delta_{y,N})\}, \tag{3.1}$$

where $\delta_{x,i} = x_i - x_{i-1}$ and $\delta_{y,i} = y_i - y_{i-1}$ $(i = 1, 2, \ldots, N)$. Discrete behaviour $B = (B_{sp}, B_{st})$ bridges P_{1-3} and M by binning mean longitudinal and lateral step displacements into five speed and five steering classes:

$$B_{sp} \in \{\texttt{fast}_2, \texttt{fast}_1, \texttt{moderate}, \texttt{slow}_1, \texttt{slow}_2\},$$
$$B_{st} \in \{\texttt{left}_2, \texttt{left}_1, \texttt{straight}, \texttt{right}_1, \texttt{right}_2\},$$

where subscripts indicate magnitude; richer semantic manoeuvres can also be represented.

3.3.2 Evaluation Metrics

DriveLM-Metrics score M, B and P_{1-3} nodes independently. Motion M uses the open-loop UniAD [3] criteria: average displacement error (**ADE**), final displacement error (**FDE**) and **collision rate**. Behaviour B reports joint, speed and steering **classification accuracy**. Stage P_{1-3} answers use two main metrics: **SPICE** [164], which computes a scene-graph tuple F-score over objects, attributes and relations, and **GPT Score**, which queries an unversioned `gpt-3.5-turbo` endpoint using the prompt archived by DriveLM [4] (Section 3.9) with reference and predicted answers (omitting the question) to output a score from 0 to 100. We selected GPT Score as a headline P_{1-3} indicator post hoc after surface n-gram metrics (BLEU, ROUGE-L, CIDEr) and Completeness diverged under context ablations (Section 3.9); the API snapshot and judge variance were not recorded, and node-wise scoring does not verify parent–child logical consistency across edges (Section 3.7).

3.4 DriveLM-Data

We instantiate GVQA in two datasets (Fig. 3.2, Table 3.1): real-world DriveLM-nuScenes and simulated DriveLM-CARLA, backed by a public evaluation server (Section 3.8).

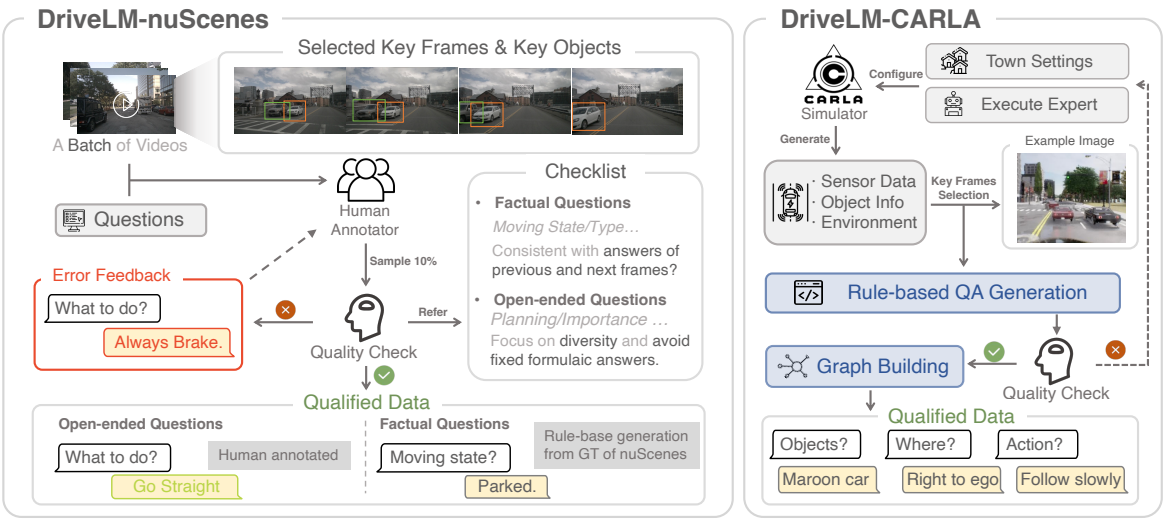


Fig. 3.2 **Annotation pipelines.** DriveLM-nuScenes combines rule-extracted perception QAs from nuScenes and OpenLane-V2 with human prediction/planning annotations and multi-round quality checks; DriveLM-CARLA uses an automated rule pipeline driven by PDM-Lite [31].

3.4.1 DriveLM-nuScenes

DriveLM-nuScenes is constructed by sampling interaction-rich nuScenes keyframes, identifying salient road users, and annotating object-grounded P_{1-3} question graphs. Some P_1 facts are extracted from nuScenes and OpenLane-V2 labels, while annotators write the remaining P_1 content and all 299,000 prediction and planning QA pairs (among 443k total in Table 3.1) using templates designed by five driving specialists [11, 15]. Inspectors audit 10% of each batch; a failed sample returns the full batch for relabelling, after which the English text receives manual cleanup and GPT-3.5 polishing (Fig. 3.2, Section 3.9). Rejection thresholds, relabelled proportions, and inter-annotator agreement were not archived. The final split contains 4,072 training and 799 validation keyframes, averaging 91.4 QA pairs per frame versus at most 15.3 previously (Table 3.1; Section 3.9).

Table 3.1 **Comparison of DriveLM-Data with prior driving-language benchmarks.** DriveLM-Data provides dense frame-level QAs across **perception, prediction and planning** linked in a **graph**. Perc., Pred., Plan.: QAs per stage (✗: absent; 144k/153k/146k are rounded to the nearest thousand whereas 91.4 QA/frame is computed from the exact unrounded QA count 445,210/4,871); †: full dataset; ‡: keyframe subset; *: hybrid rule/human; **: rule-based; -: unreported.

Dataset	Source Dataset	# Frames	Avg. QA per Frame	QAs per stage			Logic
				Perc.	Pred.	Plan.	
nuScenes-QA [121]	nuScenes	34,149	13.5	460k**	✗	✗	None
nuPrompt [149]	nuScenes	34,149	1.0	35k*	✗	✗	None
HAD [118]	HDD	25,549	1.8	25k	✗	20k	None
BDD-X [117]	BDD	26,228	1	26k	✗	✗	None
LingoQA [139]	LingoQA	28,000	15.3	-	-	-	None
DRAMA [120]	DRAMA	17,785	5.8	85k	✗	17k	Chain
Rank2Tell [119]	Rank2Tell	5,800	-	-	✗	-	Chain
DriveLM-nuScenes	nuScenes	4,871	**91.4**	144k*	153k	146k	**Graph**
DriveLM-CARLA†	CARLA	64,285	24.4	697k**	311k**	558k**	**Graph**
DriveLM-CARLA‡	CARLA	5,721	24.8	63k**	28k**	51k**	**Graph**

3.4.2 PDM-Lite: An Expert Driver for CARLA Leaderboard 2.0

We collect DriveLM-CARLA in CARLA 0.9.14 under Leaderboard 2.0 [24], whose long-horizon scenarios reduce Leaderboard 1.0 experts such as TransFuser++ [165] to a 2% Driving Score (DS) on the 8+ km validation routes. Co-author Jens Beißwenger therefore built PDM-Lite [31], adapting nuPlan's PDM-Closed [95, 111] by pairing Intelligent Driver Model (IDM) [166] longitudinal control (for lead vehicles, pedestrians, stop signs and traffic lights) with two lateral path candidates instead of 16-trajectory rollouts, scored via TransFuser++ costs [165]. Across 20 official validation routes over three seeds (60 runs by co-authors; Section 3.9), PDM-Lite reaches a pooled mean DS of 36.3 ± 28.1 (Table 3.7; [4] cited $\sim$44% from an early subset), with a route-mean SD of 23.9 and within-route three-seed SDs of 0.0 to 37.4.

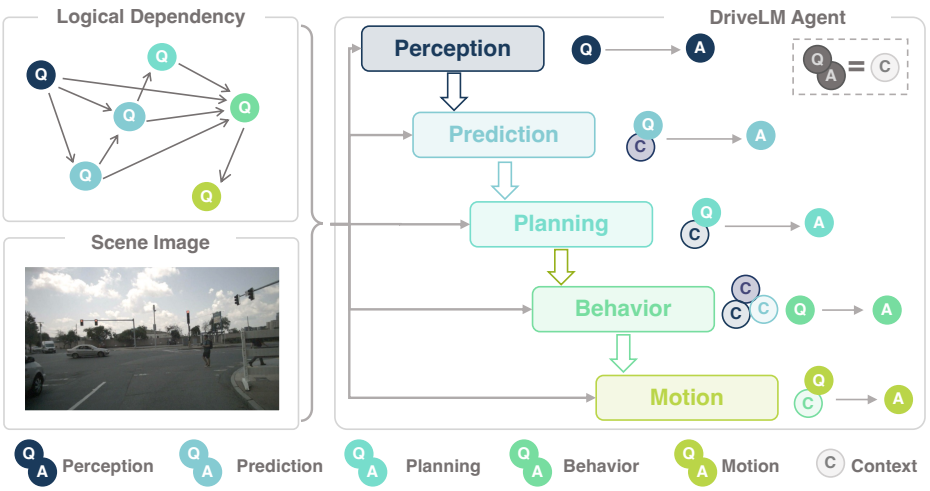


Fig. 3.3 **Pipeline of DriveLM-Agent.** From the front-camera image, a fine-tuned VLM forwards parent QAs as "Context:" prefixes along graph edges across all five stages.

3.4.3 DriveLM-CARLA

PDM-Lite drives urban, residential and rural routes at 4 FPS while logging sensors, simulator states and rule activations (Fig. 3.2), sampling keyframes when longitudinal or lateral decisions change. Privileged actor states and expert rules across 35 of the 38 Leaderboard 2.0 scenarios (excluding *InterurbanAdvancedActorFlow*, *MergerIntoSlowTraffic* and *VehicleTurningRoute*) populate templates that wire P_{1-3}, B and M QAs into a 1.6M-QA DAG (Table 3.1, Section 3.9.2).

3.5 DriveLM-Agent: A GVQA Baseline

DriveLM-Agent adapts BLIP-2 [104] via Q-Former prompting, parent-to-child text conditioning and RT-2 waypoint tokenisation [108] across three tiers (Fig. 3.3): (1) P_{1-3} object reasoning, (2) discrete behaviour classification B, and (3) continuous trajectory generation M.

3.5.1 Prompting with Context

Unlike direct pixel-to-waypoint regression [167, 168], DriveLM-Agent prepends every parent QA v_p ('$(v_p, v_c) \in E$', concatenated when $|\{v_p : (v_p, v_c) \in E\}| > 1$) under a "*Context:*" prefix to child prompt v_c. Training uses teacher forcing on all frame QAs, whereas inference rolls out $P_1 \to P_2 \to P_3 \to B \to M$ autoregressively on sampled subgraphs (Sections 3.9 and 3.9).

Context aggregation through behaviour. Distinct hazards (red lights, stop signs, braking lead cars) often require the same ego manoeuvre. Behaviour node B summarises upstream P_{1-3} QAs into a discrete command bottleneck: predicting B before M without P_{1-3} context cuts nuScenes ADE from 2.63 (BLIP-RT-2) to 1.39 (Table 3.2), whereas P_{1-3} context aids B only under transfer to Waymo (Section 3.6).

3.5.2 Trajectory Tokenisation for Motion

Following RT-2 [108], longitudinal and lateral waypoint offsets are quantised into 256 uniform bins mapped to 256 BLIP-2 vocabulary tokens (Section 3.9). Stage M reuses the BLIP-2 backbone with separate LoRA weights trained solely on motion QAs conditioned on the image and B, though any lightweight LLM [169] or command-conditioned head [66, 170] could substitute.

3.6 Experiments

We evaluate five questions: (1) open-loop VLM trajectory planning; (2) zero-shot cross-sensor transfer; (3) transfer to a withheld object category; (4) question-level ablations across P_{1-3}; and (5) context effects on P_{1-3} VQA (further ablations in Section 3.9).

Setup. Models use LoRA [171] with single seed 1234: 10 epochs at batch size 2 per GPU on DriveLM-nuScenes `train` (∼7 hours on 8 V100 GPUs), and 6 epochs on DriveLM-CARLA keyframe `train` (∼6 hours on 4 A100 GPUs; Section 3.9).

Evaluation sets and uncertainty. We test on 799 DriveLM-nuScenes `val` keyframes, 1,000 sampled Waymo `val` frames, and DriveLM-CARLA `val` (161 regular frames to within 0.01 rounding in Table 3.3) plus DriveLM-CARLA-ped (unarchived QA denominators/histograms). Single-seed runs preclude cross-seed variance; binomial 95% Wilson intervals are ±3.4 points at 60% over 799 frames and ±3.1 points at 50% over 1,000 frames.

3.6.1 VLMs for End-to-End Driving

To reduce open-loop ego-history shortcuts on nuScenes [22], DriveLM-Agent omits CAN-bus ego status and evaluates on the 799 interaction-rich keyframes (Section 3.9), though no ego-status-only baseline was run on this split.

Baselines and variants. **Command Mean** averages training trajectories sharing the frame's command ('turn left', 'turn right', 'go straight'); **UniAD-Single** adapts multi-frame surround-view UniAD [3] (shown in italics) to single-frame input; and **BLIP-RT-2** fine-tunes BLIP-2 [104] directly on waypoint tokens without B or P_{1-3}. DriveLM-Agent compares '**None**' (B without P_{1-3} context), '**Chain**' (only P_3 passed to B) and '**Graph**' (all sampled P_{1-3} QAs passed to B).

Results. Table 3.2(a) separates direct waypoint tokenisation from behaviour-conditioned planning. BLIP-RT-2 (ADE 2.63, Col. 2.77) improves on Command Mean (4.57) but trails UniAD-Single (1.80, 2.62), whereas conditioning motion on a predicted behaviour label without P_{1-3} context ('None') lowers ADE to 1.39 and collision rate to 1.67. Predicted graph context does not help in-domain: the 'Graph' variant reaches 1.74 ADE, 1.89 collision rate, and 57.49% behaviour accuracy, compared with 61.45% without graph context. The 'Chain' variant is weaker still at

Table 3.2 **Open-loop planning on DriveLM-nuScenes `val` and zero-shot transfer to 1k Waymo `val` frames.** Conditioning motion (M) on behaviour (B) lets DriveLM-Agent ('None') beat UniAD-Single in-domain, where Chain/Graph context hurts; on Waymo (where UniAD-Single loses its rear camera while VLMs use only the front view), Graph context achieves the best behaviour accuracy and ADE. Italicised UniAD uses video and CAN-bus ego status. Single-run results.

Method	Behaviour Context	Motion Context	(a) DriveLM-nuScenes `val` (in-domain)					(b) Waymo `val`, 1k frames (zero-shot)				
			Behaviour (B)			Motion (M)		Behaviour (B)			Motion (M)	
			Acc. (%)↑	Speed (%)↑	Steer. (%)↑	ADE (m)↓	Col. (%)↓	Acc. (%)↑	Speed (%)↑	Steer. (%)↑	ADE (m)↓	FDE (m)↓[†]
Command Mean	-	-	-	-	-	4.57	5.72	-	-	-	7.98	11.41
UniAD-Single	-	-	-	-	-	1.80	2.62	-	-	-	4.16	9.31
BLIP-RT-2	-	-	-	-	-	2.63	2.77	-	-	-	2.78	6.47
DriveLM-Agent	None	B	**61.45**	**72.20**	**84.73**	**1.39**	**1.67**	35.70	43.90	65.20	2.76	6.59
	Chain	B	50.43	60.32	75.34	2.07	2.08	34.62	41.28	64.55	2.85	6.89
	Graph	B	57.49	69.89	80.63	1.74	1.89	**39.73**	**54.29**	**70.35**	**2.63**	**6.17**
UniAD [3]	-	-	-	-	-	*0.80*	*0.17*	-	-	-	-	-

[†] Correcting a header misprint in Table 2 of the ECCV 2024 paper [4] (where this column was labelled `Col.`), these values are 3 s final displacement error (`FDE`, in m), matching Table 6 of the ECCV supplementary material (Table 3.9).

50.43% behaviour accuracy and 2.07 ADE. All single-frame variants remain behind the published multi-frame UniAD result (0.80, 0.17); multi-frame VLM results are reported in Section 3.9.3.

3.6.2 Generalisation Across Sensor Configurations

We evaluate the nuScenes checkpoints zero-shot on 1,000 Waymo Open Dataset [12] `val` frames using rule-generated prompts (Section 3.9; sampling seed and rule-triggered prompt counts were not archived). Because Waymo lacks a rear camera, surround-view UniAD-Single zeroes out its rear view whereas front-view VLMs keep their training layout and use 30× more parameters (3.955B versus 131.9M, Table 3.6); furthermore, model-predicted P_1 outputs determine which P_2/P_3 prompts fire.

Results. In Table 3.2(b), most of the ADE margin over UniAD-Single (4.16) already appears in BLIP-RT-2 (2.78) and context-free DriveLM-Agent (2.76) before any graph reasoning. Adding 'Graph' context trims ADE to 2.63 (matched by P_2-only context at 2.62 in Table 3.9), but raises speed accuracy from 43.90 to 54.29 (versus 41.28 for

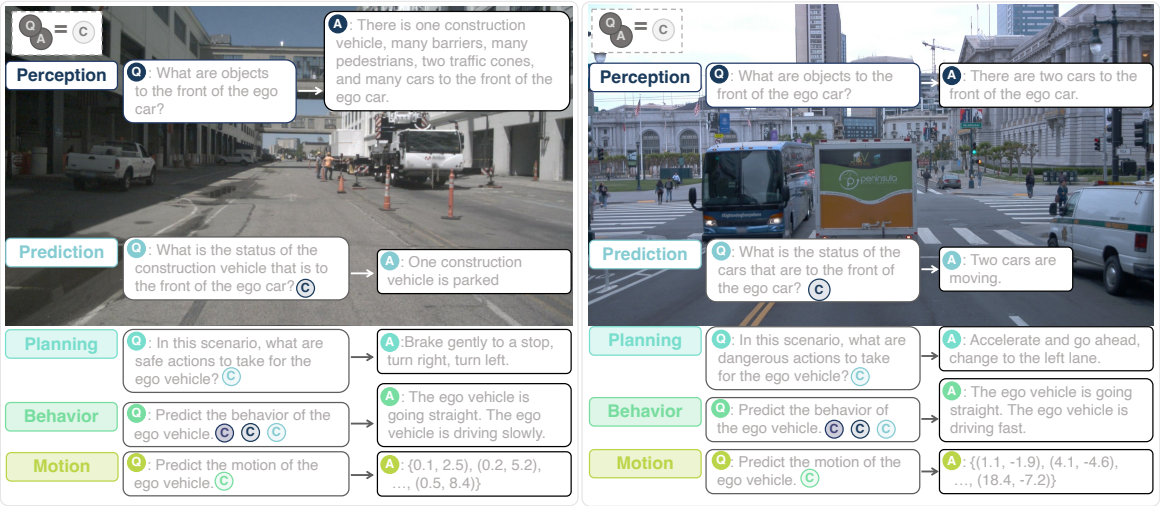


Fig. 3.4 **Qualitative examples from DriveLM-Agent** on DriveLM-nuScenes `val` (left) and Waymo `val` (right), showing questions (Q), context (C) and predicted answers (A).

'Chain'). Qualitative examples in Fig. 3.4 show inspectable intermediate QAs that remain coherent on Waymo even when P_1 is noisy [4].

3.6.3 Generalisation to Unseen Objects

Co-authors evaluated zero-shot transfer to a withheld object category by removing all pedestrian frames from DriveLM-CARLA `train/val` and assembling DriveLM-CARLA-ped solely from frames where a pedestrian crosses ahead of the ego vehicle, requiring a stop with straight steering. Because every frame in DriveLM-CARLA-ped shares this identical target action, a constant stopping rule (not separately evaluated) would score 100; accuracy on DriveLM-CARLA-ped therefore measures stopping recall and must be read alongside pedestrian-free DriveLM-CARLA accuracy.

Baselines and variants. We compare against TransFuser++ [165], which takes higher-resolution images plus LiDAR and requires dense depth, segmentation, 3D box and HD map supervision, preventing joint training on general vision datasets. By contrast, DriveLM-Agent's text interface permits co-training on DriveLM-CARLA alongside COCO [172] and GQA [173]. At test time, we evaluate two additions (word-

Table 3.3 **Transfer to unseen pedestrians on DriveLM-CARLA** (by co-authors; single runs). Adding a hand-crafted pedestrian prompt lifts DriveLM-Agent accuracy on DriveLM-CARLA-ped from 4.59 to 27.04 while lowering standard-scene accuracy from 59.63 to 52.17. Italicised 'GT' rows supply ground-truth context at test time, directly disclosing pedestrian presence under '+ Pedestrian QA'. All methods score 100.00 on steering on DriveLM-CARLA-ped because every target is `straight`.

Method	**DriveLM-CARLA** (B)			**DriveLM-CARLA-ped** (B)		
	Acc.↑	Spd.↑	Str.↑	Acc.↑	Spd.↑	Str.↑
TransFuser++ [165]	**70.19**	**73.29**	**90.68**	8.72	8.72	100.00
DriveLM-Agent	59.63	61.50	78.26	4.59	4.59	100.00
+ Pedestrian QA	52.17	55.28	77.64	**27.04**	**27.04**	100.00
DriveLM-Agent (GT)	*60.25*	*65.22*	*80.12*	*20.92*	*20.92*	*100.00*
+ Pedestrian QA	*60.25*	*65.22*	*80.12*	*92.35*	*92.35*	*100.00*

ing in Section 3.9): (1) '**+ Pedestrian QA**', which inserts a pedestrian-check P_1 prompt whose output selects a follow-up ego-action prompt appended to behaviour context; and (2) '**GT**', an oracle setting that substitutes ground-truth P_{1-3} answers at test time. Under '+ Pedestrian QA', 'GT' supplies the affirmative ground-truth answer to "Is there a person in the scene?" on every DriveLM-CARLA-ped frame and triggers its ground-truth follow-up, leaking the target condition rather than bounding achievable performance.

Results. In Table 3.3, both baselines fail on most pedestrian scenes: TransFuser++ drops from 70.19 to 8.72 and base DriveLM-Agent drops from 59.63 to 4.59. Adding '+ Pedestrian QA' raises DriveLM-CARLA-ped accuracy to 27.04 while lowering standard-scene accuracy to 52.17, still failing to brake on ∼73% of pedestrian frames (mainly from missed VLM detections; stronger VLMs [174, 175] were not tested). Because this prompt requires prior knowledge of the missing class and unablated CO-CO/GQA co-training (which includes humans), the gain reflects compliance with an injected human prior, and the 92.35 'GT' score reflects leaked oracle context. Ev-

ery model scores 100% on steering because all DriveLM-CARLA-ped scenes occur on straight roads.

3.6.4 Question-wise Analysis on DriveLM-nuScenes

To assess how specific question types affect behaviour prediction, we select three representative prompts per stage (∼60% of all P_{1-3} QAs; stage-level ablation in Section 3.9.3) and train BLIP-2 on cumulative subsets (Table 3.4):

P_{1-1}: What are the important objects in the current scene?

P_{1-2}: What is the moving status of object X?

P_{1-3}: What is the visual description of object X?

P_{2-1}: What is the future state of object X?

P_{2-2}: Would object X be in the moving direction of the ego vehicle?

P_{2-3}: What object should the ego vehicle notice first / second / third when the ego vehicle is getting to the next possible location?

P_{3-1}: What actions could the ego vehicle take based on the observation of object X?

P_{3-2}: What actions taken by the ego vehicle can lead to a collision with object X?

P_{3-3}: In this scenario, what are safe actions to take for the ego vehicle?

Results. In Table 3.4, adding P_2 and P_3 prompts (IDs 4–9) improves behaviour accuracy by 1.75–5.01 points over P_1-only context (IDs 1–3), while P_3 (IDs 7–9) adds no gain beyond P_2 (IDs 4–6), suggesting predicted traffic states already suffice for behaviour selection. Intra-stage shifts ($\leq$ 1.75 points) and the stage gap itself are comparable to single-run sampling noise (Section 3.6).

Table 3.4 **Question-level context ablation on DriveLM-nuScenes `val`** (single runs). Adding prediction and planning prompts (P_2, P_3) raises behaviour accuracy by 1.75–5.01 points over perception-only prompts (P_1), whereas intra-stage variations stay within 1.75 points.

ID	Perception			Prediction			Planning			Behaviour		
	P_{1-1}	P_{1-2}	P_{1-3}	P_{2-1}	P_{2-2}	P_{2-3}	P_{3-1}	P_{3-2}	P_{3-3}	Acc.↑	Speed↑	Steer↑
1	✓	-	-	-	-	-	-	-	-	54.69	66.83	75.22
2	✓	✓	-	-	-	-	-	-	-	55.32	67.33	74.34
3	✓	✓	✓	-	-	-	-	-	-	53.94	65.33	75.72
4	✓	✓	✓	✓	-	-	-	-	-	58.82	71.83	**80.98**
5	✓	✓	✓	✓	✓	-	-	-	-	57.07	71.96	78.97
6	✓	✓	✓	✓	✓	✓	-	-	-	57.70	72.22	79.22
7	✓	✓	✓	✓	✓	✓	✓	-	-	**58.95**	72.59	80.23
8	✓	✓	✓	✓	✓	✓	✓	✓	-	57.95	**72.97**	79.97
9	✓	✓	✓	✓	✓	✓	✓	✓	✓	57.49	69.89	80.63

Table 3.5 **Stage P_{1-3} VQA baselines** (CARLA by co-authors; single runs). 'Graph' context raises GPT Score across all four model–dataset pairs and SPICE in three, though BLIP-2 SPICE on CARLA (10.46 → 10.30) and 'GT' GPT Score on nuScenes (42.97 → 41.10) dip slightly.

Context	**DriveLM-nuScenes** (P_{1-3})				**DriveLM-CARLA** (P_{1-3})			
	BLIP-2 [104]		DriveLM-Agent		BLIP-2 [104]		DriveLM-Agent	
	SPICE↑	GPT↑	SPICE↑	GPT↑	SPICE↑	GPT↑	SPICE↑	GPT↑
None	4.34	42.97	42.56	71.39	10.46	46.37	72.71	79.67
Graph	7.71	45.21	49.54	72.51	10.30	55.03	75.26	81.78
GT	*8.19*	*41.10*	*50.29*	*72.94*	*16.18*	*57.98*	*79.07*	*83.13*

3.6.5 Performance for P_{1-3} via GVQA

We compare zero-shot BLIP-2 [104] and fine-tuned DriveLM-Agent on P_{1-3} QA across context modes: '**None**' (single-turn VQA), '**Graph**' (predicted parent answers), and '**GT**' (ground-truth parent answers; Table 3.5).

Results. In Table 3.5, scores are lower on human-annotated DriveLM-nuScenes than on template-generated DriveLM-CARLA, and fine-tuned DriveLM-Agent substantially outperforms zero-shot BLIP-2 (*e.g.*, nuScenes 'None' GPT Score 71.39 ver-

Table 3.6 **Model size and inference cost.** DriveLM-Agent updates fewer trainable weights than UniAD-Single but incurs higher total parameter count, FLOPs and latency.

Method	#Params	#Trainable	FLOPs	FPS
UniAD-Single	131.9M	58.8M	1.7T	1.8
DriveLM-Agent	3.955B	12.9M	24.2T	0.16

sus 42.97). 'Graph' context raises GPT Score across all four pairs (+1.12 to +8.66) and SPICE in three, but lowers zero-shot BLIP-2 SPICE on CARLA (10.46 → 10.30), while 'GT' lowers BLIP-2 GPT Score on nuScenes (42.97 → 41.10). Because DriveLM-Agent's nuScenes GPT Score gain is small (71.39 → 72.51) with unmeasured GPT-3.5 judge variance and post-hoc metric selection (full n-gram and Completeness suite in Section 3.9), these results provide suggestive rather than definitive support for graph context on P_{1-3}.

3.7 Discussion and Limitations

Our experiments show that graph-structured prompting hurts in-domain open-loop planning on DriveLM-nuScenes relative to context-free behaviour prediction (ADE 1.39 → 1.74, collision rate 1.67 → 1.89, behaviour accuracy 61.45 → 57.49 in Table 3.2) and yields gains only under distribution shift: improving speed accuracy on Waymo (Section 3.6.2) and stopping recall on unseen CARLA pedestrians once a class-specific prompt is added (Section 3.6.3). Several methodological and architectural limitations bound these findings.

Backbone versus graph. The transfer experiments do not isolate graph reasoning from foundation-model pre-training. On Waymo, BLIP-RT-2 and context-free DriveLM-Agent already achieve 2.78 and 2.76 ADE, respectively, compared with 4.16 for UniAD-Single; full graph context changes ADE to 2.63 (P_2-only 2.62 in Table 3.9) and primarily improves speed accuracy, from 43.90% to 54.29%. The remaining com-

parison is confounded because UniAD-Single loses its rear camera and has 131.9 million parameters, whereas DriveLM-Agent keeps its front-view layout and has 3.955 billion (Table 3.6). The CARLA result is narrower: pedestrian stopping improves to 27.04% only after adding a hand-crafted class-specific prompt alongside unablated COCO/GQA co-training, regular-scene accuracy falls from 59.63% to 52.17%, and the 92.35% 'GT' result directly reveals pedestrian presence. These are single-run findings.

Omitted baselines and consistency checks. Five diagnostic checks were omitted: (1) *majority-class baselines*, where DriveBench [16] later found ∼78.6% "Going Straight" labels in DriveLM-nuScenes (exceeding 'Chain' at 75.34 in Table 3.2 and P_1-only rows at 74.34–75.72 in Table 3.4), while a constant stop scores 100 on DriveLM-CARLA-ped; (2) *image-blind baselines* on our evaluated checkpoints; (3) *ego-status-only baselines* on the 799 keyframes; (4) *cross-edge logical consistency* between parent (P_{1-3}) and child (B, M) predictions, which node-wise DriveLM-Metrics do not test; and (5) *multi-run LLM judge variance* under a version-locked model to test whether 1–2 point GPT Score gaps (Table 3.5) exceed judge noise.

Data priors, latency, single-view input and open-loop scope. DriveLM-nuScenes reports process-level quality control without inter-annotator agreement or post-audit error rates, and subjective keyframe selection leaves "Going Straight" dominant (Section 3.9). Sequential LLM decoding runs at 0.16 FPS (8.5 tokens/s, ∼11× slower than UniAD's 1.8 FPS, Table 3.6; quantisation, token pruning [176, 177] or multi-rate scheduling could lower cost), which also made 20 Hz synchronous closed-loop CARLA evaluation (∼125 s compute per simulated second) impractical here. Furthermore, DriveLM-Agent consumes a single front-camera image without LiDAR or temporal history, yet trains and evaluates on unfiltered six-camera DriveLM-nuScenes QAs (Section 3.9), forcing non-front queries to rely on coordinate text or label priors. Finally, prompt templates remain heuristic, and P_{1-3} VQA gains translate weakly into planning improvements.

3.8 From Benchmark to Community

We released the DriveLM datasets, checkpoints and evaluation server[1], which anchored the *Driving with Language* track of the CVPR 2024 Autonomous Grand Challenge[2] (led by the thesis author). Evaluated on DriveLM-nuScenes v1.1 `val` (withheld test answers) using a weighted composite of ChatGPT score (0.4) and BLEU/ROUGE-L/CIDEr, coordinate matching and accuracy (0.2 each) from a LLaMA-Adapter V2 [174] starter kit, the track drew 978 submissions from 152 teams across 14 countries; team ADLM won the Innovation Award for aligning BEV features with an LLM. Separately, PDM-Lite [31] became a standard data-collection expert on CARLA Leaderboard 2.0 [34].

In follow-up work co-authored by the thesis author, DriveBench [16] audited 12 VLMs across 200 DriveLM-nuScenes keyframes under clean, 15 corrupted and black-image inputs, finding that models retain most of their score without images (*e.g.*, GPT-4o kept ∼95% of its clean GPT Score on text alone, whereas human perception accuracy fell from 93.33% to 33.33% in fog). DriveBench traced this shortcut to the 78.6% "Going Straight" prior and 2D camera/coordinate prompt leakage, and showed that n-gram metrics and reference-only LLM judges reward memorised templates (the fine-tuned DriveLM baseline beat GPT-4o on planning ROUGE-L, 53.12 *vs.* 6.54, while trailing Qwen2-VL-72B and GPT-4o on prediction GPT Score, 44.33 *vs.* 49.35 and 51.30). This audit shows that Table 3.5 and behaviour accuracies are vulnerable to text priors, and completes the benchmark–challenge–audit loop of Chapter 1.

3.8.1 Subsequent Developments (2023–2026)

Follow-up work has advanced three areas: (1) **Reasoning verification**, moving from fixed QA graphs and isolated node metrics to executable visual programs [178, 179], crop-grounded Visual CoT [180], and driving benchmarks that score reasoning–action consistency directly, including Reason2Drive [181], counterfactual 3D OmniDrive [182]

[1] `https://github.com/OpenDriveLab/DriveLM`
[2] `https://opendrivelab.com/challenge2024/`

and step-annotated DriveLMM-o1 [183]; (2) **Unified and closed-loop policies**, including CoT-supervised EMMA [184], RL-aligned token planners AutoVLA [185] and Alpamayo-R1 [186], closed-loop evaluations on Bench2Drive [187, 188] and CARLA (where SimLingo [189] won the 2024 CARLA Challenge using balanced DriveLM-style VQA), dual-system fast/slow planners [34, 142, 190], and text-free latent feature alignment in FLARE (co-authored by the thesis author) [35]; and (3) **Evaluation**, replacing generic LLM judges and open-loop L_2 error with trained verifiers such as Lingo-Judge [139] and rater-preference long-tail benchmarks such as WOD-E2E [191].

3.9 Dataset Statistics, Implementation Details, and Additional Results

This section documents the schema, collection workflow and statistical distributions of DriveLM-nuScenes and DriveLM-CARLA, route-by-route validation metrics for the PDM-Lite expert, formal definitions of the DriveLM-Metrics suite (including the exact GPT Score prompt), DriveLM-Agent context and trajectory tokenisation protocols, and extended Waymo and VQA ablations. Complete question templates and extra visualisations are archived in [4] and the public code repository.

3.9.1 DriveLM-nuScenes Schema, Workflow, and Statistics

DriveLM-nuScenes comprises 4,072 training and 799 validation keyframes, pairing clip-level ego-manoeuvre descriptions with six-camera 2D bounding boxes and three-stage frame QAs:

- **Perception** (P_1): 144k QAs combining automated template extraction from nuScenes [11] and OpenLane-V2 [15] with manual questions on visible scene attributes and road-user states.
- **Prediction** (P_2): 153k QAs on future ego and road-user interactions, with all answers *written manually*.

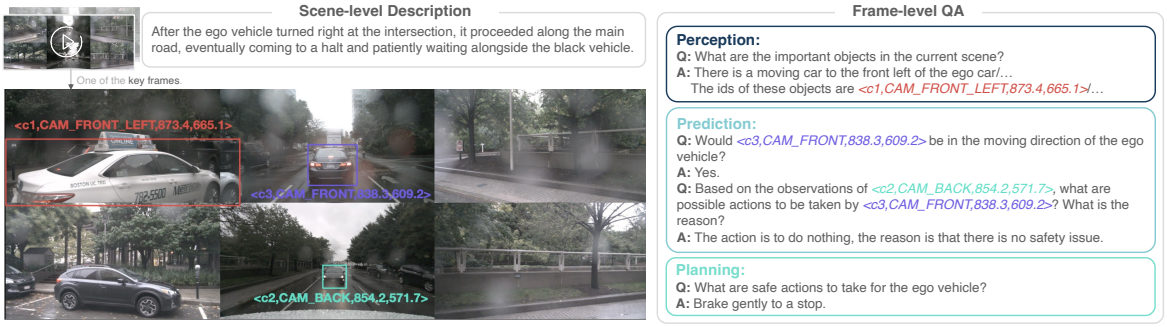


Fig. 3.5 **Schema of DriveLM-nuScenes.** Clip summaries pair with *perception*, *planning* and *prediction* QAs grounded via *c tags* with 2D box centres.

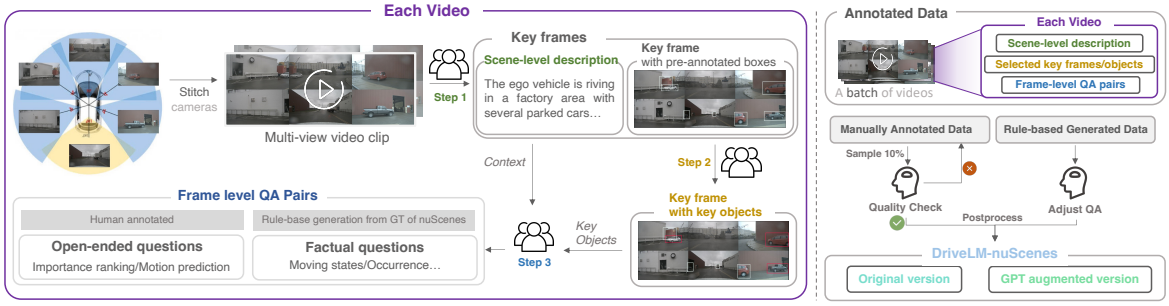


Fig. 3.6 **Left: three-stage human annotation workflow. Right: verification and translation pipeline** over 8-video batches.

- **Planning** (P_3): 146k QAs on candidate ego actions, collision risks and rationales, collected *through manual annotation*.

In total, human annotators authored all 299k prediction and planning answers (over two thirds of the 443k QAs in Table 3.1). Every referenced road user or traffic control device is indexed by a **c tag** $\langle c, CAM, x, y\rangle$ storing instance identifier c, source camera CAM and 2D bounding-box centre (x, y), linked to a per-frame metadata dictionary recording 2D box dimensions, semantic category, motion state and visual appearance (Fig. 3.5).

Licensed drivers annotate six-camera panoramas in three steps (Fig. 3.6, left): (1) *selecting keyframes* where interactions or state transitions are judged imminent and writing clip-level ego summaries (without kinematic filtering, leaving straight driving frequent as noted in Section 3.8); (2) *marking key objects* by selecting projected 2D nuScenes boxes [11] or drawing new 2D boxes for unlabelled hazards that influence

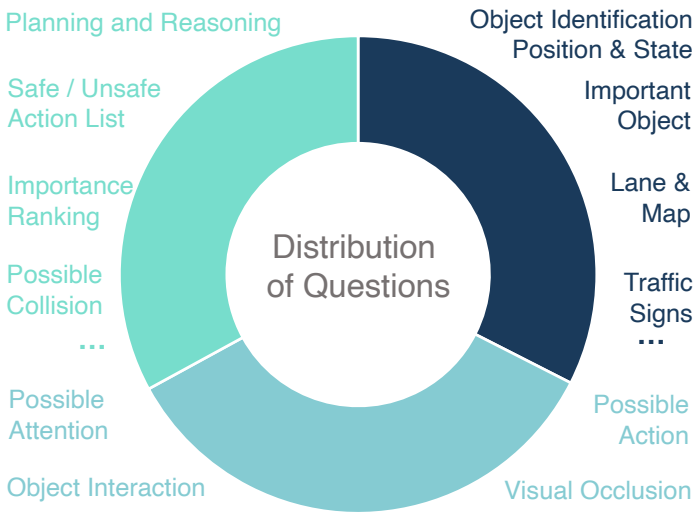


Fig. 3.7 **Breakdown of QAs across perception, prediction and planning.**

the ego trajectory; and (3) *authoring QAs* via structured multiple-choice options, a custom "Other (blank entry)" slot and free-form text, alongside rule-extracted factual QAs. Quality control operates on batches of 8 video clips (Fig. 3.6, right): automated scripts and human inspectors audit each batch, returning the entire 8-clip packet for full rework whenever manual labels fail inspection, whereas template errors are corrected directly by inspectors. Chinese annotations are translated into English using a domain terminology glossary combined with human-proofread GPT-3.5 translation; a second release adds GPT-3.5 stylistic polishing [4]. Stage-level QA proportions, fine-grained subcategory breakdowns within each stage, and key-object class and question distributions for traffic controls versus dynamic road users are shown in Figs. 3.7, 3.8 and 3.9.

3.9.2 DriveLM-CARLA Topology, PDM-Lite, and Collection

DriveLM-CARLA pairs multi-sensor simulator logs with a deterministic rule-generated QA graph (Fig. 3.10) covering road layout, dynamic vehicles, traffic signals and stop signs, and is extensible to parked vehicles, weather and speed signs. Synthetic collection eliminates manual labelling cost while providing synchronised multi-view RGB,

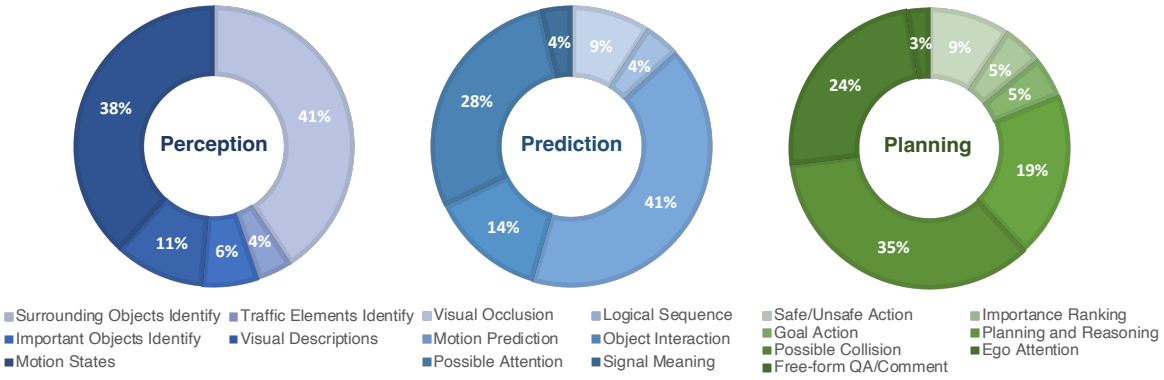


Fig. 3.8 **Fine-grained question subcategories within each stage of DriveLM-nuScenes.**

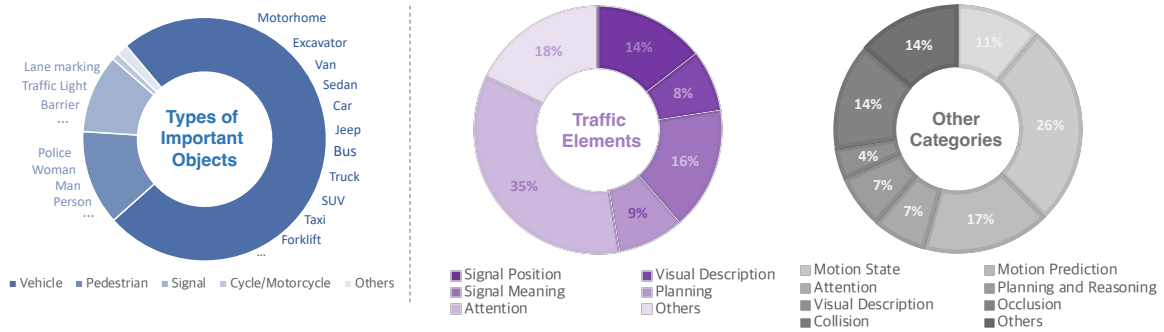


Fig. 3.9 **Left: class breakdown of key objects in DriveLM-nuScenes. Right: question-type distribution by object group.**

LiDAR, depth, and perspective/BEV semantic segmentation streams, and each object QA stores a simulator actor ID together with explicit parent–child graph edges.

Because Leaderboard 1.0 agents such as TransFuser++ [165] fail on the 38 long-horizon scenarios of CARLA 0.9.14 Leaderboard 2.0 [24], co-authors developed PDM-Lite [31], a privileged six-stage rule-based expert combining A* route following, kinematic actor forecasting, IDM longitudinal speed control, two-path lateral selection and PID actuation. Table 3.7 reports co-author evaluations of PDM-Lite across the 20 official Leaderboard 2.0 validation routes over three seeds: routes 10–19 repeat the geometry of routes 0–9 under altered weather and traffic, and because PDM-Lite reads weather-invariant simulator states, inter-run differences reflect traffic randomisation. Pooling all 60 route–seed runs gives a mean Driving Score (DS) of 36.3 ± 28.1, with

a 20-route mean SD of 23.9 (inter-seed SD of the 20-route average is not recoverable from per-route summary cells).

For dataset generation, we split the 90 Town 12 training routes (780.6 km) and 20 Town 13 evaluation routes (247.6 km) into ∼150 m segments across two $10 \times 10\,\mathrm{km}^2$ maps, discarding degenerate loops, randomising daytime weather presets, and spawning cars, trucks, vans, bicycles and motorcycles. Alongside the five sensor streams (DriveLM-Agent uses front RGB only, whereas auxiliary modalities train TransFuser++), we log privileged simulator dictionaries for:

- *Ego vehicle*: actor ID, 3D bounding box, speed and brake state.
- *Other vehicles*: actor ID, 3D box, LiDAR point count, ego distance, speed, control commands, colour, vehicle class, wheel count, light status, road/lane IDs, junction flag/distance and active route command.
- *Pedestrians*: actor ID, 3D box, LiDAR hits, age/gender attributes, ego distance, speed and lane location.
- *Traffic lights, stop signs, parked cars, landmarks and weather*: 3D boxes, ego distance, signal phase or speed limit, lane relevance and simulator weather parameters.

These dictionaries populate deterministic QA templates directly without LLM paraphrasing.

3.9.3 DriveLM-Metrics, Tokenisation, and Implementation

DriveLM-Metrics scores each node independently of parent accuracy across three stage suites. For stage P_{1-3} VQA, we compute five lexical/graph metrics (**BLEU-4** [192] for n-gram precision, **ROUGE-L** [193] for longest common subsequence recall, **METEOR** [194] for exact, stem and synonym unigram alignment with word-order penalty, **CIDEr** [195] for TF-IDF cosine similarity, and **SPICE** [164] for scene-graph tuple F-score), together with **Completeness** (the fraction of frame QAs achieving

Table 3.7 **Validation performance of PDM-Lite on the 20 official CARLA Leaderboard 2.0 routes** (by co-authors): Driving Score (DS), Route Completion (RC), Infraction Score (IS), Normalised Driving Score (NDS) and Normalised Infraction Score (NIS). Route cells show mean ± SD over 3 seeds; Avg. pools all 60 runs (36.3 ± 28.1 is the 60-run population mean and SD: 36.35, rounding to 36.4 from the one-decimal route cells, with sample route-mean SD 23.9). Routes 10–19 repeat routes 0–9 under altered weather/traffic; simulator crashes on routes 3 and 13 received floor scores.

Route	DS	RC	IS	NDS†	NIS
0	14.0 ±0.0	100.0 ±0.0	0.14 ±0.00	48.2 ±0.0	0.48 ±0.00
1	56.2 ±37.4	71.1 ±40.9	0.75 ±0.18	61.9 ±42.1	0.69 ±0.32
2	55.3 ±14.3	100.0 ±0.0	0.55 ±0.14	79.3 ±6.8	0.79 ±0.07
3	18.4 ±7.8	74.7 ±35.8	0.34 ±0.20	38.9 ±20.4	0.51 ±0.07
4	59.8 ±13.5	100.0 ±0.0	0.60 ±0.14	80.5 ±6.8	0.81 ±0.07
5	34.9 ±9.0	68.0 ±0.0	0.51 ±0.13	46.2 ±5.6	0.68 ±0.08
6	36.4 ±7.9	100.0 ±0.0	0.36 ±0.08	70.3 ±5.1	0.70 ±0.05
7	3.5 ±1.8	100.0 ±0.0	0.03 ±0.02	25.7 ±5.2	0.26 ±0.05
8	15.3 ±3.3	100.0 ±0.0	0.15 ±0.03	52.1 ±3.7	0.52 ±0.04
9	73.3 ±18.9	100.0 ±0.0	0.73 ±0.19	89.8 ±7.2	0.90 ±0.07
10	12.8 ±7.6	100.0 ±0.0	0.13 ±0.08	44.0 ±10.2	0.44 ±0.10
11	60.5 ±32.0	100.0 ±0.0	0.61 ±0.32	79.5 ±18.1	0.79 ±0.18
12	52.0 ±11.3	100.0 ±0.0	0.52 ±0.11	78.9 ±6.5	0.79 ±0.06
13	26.3 ±15.9	49.4 ±35.8	0.58 ±0.07	32.2 ±28.5	0.57 ±0.11
14	39.4 ±20.8	100.0 ±0.0	0.39 ±0.21	69.0 ±11.0	0.69 ±0.11
15	24.0 ±17.2	68.0 ±0.0	0.35 ±0.25	35.8 ±13.8	0.53 ±0.20
16	42.0 ±0.0	100.0 ±0.0	0.42 ±0.00	73.9 ±0.0	0.74 ±0.00
17	4.1 ±1.2	95.3 ±6.7	0.04 ±0.01	25.6 ±5.5	0.27 ±0.04
18	12.1 ±7.9	100.0 ±0.0	0.12 ±0.08	43.7 ±15.6	0.44 ±0.16
19	86.7 ±18.9	100.0 ±0.0	0.87 ±0.19	94.9 ±7.2	0.95 ±0.07
Avg.	**36.3** ±28.1	**91.3** ±21.1	**0.41** ±0.28	**58.5** ±25.9	**0.63** ±0.22

† In this CARLA Leaderboard 2.0 table, **NDS** stands for *Normalised Driving Score* [31], distinct from the nuScenes Detection Score (NDS) in Chapter 2.

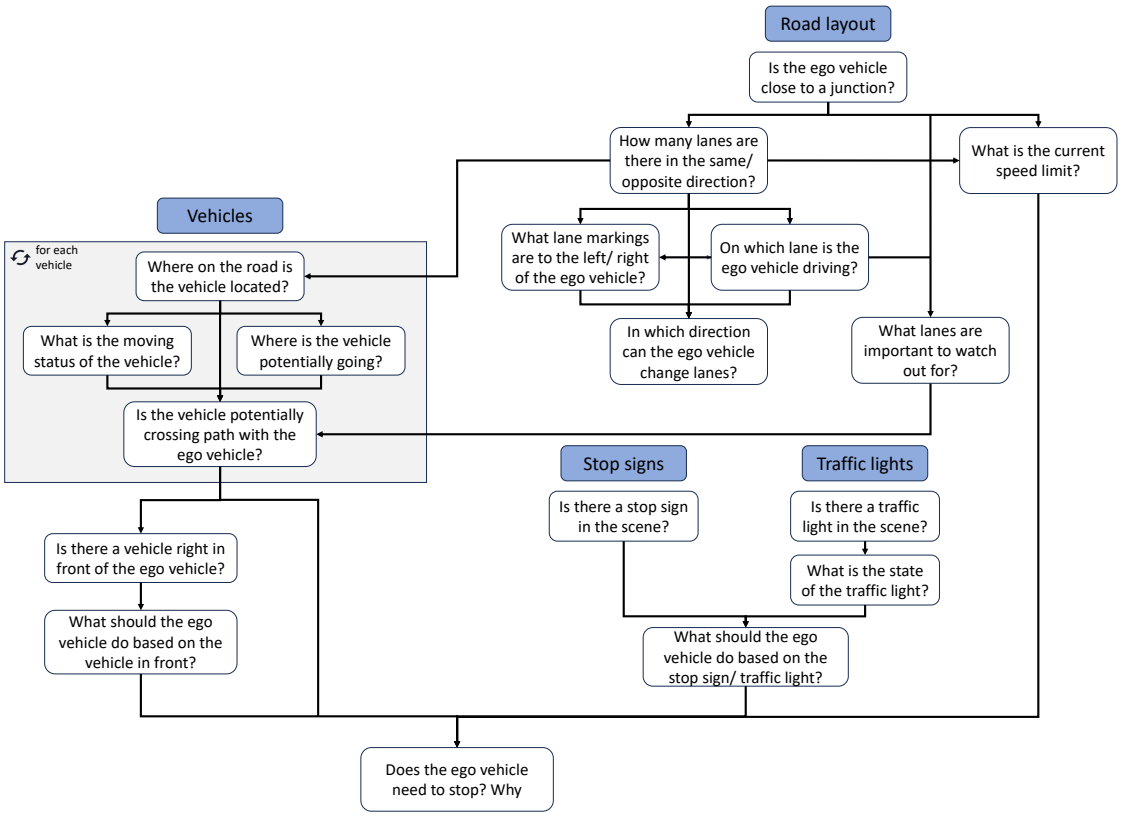


Fig. 3.10 **Topology of the DriveLM-CARLA question graph** across perception, prediction and planning.

SPICE > 0.5) and **GPT Score**, which queries `gpt-3.5-turbo` [4, 196, 197] using reference `{GT}` and prediction `{Pred}`:

> *System:* `An evaluator who rates my answer based on the correct answer.`
>
> *User:* `Rate my answer based on the correct answer out of 100, with higher scores indicating that the answer is closer to the correct answer, and you should be accurate to single digits like 62, 78, 41, etc. This is the correct answer: {GT}. This is my answer: {Pred}.`

Because the prompt omits the question text, camera views and grading rubrics, scores cluster tightly (Section 3.8) with untracked run variance. For stage B, we report classification accuracy on joint (B_{sp}, B_{st}) and marginal speed (B_{sp}) and steering (B_{st}) bins

derived from future displacements (Eq. (3.1)). For stage M, we report open-loop **ADE** (mean Euclidean error over 1 s, 2 s and 3 s), **FDE** (error at 3 s) and **collision rate** (percentage of frames with ego–obstacle box overlap across 1/2/3 s in Table 3.2), following instantaneous UniAD evaluation [3] rather than cumulative ST-P3 averaging [66][3]; results on our 799 keyframes are not directly comparable to the full 6,019-frame nuScenes `val` split.

In DriveLM-Agent, training applies teacher forcing [198, 199] across all frame QAs using LoRA [171] (constant learning rate 10^{-4}, seed 1234, default rank/scaling, 400-token cap), prepending ground-truth parent QAs to child queries along edges $e \in E$ under cross-entropy loss. Following RT-2 [108], each trajectory coordinate (x, y) is quantised into 256 uniform bins mapped to multi-digit numeric tokens in `blip-2-flan-t5-xl` (sparing single-digit tokens) between SOT and EOT markers and decoded back to metric waypoints (Fig. 3.11). Because P_{1-3} QA counts vary widely across DriveLM-nuScenes frames (variance > 260), validation samples a balanced template subset (≥ 1 prompt per stage per frame) and rolls out $P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow B \rightarrow M$ autoregressively from predicted parent answers. For zero-shot Waymo transfer (Section 3.6.2), generic P_1 prompts run on front-camera frames and detections of shared categories ("cars", "trucks", "pedestrians") trigger rule-based P_2/P_3 prompts. For DriveLM-CARLA-ped (Section 3.6.3), we insert *Is there a person in the scene?* and, when affirmative, *What should the vehicle do based on the pedestrian that is crossing the road?* (otherwise *What should the ego vehicle do?*), appending the resulting answer to the behaviour context.

Table 3.8 reports the full VQA metric suite on DriveLM-nuScenes `val`, showing that graph conditioning lowers ROUGE-L for DriveLM-Agent (66.79 → 65.58) and CIDEr/Completeness for zero-shot BLIP-2, which motivated our post-hoc selection of GPT Score. Table 3.9 expands the Waymo transfer study across individual stage contexts: behaviour accuracy rankings closely mirror ADE (top two within 0.01 m), with P_2-only context achieving the best steering accuracy (78.67) and ADE (2.62)

[3] `https://github.com/OpenDriveLab/UniAD/commit/ffac1a69a6d01c9dca5cd2ed751007896df351f8`

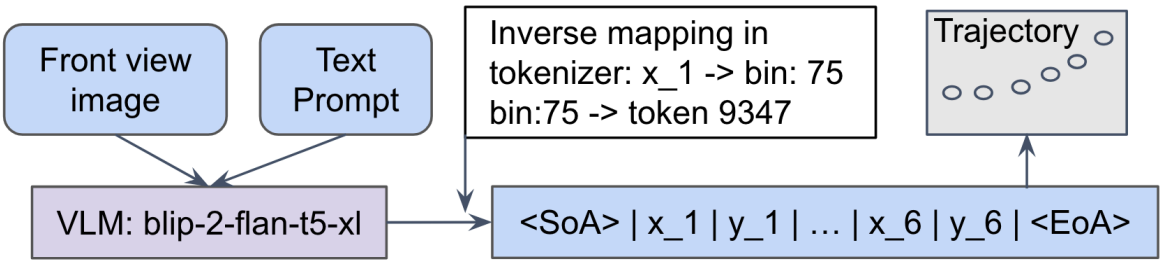


Fig. 3.11 **End-to-end pipeline of DriveLM-Agent** quantising waypoints into `blip-2-flan-t5-xl` tokens.

Table 3.8 **Full VQA metric suite on DriveLM-nuScenes `val`** (Comp.: SPICE > 0.5 percentage; single runs).

Model	**Context**	BLEU-4	METEOR	CIDEr	ROUGE-L	SPICE	GPT	Comp.
Off-the-shelf BLIP-2	None	0.022	3.317	0.1185	7.205	4.336	42.97	1.064
	Graph	0.022	3.882	0.0771	7.397	7.710	45.21	0.859
	GT	*0.022*	*4.397*	*0.0758*	*8.033*	*8.192*	*41.10*	*1.315*
DriveLM-Agent	None	51.89	35.81	2.463	66.79	42.56	71.39	30.04
	Graph	53.09	36.19	2.786	65.58	49.54	72.51	31.66
	GT	*53.06*	*36.64*	*3.069*	*66.69*	*50.29*	*72.94*	*32.41*

and full P_{1-3} context achieving the best overall behaviour (39.73) and speed accuracy (54.29). Table 3.10 substitutes a 9-epoch LLaMA-Adapter V2 (7B) [174] backbone on DriveLM-nuScenes, where single-frame performance matches BLIP-2 and multi-frame input modestly improves behaviour accuracy (57.83) and collision rate (1.69). Caching visual tokens and batching same-stage queries can reduce the latency reported in Table 3.6. More broadly, coupling driving stacks with foundation VLMs provides human-readable rationales for passengers and regulators, though hallucinations and compute cost must be addressed before safety-critical deployment.

Table 3.9 **Stage-wise context ablation for zero-shot transfer to Waymo** on 1k Waymo `val` frames (B^*: ground-truth behaviour QA context; single runs).

Method	Behaviour Context	Motion Context	Behaviour (B)			Motion (M)	
			Acc.↑	Speed↑	Steer↑	ADE↓	FDE†↓
Command Mean	-	-	-	-	-	7.98	11.41
UniAD-Single	-	-	-	-	-	4.16	9.31
BLIP-RT-2	-	-	-	-	-	2.78	6.47
DriveLM-Agent	None	B	35.70	43.90	65.20	2.76	6.59
	P_1	B	38.20	43.67	70.74	2.67	6.41
	P_2	B	39.52	44.20	**78.67**	**2.62**	6.19
	P_3	B	34.62	41.28	64.55	2.85	6.89
	P_{1-3}	B	**39.73**	**54.29**	70.35	2.63	**6.17**
BLIP-RT-2	-	B^*	*100.0*	*100.0*	*100.0*	*2.41*	*5.79*

† Printed under the header `Col.` in Table 2 of the ECCV 2024 paper [4] (Table 3.2(b)) and `FDE` in Table 6 of its supplementary material.

Table 3.10 **Alternative VLM backbones on DriveLM-nuScenes** (*: multi-frame input to LLaMA-Adapter V2).

Base VLM	Behaviour Context	Motion Context	Behaviour (B)			Motion (M)	
			Acc.↑	Speed↑	Steer↑	ADE↓	Col.↓
`blip-2-flan-t5-xl`	Graph	B	57.49	**69.89**	80.63	**1.74**	1.89
LLaMA-Adapter V2	Graph	B	55.31	61.97	81.43	1.86	2.03
LLaMA-Adapter V2*	Graph	B	**57.83**	67.21	**83.49**	1.75	**1.69**

3.10 Chapter Summary

Addressing RQ2, this chapter introduced DriveLM and Graph VQA (GVQA) to link pre-trained VLMs to driving via a directed QA graph across perception, prediction, planning, behaviour and motion. We built DriveLM-nuScenes (4,871 keyframes, 91.4 QAs/frame) and DriveLM-CARLA (1.6M QAs from PDM-Lite [31], which scores 36.3 ± 28.1 DS over 20 Leaderboard 2.0 validation routes versus 2% for TransFuser++), alongside DriveLM-Metrics and the BLIP-2 baseline DriveLM-Agent.

In single-run experiments, graph context hurt in-domain nuScenes planning relative to context-free behaviour prediction (ADE 1.39 → 1.74, collision rate 1.67 → 1.89, behaviour accuracy 61.45 → 57.49; UniAD-Single 1.80 ADE) and helped only under

distribution shift. On Waymo, most of the ADE gain over UniAD-Single (2.63 *vs.* 4.16) came from the backbone before graph conditioning (BLIP-RT-2 2.78, context-free 2.76) amidst rear-camera and 3.955B *vs.* 131.9M parameter asymmetries, while full-graph context raised speed accuracy (43.90 $\rightarrow$ 54.29) and P_2-only context matched ADE (2.62). On withheld CARLA pedestrians (by co-authors), TransFuser++ fell from 70.19 to 8.72 and DriveLM-Agent rose from 4.59 to 27.04 only with a hand-crafted pedestrian prompt (reducing regular-scene accuracy from 59.63 to 52.17; 92.35 'GT' leaks pedestrian presence). Question-level and P_{1-3} VQA ablations showed modest gains amidst 0.16 FPS latency (8.5 tokens/s), front-camera-only input, omitted diagnostic/consistency baselines, and text-shortcut vulnerabilities later exposed by DriveBench [16]. Because static weights still cannot self-correct when deployment shifts miscalibrate confidence, Chapter 4 introduces online test-time training over the five steering behaviour clusters defined here.

Chapter 4

Act I: Robust End-to-End Driving via Test-Time Training

Statement of contribution

This chapter adapts "Centaur: Robust End-to-End Autonomous Driving with Test-Time Training" [17], co-led by the thesis author (primary contact) and co-first author Kashyap Chitta during an internship at NVIDIA Autonomous Vehicle Applied Research. The author formulated test-time training for end-to-end planning, devised Cluster Entropy, integrated Centaur into Hydra-MDP, ran the NAVSIM benchmarks, curated and verified `navsafe`, and conducted the failure-detection, ablation, and qualitative studies; Kashyap Chitta co-designed the TTT formulation, regression-planner adaptation, and experimental analysis; and co-authors Zhiding Yu, Shiyi Lan, Ping Luo, Andreas Geiger, Hongyang Li, and Jose M. Alvarez contributed to project guidance, NAVSIM/Hydra-MDP integration, and manuscript writing.

While Chapters 2 and 3 examined perception and reasoning through 3D occupancy and vision-language grounding, action remains the ultimate test of an embodied agent. Because an end-to-end planner directly governs vehicle motion on public roads, any planning error risks a collision when encountering rare, safety-critical situations absent from training logs, such as unfamiliar roundabouts, ambiguous yellow signals, or sudden pedestrian crossings, a long-tail exposure known as the curse of rarity [9].

Existing end-to-end planners are often paired with fallback rules that override plans upon rule breaches or post-hoc cost optimisation over auxiliary drivable-area

maps and obstacle boxes. Those safeguards add engineering and annotation requirements and may trade progress for caution. Centaur instead asks a bounded version of RQ3a: *whether planner weights can adapt at deployment under distributional shift without ground-truth labels or a hand-crafted test-time objective.* We compare that adaptation with a simple uncertainty-triggered prior-trajectory fallback implemented for this study and with UniAD's reported post-hoc optimisation [3]. The scope is deliberately limited: NAVSIM scores independent plans per frame from logged states, so closed-loop errors cannot accumulate (Section 4.5.1), and while no rules run at test time, the base Hydra-MDP heads were trained offline to distil rule-based simulator metrics (Section 4.3.1).

We propose *Centaur* (**C**luster **En**tropy for **T**est-time tr**A**ining using **U**nce**r**tainty), which updates a pre-trained planner via test-time training (TTT) rather than overriding its outputs, taking a single gradient step averaged over a short ring buffer of recent frames to reduce trajectory-score dispersion (evaluated sequentially here, though compatible with asynchronous execution). Three contributions anchor this chapter: (i) introducing TTT for an end-to-end planner on NAVSIM (March 2025), contrasting with forecasting TTT [200] and fixed-weight search [201]; (ii) formulating *Cluster Entropy* across five lateral steering directions; and (iii) curating `navsafe`, a 229-frame diagnostic subset of `navtest` across ten safety-critical categories guided by the US National Highway Traffic Safety Administration (NHTSA) pre-crash typology [202] and ranked by baseline Hydra-MDP [203] scores on NAVSIM [23].

In NAVSIM v1.0 non-reactive 4-second replay [23] (where each frame resets to the human state, leaving closed-loop shift in Chapter 1 and long-horizon drift untested), Centaur raises Hydra-MDP from 90.3 to 92.6 PDMS in one run (human reference: 94.8). On v1.1 it reaches 92.10 ± 0.33 over three seeds (ranking first among three-seed leaderboard entries in March 2025), although the leaderboard has no matched three-seed unadapted Hydra-MDP entry. The 229-frame `navsafe` analysis (Table 4.10, Section 4.5.5) is less uniform: Centaur improves the frame-weighted mean from 49.36 to 68.86 (68.9 *vs.* 49.4), the simple category mean from 48.33 to 71.96 (72.0 *vs.* 48.3),

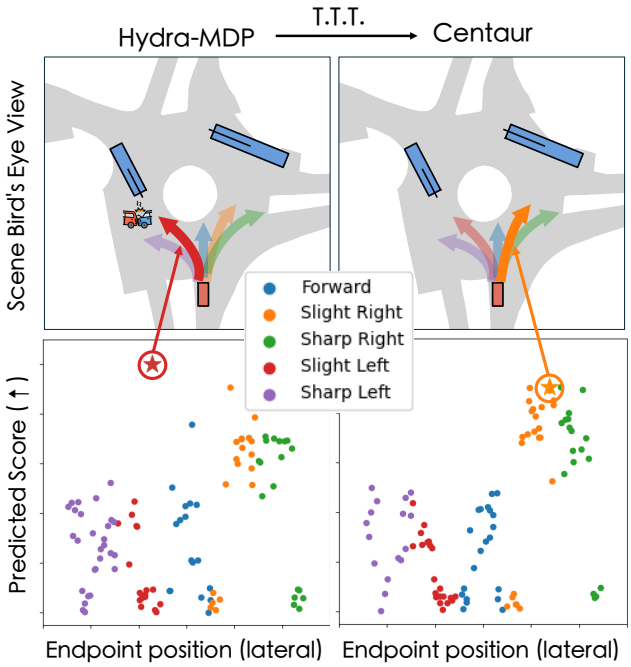


Fig. 4.1 **Minimising directional score entropy via test-time training (TTT).** The trajectory-scoring planner Hydra-MDP [203] scores candidate trajectories on `navtest` [23] across five directional clusters (★: top-scoring plan). **Left:** Entering a roundabout, the unadapted model picks an isolated 'slight left' outlier despite low cluster support. **Right:** Minimising Cluster Entropy via TTT sharpens the score distribution toward 'slight right', avoiding oncoming traffic.

and the preprint Overall from 56.47 to 74.14, but the subset was selected using Hydra-MDP scores and performance on yielding falls from 71.6 to 59.3 while trailing human logs (94.65). NAVSIM derives from OpenScene [29] (Chapter 2) and nuPlan [95], and Cluster Entropy reuses DriveLM's five lateral steering bins [4] (Chapter 3) without longitudinal speed bins. Sections 4.1–4.9 cover motivation, related work, method, `navsafe`, experiments, analysis, discussion, supplementary evaluations (Section 4.8), and summary.

4.1 Motivation

Operating safely alongside human traffic requires handling rare interactions and unfamiliar road geometries [9, 204]. While end-to-end neural planners scale directly with

driving logs [3, 98, 101, 203, 205, 206], deployed policies are routinely paired with rule-based fallback modules that override the network under hazard conditions [101, 207, 208] or post-hoc cost optimisers [3, 209] that require expensive intermediate perceptual annotations such as drivable surfaces and motion forecasts, undercutting end-to-end learning from logged ego poses [167, 170]. Our goal is to improve deployment reliability in a purely data-driven manner at inference time without hand-tuned fallback rules or test-time perceptual supervision, while noting that our base planner distils rule-based simulation scores offline (Section 4.3.1).

Centaur achieves this online adaptation via test-time training [210–213], updating score decoder weights by gradient descent on an unsupervised uncertainty loss over recent observations (evaluated sequentially in Section 4.6.3, though compatible with asynchronous execution). Whereas classic trajectory regression policies output a single continuous waypoint sequence [3, 22, 101, 205], discrete trajectory-scoring planners [203, 214, 215] evaluate a fixed candidate vocabulary analogous to classification [213, 216], enabling Cluster Entropy over five directional trajectory buckets.

In Fig. 4.1, a single TTT step on Hydra-MDP [203] shifts selection from an isolated 'slight left' outlier to the 'slight right' cluster, preventing a turn into oncoming traffic. Cluster Entropy can also flag high-risk frames at a fixed threshold, attaining the highest true-positive rate among tested uncertainty estimators, though its overall accuracy remains below a never-flag baseline and threshold-free curves were not logged (Section 4.5.3). Lastly, because aggregate test averages obscure rare failures, we curate `navsafe`, a 229-frame NHTSA-guided subset of `navtest` that exposes shared vulnerabilities across planners (Section 4.4).

4.2 Related Work

End-to-end autonomous driving. Modern end-to-end architectures map raw multi-view camera and LiDAR streams directly to future waypoints or control

commands with a single trainable network [3, 98, 165, 167, 168, 203, 217–224], including the 3D occupancy pipeline in Chapter 2 [13].

End-to-end planner deployment. Deployed neural policies typically attach rule-based verifiers or cost maps: UniAD [3] refines trajectories against an obstacle cost map, TransFuser [101] triggers hard braking on LiDAR obstacles, and SafetyNet [207] validates kinematic feasibility before falling back to a rule controller. Centaur invokes no hand-crafted rules at test time, though its base policy distils rule-based simulation scores offline (Section 4.3.1).

Test-time compute and training. Allocating test-time computation strengthens reasoning [225], while test-time adaptation and training (TTT) [211] improve robustness in image recognition [210, 216, 226, 227], multimodal alignment [228, 229], navigation [213], motion forecasting [200], manipulation [230], and language modelling [231]. Unlike forecasting TTT (T4P [200]) or fixed-weight trajectory search (Diffusion-ES [201], TOAD [232]), Centaur adapts end-to-end planner decoder weights directly on NAVSIM (see Chapter 1 and Section 4.8).

4.3 Method

Fig. 4.2 summarises the Centaur pipeline: trajectory scoring (Section 4.3.1), Cluster Entropy (Section 4.3.2), and gradient-buffered TTT (Section 4.3.3).

4.3.1 Preliminaries

An end-to-end driving policy π maps sensor observations $\mathbf{x}_0$ and discrete route command $\mathbf{c}_0$ at step 0 to N Bird's Eye View (BEV) waypoints $\tilde{\mathbf{t}} = (\mathbf{p}_0, \mathbf{p}_1, \ldots, \mathbf{p}_N)$. While TTT can also adapt continuous trajectory regression policies by attaching evidential heads (see Sections 4.8 and 4.8 for UniAD [3] and TransFuser [101]), we focus pri-

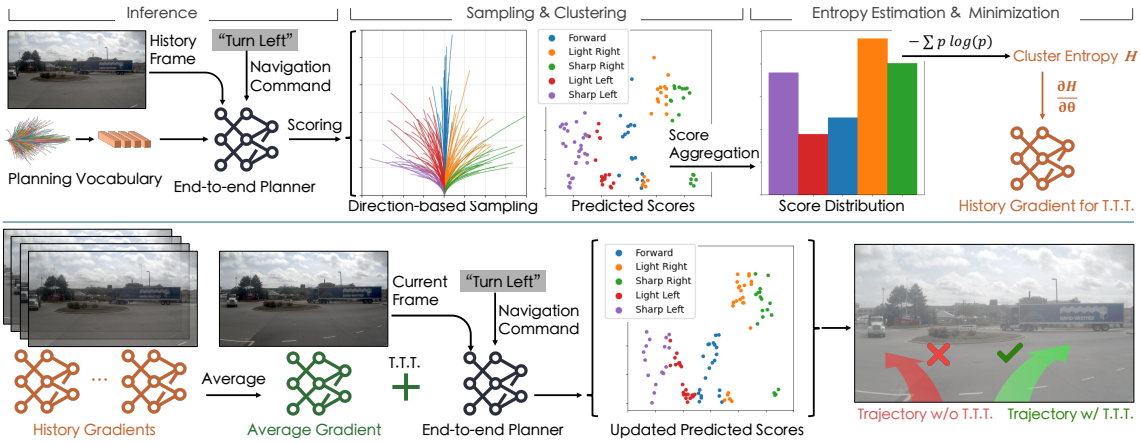


Fig. 4.2 **Overview of test-time training (TTT) in Centaur. Top:** At each deployment frame, a fixed subset of M candidate trajectories is grouped into five directional clusters to compute Cluster Entropy and its gradient via backpropagation. **Bottom:** Gradients from the F preceding evaluation steps are buffered and averaged for a single gradient step from base decoder weights θ (Algorithm 1).

marily on trajectory scoring planners such as Hydra-MDP [203], VADv2 [218], and RAD [233].

Trajectory scoring. Rather than regressing continuous coordinates, a scoring planner evaluates a fixed vocabulary of k candidate trajectories $\{\mathbf{t}_j\}_{j=1}^{k}$, predicting score feature vectors $\{s_j\}_{j=1}^{k}$ that a combining function ϕ maps to a scalar utility to pick the highest-scoring plan:

$$\{s_j\}_{j=1}^{k} = \{\pi_\theta(\mathbf{t}_j, \mathbf{x}_0, \mathbf{c}_0)\}_{j=1}^{k}, \tag{4.1}$$

$$\hat{\mathbf{t}}_0 = \underset{\mathbf{t}_j}{\operatorname{argmax}} \{\phi(s_j)\}_{j=1}^{k}. \tag{4.2}$$

Offline knowledge distillation trains the weights θ over a dataset $D_{kd} = \{(\mathbf{t}, \mathbf{x}, \mathbf{c}), \dots\}$ rolled out in a driving simulator [24, 95] or world model [234, 235]:

$$\underset{\theta}{\operatorname{argmin}}\, \mathbb{E}_{(\mathbf{t},\mathbf{x},\mathbf{c})\sim D_{kd}} \left[\mathcal{L}_{kd}\Big(e(\mathbf{t}, \mathbf{x}, \mathbf{c}), \pi_\theta(\mathbf{t}, \mathbf{x}, \mathbf{c})\Big)\right], \tag{4.3}$$

using binary cross-entropy $\mathcal{L}_{kd}$ on $(0, 1)$-scaled sub-scores. In Hydra-MDP, the teacher e evaluates the rule-based NAVSIM sub-scores (Section 4.5.1) alongside a human imitation loss (Section 4.8.1), so the adapted decoder heads approximate rule-based metrics distilled offline.

4.3.2 Cluster Entropy

While Eqs. (4.1)–(4.2) resemble k-way classification, minimising raw k-way Shannon entropy over-penalises dense vocabulary neighbourhoods around common manoeuvres such as cruising straight. Cluster Entropy avoids this density artefact in three stages without requiring teacher e at deployment.

Trajectory candidates. Offline, we draw a fixed subset of $M \ll k$ candidates $\{\mathbf{t}_m\}_{m=1}^{M}$ weighted by their mean expert score e across `navtrain` (Section 4.8), which under-represents rare manoeuvres. Computing Shannon entropy directly over these M scores defines our Full Entropy baseline.

Direction-based sampling. From the M sampled candidates, Centaur defines five directional anchors by endpoint lateral offset inspired by DriveLM's five steering categories [4] (Chapter 3): `sharp left` and `sharp right` (most extreme left and right lateral endpoint displacements), `slight left` and `slight right` (closest to half of those extremes), and `forward` (minimal lateral displacement). Because longitudinal speed bins are omitted, braking and cruising along the same lane share a directional bucket, leaving stop-or-go ambiguity unrepresented (Section 4.7).

Score aggregation. Assigning each of the M candidates to its closest anchor in Euclidean (L_2) waypoint distance partitions the subset into five directional clusters $\mathcal{C}_1, \ldots, \mathcal{C}_5$ (Fig. 4.2, top). Summing post-sigmoid positive composite scores $\phi(s_m) > 0$

within each cluster $\mathcal{C}_a$ yields unnormalised cluster mass $score_a = \sum_{m:\,\mathbf{t}_m \in \mathcal{C}_a} \phi(s_m)$ and Shannon entropy $H(\mathbf{x}) \in [0, \ln 5]$ (with $0 \log 0 = 0$):

$$p_a = \frac{score_a}{\sum_{a'=1}^{5} score_{a'}}, \qquad H(\mathbf{x}) = -\sum_{a=1}^{5} p_a \log p_a. \tag{4.4}$$

Because offline cluster cardinalities are unequal, summing within clusters retains a mild density bias whose exact size distribution was not separately audited.

4.3.3 Test-Time Training

Following single-step TTT [213], Centaur evaluates $H(\mathbf{x}_r; \theta)$ at the base decoder weights θ on each frame, backpropagates buffered gradients $g_r = \nabla_\theta H(\mathbf{x}_r; \theta)$, and averages the last $F = 4$ gradients in a ring buffer (Fig. 4.2, bottom) to take a single step of size η from the base decoder weights θ:

$$\hat{\theta}_i = \theta - \eta \left\{ \frac{\partial H}{\partial \theta} \right\}_{avg}, \tag{4.5}$$

dampening isolated high-scoring outliers in weakly supported directions (Fig. 4.1), though entropy minimisation can reinforce an erroneous dominant cluster when the base policy is confidently wrong (Section 4.7). Algorithm 1 details the update loop.

Operational semantics and deployment. Algorithm 1 defines an episodic reset rule with four properties. First, every step i resets to the base decoder weights θ before applying Eq. (4.5), preventing weight drift across long sequences and limiting memory to the $F = 4$ buffered gradients. Second, step i depends only on the four preceding evaluation steps ($\hat{\theta}_1 = \theta$), which in NAVSIM come from logged human states rather than states visited by the planner's own past actions. Third, the ring buffer $\mathcal{B}$ does not reset across clips. Fourth, adaptation modifies only the score decoder parameters θ while visual and LiDAR backbones remain frozen. Because $\mathcal{B}$ consumes only past steps, backpropagation can execute asynchronously alongside inference, though our current benchmarks run it sequentially (Section 4.6.3).

Algorithm 1: Test-time training in Centaur

Input: pre-trained scoring policy π_θ with adaptable decoder weights θ (perception backbones frozen); vocabulary $\{\mathbf{t}_j\}_{j=1}^{k}$; M sampled candidates partitioned around five directional anchors (Section 4.3.2); gradient buffer size F; step size η

Output: selected plan $\hat{\mathbf{t}}_0$ at each evaluation step

$\mathcal{B} \leftarrow \emptyset$ `// FIFO ring buffer of past gradients`

foreach *evaluation step i with sensor observation $\mathbf{x}_0$ and route command $\mathbf{c}_0$* **do**

  if $\mathcal{B} \neq \emptyset$ **then**

    $\hat{\theta}_i \leftarrow \theta - \eta \left\{ \frac{\partial H}{\partial \theta} \right\}_{avg}$ using the mean gradient across $\mathcal{B}$ `// Eq.` (4.5)

  else

    $\hat{\theta}_i \leftarrow \theta$

  Evaluate vocabulary scores under $\pi_{\hat{\theta}_i}$ and pick $\hat{\mathbf{t}}_0$ `// Eqs.` (4.1)-(4.2)

  `// can execute asynchronously on parallel compute`

  Evaluate Cluster Entropy $H(\mathbf{x}_0; \theta)$ over the M candidate scores at base weights θ `// Eq.` (4.4)

  Backpropagate $g_i = \nabla_\theta H(\mathbf{x}_0; \theta)$ at θ and append to $\mathcal{B}$, retaining at most F recent steps

4.4 navsafe: A Safety-Critical Scenario Set

To probe planner behaviour on rare hazards, we curate `navsafe`, an internal 229-frame diagnostic subset of `navtest` drawn from NAVSIM [23], OpenScene [29] (Chapter 2), and nuPlan [95].

NAVSIM and construction. NAVSIM filters nuPlan logs for dynamic manoeuvres, comprising 1,192 training logs (`navtrain`, 103,288 frames) and 136 test logs (`navtest`, 12,146 frames), where policies map 1.5 seconds (4 frames) of multi-camera, LiDAR, and ego history to a 4-second plan. Guided by the NHTSA pre-crash typology [202, 237] (whose five most prevalent categories are "Lead Vehicle Stopped", "Control Loss Without Prior Vehicle Action", "Vehicle(s) Turning at Non-Signalized Junctions", "Lead Vehicle Decelerating", and "Road Edge Departure Without Prior Vehicle Maneuver"), we follow a five-step loop:

Step 0. Rank `navtest` clips by the mean PDMS of unadapted Hydra-MDP [203].

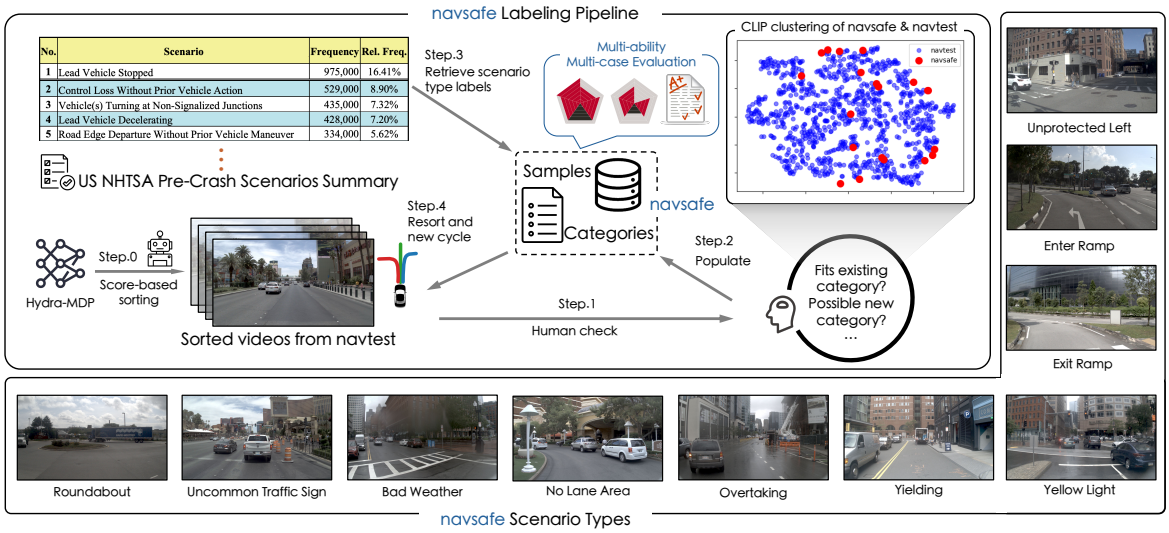


Fig. 4.3 **Curation workflow of `navsafe`.** `navtest` clips are ranked by Hydra-MDP PDMS (Step 0), inspected by human annotators (Step 1), and grouped into ten scenario classes (Steps 2–4) guided by the NHTSA pre-crash typology [202] (top-left inset), with a qualitative 2D OpenCLIP [236] projection of `navtest` (blue) and `navsafe` (red) frames at top right.

Step 1. Manually inspect low-scoring clips and assign frames to `navsafe` categories.

Step 2. Add verified frames to the corresponding scenario buckets.

Step 3. Consult the NHTSA typology to introduce uncovered scenario buckets.

Step 4. Re-rank the remaining `navtest` clips and repeat.

Categories and methodological caveats. The 229 frames span 10 diagnostic categories: “Roundabout”, “Yellow light, rush or wait?”, “Exit ramp”, “Unprotected left”, “Enter ramp”, “Uncommon traffic sign”, “Overtaking with lane change”, “No lane area (*e.g.* parking lot)”, “Bad weather”, and “Yielding (*e.g.* to pedestrians)”. Three caveats govern interpretation: ranking clips by Hydra-MDP PDMS in Step 0 induces regression to the mean in favour of modified Hydra-MDP variants without an independent seed control; `navsafe` samples the tail of `navtest` rather than an unseen domain (with qualitative 2D OpenCLIP [236] periphery in Fig. 4.3); and human category assignments lack a formal inter-annotator agreement study.

4.5 Experiments

We evaluate Centaur on NAVSIM [23]: full `navtest`, the multi-seed leaderboard, and the 229-frame `navsafe` subset (Section 4.4).

4.5.1 Experimental Setup

Metrics and replay scope. Under NAVSIM v1 non-reactive log replay, each 4-second plan is scored by the Predictive Driver Model Score (PDMS):

$$\text{PDMS} = 100 \cdot \text{NC} \cdot \text{DAC} \cdot \frac{5\,\text{TTC} + 2\,\text{C} + 5\,\text{EP}}{12}, \tag{4.6}$$

combining No at-fault Collision (NC), Drivable Area Compliance (DAC), 1-second Time-to-Collision (TTC), Comfort (C), and Ego Progress (EP), where $\text{NC}, \text{DAC}, \text{TTC}, \text{C}, \text{EP} \in [0, 1]$ in the evaluator product and are displayed as percentages ($\times 100$) in tables [23]. Resetting each frame to the human state precludes closed-loop shift or drift (Section 4.7), and logged human PDMS is a reference rather than a strict bound (EP is normalised by PDM-Closed [111]).

Base model and deployment. Hydra-MDP [203] fuses ResNet34 [73] BEV LiDAR and V2-99 [238] camera features via TransFuser blocks [101], scoring $k = 8192$ trajectories across five distilled PDMS sub-score heads (Section 4.3.1); we also test ResNet34 TransFuser [101] in Sections 4.6.2 and 4.8. Our prior-trajectory fallback caches the top 20 of $k = 8192$ trajectories by `navtrain` PDMS and substitutes the nearest (L_2) prior whenever uncertainty exceeds a Table 4.2 cutoff, without the feasibility checks or PDM-Closed [111] rules of SafetyNet [207].

Uncertainty measures and implementation. Alongside Cluster Entropy, we test pairwise inter-head **KL Divergence** over $k = 8192$ trajectories, **Full Entropy** [239] over $M = 100$ candidates, and **Semantic Entropy** (**Hydra-SE**) clustering $M = 100$ candidates in score space [240] (Section 4.8). Single-seed ablations (Tables 4.1 and 4.6)

Table 4.1 **Impact of TTT on `navtest` (NAVSIM v1.0, single seed).** TTT lifts PDMS under all uncertainty measures, led by Cluster Entropy. Prior fallback rows are audited against Table 4.2 in Section 4.5.2; three-seed v1.1 results appear in Table 4.3.

Model	Deployment	Uncertainty	NC ↑	DAC ↑	EP ↑	C ↑	TTC ↑	PDMS ↑
Hydra-MDP [203]	-	-	98.4	97.8	**86.5**	100.0	93.9	90.3
	Fallback to prior trajectories	KL Divergence	99.1	99.2	9.6	100.0	96.7	63.1
		Full Entropy	99.6	99.4	13.2	100.0	99.1	65.9
		Semantic Entropy	**99.7**	**99.5**	10.4	100.0	98.9	65.3
		Cluster Entropy	**99.7**	**99.5**	8.2	100.0	**99.3**	64.2
	TTT	KL Divergence	98.9	98.1	86.0	100.0	94.4	91.5
		Full Entropy	99.3	98.5	85.7	100.0	97.1	91.8
Hydra-SE	TTT	Semantic Entropy	99.2	98.5	85.7	100.0	97.1	91.8
Centaur (Ours)	TTT	Cluster Entropy	99.5	98.9	85.9	100.0	98.0	**92.6**
Human Trajectory	-	-	*100.0*	*100.0*	*87.5*	*99.9*	*100.0*	*94.8*

use NAVSIM v1.0; the three-seed leaderboard (Table 4.3) uses NAVSIM v1.1. Hydra-MDP trains on `navtrain` for 20 epochs on 8 A100 GPUs (batch 256, AdamW [97] lr 10^{-4}). TTT uses $M = 100$, $F = 4$, and $\eta = 10^{-4}$ on the decoder (η, M, F, τ tuned on `navtest`; Section 4.8.2).

4.5.2 Impact of Test-Time Training on `navtest`

Deployment strategy and uncertainty measure. In Table 4.1, the prior-trajectory fallback improves NC, DAC, and TTC but reduces PDMS to 63.1–65.9 because the reported EP values fall below 15. These rows should be treated as indicative rather than quantitative: the failure-detector statistics in Table 4.2 imply trigger rates of 29–33%, whereas EP values of 8.2–13.2 would require overrides on roughly 73–78% of frames, and the per-frame logs needed to resolve the discrepancy are unavailable (Section 4.8). TTT does not show the same progress collapse: Cluster Entropy raises TTC from 93.9 to 98.0 while retaining EP at 85.9, producing 92.6 PDMS (+2.3 over Hydra-MDP and +0.8 over Hydra-SE at 92.6 vs. 91.8 in Table 4.1, or +0.23 across three v1.1 seeds in Section 4.5.4) in the single v1.0 run. All four tested TTT objectives improve on the unadapted score (91.5 KL, 91.8 Full/Semantic, 92.6 Cluster; Section 4.6.1), while their relative ordering remains small compared with the larger adaptation effect.

Table 4.2 **Failure identification on `navtest`** (audited in Section 4.5.3).

Uncertainty	**Threshold**	**TPR ↑**	**Acc. ↑**
KL Divergence	1500	41.3	68.3
Full Entropy	0.8	61.7	70.4
Semantic Entropy	0.8	62.5	72.2
Cluster Entropy (Ours)	0.8	**62.8**	**73.6**
Select All	-	*100.0*	*15.6*
Select None	-	*0.0*	*84.4*

Table 4.3 **`navtest` leaderboard** (NAVSIM v1.1, March 2025, 3 seeds). *Hydra-MDP is a single run of the 2024 3-model ensemble.

Model	**NC ↑**	**DAC ↑**	**EP ↑**	**C ↑**	**TTC ↑**	**PDMS ↑**
Centaur (Ours)	99.23 ± 0.35	$\mathbf{98.72 \pm 0.22}$	$\mathbf{85.96 \pm 0.77}$	99.97 ± 0.03	97.17 ± 1.06	$\mathbf{92.10 \pm 0.33}$
Hydra-SE	$\mathbf{99.32 \pm 0.05}$	98.58 ± 0.12	85.42 ± 0.07	99.96 ± 0.01	$\mathbf{97.30 \pm 0.23}$	91.87 ± 0.19
Hydra-MDP* [203]	99.07	98.29	85.20	**100.0**	96.56	91.26
TransFuser [101]	97.78 ± 0.10	92.63 ± 0.48	78.88 ± 0.25	99.98 ± 0.01	92.89 ± 0.21	83.88 ± 0.45
LTF [101]	97.68 ± 0.11	92.29 ± 0.45	78.33 ± 0.53	99.99 ± 0.01	93.10 ± 0.19	83.52 ± 0.55
Ego Status MLP [23]	93.09 ± 0.12	78.26 ± 0.98	63.20 ± 0.53	99.97 ± 0.02	84.02 ± 0.61	66.40 ± 0.94

4.5.3 Failure Identification

Table 4.2 tests binary failure detection at fixed cutoffs ($0.8 \approx 0.5 \ln 5$ for entropy, 1500 for KL). Cluster Entropy achieves the top TPR (62.8%) and accuracy (73.6%), matching the TTT PDMS rank order in Table 4.1 (within 0.8 PDMS across entropy variants). However, at the 15.6% Select All base rate, Select None achieves 84.4% accuracy, single cutoffs are uncalibrated across estimators, and AUROC/AUPRC were not logged. Moreover, 15.6% zero-PDMS frames would cap Hydra-MDP PDMS at 84.4 < 90.3, while 100.0 C, 98.4 NC, and 97.8 DAC (Eq. (4.6)) cap strict PDMS $= 0$ frames at 3.8% (Section 4.8); thus the 15.6% Hydra-MDP and 24.1% TransFuser (Table 4.8) rates reflect a broader low-score indicator, and fallback PDMS moves with rather than against implied trigger rates (Sections 4.8 and 4.8).

Table 4.4 **`navsafe` PDM scores** across **RDBT** (Roundabout), **YLLT** (Yellow light), **EXR** (Exit ramp), **UNPL** (Unprotected left), **ENR** (Enter ramp), **UNTS** (Uncommon traffic sign), **OTLC** (Overtaking with lane change), **NLA** (No lane area), **BWTH** (Bad weather), and **YLD** (Yielding). Preprint Overall [17] (74.14 vs. 56.47, 17.67 gap) differs from the frame-weighted (49.36 vs. 68.86, or 49.4 vs. 68.9) and simple (48.33 vs. 71.96, or 48.3 vs. 72.0) means in Table 4.10 (Section 4.4).

	RDBT	YLLT	EXR	UNPL	ENR	UNTS	OTLC	NLA	BWTH	YLD	Overall
Num. frames	7	23	45	28	51	12	8	5	39	11	229
Constant Velocity [23]	0.0	3.8	3.6	0.0	2.9	7.4	0.0	6.4	6.3	3.8	3.41
Ego Status MLP [23]	28.9	22.7	26.0	8.9	37.7	28.0	35.0	8.8	33.4	22.1	24.10
LTF [101]	42.8	**70.2**	43.8	59.4	70.6	55.3	55.8	31.5	62.8	47.5	53.74
TransFuser [101]	47.3	55.1	35.6	55.7	66.3	51.7	56.2	42.7	55.2	61.6	51.87
Hydra-MDP [203]	32.6	34.3	37.7	64.9	65.5	59.6	43.3	38.1	35.7	71.6	56.47
Hydra-SE	62.2	53.9	50.5	57.8	72.0	65.6	74.0	57.6	**64.4**	**84.6**	62.84
Centaur (Ours)	**76.5**	65.0	**61.4**	**73.6**	**75.0**	**85.2**	**83.6**	**79.8**	60.2	59.3	**74.14**
Human Trajectory	*96.4*	*97.6*	*95.2*	*94.3*	*97.2*	*95.8*	*94.1*	*93.7*	*90.7*	*91.5*	*94.65*

4.5.4 `navtest` Leaderboard

Across 3 seeds on the official NAVSIM v1.1 leaderboard [23] (Table 4.3, March 2025), single-model Centaur ranks first at 92.10 ± 0.33 PDMS, 0.23 ahead of Hydra-SE (91.87 ± 0.19; Section 4.7). Because Hydra-MDP* is a single 3-model ensemble run [203], no matched three-seed unadapted baseline appears here (see Section 4.8 for v1.0).

4.5.5 Scenario-Level Evaluation on `navsafe`

On `navsafe` (Table 4.4), all unadapted planners score below 50 PDMS on EXR and NLA (though five categories have 5–12 frames), Hydra-MDP nears Ego Status MLP on RDBT and OTLC, and LTF tops TransFuser on YLLT and ENR. Centaur leads seven of ten categories, topping Hydra-MDP and Hydra-SE (62.84) on the preprint Overall (74.14 *vs.* 56.47) as well as the frame-weighted (68.9 *vs.* 49.4) and simple (72.0 *vs.* 48.3) means in Table 4.10 (where Hydra-MDP trails LTF and TransFuser). Still, sorting by Hydra-MDP induces regression to the mean (Section 4.4), Centaur drops from 71.6 to 59.3 on YLD and trails on BWTH and YLLT (Section 4.7), and all policies remain far below human logs (94.65).

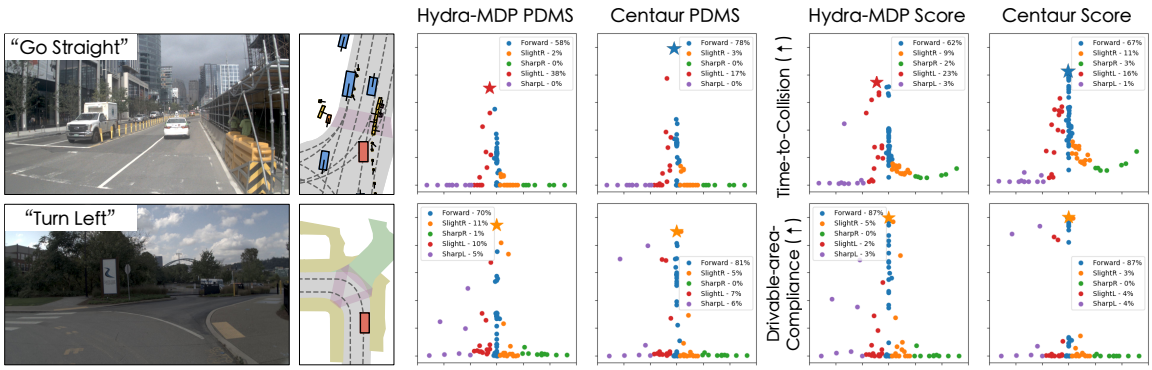


Fig. 4.4 **Qualitative results** (★: top-scoring plan ordered by lateral endpoint). **Top:** TTT shifts selection from 'Slight Left' to 'Forward', improving TTC. **Bottom:** TTT fails to overturn an overconfident choice.

4.6 Analysis

We examine trajectory shifts, regression transfer, hyper-parameters, and runtime (see Section 4.8).

4.6.1 Qualitative Results

In Fig. 4.4 (top), Cluster Entropy TTT shifts Hydra-MDP from a tight-TTC 'Slight Left' plan to a safer 'Forward' trajectory; in Fig. 4.4 (bottom), TTT lifts DAC on two 'Sharp Left' plans without overturning the initial choice (dataset-wide switch rates were not logged).

4.6.2 Generality and Design Choices

Regression planners and explicit representations. Adding an evidential head (Section 4.8) to TransFuser [101] with Semantic Entropy (TransFuser-SE) lifts single-seed NAVSIM v1.0 PDMS from 83.4 to 85.7 (+2.3; Table 4.6), compared with 84.2 under direct evidential TTT and roughly 60 (EP < 15) under the prior fallback (Table 4.7; audited in Section 4.8). In Table 4.6, UniAD [3] cost optimisation drops PDMS from 83.4 to 75.2 across all five sub-scores (NC 97.8 to 95.4, DAC 91.9 to 86.0, TTC 92.9 to 87.3, EP 78.8 to 71.9, C 100.0 to 99.3), while adding 3D occupancy

to PARA-Drive [205] shifts PDMS from 84.0 to 83.6 (unlike the open-loop setting of Chapter 2).

Gradient buffer and backbone. In Table 4.7, singular-value decomposition (SVD) gradient projection [213] on TransFuser-SE scores 83.8 vs. 85.7 for uniform averaging; sweeping $F \in \{1, 2, 3, 4\}$ on Hydra-SE yields 91.4, 91.3, 91.5, and 91.8 PDMS (a 0.5-point spread above Hydra-MDP's 90.3); and replacing V2-99 with ViT-L in Hydra-MDP gives 89.9 vs. 90.3 unadapted, 90.9 vs. 91.5 under KL TTT, and 67.4 vs. 63.1 under KL fallback (Section 4.5.2).

4.6.3 Computational Cost

On one A100 GPU (Table 4.7), unadapted Hydra-MDP runs at 4.1 Hz (243.9 ms per frame) versus 103.1 ms for ResNet-34 TransFuser (roughly 7 PDMS lower). Synchronous TTT runs at 2.8–3.2 Hz (312.5–357.1 ms on one A100, with Cluster Entropy's exact overhead matching the 100-candidate entropy pass), above NAVSIM's 2 Hz rate (4 frames in 1.5 s). Asynchronous updates (Section 4.3.3) or uncertainty-gated Hydra-SE^{4} (3.6 Hz / 279.6 ms at 91.3 PDMS) reduce this overhead.

4.7 Discussion and Limitations

Evidence and fallback comparison. Under NAVSIM v1 non-reactive replay, TTT lifts single-seed Hydra-MDP from 90.3 to 92.6 PDMS (TTC 93.9 to 98.0, EP 86.5 to 85.9), reaches 92.10 $\pm$ 0.33 across three v1.1 seeds (without a matched three-seed Hydra-MDP baseline), and adds 0.8 to 2.3 PDMS across base planners (or 0.4 with SVD gradient projection; Tables 4.1, 4.6, and 4.7) using offline rule-distilled heads. Whereas our 20-prior fallback cut EP below 15 (PDMS 63.1–65.9 on Hydra-MDP and 59.9–67.4 across all tested planners, with mismatched trigger rates in Section 4.5.2) and UniAD's cost optimiser reduced all sub-scores, full rule verifiers like SafetyNet [207] remain untested here. On `navsafe`, Centaur's lead over Hydra-MDP (17.67 points

in preprint Overall, 68.9 *vs.* 49.4 frame-weighted, and 72.0 *vs.* 48.3 simple mean in Table 4.10) is confounded by Hydra-MDP selection bias (Section 4.4) and accompanied by a drop on YLD (71.6 to 59.3).

Cluster Entropy and failure detection. Across three v1.1 seeds, Centaur leads Hydra-SE by 0.23 PDMS (92.10 ± 0.33 *vs.* 91.87 ± 0.19 in Table 4.3), within inter-seed variance, and all four objectives in Table 4.1 gain 1.2 to 2.3 PDMS; thus the primary gain comes from generic test-time score sharpening, while Cluster Entropy provides directional interpretability (Figs. 4.1 and 4.4). As a failure detector (Table 4.2), Cluster Entropy's 62.8% TPR comes with 73.6% accuracy (below the 84.4% Select None baseline at a 15.6% non-zero failure rate) and no AUROC/AUPRC curves (Section 4.5.3).

Threats to validity. Four threats qualify our findings:

1. **Evaluation protocol:** NAVSIM v1 resets each frame to logged human states with non-reactive traffic and Episodic weight resets to θ (Section 4.3.3), leaving closed-loop compounding errors, oscillation, and continual weight drift untested; hyper-parameters and `navsafe` were also selected on `navtest` without multi-seed confidence bounds on v1.0 ablations.

2. **Confounding mechanisms:** Single-step TTT ($\eta = 10^{-4}$) could reflect generic score re-calibration of heads distilled from NC, DAC, EP, C, and TTC on pre-filtered `navtrain` candidates (Section 4.8); static pre-test sharpening, unrelated-clip buffers, and independent NAVSIM v2 [241] metrics were not tested.

3. **Objective expressivity:** Lateral-only anchors cannot represent longitudinal stop-or-go uncertainty (YLD, YLLT, and NHTSA "Lead Vehicle Stopped", where Centaur drops from 71.6 to 59.3 on YLD): while Cluster Entropy groups candidates by full-trajectory Euclidean L_2 distance around five endpoint lateral anchors (conflating stopping and cruising along the same path), Semantic Entropy (Hydra-SE,

which reaches 84.6 on YLD) clusters in the 5D predicted sub-score space where EP and TTC separate stopping from cruising, and entropy minimisation can reinforce confident errors [212] (Fig. 4.4).

4. **`navsafe` curation:** Sorting `navtest` by Hydra-MDP PDMS induces regression to the mean without an independent seed control, five of ten categories have 5–12 frames without inter-annotator agreement, and Table 4.10 audits the preprint Overall column of Table 4.4.

Limitations. Future work must test closed-loop simulators, NAVSIM v2 [241], and vehicle hardware under asynchronous compute (Section 4.6.3), address regressions on YLD (59.3 *vs.* 71.6 Hydra-MDP, 84.6 Hydra-SE, 61.6 TransFuser), BWTH (60.2 *vs.* 64.4 Hydra-SE, 62.8 LTF), YLLT (65.0 *vs.* 70.2 LTF), and `navtest` EP (85.9 *vs.* 86.5), and extend adaptation beyond frozen perception backbones (Chapter 6).

4.8 Implementation Details and Supplementary Evaluations

This section provides supplementary context on related literature, mathematical formulations for regression and scoring policies, extended ablations, and arithmetic consistency audits.

4.8.1 Related Literature and Methodological Formulations

Context and uncertainty formulations. Hydra-MDP [203] pairs TransFuser [101] with rule-based PDM [111] distillation; unlike model ensembles [242, 243], Centaur updates decoder parameters online via TTT. Semantic Entropy follows paraphrase clustering in language models [240] by grouping trajectories in feature space prior to computing entropy. For scenario evaluation, whereas open-loop ADE/FDE [3, 22, 219] omits rule compliance, NAVSIM [23] scores non-reactive

Table 4.5 **Summary of mathematical symbols and definitions in Chapter 4.**

Notation	Description
$\mathcal{L}_{im}$	Imitation objective
$\mathcal{L}_{kd}$	Distillation objective
$\mathbf{t}$	Ego trajectory
$\mathbf{p}$	BEV waypoint coordinate
π	Planning policy network
$\mathbf{x}$	Multi-sensor observation
$\mathbf{c}$	High-level route command
θ	Trainable decoder weights
H	Directional Cluster Entropy
M	Sampled candidate count
$score_a$	Aggregated cluster score
k	Vocabulary trajectory count
$\gamma, \alpha, \beta, \upsilon$	NIG distribution parameters
F	Gradient buffer horizon
η	Test-time step size
τ	Feature-distance cutoff

replay, Bench2Drive [187] evaluates 44 closed-loop scenario families, InterPlan [244] synthesises rare hazards, and `navsafe` isolates 229 low-scoring `navtest` frames (Section 4.4). For continuous regression policies, Deep Evidential Regression [245] predicts target coordinates alongside conjugate prior parameters in a single forward pass. Table 4.5 summarises the notation used across Chapter 4.

Evidential regression planner. Deep Evidential Regression [245] predicts Normal Inverse Gamma (NIG) parameters (α, β, γ, υ) for each scalar target y_i of π_θ:

$$\begin{aligned} (y_1, \ldots, y_N) &\sim \mathcal{N}(\mu, \sigma^2), \\ \mu \sim \mathcal{N}(\gamma, \sigma^2 \upsilon^{-1}), \qquad & \sigma^2 \sim \text{InvGamma}(\alpha, \beta), \end{aligned} \tag{4.7}$$

with Gaussian $\mathcal{N}(\cdot)$, inverse-gamma distribution InvGamma(α, β), predictive mean $(\gamma_0, \gamma_1, \ldots, \gamma_N) = \pi_\theta(\mathbf{x}_0) = \hat{\mathbf{t}}_0$, and imitation loss $\mathcal{L}_{im}$ defined as the Negative Log-

Likelihood of the marginal Student-t distribution plus the evidence regulariser of Amini *et al.* [245] over $(\mathbf{x}, \mathbf{t}, \mathbf{c}) \in D_{im}$:

$$\underset{\theta}{\operatorname{argmin}}\, \mathbb{E}_{(\mathbf{x},\mathbf{t},\mathbf{c})\sim D_{im}} \left[\mathcal{L}_{im}\Big(\mathbf{t}, \pi_\theta(\mathbf{x}, \mathbf{c})\Big) \right]. \tag{4.8}$$

For an N-step vector trajectory $\vec{\mathbf{t}} = (\mathbf{p}_0, \mathbf{p}_1, \ldots, \mathbf{p}_N)$ parameterised by $(\vec{\alpha}, \vec{\beta}, \vec{\gamma}, \vec{\upsilon})$:

$$\begin{aligned} (\vec{\mathbf{t}}_1, \ldots, \vec{\mathbf{t}}_N) &\sim \mathcal{N}(\vec{\mu}, \vec{\sigma^2}), \\ \vec{\mu} \sim \mathcal{N}(\vec{\gamma}, \sigma^2 \vec{\upsilon}^{-1}), &\qquad \vec{\sigma^2} \sim \text{InvGamma}(\vec{\alpha}, \vec{\beta}), \end{aligned} \tag{4.9}$$

with mean plan $\vec{\gamma} = \pi_\theta(\mathbf{x}_0) = \hat{\mathbf{t}}_0$ and vectorised imitation objective over $(\mathbf{x}, \vec{\mathbf{t}}, \mathbf{c}) \in D_{im}$:

$$\underset{\theta}{\operatorname{argmin}}\, \mathbb{E}_{(\mathbf{x},\vec{\mathbf{t}},\mathbf{c})\sim D_{im}} \left[\mathcal{L}_{im}\Big(\vec{\mathbf{t}}, \pi_\theta(\mathbf{x}, \mathbf{c})\Big) \right]. \tag{4.10}$$

Scoring objective and Semantic Entropy. Following VADv2 [218], Hydra-MDP [203] combines the multi-target distillation loss $\mathcal{L}_{kd} = \sum_{j=1}^{k} \mathcal{L}_{kd}\Big(e(\mathbf{t}_j, \mathbf{x}, \mathbf{c}), \pi_\theta(\mathbf{t}_j, \mathbf{x}, \mathbf{c})\Big)$ (Eq. (4.3)) and human imitation regularisation $\mathcal{L}_{im}$:

$$\mathcal{L} = \mathcal{L}_{im} + \mathcal{L}_{kd}, \tag{4.11}$$

where $\mathcal{L}_{im}$ is the vocabulary-level cross-entropy [203] over the k candidates $\{\mathbf{t}_j\}_{j=1}^{k}$ with target distribution given by softmax-normalised negative L_2 distances to the logged trajectory $\tilde{\mathbf{t}}$. For entropy estimation, $M = 100$ fixed candidates are sampled offline from the 8192 vocabulary weighted by mean `navtrain` PDMS (Algorithm 1) around five lateral DriveLM [4] anchors (Sections 4.3.2 and 4.7). Semantic Entropy assigns each of the M candidates to the closest of the five anchors by feature-space L_2 distance $D(\cdot, \cdot)$ when $D \leq \tau$ (falling back to trajectory-space L_2 distance if $D > \tau$) before computing Shannon entropy H: feature vectors are predicted sub-scores for scoring policies, or 2D pairs $\{L_\gamma, L_\mu\}$ (L_2 distances to γ and the closest of N samples $\mu \sim \mathcal{N}(\gamma, \sigma^2 \upsilon^{-1})$ with $\sigma^2 \sim \text{InvGamma}(\alpha, \beta)$) for evidential regression policies.

Table 4.6 **Single-seed NAVSIM v1.0 results** (2024 Challenge protocol; cf. Table 4.3).

Model	Deployment	NC ↑	DAC ↑	EP ↑	C ↑	TTC ↑	PDMS ↑
UniAD [3]	None	97.8	91.9	78.8	100.0	92.9	83.4
	Optimisation	95.4	86.0	71.9	99.3	87.3	75.2
PARA-Drive [205]	None	97.9	92.4	79.3	99.8	93.0	84.0
+ Occupancy Head	None	97.8	92.0	78.3	99.7	93.7	83.6
TransFuser [101]	None	97.1	91.9	79.3	100.0	91.9	83.4
TransFuser-SE	TTT	97.9	92.9	80.1	100.0	94.1	85.7
DiffusionDrive [224]	None	98.2	96.2	82.2	100.0	94.7	88.1
Hydra-MDP [203]	None	98.4	97.8	86.5	100.0	93.9	90.3
Hydra-SE	TTT	99.2	98.5	85.7	100.0	97.1	91.8
Centaur	TTT	99.5	98.9	85.9	100.0	98.0	92.6

4.8.2 Hyper-Parameters, Baselines, and Extended Ablations

Implementation and baseline design. Hydra-MDP evaluates 8192 candidate trajectories [203], whereas TransFuser regresses 40 waypoints at 10 Hz (4 s horizon) with an evidential head [245] trained on one A100 GPU (batch size 64, learning rate 10^{-4}), using $\tau = 0.06$ for Hydra-SE and $\tau = 0.02$ for TransFuser-SE tuned on `navtest` (Section 4.5.1). Three uncertainty and deployment baselines complement Cluster Entropy: (i) **KL Divergence**, motivated by inter-head sub-score divergence (particularly between NC and DAC) on failed versus successful scenes (Fig. 4.5); (ii) **Evidential uncertainty**, providing single-pass epistemic and aleatoric estimates for continuous regression in TransFuser; and (iii) **Prior fallback**, which flags frames exceeding the Table 4.2 cutoff and selects the nearest (L_2) trajectory among 20 `navtrain` priors without explicit perceptual maps (audited in Section 4.8).

Supplementary ablations and latency. On single-seed NAVSIM v1.0 (Table 4.6), Centaur achieves the highest PDMS (92.6) and TransFuser-SE gains +2.3 PDMS over base TransFuser, whereas UniAD [3] cost optimisation reduces all five sub-scores and PARA-Drive [205] with an occupancy head shifts from 84.0 to 83.6. Measured on a single A100 GPU (driver 535.54.03), 16-core AMD EPYC 9124 CPU, and 500 GB

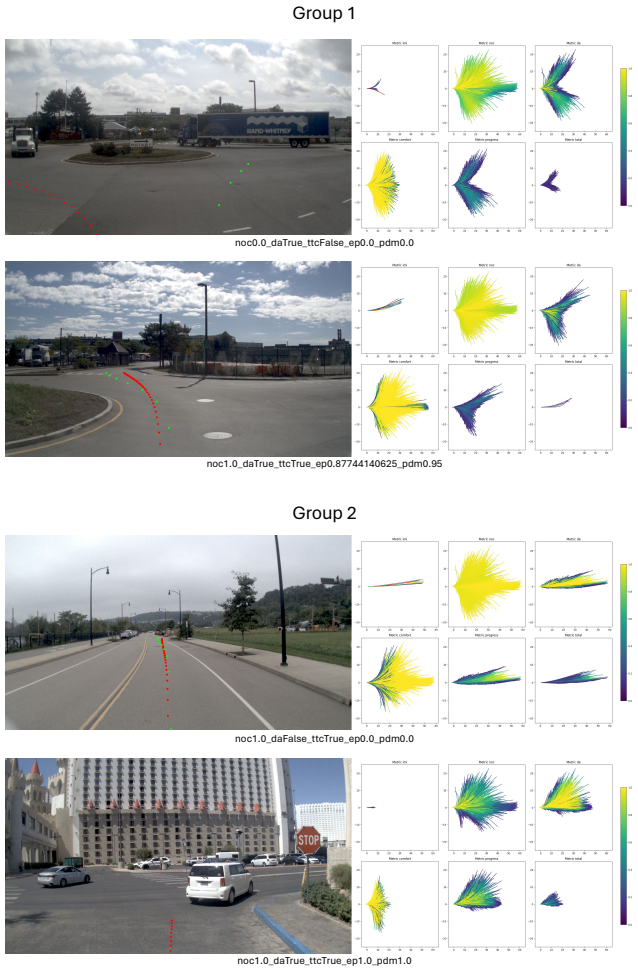


Fig. 4.5 **Score distributions on failed (top) vs. successful (bottom) pairs.**

RAM (Table 4.7), ResNet-34 TransFuser runs in 103.1 ms versus 243.9 ms for V2-99 Hydra-MDP ($\approx$ 7 PDMS higher); SVD gradient filtering (TTT*) [213] underperforms uniform gradient averaging on TransFuser-SE (83.8 *vs.* 85.7), V2-99 outperforms ViT-L except under KL fallback, varying buffer horizon $F \in \{1, 2, 3, 4\}$ spans 0.5 PDMS, and every threshold in Table 4.8 remains below the Select None accuracy baseline.

4.8.3 Arithmetic Consistency Audits

We audit internal arithmetic across Tables 4.1, 4.2, 4.4, 4.6, 4.7, and 4.8.

Table 4.7 **NAVSIM v1.0 ablations and runtime.** Fallback is audited in Section 4.8; TTT* uses SVD; Hydra-SE$^{\mathbf{f}}$ buffers f steps; Hydra-SE$^{\mathbf{4}}$ gates TTT by uncertainty.

Model	Deployment	Uncertainty	NC ↑	DAC ↑	EP ↑	C ↑	TTC ↑	PDMS ↑	Latency (ms) ↓
TransFuser [101]	-	-	97.1	91.9	79.3	100.0	91.9	83.4	103.1
	Fallback	Evidential	98.8	97.4	13.1	100.0	97.9	60.6	108.7
	Fallback	Semantic Entropy	99.2	97.2	14.6	99.8	96.5	59.9	109.9
	TTT	Evidential	97.9	92.5	79.6	100.0	92.9	84.2	149.3
TransFuser-SE	TTT	Semantic Entropy	97.9	92.9	80.1	100.0	94.1	85.7	153.8
	TTT*		97.4	89.3	78.3	100.0	91.1	83.8	157.6
Hydra-MDP [203]	-	-	98.4	97.8	86.5	100.0	93.9	90.3	243.9
	Fallback	KL Divergence	99.1	99.2	9.6	100.0	96.7	63.1	263.2
	Fallback	Semantic Entropy	99.7	99.5	10.4	100.0	98.9	65.3	256.4
	TTT	KL Divergence	98.9	98.1	86.0	100.0	94.4	91.5	357.1
Hydra-MDP-ViT-L	-	-	98.4	97.7	85.0	100.0	94.5	89.9	237.2
	Fallback	KL Divergence	99.7	99.2	10.4	100.0	98.9	67.4	259.5
	Fallback	Semantic Entropy	99.6	97.5	11.8	100.0	94.1	61.0	255.7
	TTT	KL Divergence	99.1	98.2	89.6	100.0	96.7	90.9	354.1
Hydra-SE$^{\mathbf{4}}$	TTT	Semantic Entropy	99.2	98.5	85.7	100.0	97.1	91.8	312.5
Hydra-SE$^{\mathbf{3}}$			98.9	98.7	86.7	100.0	95.1	91.5	
Hydra-SE$^{\mathbf{2}}$			98.8	98.6	86.5	100.0	95.0	91.3	
Hydra-SE$^{\mathbf{1}}$			98.9	98.6	86.7	100.0	95.1	91.4	
Hydra-SE$^{\mathbf{4}}$	TTT	Semantic Entropy	98.6	98.1	86.3	100.0	94.9	91.3	279.6

Failure base rate and fallback trigger proportions. Because PDMS $\in [0, 100]$, a 15.6% fraction of strict PDMS = 0 frames (Table 4.2) would bound mean Hydra-MDP PDMS at $84.4 < 90.3$ (Table 4.1), while Eq. (4.6) with 98.4 NC and 97.8 DAC bounds zero-PDMS frames at $1.6\% + 2.2\% = 3.8\%$ (for TransFuser, 24.1% bounds PDMS at $75.9 < 83.4$, and 97.1 NC with 91.9 DAC allows at most $2.9\% + 8.1\% = 11.0\%$). Furthermore, given base rate p, TPR, and accuracy a, the flagged fraction $1-p-a+2p\,\mathrm{TPR}$ and false-positive rate $(1-p-a+p\,\mathrm{TPR})/(1-p)$ imply 29.0%–33.3% triggered frames in Table 4.9 (Sections 4.5.1 and 4.8.2), which bounds Hydra-MDP EP at ≥ 53.2 against the reported 8.2–13.2 (73.3%–78.3% trigger rates required; reported PDMS $63.1, 64.2, 65.3, 65.9$ also increases with flagging rate). Similarly, TransFuser-SE at threshold 0.8 flags 29.8% of frames, bounding EP at 49.5 versus the reported 14.6.

`navsafe` summary aggregations. Across the 229 `navsafe` frames (Table 4.4), the preprint Overall column lies within 1.5 points of the category mean on six rows, but exceeds the unweighted and frame-weighted means by 8.14 and 7.11 points for Hydra-MDP and by 2.18 and 5.28 points for Centaur (Table 4.10), increasing Centaur's

Table 4.8 **Failure classification across models and cut-offs** (cf. Table 4.2; audited in Section 4.8).

Uncertainty	**Model**	**Thresh.**	**TPR ↑**	**Acc. ↑**
Evidential	TransFuser [101]	0.1	47.2	34.9
		0.3	32.6	48.2
		0.5	26.9	59.5
		0.7	23.5	62.8
		0.9	7.4	71.2
KL Divergence	Hydra-MDP [203]	1500	41.3	68.3
Semantic Entropy	TransFuser [101]	0.8	35.3	63.1
	Hydra-MDP [203]	0.2	75.2	61.9
		0.5	73.3	62.4
		0.8	62.5	72.2
		1.1	41.7	73.2
		1.4	10.9	82.5
Select All	*TransFuser* [101]	-	*100.0*	*24.1*
Select None		-	*0.0*	*75.9*
Select All	*Hydra-MDP* [203]	-	*100.0*	*15.6*
Select None		-	*0.0*	*84.4*

margin over Hydra-MDP from 17.67 to 23.63 (72.0 *vs.* 48.3 simple mean) or 19.50 (68.9 *vs.* 49.4 frame-weighted mean; Section 4.4).

Table 4.9 **Implied fallback trigger proportions at the cut-offs of Table 4.2**, derived from Tables 4.1 and 4.2 at $p = 15.6\%$ (in %).

Uncertainty	TPR	Acc.	Flagged	FPR	EP bound	Reported EP	Needed
KL Divergence	41.3	68.3	29.0	26.7	57.5	9.6	76.9
Full Entropy	61.7	70.4	33.3	28.0	53.2	13.2	73.3
Semantic Entropy	62.5	72.2	31.7	26.0	54.8	10.4	76.1
Cluster Entropy	62.8	73.6	30.4	24.4	56.1	8.2	78.3

Table 4.10 **Comparison of `navsafe` summary aggregations** from Table 4.4 (frame-weighted accurate to ±0.05).

Planner	Reported Overall	Mean	Frame-weighted
Constant Velocity [23]	3.41	3.42	3.52
Ego Status MLP [23]	24.10	25.15	27.39
LTF [101]	53.74	53.97	58.46
TransFuser [101]	51.87	52.74	53.52
Hydra-MDP [203]	56.47	48.33	49.36
Hydra-SE	62.84	64.26	62.65
Centaur	74.14	71.96	68.86
Human Trajectory	*94.65*	*94.65*	*94.83*

4.9 Chapter Summary

Addressing RQ3a on per-frame logged states with offline rule-distilled heads, this chapter introduced Centaur, which resets decoder weights each step and applies one buffered ($F = 4$) gradient step on Cluster Entropy across five lateral directions, alongside the 229-frame NHTSA-motivated `navsafe` subset of `navtest`.

Under NAVSIM v1 non-reactive replay, Centaur lifts single-seed Hydra-MDP from 90.3 to 92.6 PDMS (TTC 93.9 to 98.0, EP 86.5 to 85.9) while prior fallback drops PDMS to 63.1–65.9 on Hydra-MDP (59.9–67.4 across all tested planners, EP < 15), and places first on the three-seed v1.1 leaderboard at 92.10 ± 0.33 PDMS (0.23 ahead of Hydra-SE, with +1.2 to +2.3 PDMS across the four Hydra-MDP objectives, +0.8 to +2.3 across base planners, or +0.4 with SVD gradient projection, and +2.3 on TransFuser-SE, whereas UniAD cost refinement drops PDMS from 83.4 to 75.2). At

a fixed failure cutoff, Cluster Entropy reaches 62.8% TPR and 73.6% accuracy (vs. 84.4% Select None at 15.6% base rate). On `navsafe`, Centaur leads on frame-weighted (68.9 *vs.* 49.4), simple (72.0 *vs.* 48.3), and preprint Overall (74.14 *vs.* 56.47) means (Table 4.10), but drops from 71.6 to 59.3 on yielding and trails human logs (94.65). Open-loop resets, validation-free tuning, metric-distilled calibration, and lateral-only clustering remain key limitations. Chapter 5 next examines robotic manipulation under noisy demonstrations and hardware shift.

Chapter 5

Act II: Resource-Aware Robust Manipulation

Statement of contribution

Adapted from our alphabetically ordered seventeen-author preprint [18], this chapter reflects my work as co-project lead alongside Li Chen and as one of seven core contributors. I formulated the distributional framework across P_{train}, Q_{model}, and P_{test}, co-developed Model Arithmetic with Modi Shi, and co-developed Train–Deploy Alignment (covering heuristic DAgger recovery, spatiotemporal augmentation, and chunk-wise smoothing) with Checheng Yu, Kaiyang Wu, and Yibo Yuan. I also co-designed the physical benchmarks, assisted with data capture and robot maintenance, and led manuscript writing and figure design. Co-author Lirui Zhao developed Stage Advantage, which is included here to keep the system presentation self-contained; Hongyang Li and Ping Luo provided compute resources and academic guidance.

Chapter 4 showed in non-reactive driving simulation that an end-to-end planner can adjust frame-level predictions at test time by lowering predictive uncertainty without ground-truth supervision [17]. Extending the Act pillar to physical robotics, RQ3(b) asks how a learned controller can stay dependable once test-time conditions diverge from offline training in *real-world robot manipulation*. Deformable cloth manipulation entails contact-rich dynamics across a high-dimensional configuration space from arbitrary crumpled initial states, whereas human teleoperation samples only a narrow fraction of feasible trajectories. Practical resource limits add further con-

straints: recording demonstrations consumes substantial operator and robot time, post-training a large vision-language-action (VLA) model requires considerable GPU compute, and closed-loop control delays plus mechanical limits cause physically executed motions to drift from offline predictions.

Resource limits make data and model scale only part of the manipulation problem. We analyse reliability through three linked objects across the learning loop: the recorded human demonstrations P_{train}, the policy-induced trajectory distribution Q_{model}, and the trajectories produced on hardware P_{test}. In the terminology of Chapter 1 (Section 1.1), these correspond to training data p_{train}, policy-induced behaviour, and the physically executed component of deployment p_{deploy}. They are conceptual distributions rather than fitted density models (Section 5.3.2), so the experiments test interventions associated with each mismatch rather than measured reductions in distributional divergence. The χ_0[1] framework links this analysis to three modules:

(a) *Model Arithmetic* (MA) fits separate checkpoints on disjoint subsets of P_{train} and interpolates their weights using convex coefficients tuned on out-of-distribution (OOD) DAgger recovery rollouts, reconciling Q_{model} with P_{train} without enlarging the primary training corpus (though gathering the validation rollouts still requires DAgger interventions).

(b) *Stage Advantage* (SA), designed by co-author Lirui Zhao, regresses relative task progress directly from observation pairs conditioned on discrete semantic stages and converts the prediction into binary weights for advantage-weighted imitation, guiding Q_{model} toward productive actions with lower temporal variance than subtracting independent state-value estimates.

(c) *Train–Deploy Alignment* (TDA) shifts P_{train} toward P_{test} via heuristic DAgger recovery collection and spatiotemporal augmentation while mitigating inference-to-actuation latency at run time through temporal chunk-wise smoothing.

[1]Project page: `https://mmlab.hk/research/kai0`. Code: `https://github.com/OpenDriveLab/KAI0`.

We evaluate χ_0 on two bimanual platforms across garment flattening, folding, conditional sorting, and rack hanging. The full system is compared with the base policy on Task A (T-shirt flattening and folding), while Tasks B and C provide module-level tests. In 30 Task A rollouts (with values read from the plotted evaluation summaries; Section 5.5.4), completion rises from approximately 8/30 (27%; 95% Wilson interval 14–44%) for the $\pi_{0.5}$ baseline to 29/30 (97%; 83–99%) for χ_0 (Table 5.2). Post-training uses a pre-trained $\pi_{0.5}$ backbone, about 20 hours of base demonstrations per task, and eight A100 GPUs; total compute, robot time, and operator time across all sub-experiments were not separately recorded. Because targeted DAgger recovery data supply the largest individual gain (Section 5.7.3) and no matched-budget data-scaling baseline was run, the experiment supports alignment as a useful complement to scale, not as evidence that alignment is the dominant bottleneck (Section 5.8).

The rest of this chapter covers motivation (Section 5.1), prior literature (Section 5.2), problem formulation (Section 5.3), the three alignment modules (Section 5.4), hardware and benchmark protocols (Section 5.5), system-level and component-level experiments (Sections 5.6 and 5.7), limitations (Section 5.8), implementation details and supplementary ablations (Section 5.9), and conclusions (Section 5.10).

5.1 Motivation

Learned driving systems now operate on public roads in both rider-only automated services and supervised driver-assistance deployments [246–248], whereas physical manipulation in open-ended environments remains fragile because controllers must act over high-dimensional joint spaces under wide variations in cloth geometry and lighting. Recent robotic foundation efforts have scaled VLA backbones on large compute clusters [6, 249–254] and assembled large teleoperation datasets such as AgiBot World [5] (co-authored by the author, comprising over one million trajectories across more than 100 robots). While network capacity and dataset scale are necessary, they

do not by themselves ensure reliable closed-loop execution: a key barrier is *the distributional mismatch across data collection (P_{train}), policy-induced trajectory distribution (Q_{model}; Eq. (5.3)), and physical deployment (P_{test})*, which affects task completion, motion continuity, hourly throughput, and corrective retry counts [5, 252, 255–257].

In physical rollouts, observed failures fall into three systematic mismatches across these distributions, as shown in Fig. 5.1. First, because deformable cloth spans a high-dimensional state space, P_{train} covers only a sparse set of valid trajectories, biasing Q_{model} toward narrow demonstration habits. Second, computation and communication delays between policy inference (Q_{model}) and physical actuation (P_{test}) introduce temporal offsets that disrupt action sequences at chunk boundaries [258, 259]. Third, because expert demonstrations rarely visit error states, minor disturbances during execution in P_{test} trigger compounding deviations from which the policy cannot recover [252, 260, 261]. Standard remedies such as broader data collection [5, 262, 263], synthetic augmentation [264–266], or deployment-time adaptation [210, 212, 230] are difficult to apply directly when teleoperation is expensive, control loops are latency-critical, and VLA post-training is compute-intensive.

To counter these three discrepancies under practical robotics budgets, χ_0 pairs each distributional mismatch with a targeted module (bottom panel of Fig. 5.1): (a) **Model Arithmetic (MA)** interpolates parameters across checkpoints trained on disjoint splits of P_{train} to combine behavioural modes that a single training run can miss; (b) **Stage Advantage (SA)**, contributed by co-author Lirui Zhao, partitions a multi-step task into semantic stages and supplies stage-conditioned advantage weights for behaviour cloning [267], yielding a less noisy signal than a $\pi^*_{0.6}$-style value-differencing baseline [252] (see Section 5.7.2); and (c) **Train–Deploy Alignment (TDA)** expands P_{train} toward P_{test} via heuristic DAgger recovery trajectories and spatiotemporal augmentation while applying temporal chunk-wise smoothing at run time to counter inference delay, outperforming RTC-only control [258] in throughput under two of three augmentation regimes on Task A (see Section 5.7.3). Across 30 evaluation rollouts per setting taken from the plotted summaries (where differences

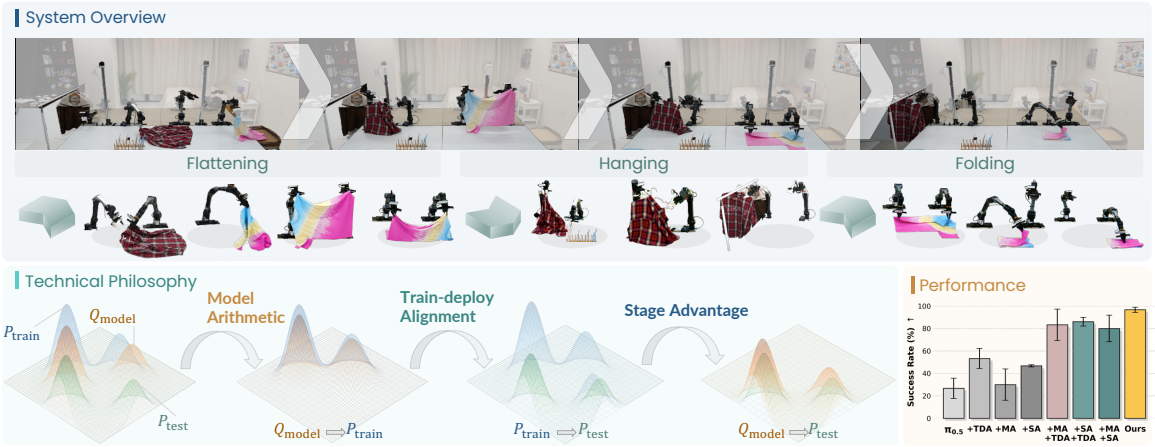


Fig. 5.1 **Top: platform overview.** Two ALOHA-style bimanual stations collaborate on multi-stage garment flattening, folding, and rack hanging. **Bottom: distributional alignment and Task A summary.** Across the learning loop (P_{train}, Q_{model}, P_{test}), MA reconciles Q_{model} with P_{train}, TDA bridges P_{train} and P_{test}, and SA steers Q_{model} toward P_{test}. On Task A (Fig. 5.7), χ_0 lifts $\pi_{0.5}$ [6] completion from roughly 27% (8/30) to roughly 97% (29/30) over 30 rollouts (Table 5.2).

of up to four successes fall within binomial sampling variation; Section 5.5.4), our primary findings are:

1. **Integrated system.** On Task A, combining all three modules lifts $\pi_{0.5}$ completion from roughly 27% to roughly 97%; without an equal-volume unaugmented baseline, this gain does not separate distributional alignment from total data volume.

2. **Train–Deploy Alignment.** DAgger recovery data provide the largest standalone improvement, raising $\pi_{0.5}$ completion from roughly 23% to 87–93% on Task A and from roughly 10% to 43–53% on Task C. Heuristic DAgger raises retry counts across all tested settings, whereas standard DAgger leaves retries nearly unchanged except on π_0 in Task A. Spatiotemporal augmentation alone shows mixed effects on Task A; temporal chunk-wise smoothing beats RTC in completion rate under two of three augmentation conditions, and combining both yields the strongest unaugmented controller.

3. **Model Arithmetic.** On Task C, every weight-merged policy exceeds both the single-best subset checkpoint and the full-data checkpoint in mean completion rate and throughput, though only the OOD-validated greedy merge clears sampling noise over the full-data baseline. Across Tasks A and B, merging surpasses unmerged baselines on most metrics. On Task A, MA alone leaves $\pi_{0.5}$ completion nearly unchanged (roughly 30% vs. 27%) and lowers the rubric score (35 vs. 41), whereas pairing MA with TDA or SA adds roughly 30 percentage points in completion rate.

4. **Stage Advantage.** Across all three tasks, Stage Advantage produces smoother temporal advantage traces than value differencing (without ground-truth verification). In closed-loop rollouts, completion rises on Task A (roughly 80% vs. 67%, within sampling noise); on Task B, Stage Advantage reduces retries and raises the rubric score but trails in completion rate (roughly 17% vs. 23%) and throughput; on Task C, completion rate, throughput, and score all decrease.

5.2 Related Work

5.2.1 Imitation Learning and Real-World Policy Deployment

Imitation learning for robot manipulation spans compact transformer policies [268–271] and pre-trained VLA backbones [6, 108, 252, 272–275], with the π series [6, 249, 252] demonstrating strong multi-robot transfer. Because collecting large demonstration corpora [5, 262, 263, 276, 277] requires major hardware and operator investment, prior work explores interactive DAgger collection [8, 252, 260, 278], visual and kinematic augmentation [266, 279–281], and run-time trajectory blending or inpainting to counter inference delay [258, 259, 268]. Whereas existing studies typically address data collection, policy training, or run-time control separately [282, 283], χ_0 jointly targets all three pairwise gaps among $(P_{\text{train}}, Q_{\text{model}}, P_{\text{test}})$. In complementary co-authored work, PerturBot [37] diagnoses how fine-tuned VLA policies rely on visual,

language, and motor shortcut priors via an offline `GroundFscore` diagnostic. PerturBot then breaks these shortcuts through perturbative training, complementing χ_0's system-level distributional alignment across $(P_{\text{train}}, Q_{\text{model}}, P_{\text{test}})$.

5.2.2 Model Merging and Weight Interpolation

Interpolating parameters across checkpoints trained with varied hyper-parameters or fine-tuning splits improves out-of-distribution generalisation across vision, language, motion planning, and embodied control [284–290]. While RETAIN [291] merges VLA checkpoints to preserve general pre-trained capabilities and standard recipes tune mixing weights on in-domain splits, our Model Arithmetic module targets narrow demonstration coverage by selecting convex combinations (across uniform averaging, inverse-loss weighting, gradient optimisation, and greedy soup construction) using an OOD validation set of DAgger recovery rollouts.

5.2.3 Advantage Estimation for Long-Horizon Tasks

Conditioning imitation learning on returns, values, or advantages helps guide multi-step decision-making [292–296], with advantage-weighted regression (AWR) upweighting high-advantage transitions [267] and $\pi^*_{0.6}$ computing VLA advantages from distributional value differences $V(s') - V(s)$ [252]. Because subtracting two independent frame-level value predictions compounds estimation noise, Stage Advantage (developed by co-author Lirui Zhao) regresses relative progress directly from paired frames conditioned on semantic task stages [18], producing a smoother signal (see Section 5.7.2) that is thresholded into a binary optimality indicator [252].

5.3 Distributional Inconsistencies in Robot Learning

5.3.1 Preliminaries

We model manipulation over horizon H with state space $\mathcal{S}$, action space $\mathcal{A}$, initial state distribution $\mu(s_0)$, and transition kernel $s_{t+1} \sim T(\cdot \mid s_t, a_t, \xi)$ under environment parameters ξ. Under policy $\pi(a \mid s; \phi)$, a trajectory $\tau = (s_0, a_0, s_1, a_1, \ldots, s_H)$ has density [297, 298]:

$$P_\pi(\tau) = \mu(s_0) \prod_{t=0}^{H-1} \pi(a_t \mid s_t; \phi)\, T(s_{t+1} \mid s_t, a_t, \xi), \tag{5.1}$$

written as P whenever π is clear from context.

5.3.2 Three Distributions of the Robot Learning Cycle

Denoting successful task trajectories by $\mathcal{G}$ and a base kinematic trajectory measure by $P_0(\tau)$, the target trajectory distribution is

$$P_{\text{real}}(\tau) \propto P_0(\tau)\, \mathbb{I}(\tau \in \mathcal{G}). \tag{5.2}$$

Relative to P_{real}, we distinguish three distributions in the learning loop:

- **Training distribution (P_{train})**: the empirical trajectory distribution of human teleoperation demonstrations, yielding dataset $\mathcal{D} = \{\tau^{(i)}\}_{i=1}^{N}$.
- **Model distribution (Q_{model})**: the policy-induced trajectory distribution $Q_{\text{model}}(\tau) = P_{\pi(\cdot\mid\cdot;\hat{\phi})}(\tau)$ induced via Eq. (5.1) by rolling out the learned policy $\pi(a \mid s; \hat{\phi})$ fitted by maximum likelihood on $\mathcal{D}$ [8, 299]:

$$\hat{\phi} = \operatorname*{argmax}_{\phi} \sum_{\tau \in \mathcal{D}} \sum_{t} \log \pi(a_t \mid s_t; \phi), \tag{5.3}$$

so that Q_{model} lives on the same trajectory space as P_{train} and P_{test}, with extrapolation outside $\mathcal{D}$ reflecting architectural priors and pre-trained weights (for flow-matching policies such as $\pi_{0.5}$ [6, 249], maximum-likelihood training in Eq. (5.3) is instantiated by minimising the conditional flow-matching velocity-field regression loss over action chunks).

- **Test distribution (P_{test})**: the closed-loop rollout distribution when Q_{model} acts through a hardware execution operator $I(\tilde{a}_t \mid a_t, s_t)$ mapping predicted actions a_t to applied commands $\tilde{a}_t$ under latency and actuator bounds:

$$P_{\text{test}}(\tau) = \mu(s_0) \prod_{t=0}^{H-1} \pi(a_t \mid s_t; \hat{\phi})\, \tilde{T}(s_{t+1} \mid s_t, a_t, \xi), \tag{5.4}$$

$$\tilde{T}(s_{t+1} \mid s_t, a_t, \xi) \triangleq \mathbb{E}_{\tilde{a}_t \sim I(\cdot \mid a_t, s_t)} \Big[T(s_{t+1} \mid s_t, \tilde{a}_t, \xi) \Big]. \tag{5.5}$$

Because we do not fit explicit density estimators or compute divergences among P_{train}, Q_{model}, and P_{test}, these definitions serve as a conceptual lens for each module's objective rather than measured divergence reductions, and the t-SNE projections (Fig. 5.5) are qualitative.

5.3.3 Three Systematic Inconsistencies

Observed rollout failures correspond to three recurring mismatches among these distributions: (i) **coverage deficiency**, where sparse sampling of P_{real} by P_{train} causes Q_{model} to overfit narrow demonstration modes; (ii) **temporal mismatch**, where visually similar cloth states at different task stages confuse Q_{model} while inference latency offsets applied commands in P_{test}; and (iii) **failure cascade**, where the lack of corrective manoeuvres in P_{train} leaves the policy unable to recover once disturbances push trajectories off course in P_{test}.

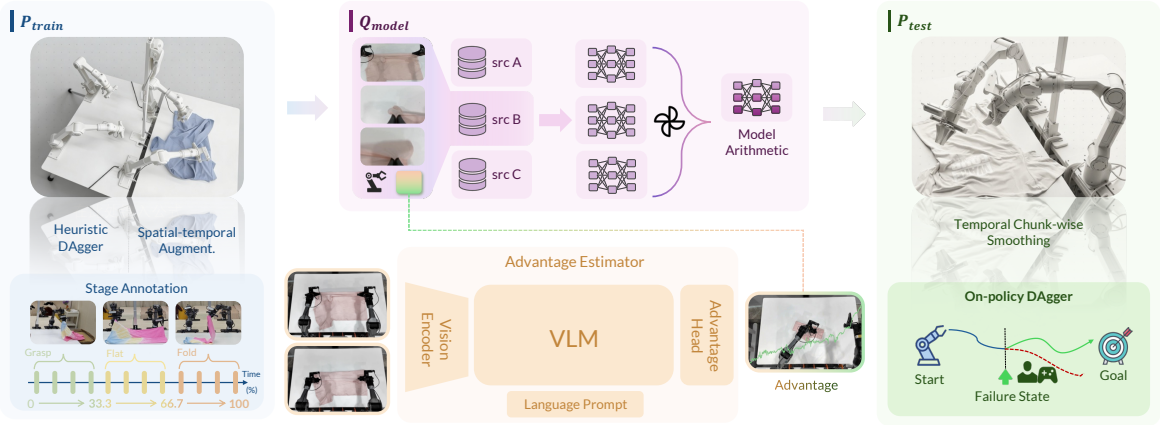


Fig. 5.2 **Architecture of χ_0. Left** (P_{train}): heuristic DAgger recovery collection, spatiotemporal augmentation, and stage annotations. **Middle** (Q_{model}): Model Arithmetic weight interpolation and Stage Advantage conditioning. **Right** (P_{test}): temporal chunk-wise smoothing and on-policy DAgger feedback.

5.4 Method

5.4.1 Overview of the Framework

To counter the three mismatches of Section 5.3.3 within practical data and compute budgets, χ_0 coordinates three modules across the learning pipeline (Fig. 5.2). Model Arithmetic (Section 5.4.2) expands policy coverage in weight space by interpolating checkpoints trained on disjoint data partitions. Stage Advantage (Section 5.4.3) resolves temporal ambiguity during imitation learning via stage-conditioned advantage estimation, providing lower-variance supervision across multi-stage tasks. Train–Deploy Alignment (Section 5.4.4) connects physical execution back to training by combining run-time chunk smoothing with targeted recovery data and spatiotemporal augmentation.

5.4.2 Model Arithmetic

Under modest teleoperation budgets, P_{train} samples the state space sparsely, biasing a single fine-tuned checkpoint toward narrow manipulation routines. How can we reduce this coverage bias in Q_{model} without collecting additional raw demonstrations? Unlike

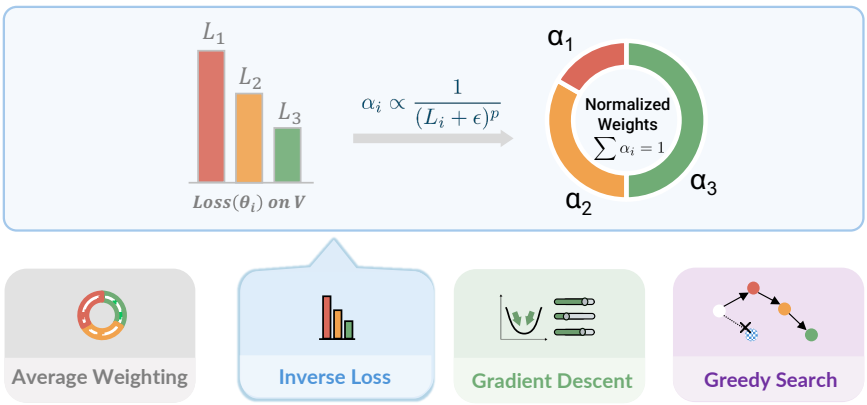


Fig. 5.3 **Weight-merging schemes in Model Arithmetic.** Checkpoints trained on disjoint subsets are merged via convex weight interpolation using inverse-loss weighting (**top**) or uniform averaging, gradient optimisation, and greedy selection (**bottom**).

mixture-of-experts architectures that require learned routers [300, 301] or output ensembles that multiply forward-pass latency [242], Model Arithmetic (MA) combines multiple checkpoints into a single weight vector at zero run-time overhead. Specifically, we partition $\mathcal{D} \sim P_{\text{train}}$ randomly into n disjoint subsets $\{\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_n\}$, fine-tune an independent checkpoint θ_i on each subset, and form the deployed parameters via convex combination:

$$\theta_{\text{merged}} = \sum_{i=1}^{n} \alpha_i \theta_i, \qquad \text{s.t.} \quad \alpha_i \geq 0, \quad \sum_{i=1}^{n} \alpha_i = 1, \tag{5.6}$$

where the coefficients $\{\alpha_i\}$ are selected to reduce validation loss on a held-out set, consolidating complementary behavioural modes across partitions.

Out-of-distribution validation. Rather than tuning $\{\alpha_i\}$ only on in-domain demonstrations, we construct an OOD validation split from DAgger recovery rollouts [8, 278] gathered by running the subset policies on the physical robot with human corrective interventions. Although MA does not enlarge the primary training splits, collecting this OOD validation set still entails physical rollouts and operator time (Section 5.8).

Merging strategies. Using this validation set, we evaluate four weight-souping rules [284] (Fig. 5.3; Sections 5.7.1 and 5.9):

- *Average weighting* [284] assigns uniform coefficients $\alpha_i = 1/n$.
- *Inverse-loss weighting* [302] weights each checkpoint i inversely to its validation loss L_i with scalar hyper-parameters δ and p:

$$\alpha_i \propto \frac{1}{(L_i + \delta)^p}. \tag{5.7}$$

- *Gradient descent* [18] (plus an adaptive-step variant) parameterises the simplex weights via a softmax map and directly minimises the validation loss $\mathcal{L}_{\text{val}}$ of θ_{merged}:

$$\boldsymbol{\alpha} = \text{softmax}(\mathbf{w}), \qquad \min_{\mathbf{w}} \ \mathcal{L}_{\text{val}}\Big(\sum_i \alpha_i \theta_i\Big). \tag{5.8}$$

- *Greedy search* [284] incrementally adds the candidate checkpoint that achieves the largest validation-loss drop when uniformly averaged with the active set.

5.4.3 Stage Advantage

Developed by co-author Lirui Zhao (see the contribution statement), Stage Advantage (SA) addresses temporal ambiguity in P_{test} when visually similar cloth states recur at different phases of a multi-stage sequence, providing a low-variance, stage-aware progress signal for policy training [18].

Value differencing vs. direct stage-conditioned advantage. Prior VLA methods such as $\pi^*_{0.6}$ [252] pair AWR [295, 296] with state-value differences,

$$A(s, a) = V(s') - V(s), \tag{5.9}$$

which compounds per-frame errors e, e' with variance $\text{Var}(e) + \text{Var}(e') - 2\,\text{Cov}(e, e')$ ($\text{Cov}(e, e') \approx 0$) and becomes ill-posed when separate stages look visually similar. SA

instead regresses relative timestamp progress directly from frame pairs (s, s') ($s' = s_{t+\Delta}$ across varied step offsets Δ) via a VLM estimator f_θ (Fig. 5.2):

$$A(s, a) = f_\theta(s, s'), \tag{5.10}$$

inducing positive error correlation across the pair to lower temporal variance (Section 5.7.2). To disambiguate similar cloth states across S manual semantic stages, $f_\theta(s, s' \mid g)$ acts as a stage-conditioned multi-step progress predictor conditioned on the normalised stage index g (written A_{stage} following the paper's terminology):

$$A_{\text{stage}}(s, a, g) = f_\theta(s, s' \mid g), \qquad g \in \left\{0, \frac{1}{S}, \ldots, \frac{S-1}{S}\right\}. \tag{5.11}$$

Figure 5.4 plots the resulting cumulative progress curves across Tasks A–C (Section 5.5.2). Following the binary optimality construction in $\pi^*_{0.6}$ [252], A_{stage} is converted into a binary optimality indicator for advantage-weighted behaviour cloning:

$$I = \mathbf{1}[A_{\text{stage}} > \epsilon], \tag{5.12}$$

where ϵ is set at the 70th percentile so the top fraction $q = 0.3$ of transitions receives $I = 1$, upweighting productive demonstration segments in P_{train} to reduce stalling in P_{test} (Sections 5.7.2 and 5.9).

5.4.4 Train–Deploy Alignment

During physical deployment, forward-pass and communication delays between Q_{model} and P_{test} create velocity and position jumps at action-chunk boundaries. Whereas prior controllers smooth or inpaint chunks at run time [258, 259, 268], χ_0 applies *temporal chunk-wise smoothing* to maintain continuous actuation across chunk updates (Fig. 5.5, bottom left; Algorithm 2). Given the unconsumed suffix $\mathbf{a}^{\text{old}}$ of the previous action chunk starting at index k (so a^{old}_j is the j-th remaining action in the buffer) and a new chunk $\mathbf{a}^{\text{new}}$, we discard the stale prefix of length $d = \min(k, d_{\max})$ from $\mathbf{a}^{\text{new}}$ (when the

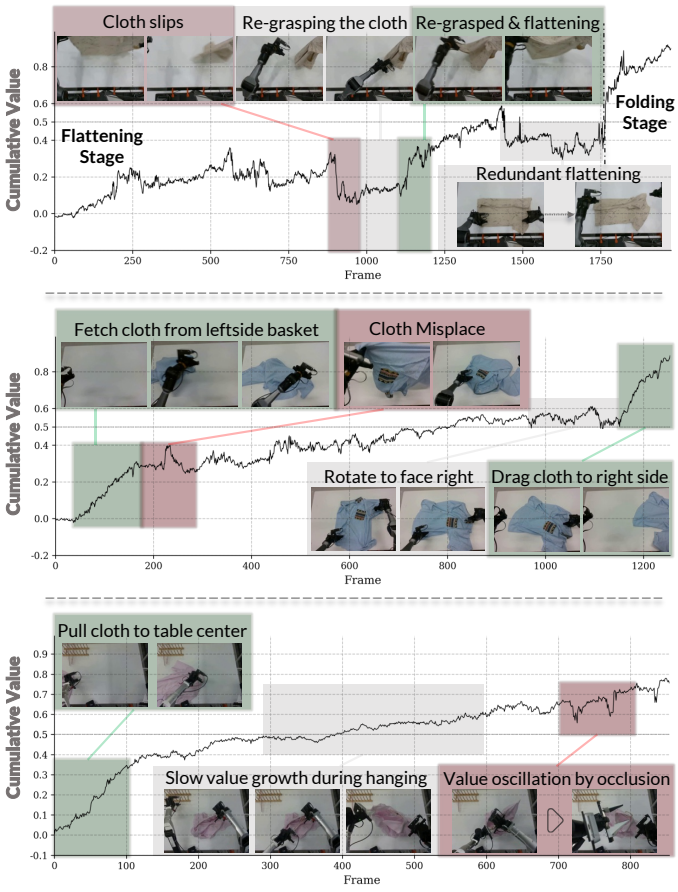


Fig. 5.4 **Cumulative progress curves from Stage Advantage.** Green and red shading indicate positive and negative predicted advantage on Task A (cloth slip and recovery, **top**), Task B (retrieval and placement adjustment, **middle**), and Task C (rack pull-over under occlusion, **bottom**).

early-return branch $d \geq |\mathbf{a}^{\text{new}}|$ fires, k continues incrementing along the unconsumed buffer), pad $\mathbf{a}^{\text{old}}$ to a minimum overlap $m_{\min}$, and linearly cross-fade the remaining suffix $\mathbf{a}^{\text{new}}_{\text{rem}}$ with $\mathbf{a}^{\text{old}}$ over L overlapping steps via decaying weights w_i. Unlike ACT temporal ensembling [268] (which queries the model at every control step to average all historical chunks) or RTC [258] (which inpaints chunks during sampling conditioned on committed actions), chunk-wise smoothing post-processes consecutive buffers without extra model evaluations and combines orthogonally with RTC (Section 5.7.3).

Closing the loop from deployment to training. To fold error states from P_{test} back into P_{train} with minimal collection overhead, TDA uses two data-side mechanisms

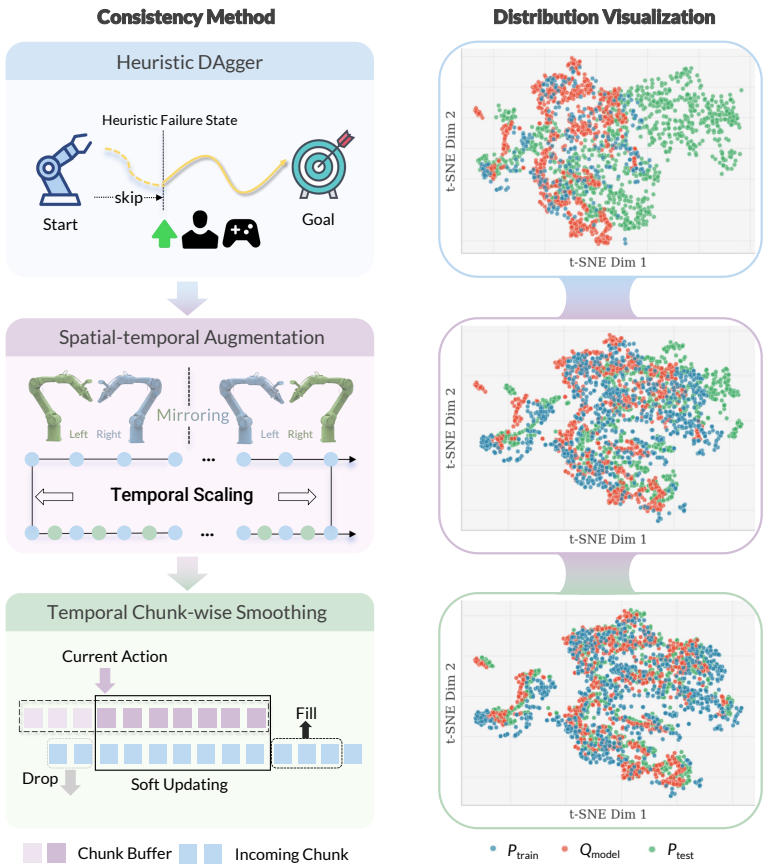


Fig. 5.5 **Train–Deploy Alignment mechanisms (left) and qualitative t-SNE trajectory projections (right)** illustrating closer overlap among distributions as each mechanism is added.

(Section 5.9): (1) whereas interactive DAgger collects expert labels on states visited by the learned policy [8, 278], *heuristic DAgger* places the garment directly into hand-crafted fault states (such as slipped grasps or half-dropped cloth) and records human recovery trajectories without running the learned policy (Fig. 5.5, top left), concentrating targeted data expansion at failure states; and (2) *spatiotemporal augmentation* horizontally mirrors camera frames and joint actions with left-right arm swap [281] while subsampling frames to synthesise varied execution speeds.

Algorithm 2: Temporal chunk-wise smoothing

Input: unconsumed suffix $\mathbf{a}^{\text{old}}$ of previous action chunk starting at index k (a_j^{old} is the j-th remaining action), consumption index k, new chunk $\mathbf{a}^{\text{new}}$, drop threshold $d_{\max}$, minimum overlap $m_{\min}$

Output: updated buffer $\mathbf{a}^{\text{buf}}$ and updated index k

$d \leftarrow \min(k, d_{\max})$; `// stale prefix length`

if $d \geq |\mathbf{a}^{\text{new}}|$ **then**

  return $\mathbf{a}^{\text{old}}, k$; `// skip outdated chunk;` k `continues incrementing along` $\mathbf{a}^{\text{old}}$

$\mathbf{a}_{\text{rem}}^{\text{new}} \leftarrow (a_d^{\text{new}}, \dots)$; `// unconsumed new actions`

if $|\mathbf{a}^{\text{old}}| < m_{\min}$ **then**

  pad $\mathbf{a}^{\text{old}}$ by repeating its final command;

$L \leftarrow \min(|\mathbf{a}^{\text{old}}|, |\mathbf{a}_{\text{rem}}^{\text{new}}|)$; `// blending window`

for $i \leftarrow 0$ **to** $L - 1$ **do**

  $w_i \leftarrow 1 - i / \max(L - 1, 1)$;

  $\tilde{a}_i \leftarrow w_i \, a_i^{\text{old}} + (1 - w_i) \, a_{\text{rem},i}^{\text{new}}$;

$\mathbf{a}^{\text{buf}} \leftarrow (\tilde{a}_0, \dots, \tilde{a}_{L-1}) \,\|\, (\text{suffix of } \mathbf{a}_{\text{rem}}^{\text{new}})$;

$k \leftarrow 0$; `// reset buffer index`

return $\mathbf{a}^{\text{buf}}, k$;

5.5 System and Data Setup

5.5.1 Robot Platform

Our dual-station bimanual setup (Fig. 5.6; Section 5.9.1) comprises one station with AgileX Piper arms and one with ARX X5 arms. Each station mounts two 6-DoF manipulators with 1-DoF parallel-jaw grippers and three 640×480 Intel RealSense D435i RGB cameras (one overhead static camera plus two wrist cameras), with policy inference running on a local NVIDIA RTX 4090 GPU workstation.

5.5.2 Evaluation Tasks

We evaluate χ_0 across three garment manipulation benchmarks ordered by difficulty (Table 5.1):

- **Task A: T-shirt flattening and folding (easy).** Streamlined from the π-series laundry benchmark [6, 249, 252]: unfold a crumpled T-shirt from an ar-

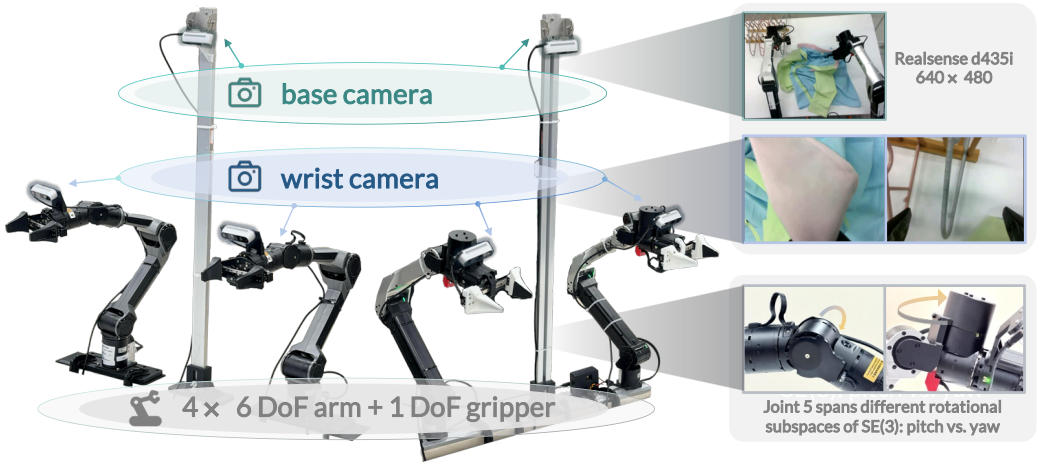


Fig. 5.6 **Physical layout** of the dual-station bimanual manipulation platform.

bitrary initial pose and fold it near the workspace centre within a 180-second horizon.

- **Task B: conditional retrieval and sorting (medium).** Extending [6, 249, 252] with category-dependent routing: fetch and flatten an arbitrarily posed garment within a 180-second cutoff, folding and placing T-shirts at the upper-left corner while passing collared shirts to the right-hand workspace.

- **Task C: garment hanging (hard).** Following GR-3 [303]: receive a flattened collared shirt from Task B, dress it onto a hanger rack, and suspend it stably on the rail without slipping (no fixed episode time cutoff is recorded in the summary logs).

5.5.3 Metrics

Each policy configuration runs for 30 rollouts (10 rollouts across each of three test garments) and is scored on four summary metrics:

- **Success rate (SR, % ↑)**: percentage of rollouts that satisfy the full task completion criteria.

- **Throughput (TP ↑)**: projected count of completed tasks per hour of operation.

Table 5.1 **Characteristics of the three cloth manipulation benchmarks.** Semantic sub-goals (S) match Stage Advantage (Section 5.4.3); episode counts and mean durations cover both base demonstrations and shorter DAgger recovery clips, with no fixed cutoff logged for Task C.

Task	Stages (S)	Time limit	Episodes	Mean duration
A (easy): T-shirt flattening and folding	2: flattening, folding	180 s	2668	30.41 s
B (medium): conditional retrieval and sorting	4: retrieving, flattening, folding, handover	180 s	3519	34.40 s
C (hard): garment hanging	3: retrieving, dressing the rack, hanging	–	2988	39.01 s

- **Retry cost (↓)**: mean number of corrective re-attempts per rollout during closed-loop execution.
- **Average score (↑)**: partial-credit score normalised to a 100-point scale under a milestone rubric (sub-goal weights in Table 5.4, Section 5.9).

Stage Advantage ablations additionally report the smooth frame ratio (**SFR** ↑) and mean squared temporal difference (**MSTD** ↓), which quantify frame-to-frame trajectory smoothness rather than ground-truth progress accuracy (since a constant predictor has zero temporal variation).

5.5.4 Evaluation Protocol and Uncertainty

Closed-loop rollouts run on the two physical stations of Section 5.5.1 in the demonstration workspace with garments tossed into arbitrary starting poses under randomised lighting (Section 5.9.1). Because archived logs store only aggregate rollout summaries rather than the per-episode initial-state matching, operator blinding, scheduling, and garment-overlap metadata recommended by Kress-Gazit *et al.* [27], our experiments measure closed-loop reliability within the training workspace rather than zero-shot transfer to unseen rooms, hardware, or garments.

Table 5.2 and Figs. 5.7–5.12 report metrics read from the plotted evaluation summaries (accurate to ≈1 axis unit), paired with reconstructed integer success counts (in 3.3-point steps for $n = 30$, where 27% is 8/30), two-sided Fisher exact p-values, and 95% Wilson intervals. Plotted whiskers labelled "accumulative standard error" are omitted from comparisons because that quantity is undocumented and sometimes smaller than binomial variance permits (e.g., the +SA bar at 47%, or 14/30, shows a ±1-point whisker whereas the standard error across the three 10-trial garment strata is at least ≈3 points and the pooled binomial standard error is near 9 points). At $n = 30$, a 95% Wilson interval around 50% spans ≈34 points, so differences of four or fewer successes never clear sampling noise ($p > 0.1$); nominal 5% differences in multi-factor grids (Figs. 5.8 and 5.12) also face multiple-testing inflation, and throughput, retries, and average score are point means.

5.5.5 Data Collection and Training

Roughly 20 hours of human teleoperated demonstrations are collected per benchmark across varied garment colours, fabrics, poses, and lighting; combining the episode counts and mean durations in Table 5.1 yields roughly 22.5, 33.6, and 32.4 hours of trajectories (base demonstrations plus DAgger recovery clips) for Tasks A–C. We fine-tune all parameters of $\pi_{0.5}$ [6] on 8×A100 GPUs under flow matching [249] (Table 5.3, Section 5.9; resource caveats in Section 5.8).

5.6 Experiments

Physical rollouts on the dual-station platform of Section 5.5 address four questions:

1. **System efficacy**: how the three modules interact in the eight-way factorial evaluation on Task A (Section 5.6.2).

2. **Model Arithmetic**: whether weight merging of subset checkpoints outperforms both the strongest subset checkpoint and pooled training, and whether OOD DAgger validation beats in-domain validation (Section 5.7.1).

3. **Stage Advantage**: whether direct stage-conditioned progress estimation yields smoother supervision traces and stronger rollouts than a $\pi^*_{0.6}$-style RECAP value-differencing baseline [252] (Section 5.7.2).

4. **Train–Deploy Alignment**: how heuristic DAgger compares with standard on-policy DAgger, and how run-time chunk smoothing interacts with spatiotemporal augmentation (Section 5.7.3).

5.6.1 Baselines and Ablation Design

Base policy. We adopt $\pi_{0.5}$ as our primary backbone and π_0 in secondary comparisons, as only these two open-weight policies achieved non-trivial completion on our cloth tasks; GO-1 from AgiBot World [5] (co-authored by the author), X-VLA [251], DexVLA [304], UniVLA [274], and OpenVLA [273] failed to complete the manipulation sequences reliably (Section 5.9).

Module ablation baselines. For **MA**, we benchmark the five coefficient-selection rules of Section 5.4.2 (with in-domain or OOD DAgger validation) against the *single-best* subset policy and the pooled *full-data* policy. For **SA**, we compare stage-agnostic direct regression (Direct, Eq. (5.10)) and stage-conditioned regression (Direct+Stage, Eq. (5.11)) against a $\pi^*_{0.6}$-style RECAP value-differencing baseline [252] sharing the PaliGemma backbone of $\pi_{0.5}$ [6] that regresses point state values on offline frames over a 50-step lookahead (rather than full RECAP's distributional value head over demonstrations, on-policy rollouts, and human interventions; Section 1.1). For **TDA**, we compare temporal chunk-wise smoothing against synchronous and asynchronous chunk execution [259], ACT-style temporal ensembling [268], and RTC [258] across

spatial mirroring and temporal frame skipping, and compare standard on-policy DAgger [8] with heuristic DAgger on π_0 [249] and $\pi_{0.5}$ [6].

5.6.2 System Efficacy Breakdown

Figure 5.7 and Table 5.2 report the eight-way factorial ablation on Task A using the best module settings from Section 5.7:

- **TDA** boosts completion rate by roughly 17 to 53 percentage points across every pairing; its effect on retry count depends on the partner module, rising when paired with +MA (from ≈1.8 to 2.6) or +SA (from ≈1.1 to 2.3) yet dropping when added to bare $\pi_{0.5}$ (from ≈3.0 to 1.7) or +MA+SA, consistent with recovery data encouraging active re-grasping after slips.
- **SA** raises both completion rate and hourly throughput across all pairings, with configurations containing SA achieving the fastest task completions overall.
- **MA** in isolation leaves completion rate nearly unchanged relative to $\pi_{0.5}$ (roughly 30% vs. 27%, or 9 vs. 8 of 30 trials) and lowers the rubric score (35 vs. 41), but adds roughly 30 percentage points in completion rate when paired with +TDA (16 to 25 of 30; uncorrected Fisher $p \approx 0.025$) or +SA (14 to 24 of 30; $p \approx 0.015$).
- **Full** χ_0 achieves the highest completion rate (roughly 97%, 29/30) and rubric score (≈97) while matching +SA+TDA in throughput (66 vs. 68 completions/h). At $n = 30$ trials, 29/30 cannot be statistically separated from the 24 to 26 successes of two-module pairings (Fisher $p \geq 0.10$), nor does any single module clear the sampling threshold over bare $\pi_{0.5}$ ($p \geq 0.06$); statistically decisive gains require combining at least two modules.

5.6.3 Quantitative Summary

Table 5.2 compiles the Task A factorial summary means from Fig. 5.7 alongside reconstructed success counts and 95% Wilson confidence intervals for $n = 30$ trials

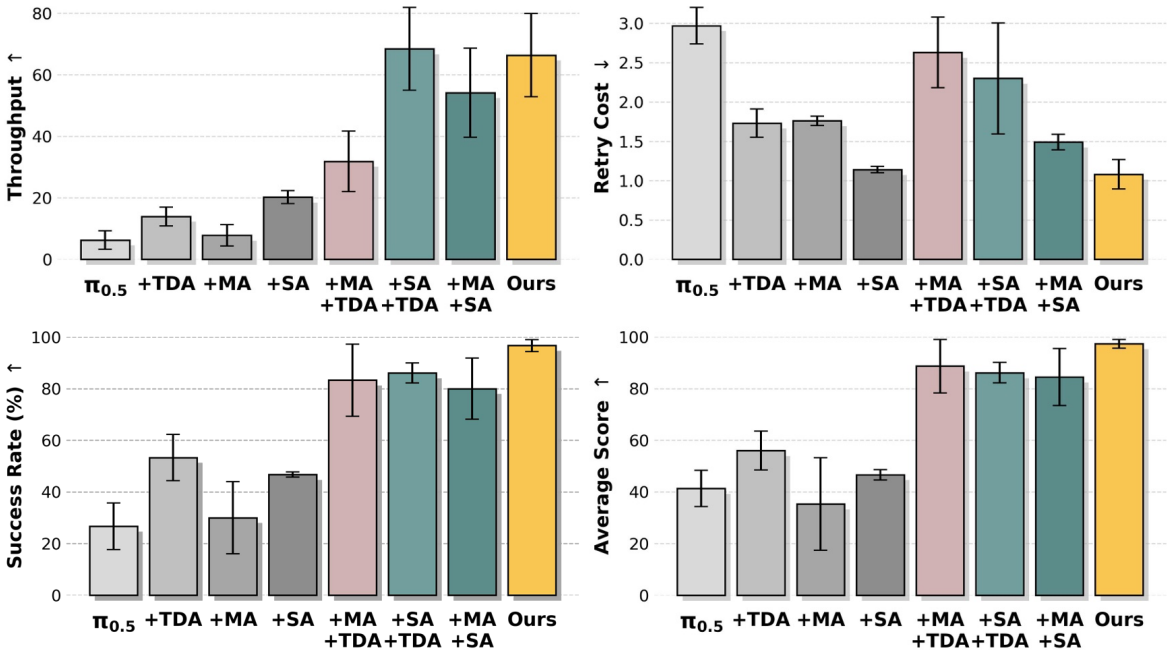


Fig. 5.7 **Factorial module breakdown of χ_0 on Task A** (30-trial means per configuration; Table 5.2). Both TDA and SA boost completion rates across all pairings, whereas MA provides gains only when paired with TDA or SA. Whiskers reflect the plotted evaluation summaries (Section 5.5.4).

(Section 5.5.4). Relative to $\pi_{0.5}$, the full χ_0 pipeline lifts completion from roughly 27% (8/30; Wilson interval 14–44%) to roughly 97% (29/30; 83–99%; two-sided Fisher exact $p \approx 1.5 \times 10^{-8}$, a relative gain of about 260%, +70 percentage points), scales hourly throughput roughly elevenfold (≈ 6 to ≈ 66 tasks/h), and cuts retry frequency by about 63% (≈ 3.0 to ≈ 1.1 retries). Across the multiple ablation comparisons in Section 5.7, p-values are reported unadjusted as exploratory diagnostics at $n = 30$, whereas the primary system-level test on Task A ($p \approx 1.5 \times 10^{-8}$) survives any multiplicity correction.

5.7 Ablation Studies

Cross-figure consistency. The module studies (Figs. 5.8–5.12 and 5.14–5.18, including the in-chunk smoothing baselines in Figs. 5.16–5.18) were conducted in separate sessions with different checkpoints and dataset snapshots, so nominal baselines

Table 5.2 **Factorial component results on Task A (T-shirt flattening and folding).** Values report 30-trial evaluation summaries (10 rollouts per garment across three garments) from Fig. 5.7, paired with reconstructed success counts and 95% Wilson intervals (Section 5.5.4); throughput, retry cost, and 100-point rubric score are point means.

Configuration	Success rate (%) ↑	Successes (of 30)	95% Wilson interval (%)	Through-put (tasks/h) ↑	Retry cost ↓	Average score ↑
$\pi_{0.5}$	≈27	≈8	14–44	≈6	≈3.0	≈41
+TDA	≈53	≈16	36–70	≈14	≈1.7	≈56
+MA	≈30	≈9	17–48	≈8	≈1.8	≈35
+SA	≈47	≈14	30–64	≈20	**≈1.1**	≈47
+MA+TDA	≈83	≈25	66–93	≈32	≈2.6	≈89
+SA+TDA	≈87	≈26	70–95	**≈68**	≈2.3	≈86
+MA+SA	≈80	≈24	63–90	≈54	≈1.5	≈84
χ_0 (full system)	**≈97**	**≈29**	83–99	≈66	**≈1.1**	**≈97**

are not interchangeable across figures. On Task A, for example, $\pi_{0.5}$ completion ranges from about 23% (Fig. 5.10) to 27% (Fig. 5.7) and 37% (synchronous in Fig. 5.12). Likewise, heuristic DAgger reaches 87% completion with 8.6 retries in Fig. 5.10, whereas the factorial '+TDA' condition reaches 53% with 1.7 retries in Fig. 5.7; unaugmented chunk-wise smoothing incurs ≈12 retries and 77% completion in Fig. 5.12 (70% in Fig. 5.16a) versus 1.1 retries for the full system in Table 5.2; and the Stage Advantage (Fig. 5.9) and Model Arithmetic (Fig. 5.14a) figures use still stronger references (67–83%). Task C baselines similarly range from about 10% (Figs. 5.11 and 5.15) to 20% (Fig. 5.8) and 37% (Fig. 5.18a). These shifts preclude cross-figure effect estimates. All ablation claims below are therefore restricted to comparisons made within the same figure and evaluation session.

5.7.1 Model Arithmetic

On Task C (Fig. 5.8), every weight-merged policy exceeds both the single-best subset checkpoint (≈7/30) and the full-data policy (≈6/30, 20%) in mean completion rate and throughput, though some merge rules raise retries and only OOD-validated greedy search (≈15/30, 50%) clears the nominal 5% threshold over full-data (uncorrected

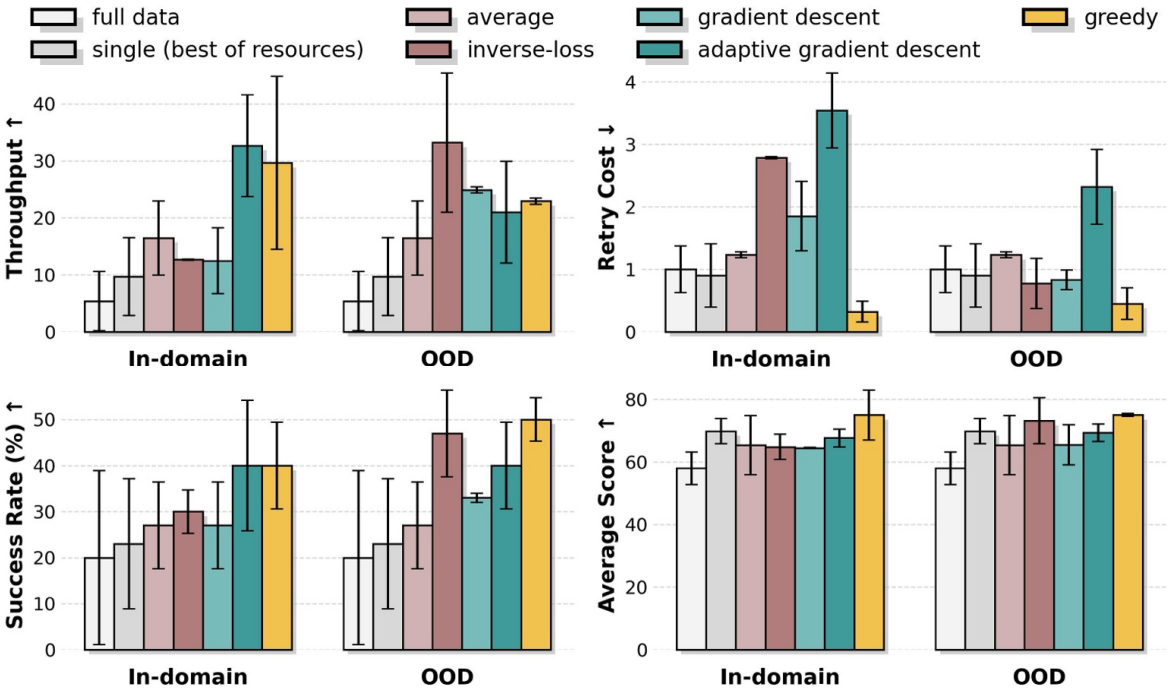


Fig. 5.8 **Model Arithmetic comparison on Task C** (30 rollouts per bar). All weight-merged policies exceed both unmerged baselines in mean completion rate and throughput, with OOD-validated greedy search achieving the highest completion rate (Section 5.7.1). Whiskers reflect the plotted evaluation summaries (Section 5.5.4).

Fisher $p \approx 0.03$; $p \approx 0.06$ vs. single-best). On the partial-credit rubric, only greedy search and OOD inverse-loss weighting beat the single-best candidate (score ≈70); why linear interpolation matches or beats pooled training remains an open question, potentially linked to VLA weight redundancy [305].

Replacing in-domain validation with OOD DAgger validation lifts completion for three of four loss-guided rules (inverse-loss from ≈30% to 47%, gradient descent from 27% to 33%, greedy search from 40% to 50%, or 12 vs. 15 of 30, Fisher $p \approx 0.60$, with adaptive gradient descent near 40%) and reduces retries under three rules despite minor throughput dips under adaptive gradient descent and greedy search. Across both validation splits on Task C, greedy search posts the highest (or tied-highest) completion rate, top score, and lowest retries. Parallel ablations on Tasks A and B appear in Section 5.9, and Section 5.9 notes the in-domain inverse-loss rule used in Table 5.2.

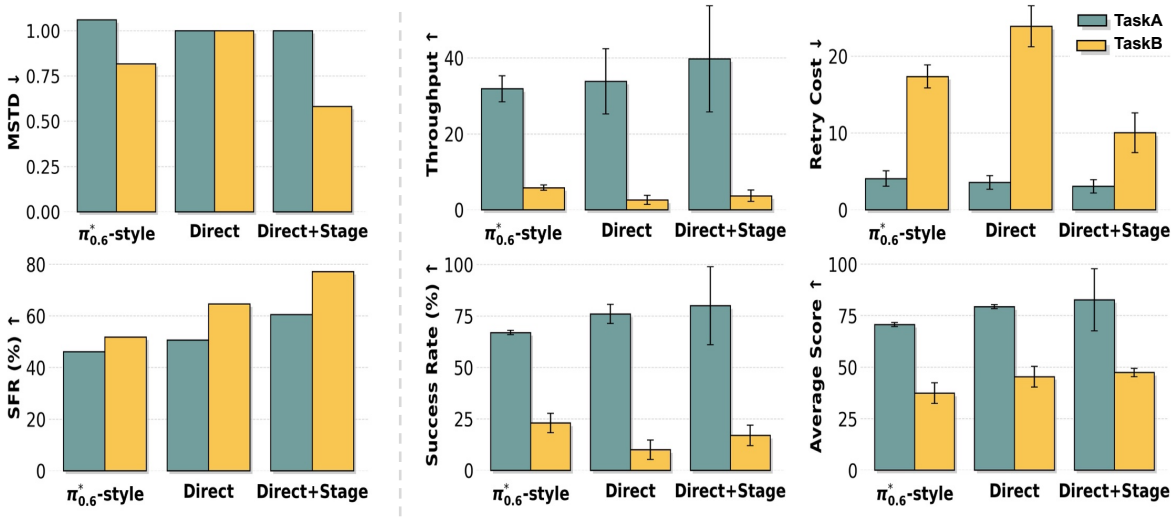


Fig. 5.9 **Stage Advantage evaluation on Tasks A and B** (developed by a co-author; 30 trials per bar). **Left**: temporal smoothness via SFR and MSTD. **Right**: closed-loop rollout metrics.

5.7.2 Stage Advantage

Figure 5.9 compares Stage Advantage (Direct+Stage, co-author module) against the $\pi^*_{0.6}$-style value-differencing baseline and stage-agnostic Direct regression on Tasks A and B. Direct+Stage yields smoother advantage traces than value differencing: SFR rises from ≈46% to 60% on Task A and from 52% to 77% on Task B, while MSTD drops from 0.82 to 0.58 on Task B and stays comparable on Task A (1.0 vs. 1.06). Stage-agnostic Direct regression raises SFR on Task B but worsens MSTD and posts the lowest completion rate and highest retries, supporting stage conditioning. In closed-loop rollouts, Direct+Stage improves all four metrics on Task A, though the completion gain from ≈20 to 24 of 30 (67% to 80%) lies within binomial noise (Fisher $p \approx 0.38$). On Task B, Direct+Stage lowers retries (≈17 to 10) and lifts the rubric score (≈37 to 48) but trails in completion rate (17% vs. 23%) and throughput (4 vs. 6); on Task C (Section 5.9), it similarly cuts retries while trailing in completion, throughput, and score, showing that lower training loss (Section 5.9.1) and temporal smoothness do not guarantee stronger closed-loop execution.

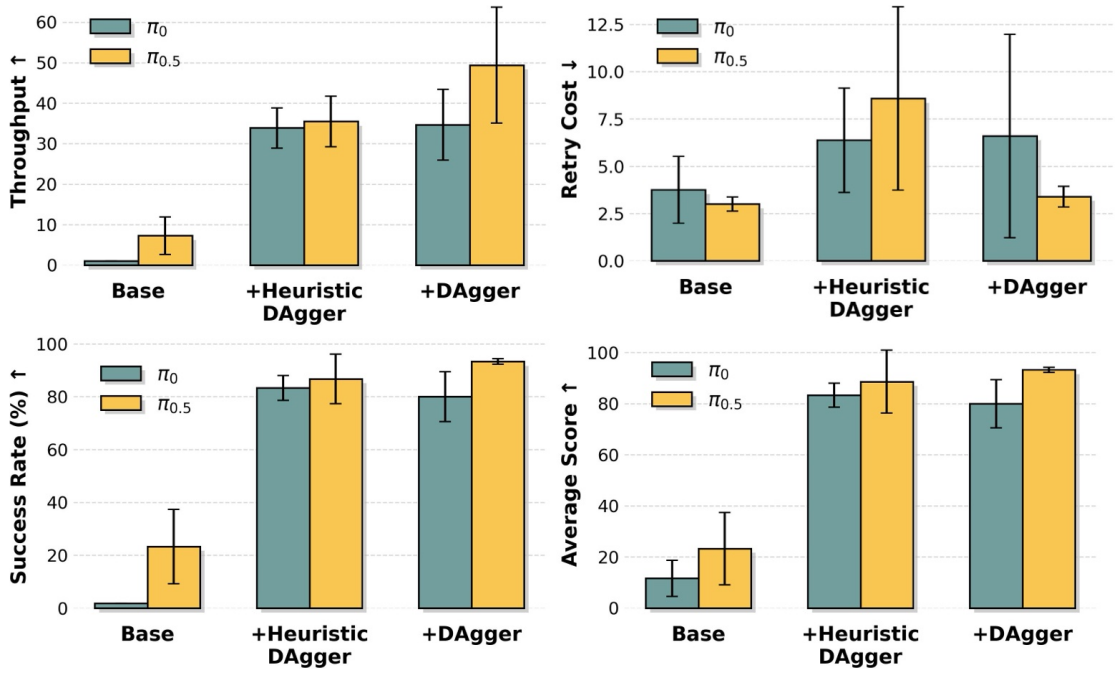


Fig. 5.10 **Comparison of heuristic and standard DAgger on Task A** (30 rollouts per bar). Both recovery-collection schemes boost completion, throughput, and rubric score, with heuristic DAgger incurring more retries and standard DAgger achieving higher throughput and fewer retries on $\pi_{0.5}$.

5.7.3 Train–Deploy Alignment

Heuristic vs. standard DAgger. Across $\pi_{0.5}$ and π_0 on Tasks A and C (Figs. 5.10 and 5.11), DAgger recovery data yield the largest standalone gain in this chapter. Heuristic DAgger lifts $\pi_{0.5}$ completion from ≈23% to 87% (7 vs. 26 of 30) and π_0 from near zero to ≈83% on Task A, and raises $\pi_{0.5}$ from ≈10% to 43% (3 vs. 13 of 30) on Task C, while increasing retries across all settings (e.g., ≈3.0 to 8.6 on Task A for $\pi_{0.5}$) as the policy actively re-grasps after slips. Standard on-policy DAgger attains comparable $\pi_{0.5}$ completion (93% or 28/30 on Task A; 53% or 16/30 on Task C, within sampling noise) with fewer retries (3.4 vs. 8.6 on Task A; 1.5 vs. 2.4 on Task C), higher Task C score (83 vs. 62), and higher Task A throughput (50 vs. 36); exact operator and robot collection hours were not logged for either protocol.

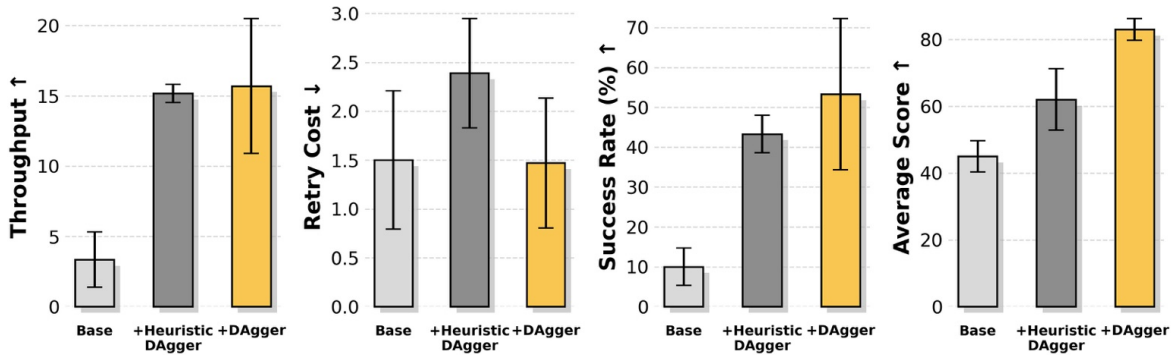


Fig. 5.11 **Comparison of heuristic and standard DAgger on Task C** (30 rollouts per bar). Both recovery schemes raise completion rates while heuristic DAgger increases retry frequency.

Control strategies and augmentation. Figure 5.12 compares temporal chunk-wise smoothing against competing controllers across four augmentation regimes on Task A. Without augmentation, chunk-wise smoothing beats temporal ensembling and RTC in mean completion rate (77% vs. 53% and 60%, or 23 vs. 16 and 18 of 30; Fisher $p \approx 0.10$ and 0.27), and combining chunk-wise smoothing with RTC achieves the top overall result (90%, 27/30). Used without RTC, however, chunk-wise smoothing incurs the highest retries in the unaugmented and temporally scaled (T-scale) regimes (12–13 vs. 2–6) and trails RTC under T-scale alone (50% vs. 73%) before regaining its lead when spatial mirroring (S-mirror) is added (see Section 5.9 for Tasks B and C and joint-space ablations).

5.8 Discussion and Limitations

By combining Model Arithmetic, Stage Advantage, and Train–Deploy Alignment, χ_0 lifts Task A completion of $\pi_{0.5}$ from roughly 27% to 97%, subject to three main caveats:

- **Alignment vs. data volume and resource accounting.** Lacking an iso-budget scaling baseline trained on an equal number of operator and robot hours of standard demonstrations, the Task A study does not separate distributional alignment from targeted data volume, especially since DAgger recovery clips

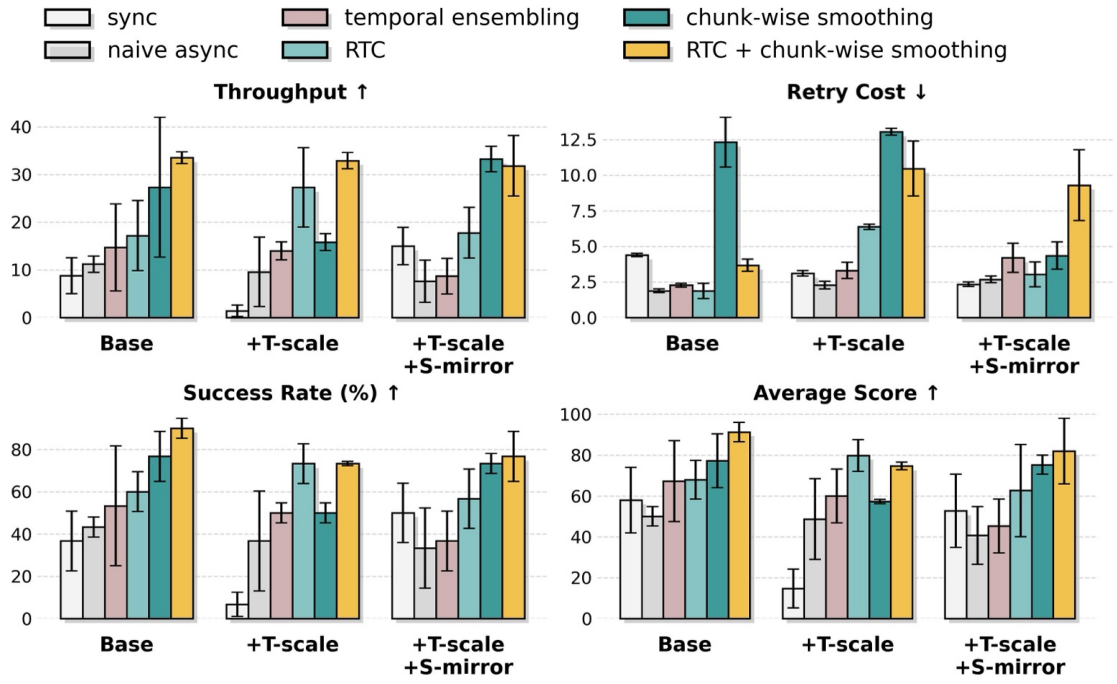


Fig. 5.12 **Execution controllers and spatiotemporal augmentation on Task A** (30 rollouts per setting; T-scale: frame-skipping speed augmentation; S-mirror: horizontal mirroring with arm swap).

drive the largest single gain (Section 5.7.3) and only Task A received the full eight-way factorial evaluation. Moreover, while $\pi_{0.5}$ post-training uses roughly 20 demonstration hours per task and 8 A100 GPUs over 80,000 steps (Tables 5.1 and 5.3), cumulative GPU, robot, and operator hours across the four MA checkpoints, the SA estimator, and DAgger rollouts were not itemised separately.

- **Evaluation scope, statistics, and generalisation.** All 30-trial rollouts took place in the training workspace without logged blinding, scheduling, or unseen-garment splits (Section 5.5.4), measuring in-workspace closed-loop reliability rather than zero-shot transfer to unseen rooms, embodiments, or cloth categories; cross-session baseline shifts and 30-trial binomial variance (Section 5.7) further restrict statistical resolution to large effects, and retention of general pre-trained VLA skills, rigid-body transfer, and cross-task merging were not tested.

- **Data valuation, advantage supervision, and failure modes.** Demonstration cleanliness informally shifted completion rates between roughly 20% and 60% under fixed training settings (Section 5.9), motivating offline data-utility proxies. Furthermore, Stage Advantage assumes monotonic timestamp progress without human advantage labels, and neither MA nor SA was separately benchmarked against the two main rollout failure modes of imprecise 3D grasping and mid-episode stagnation (Sections 5.9 and 5.9).

5.9 Implementation Details and Supplementary Ablations

5.9.1 Module Implementation, Hardware, and Evaluation Rubric

Nomenclature, baseline screening, and data quality. **K**inetics-**A**ligned-**I**ntelligence (KAI_0, written χ_0 after the Greek letter χ in parallel with the π family) aligns demonstration collection (P_{train}) and physical deployment (P_{test}) with the policy-induced trajectory distribution (Q_{model}). In pilot screening on our 20-hour corpus, X-VLA [251], DexVLA [304], OpenVLA [273], UniVLA [274], and GO-1 (from AgiBot World [5], co-authored by the thesis author) all scored near zero over 30 rollouts despite matched camera views and converged losses, whereas π_0 and $\pi_{0.5}$ showed stronger post-training plasticity. Because demonstration quality shifted completion between 20% and 60% under identical settings, we use open-loop *replay-ability* from a comparable initial state to screen corrupted logs and hardware drift between P_{train} and P_{test}. At run time, whereas SmolVLA [306] switches directly between asynchronous chunks and A2C2 [307] adds auxiliary heads, χ_0 blends overlapping chunks linearly (Algorithm 2) without architectural changes.

Module specifications and DAgger protocol. Model Arithmetic partitions demonstrations into four disjoint splits; Table 5.2 uses in-domain inverse-loss weighting, whereas ablations compare all five rules against the lowest-validation-loss *single-best* checkpoint and the pooled *full-data* model. For Stage Advantage (by co-author Lirui Zhao; cf. VLA/robot RL post-training [308–323] and benchmark suites used to evaluate long-horizon transfer [324–326]), given frame pairs $(s_t, s_{t+\Delta})$ from an episode of length T_e, stage-agnostic direct regression predicts the normalised step offset:

$$\hat{A}_{\text{direct}}(s_t, s_{t+\Delta}) = f_\theta(s_t, s_{t+\Delta}) \approx \frac{\Delta}{T_e}, \tag{5.13}$$

stage-conditioned regression conditions on $g \in \{0, 1/S, \ldots, (S-1)/S\}$ across $S \in \{2, 4, 3\}$ stages on Tasks A–C:

$$\hat{A}_{\text{stage}}(s_t, s_{t+\Delta}, g) = f_\theta(s_t, s_{t+\Delta} \mid g), \tag{5.14}$$

and 70th-percentile thresholding ϵ_{1-q} ($q = 0.3$) yields the binary indicator:

$$I_t = \mathbf{1}\big[\hat{A}_{\text{stage}}(s_t, s_{t+\Delta}, g) > \epsilon_{1-q}\big]. \tag{5.15}$$

Stage Advantage converges to a lower terminal loss than the $\pi^*_{0.6}$-style baseline (Fig. 5.13). Standard DAgger (1) launches an autonomous rollout, (2) halts after > 10 consecutive missed grasps or stagnant loop steps while freezing observations and holding joints for human takeover, and (3) logs both segments; heuristic DAgger directly places the cloth and arms into typical halt states to record expert recoveries. On the dual-station rig (Fig. 5.6; AgileX Piper with pitch joint 5, ARX X5 with yaw joint 5), cameras stream at 30 Hz under a standard operating protocol and joint controllers run at 100–200 Hz on an Ubuntu 20.04 workstation. Tables 5.3 and 5.4 list the $\pi_{0.5}$ fine-tuning hyper-parameters and the 100-point milestone rubric.

Table 5.3 Fine-tuning configuration for $\pi_{0.5}$ post-training (not a controlled from-scratch budget; Section 5.8).

Hyper-parameter	**Value**
Data and input	
Expert demonstration hours	$\sim$20 h per task
Action chunk length K	50
Execution frequency	100 Hz
Optimisation	
Training steps	80,000
Batch size	128
Optimiser	AdamW
Learning rate	2.5×10^{-5}
Cosine decay steps	10,000
Conditioned noise level σ	[0.001, 1.0]
Gradient clip	1.0
Module-specific	
MA: number of checkpoints	4
SA: fraction of samples labelled positive q	0.3
Infrastructure	
Training GPUs	8 $\times$ A100
Inference GPUs	RTX 4090

5.9.2 Extended Module Ablations and Failure Analysis

Model Arithmetic on Tasks A and B, and Stage Advantage on Task C. Over 30 rollouts per variant on Tasks A and B (Fig. 5.14), weight-merged policies surpass the full-data and single-best baselines on most metrics. On Task A (Fig. 5.14a), uniform averaging trails slightly in completion rate (80% *vs.* 83%, within 3 of 30 rollouts) and several merges yield lower throughput; on Task B (Fig. 5.14b), every merged policy improves completion (70%–90% *vs.* 47% and 60%), throughput, and retry count. Because two average-score bars in Fig. 5.14b (full-data at 16 and in-domain inverse-loss at 6) fall below their completion rates (47% and 76%) due to a plotting discrepancy (completed episodes must score 100 under Table 5.4), Task B comparisons rely on completion, throughput, and retries. On Task C (Fig. 5.15, 30 rollouts), Stage Advantage improves smoothness over the value-difference baseline

Table 5.4 **Milestone point allocation for the average score metric (normalised to 100).**

Task	Sub-goal	Score
Task A (easy)	Flatten garment	+40
	1st fold	+20
	2nd fold	+20
	3rd fold	+20
Task B, T-shirt (medium)	Retrieve & flatten	+40
	1st fold	+15
	2nd fold	+15
	3rd fold	+15
	Stack to top-left	+15
Task B, shirt (medium)	Retrieve from basket	+30
	Flatten	+50
	Pull to right-side table	+20
Task C (hard)	Pull garment rightward	+15
	Grasp collar	+15
	Grasp hanger	+15
	Insert hanger into sleeve	+20
	Hook left collar on hanger	+20
	Hang on standing rack	+15

(MSTD 0.77 *vs.* 1.0; SFR 76% *vs.* 73%) and lowers retries, but completion (3% *vs.* 10%, 1 *vs.* 3 of 30 rollouts, within sampling noise), throughput, and score decline because $\pi_{0.5}$ lacks garment-hanging priors.

Train–Deploy Alignment across tasks, action spaces, and failure modes. Across 30 rollouts per bar in Figs. 5.16, 5.17, and 5.18, temporal chunk-wise smoothing outperforms synchronous and in-chunk baselines across nearly all Task A settings for both absolute (Fig. 5.16a) and delta (Fig. 5.16b) joint targets, with RTC adding further gains in 3 of 4 absolute-joint regimes versus 1 delta-joint regime. On Tasks B and C (Figs. 5.17 and 5.18), completion peaks near 30% and 53%, chunk-wise smoothing shows no consistent edge, and performance remains sensitive to absolute versus delta joint representations during tight Task C sleeve insertion. Finally, physical rollouts fail

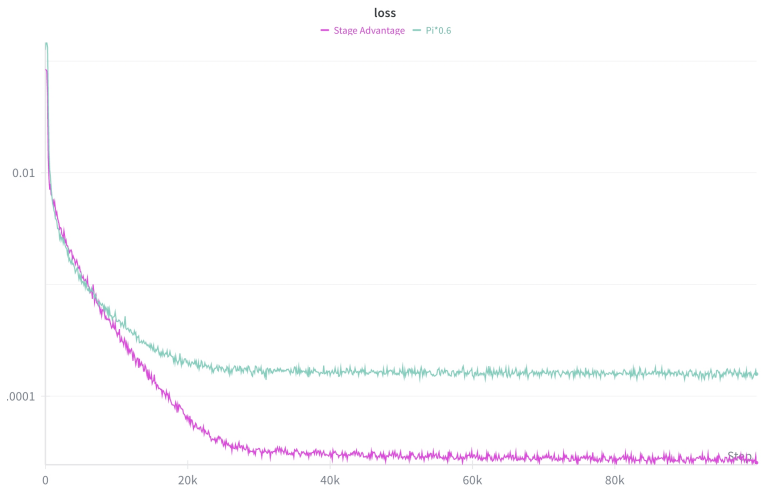
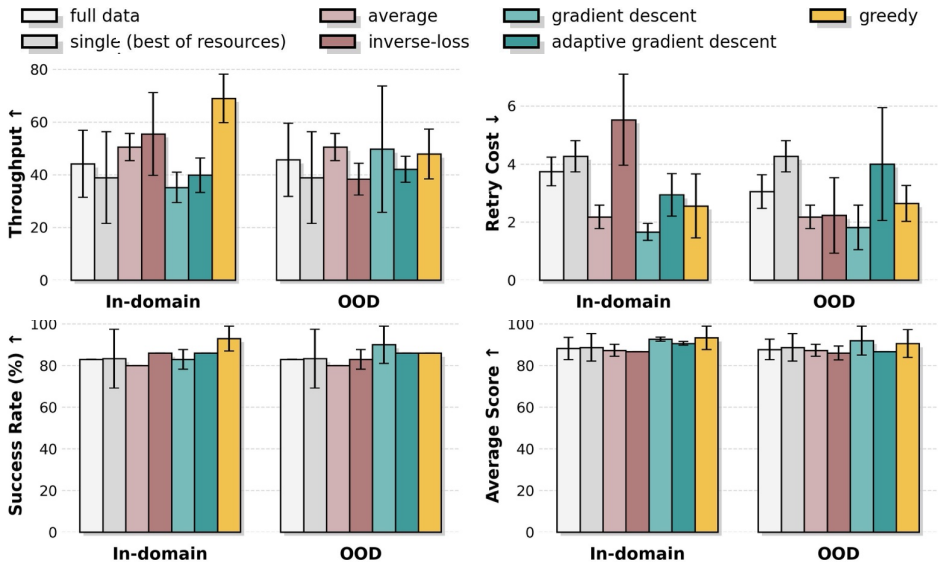
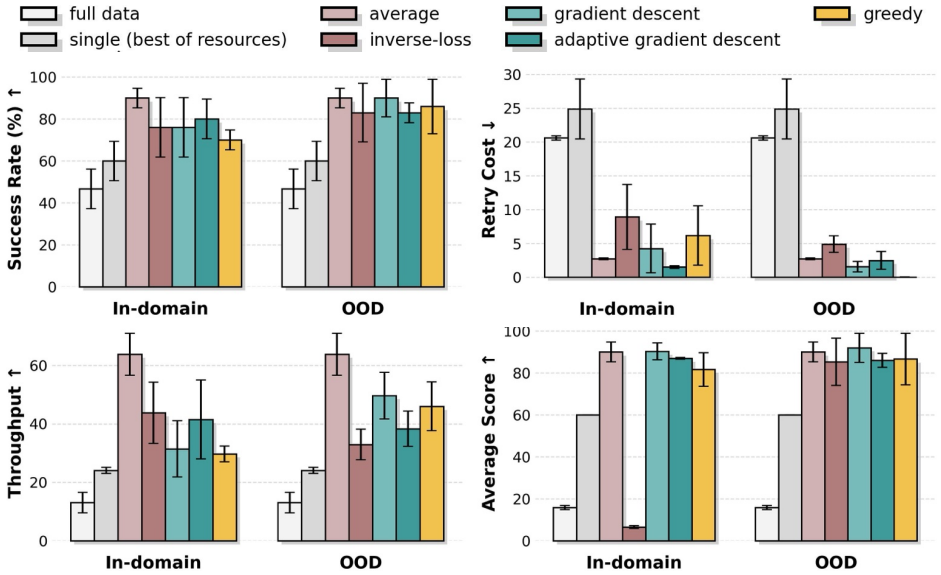


Fig. 5.13 **Loss comparison between SA (by co-author Lirui Zhao; Chapter 5) and the $\pi^*_{0.6}$-style baseline.** SA converges to a lower terminal loss.

primarily through spatial misalignment (missed cloth grasps) and policy stagnation (repetitive motion loops); pooling 1026 Task A failure events across tested policies by task stage (Fig. 5.19) shows that flattening errors dominate (1010 of 1026 instances), led by prolonged re-grasping (874) and redundant flattening (85), explaining why DAgger and heuristic DAgger interventions after missed grasps or stalled loops yield large gains on Task A. Training scripts, datasets (CC BY-NC-SA 4.0), and fine-tuned χ_0 weights and code (Apache-2.0) are released at `https://github.com/OpenDriveLab/KAIO`.



(a) Task A results



(b) Task B results

Fig. 5.14 **Model Arithmetic on Tasks A and B** (30 rollouts per bar; Section 5.5.4). Merged policies beat full-data and single-best baselines on most metrics and lower all Task B retries (see Section 5.9 on panel b score bars).

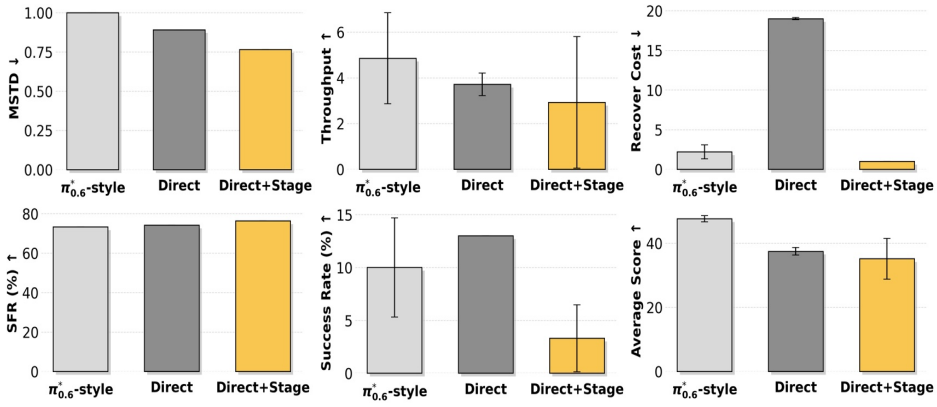
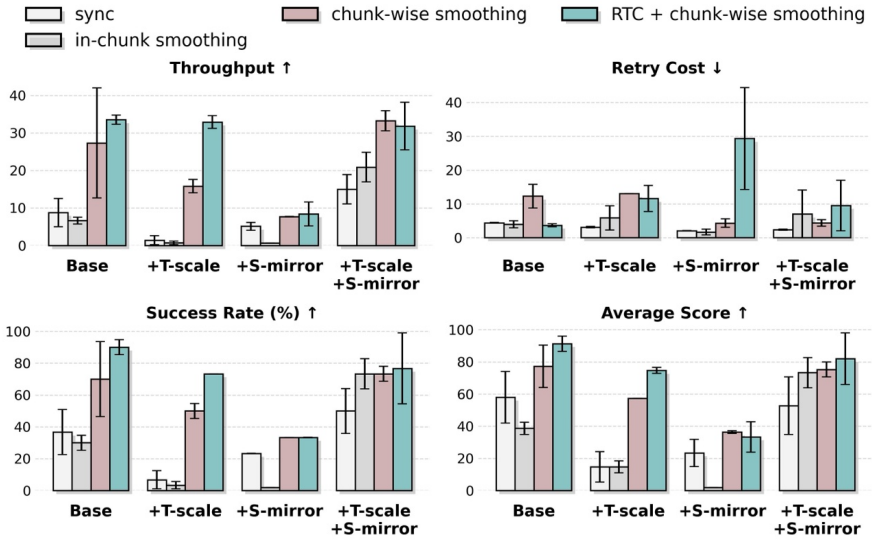
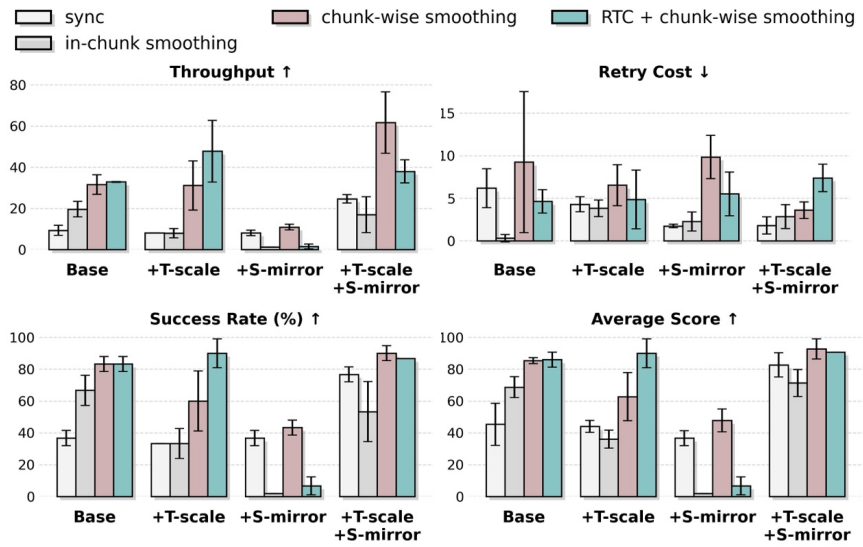


Fig. 5.15 **Stage Advantage on Task C** (co-author module, Chapter 5; 30 rollouts). SA improves MSTD, SFR, and retries ("Recover Cost"), whereas Direct+Stage trails in completion, throughput, and score.

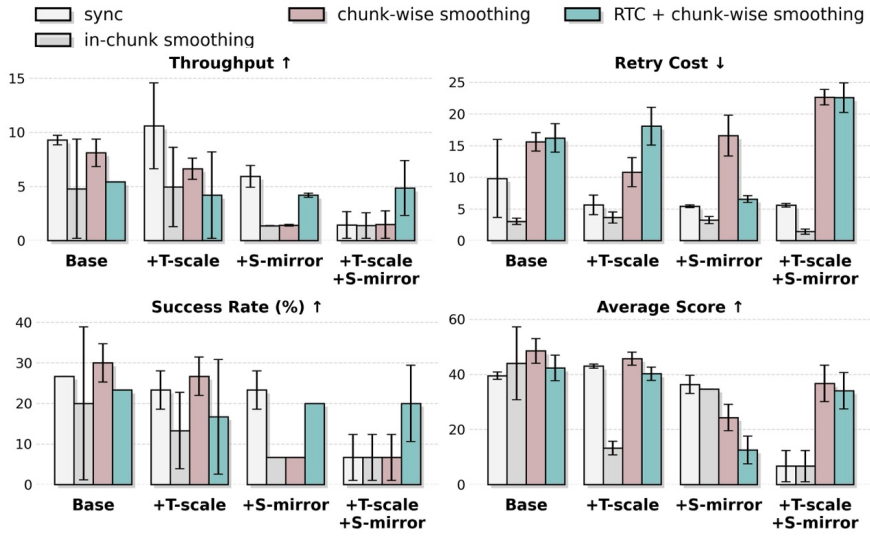


(a) Absolute targets

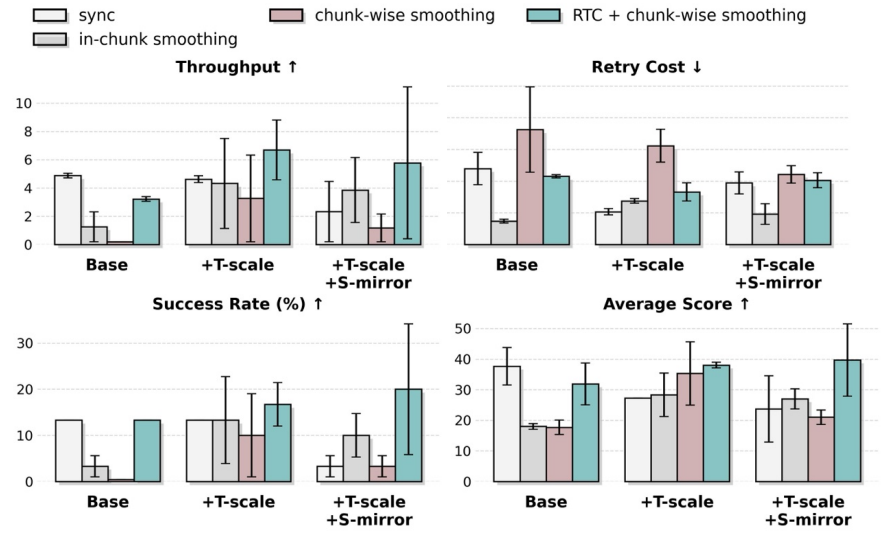


(b) Delta increments

Fig. 5.16 **Task A control and augmentation across joint spaces** (30 rollouts per bar). Chunk-wise smoothing leads in nearly all regimes for absolute (a) and delta (b) joints; RTC helps in 3 of 4 absolute regimes versus 1 delta regime.

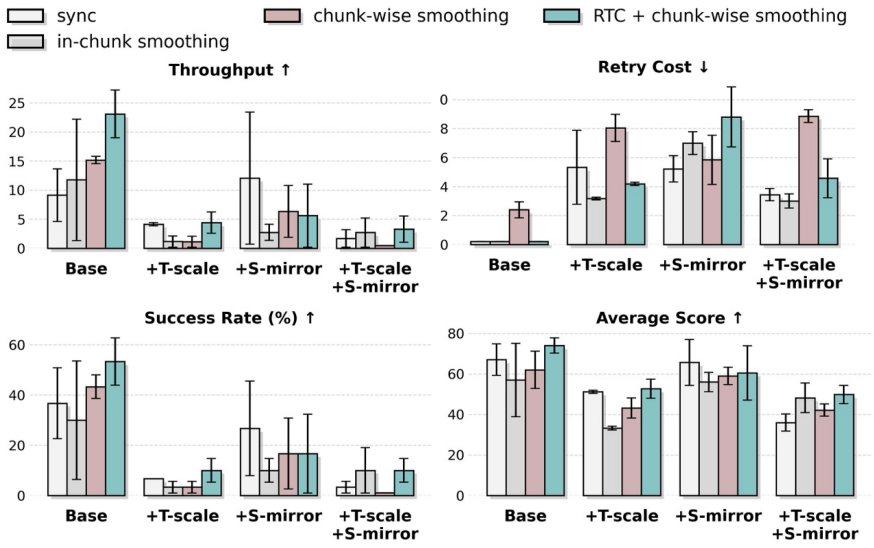


(a) Absolute targets

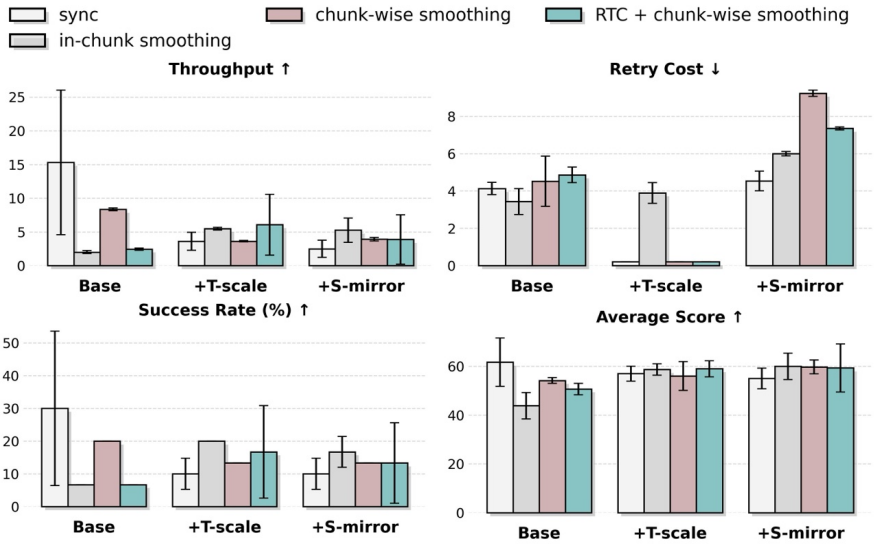


(b) Delta increments

Fig. 5.17 **Task B control and augmentation** (30 rollouts per bar). Delta joints trail absolute joints, completion stays near or below 30%, and chunk-wise smoothing gives no consistent margin.



(a) Absolute targets



(b) Delta increments

Fig. 5.18 **Task C control and augmentation** (30 rollouts per bar). Tight hanger-insertion tolerances keep completion modest and sensitive to joint parameterisation.

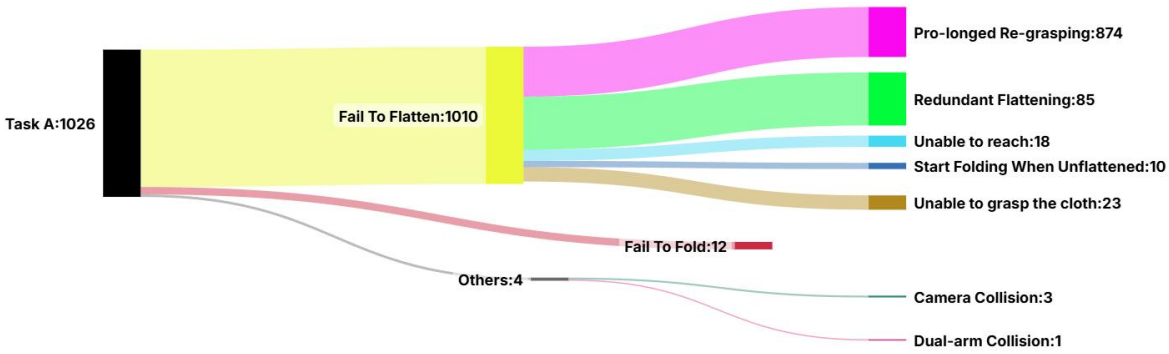


Fig. 5.19 **Task A failure breakdown.** Distribution of 1026 failure events pooled across tested policies; flattening errors dominate (1010 instances), led by prolonged re-grasping (874).

5.10 Chapter Summary

Addressing RQ3(b), this chapter investigated how bimanual manipulation policies can maintain closed-loop reliability under practical data and compute limits when deployment conditions diverge from offline training. Organising the learning loop around the demonstration distribution P_{train}, the policy-induced trajectory distribution Q_{model}, and the physical rollout distribution P_{test}, χ_0 pairs each pairwise mismatch with a dedicated module: Model Arithmetic interpolates subset-trained checkpoints using merge weights tuned on OOD DAgger validation rollouts; Stage Advantage (contributed by a co-author) supplies stage-conditioned progress labels for advantage-weighted behaviour cloning; and Train–Deploy Alignment combines heuristic DAgger recovery collection, spatiotemporal augmentation, and run-time temporal chunk-wise smoothing.

In the eight-way factorial study on Task A using the dual-station bimanual platform, χ_0 lifted the 30-trial completion rate of $\pi_{0.5}$ from roughly 27% to 97% (8 vs. 29 successes, an about 260% relative gain and +70 percentage points alongside a roughly elevenfold increase in hourly throughput; Table 5.2), driven primarily by targeted DAgger recovery data together with complementary multi-module interactions across MA, SA, and run-time smoothing+RTC control. Taken together with the autonomous driving study in Chapter 4, where Centaur adapts online at test time via uncertainty-guided updates in non-reactive log replay, χ_0 shows how routing physical rollout feedback back into data collection, checkpoint selection, and run-time control closes the loop between deployment behaviour and policy learning. Chapter 6.2 synthesises these findings into broader design principles for self-correcting embodied systems.

Chapter 6

Conclusion

6.1 Summary of Contributions

This dissertation has investigated how physical agents can be made robust to real-world conditions absent from their training distributions, a challenge we termed the curse of reality, across the perceive–reason–act loop. The research was organised around three progressive stages and four core technical contributions, with every claim evaluated at an explicit level of the evaluation-fidelity ladder (L0 open-loop log evaluation, L1 non-reactive short-horizon rollout simulation, L2 reactive closed-loop simulation, and L3 physical robot deployment).

Stage 1: Perceiving 3D Scenes as Volumetric Occupancy.

The first stage confronted the representational bottleneck of 3D bounding boxes and flat bird's-eye-view grids, which presuppose rigid cuboids or planar surfaces and struggle to represent articulated vehicles, overhanging loads, or arbitrary 3D obstacles.

OccNet, OpenOcc, and OpenScene (Chapter 2). Stage 1 replaced cuboid and planar scene abstractions with camera-predicted 3D semantic occupancy. OccNet's cascade decoder reaches 26.98 mIoU and 41.08 IoU_{geo} on OpenOcc (and 60.46 mIoU on camera-based nuScenes LiDAR segmentation), while the OpenOcc and OpenScene releases provide dense nuScenes supervision and more than 120 hours of compact nuPlan logs, respectively. The representation better fits the labelled geometry of ir-

regular instances than voxelised boxes (widening from 2.21 to 3.51 mIoU as voxel size tightens from 0.50 m to 0.25 m) and supports transfer to detection and segmentation. Its planning evidence is much weaker: the same-model open-loop comparison changes collision rates by only 0.06–0.24 percentage points in single runs, and two NAVSIM occupancy mechanisms (Chapter 4) do not improve PDMS. RQ1 is therefore answered at the level of geometric representation, not universal planning benefit or open-world semantics; the vocabulary remains closed at 16 classes and no distribution-shift experiment was run.

Stage 2: Structured Reasoning with Vision-Language Models.

Because 3D geometry specifies what occupies space but not why a traffic configuration matters or how the ego vehicle should respond, the second stage investigated how pre-trained vision-language models (VLMs) can be grounded in driving scenes to bridge perception, prediction, and planning.

DriveLM (Chapter 3). Stage 2 introduced Graph Visual Question Answering (GVQA), two DriveLM datasets (4,871 nuScenes keyframes and ∼1.6 million CARLA QA pairs, hosting 152 teams in the CVPR 2024 Challenge), and the DriveLM-Agent baseline to make perception-to-planning dependencies explicit. In zero-shot nuScenes-to-Waymo transfer, full graph context reaches 2.63 m ADE and raises speed accuracy from 43.90% to 54.29%. The result does not isolate graph reasoning: a waypoint-only BLIP-2 model already reaches 2.78 m (and context-free DriveLM-Agent 2.76 m) against 4.16 m for UniAD-Single, and the comparison is confounded by a large parameter gap (3.955 billion versus 131.9 million) and asymmetric camera inputs. Graph context also worsens in-domain behaviour accuracy (61.45% to 57.49%), ADE (1.39 m to 1.74 m), and collision rate (1.67% to 1.89%). On withheld CARLA pedestrians, recovery requires a class-specific prompt (lifting accuracy from 4.59% to 27.04% vs. 59.63% in regular scenes) and remains far from reliable. RQ2 therefore supports structured language as an inspectable interface with selective transfer benefits, while leaving visual grounding, real-time inference (0.16 FPS), and causal reasoning unresolved.

Stage 3: Reliable Action via Test-Time Adaptation and Train–Deploy Alignment.

The third stage addressed RQ3 at the action level, investigating how learned policies can maintain reliability when deployment conditions diverge from offline training in both autonomous driving and physical robot manipulation.

Centaur (Chapter 4) addressed RQ3(a) by adapting trajectory-scoring end-to-end driving planners online via a single gradient step of test-time training (TTT) minimising Cluster Entropy, a label-free uncertainty measure over five lateral driving-direction clusters, alongside the 229-frame, ten-category `navsafe` diagnostic subset of `navtest`. Evaluated in non-reactive L1 NAVSIM v1 simulation on logged frames (where per-frame resets prevent compounding errors), Centaur raised single-seed Hydra-MDP from 90.3 to 92.6 PDMS on v1.0 and achieved 92.10 ± 0.33 PDMS across three seeds on the official NAVSIM v1.1 leaderboard (ranking first among three-seed entries in March 2025). All four TTT objectives on Hydra-MDP gain 1.2–2.3 PDMS (0.8–2.3 PDMS across base planners, or 0.4 with SVD gradient projection) whereas uncertainty-triggered fallback to prior trajectories degraded all planners to 59.9–67.4 PDMS (63.1–65.9 PDMS on Hydra-MDP). However, the three-seed margin over standard semantic entropy is small (92.10 ± 0.33 vs. 91.87 ± 0.19, 0.23 PDMS) and lacks a matched three-seed unadapted baseline; standalone failure flagging accuracy (73.6% at 62.8% recall) trails a trivial never-flag baseline (84.4%); and on `navsafe` (selected using base-planner scores rather than held out), Centaur improved frame-weighted mean PDMS from 49.4 to 68.9 but regressed on yielding (59.3 vs. 71.6) due to lateral-only clustering.

χ_0 **(Chapter 5)** addressed RQ3(b) at L3 fidelity on physical bimanual robots by formalising long-horizon manipulation failures as distributional inconsistencies across human demonstrations P_{train}, policy-induced trajectory distribution Q_{model}, and hardware-executed trajectories P_{test}. Combining Model Arithmetic (weight-space merging across subset checkpoints; author contribution), Stage Advantage (stage-progress advantage estimation; co-author component), and Train–Deploy Alignment

(targeted DAgger recovery collection, spatiotemporal augmentation, heuristic chunk smoothing, and real-time chunking; author contribution) under a bounded post-training budget (∼20 hours of base demonstrations per task, or 22.5 to 33.6 hours including targeted DAgger recovery rollouts, on eight A100 GPUs), χ_0 lifted 30-trial real-robot T-shirt flattening and folding (Task A) completion for $\pi_{0.5}$ from ∼27% to ∼97% (8/30 vs. 29/30 successes, Fisher exact $p \approx 1.5 \times 10^{-8}$; Wilson 95% confidence intervals 14–44% vs. 83–99%) and increased hourly throughput roughly elevenfold ($\approx$ 6 to $\approx$ 66 tasks/h) across long-horizon cyclic rollouts, with targeted DAgger recovery data providing the largest single gain.

6.2 Unified Perspective: Towards Self-Correcting Physical Agents

Looking across the perceive–reason–act loop, the individual chapters point toward a unified architectural perspective (RQ4). Each stage of this dissertation exposed a characteristic boundary of static offline learning: a geometry-open occupancy grid (Chapter 2) represents arbitrary 3D obstacle shapes but does not interpret traffic intent; a vision-language reasoning graph (Chapter 3) transfers semantic priors across camera rigs and unseen categories when properly prompted, yet runs at 0.16 FPS and can rely on language shortcuts [16]; per-frame test-time planner training (Chapter 4) refines trajectory scoring without labels at L1, but relies on frozen perception features and lateral-only uncertainty; and offline train–deploy alignment (Chapter 5) achieves high closed-loop reliability on real robots at L3, but still depends on human teleoperators to supply recovery trajectories.

Fig. 6.1 synthesises these complementary strengths into a five-element architecture for *self-correcting physical agents*, in which perception, reasoning, and action monitor and correct one another across deployment timescales. Because no integrated end-to-end self-correcting agent was constructed in a single experiment, Table 6.1 grades the empirical status and weakest link of each element across the dissertation:

Table 6.1 **Evidence grading for the five elements of a self-correcting physical agent (Fig. 6.1),** distinguishing author-led chapter contributions from co-authored works.

Element	Evidence	Source	Fidelity	Status	Weakest link
Open, queryable representation	occupancy expresses irregular shapes of known classes and slightly lowered open-loop collision rates (Ch. 2); planning-oriented design (UniAD [3])	led; co-authored	L0	hypothesised (related capability: open geometry)	planning gain of 0.06–0.24 percentage points in single runs, not statistically established; closed semantics; no benefit shown at L1
Fast and slow decisions	reasoning helps under a sensing shift but is slow (Ch. 3); asynchronous large and small models (ETA [34]); latent distillation (FLARE [35])	led; co-authored	L0; L2 (ETA)	hypothesised	no two-speed system is built in the core chapters; DriveLM-Agent runs at 0.16 frames per second
Competence monitor	label-free behaviour-mode uncertainty (Cluster Entropy, Ch. 4); supervised stage progress used in training (Stage Advantage, co-author component of Ch. 5)	led	L1; L3	weak	flags failures less accurately than never flagging (73.6% against 84.4%); no AUROC; stage labels required
Adaptation at deployment	test-time training of the planner (Ch. 4)	led	L1	demonstrated on logged frames in non-reactive simulation	errors cannot compound; single-run headline without a matched three-seed baseline; compared only with a fallback to prior trajectories
Learning loop	recovery data and train–deploy alignment (Ch. 5); benchmark iterations (Ch. 1); self-directed learning [38]	led; co-authored	L3; –	partial; agent-level loop hypothesised	recovery data are collected by humans; no agent collects and learns from its own failures

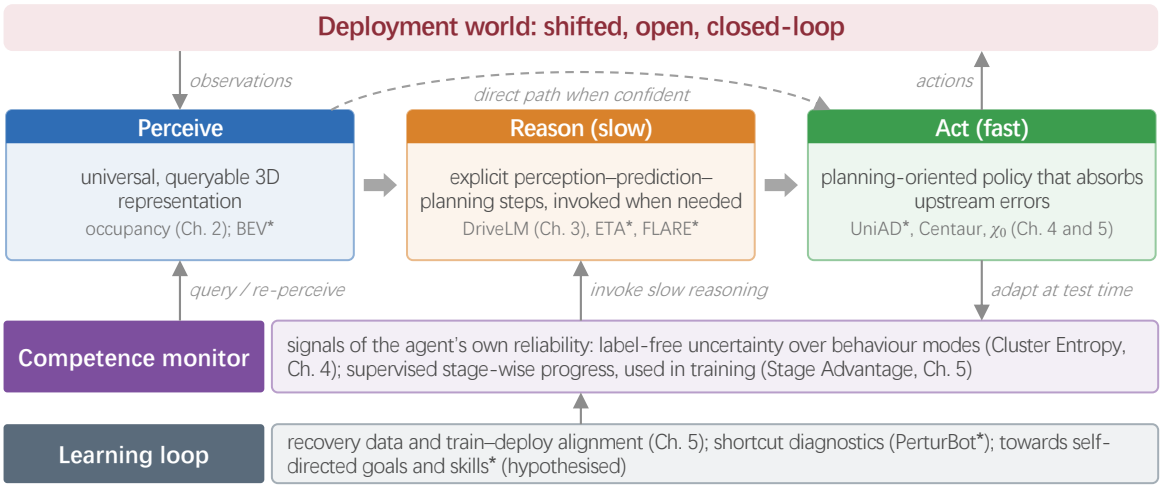


Fig. 6.1 Proposed anatomy of a self-correcting physical agent synthesising the perceive–reason–act contributions of this dissertation (* denotes co-authored works; BEV refers to BEVFormer [1]). Evidence for each architectural element comes from L0/L1 driving evaluations or offline-aligned L3 robot trials and is graded in Table 6.1.

1. An open, queryable scene representation. Downstream reasoning and control modules should query a 3D representation that does not presuppose which physical geometries matter (Chapter 2), trained end-to-end toward planning so upstream perception errors can be absorbed downstream as in the co-authored UniAD [3]. Chapter 2 establishes geometry-open 3D occupancy at L0 across multiple tasks, but its semantic vocabulary remains closed at 16 classes, its open-loop nuScenes planning gain over boxes is small (0.06–0.24 percentage points), and dynamic re-querying triggered by a runtime monitor has not yet been tested.

2. Fast and slow decision pathways. Because explicit VLM graph reasoning improved speed decisions under zero-shot sensor shift in Chapter 3 but operates at 0.16 FPS, a physical agent must decouple high-frequency reactive control from slow deliberative reasoning. As explored in the co-authored ETA [34] (which runs a slow VLM asynchronously ahead of a fast planner in L2 CARLA simulation, lifting closed-loop driving score from 64.22 at 931 ms to 69.53 at 50 ms) and FLARE [35] (which aligns planner latents with future-aware VLM representations during training without

test-time text generation), pairing fast visuomotor execution with selective or distilled VLM deliberation reconciles semantic generalisation with real-time control rates.

3. An online competence monitor. Self-correction requires an internal signal indicating when the fast policy is uncertain or stalls. Chapter 4 introduced Cluster Entropy over directional driving modes as a label-free test-time uncertainty signal, while Chapter 5 introduced supervised Stage Advantage (by co-author L. Zhao) to estimate fine-grained sub-task progress during training. However, standalone online failure detection remains the weakest link in Table 6.1: at a fixed threshold on `navtest`, Cluster Entropy flags 62.8% of failures but achieves 73.6% overall accuracy compared with 84.4% for a trivial never-flag rule due to the 15.6% base failure rate, and Stage Advantage requires stage annotations. Developing a well-calibrated, threshold-free competence monitor that arbitrates between fast execution, slow VLM reasoning, and test-time adaptation remains open.

4. Online adaptation at deployment. When the competence monitor detects out-of-distribution ambiguity, the agent must adjust its behaviour without waiting for offline retraining. Chapter 4 demonstrated at L1 fidelity that single-step gradient updates on Cluster Entropy improve trajectory scoring across multiple planner backbones, outperforming both an uncertainty-triggered fallback to prior trajectories and UniAD's occupancy cost post-optimisation. Extending online parameter adaptation from per-frame resets to multi-step closed-loop episodes without weight drift or category-specific regressions (such as the yielding drop on `navsafe`) is the next step toward deployment-time self-correction.

5. A continual learning loop that outlives deployment. Finally, residual failures that cannot be resolved by online adaptation should flow back into targeted recovery data, weight-space model merging, execution alignment, and updated diagnostic benchmarks. Across research projects, this loop took the form of our benchmark-driven cycle (where OpenOcc led to OpenScene and NAVSIM, and DriveLM led to

DriveBench [16] and FLARE [35]); within a single robot deployment, Chapter 5 closed the loop through human-collected DAgger recovery trajectories, Model Arithmetic, and Train–Deploy Alignment. Replacing human teleoperators in that loop with autonomous failure recovery and skill practice is the central thesis of our co-authored self-directed learning perspective [38].

6.3 Limitations and Future Directions

The conclusions of this dissertation are bounded by several methodological and empirical limitations that directly motivate future research:

Limitations. Four limits bound the thesis-wide conclusions. First, *driving evidence stops at logged-frame evaluation or non-reactive rollout*: unlike the L3 physical robot experiments in Chapter 5, the L0 (Chapters 2 and 3) and L1 (Chapter 4) driving protocols cannot reveal compounding control errors, reactive traffic responses, or multi-step parameter drift. Second, *the shift studies contain important confounds*: Chapter 2 tests unusual geometry within known classes without distribution shift, Chapter 3's Waymo comparison mixes model scale with camera asymmetry, its CARLA pedestrian result uses an oracle-guided class-specific query, and `navsafe` in Chapter 4 is selected with the base planner's scores rather than held out. Third, *statistical and resource precision is uneven*: most driving results are single-run, several multi-seed margins (such as 0.23 PDMS between Cluster Entropy and semantic entropy over three seeds) are small, and the 30-trial robot results are reconstructed from plotted summaries without a matched-budget data-scaling study or complete operator and compute accounting. Finally, *the proposed safeguards remain incomplete*: OpenOcc is closed-vocabulary (16 classes), language models can exploit textual priors rather than causal visual evidence [16, 37], and unsupervised test-time weight updates have neither formal safety bounds nor uniform gains across scenario categories such as yielding.

Overcoming these limitations points to four primary directions for future work:

Open, predictive 4D world models. Future scene representations should extend 3D occupancy from a closed 16-class taxonomy to open-vocabulary 3D semantics via vision-language distillation [87, 327] and self-supervised temporal rendering [86], while scaling 4D occupancy forecasting on OpenScene [29, 90]. Crucially, predictive occupancy world models should be evaluated by whether they improve closed-loop planning under geometric and environmental shift in reactive simulation (L2) rather than open-loop L0 displacement metrics alone.

Grounded, adaptive fast–slow reasoning. To eliminate linguistic shortcuts in VLM and VLA policies [16, 37], future reasoning benchmarks and training pipelines should enforce class-balanced action distributions, blind and ego-status-only input ablations, and causal perturbation checks verifying that corrupting intermediate perception or prediction answers alters downstream actions. Architecturally, slow VLM reasoning should be invoked selectively by a calibrated competence monitor or distilled into fast latent policy representations [34, 35] to satisfy onboard latency budgets.

Lifelong closed-loop test-time adaptation with safety bounds. Test-time planner training (Chapter 4) should be extended from episodic single-frame updates in non-reactive NAVSIM v1 to continuous multi-step adaptation in reactive closed-loop simulators and 3D Gaussian-splatting pseudo-simulation (NAVSIM v2 [241]). Incorporating longitudinal speed modes alongside lateral steering clusters in Cluster Entropy, pairing online weight updates with scene-aware safety filters, and establishing formal bounds on multi-step parameter drift will be essential to prevent category-specific regressions such as yielding failures.

Self-directed robot learning and matched-budget scaling laws. In embodied manipulation, future work should systematically compare distributional alignment (χ_0) against raw demonstration scaling [5] at equal total robot, operator, and compute budgets. More fundamentally, moving beyond human teleoperation and human-triggered DAgger interventions requires robots that autonomously detect execution

failures, propose sub-goals, practice recovery skills in the physical world, and evaluate their own progress via learned value functions [38].

No finite training dataset will ever record every obstacle geometry, traffic interaction, sensor configuration, or deformable object state that the physical world presents. Rather than treating out-of-distribution failures as edge cases to be solved by offline dataset scale alone, this dissertation has argued for designing and evaluating physical agents around the inevitability of deployment shift: representing 3D geometry without rigid cuboids, structuring multimodal reasoning so intermediate steps can be inspected and grounded, adapting planner predictions to test-time uncertainty, and aligning demonstration data, policy weights, and hardware execution across the training–deployment loop. Having taken concrete steps across open-loop logs, non-reactive rollouts, and physical robots, closing the full perceive–reason–act self-correction loop in lifelong real-world deployment remains the central frontier ahead.

Appendix A

Supplementary Material

A.1 LLM Prompts and Question Templates for DriveLM

We present the structured prompting templates, coordinate tokenisation rules, and evaluation prompts used in Chapter 3 for Graph Visual Question Answering (GVQA) across DriveLM-nuScenes, DriveLM-CARLA, and DriveLM-Agent. Each frame graph $G = (V, E)$ organises question–answer pairs across five sequential reasoning stages: perception (P_1), prediction (P_2), planning (P_3), behaviour (B), and motion (M).

A.1.1 Graph VQA Question Templates Across Reasoning Stages

In DriveLM-nuScenes and DriveLM-CARLA, every object-centric node $v \in V$ references a key object via a grounded identifier tag $\langle c, \mathit{cam}, x, y \rangle$, where $c \in \{\texttt{c1}, \ldots, \texttt{c6}\}$ indexes the key objects in the frame, *cam* specifies one of the six surround-view cameras (`CAM_FRONT`, `CAM_FRONT_LEFT`, `CAM_FRONT_RIGHT`, `CAM_BACK`, `CAM_BACK_LEFT`, or `CAM_BACK_RIGHT`) in which the object's 2D bounding-box centre appears, and (x, y) denotes the pixel coordinates of the 2D box centre scaled to a 1600×900 image plane.

Below we list the canonical question templates and answer formats for each stage of the reasoning graph:

```
================================================================================
Stage P1: Perception (Object Identification, Status, and Scene Attributes)
================================================================================
Q1 (Key-object identification):
  "What are the important objects in the current scene? Those objects will be
   considered for the future reasoning and driving decision."
A1 (Grounded object list):
  "There is a <category> <c_i, CAM_VIEW, x_i, y_i> [and a <category>
   <c_j, CAM_VIEW, x_j, y_j> ...]."

Q2 (Object visual description and motion state):
  "What is the moving status of object <c_i, CAM_VIEW, x_i, y_i>? Please select
   from [stopped, moving, parked,jaywalking, signalling, ...]."
A2 (Visual and kinematic state):
  "<c_i, CAM_VIEW, x_i, y_i> is a <colour/type> <category> that is <status>
   [in the <lane_position> / ahead of the ego vehicle]."

Q3 (Traffic signal and road-rule perception):
  "What is the status of the traffic light <c_k, CAM_FRONT, x_k, y_k> controlling
   the ego vehicle's lane?"
A3 (Signal phase):
  "The traffic light <c_k, CAM_FRONT, x_k, y_k> is <red / green / yellow>."

================================================================================
Stage P2: Prediction (Future Motion and Multi-Object Interaction)
================================================================================
Q4 (Object future trajectory and intention):
  "Where might object <c_i, CAM_VIEW, x_i, y_i> move in the next few seconds,
   and what is its likely intention?"
A4 (Predicted motion):
  "<c_i, CAM_VIEW, x_i, y_i> will <continue straight / turn left / turn right /
   remain stationary / cross the crosswalk> in the <lane_region>."
```

```
Q5 (Pairwise object--ego or object--object interaction):
  "Would object <c_i, CAM_VIEW, x_i, y_i> affect the trajectory of object
   <c_j, CAM_VIEW, x_j, y_j> or the ego vehicle?"
A5 (Interaction rationale):
  "Yes, because <c_i, CAM_VIEW, x_i, y_i> is <merging into the ego lane /
   crossing ahead of the ego vehicle / stopped in the path of c_j>."

================================================================================
Stage P3: Planning (Action Selection, Object Prioritisation, and Risk Check)
================================================================================
Q6 (High-level driving action rationale):
  "What is the next driving action of the ego vehicle given the surrounding
   traffic, and why?"
A6 (Goal-conditioned planning decision):
  "The ego vehicle should <keep going at the same speed / decelerate slowly /
   brake to a stop / steer slightly left to yield to c_i>."

Q7 (Action-conditioned collision risk verification):
  "If the ego vehicle executes [<candidate_action>], would it risk colliding
   with object <c_i, CAM_VIEW, x_i, y_i>?"
A7 (Counterfactual safety assessment):
  "<Yes / No>, if the ego vehicle [<candidate_action>], it <would / would not>
   collide with <c_i, CAM_VIEW, x_i, y_i> because <spatial_reason>."

================================================================================
Stage B: Behaviour Classification (Discretised Speed and Steering Bins)
================================================================================
Q8 (Discretised speed-and-steering decision):
  "Predict the behaviour of the ego vehicle. The speed options are:
   [stopping, decelerating to stop, decelerating Slowly, decelerating fast,
    keeping speed, accelerating slowly, accelerating fast].
   The steering options are:
   [going straight, turning slightly left, turning left, turning slightly right,
    turning right, changing to the left lane, changing to the right lane]."
A8 (Joint behaviour tuple):
```

```
  "The ego vehicle is <B_sp> and <B_st>."

================================================================================
Stage M: Motion Planning (Tokenised 3-Second Ego Waypoint Sequence)
================================================================================
Q9 (Conditioned trajectory rollout):
  "Given the behaviour [<B_sp>, <B_st>], predict the future 3-second waypoints
   of the ego vehicle at 0.5-second intervals."
A9 (Quantised 256-bin coordinate tokens between <SOT> and <EOT>):
  "<SOT> <x_1> <y_1> <x_2> <y_2> <x_3> <y_3>
         <x_4> <y_4> <x_5> <y_5> <x_6> <y_6> <EOT>"
```

A.1.2 GPT Score Evaluation Prompt and Autoregressive Context Formatting

Below we record (i) the autoregressive graph-context prompt constructed by DriveLM-Agent when querying child nodes along directed edges $e = (u, v) \in E$, (ii) the rule-based cascade prompts used for zero-shot transfer to Waymo Open Dataset and unseen pedestrian categories in DriveLM-CARLA, and (iii) the exact evaluation prompt sent to `gpt-3.5-turbo` when computing the GPT Score metric in Table 3.8.

```
================================================================================
1. Autoregressive Graph-Context Formatting in DriveLM-Agent
================================================================================
Input prompt to BLIP-2 / Flanagan-T5-XL or LLaMA-Adapter V2 for node v:
  "Context:
   Q: <Question(u_1)> A: <Predicted_or_GT_Answer(u_1)>
   ...
   Q: <Question(u_k)> A: <Predicted_or_GT_Answer(u_k)>
   Question: <Question(v)>
   Answer:"
(During training, teacher forcing supplies ground-truth parent answers A(u_i);
 during inference, predicted parent answers are substituted along edges in E.)
```

```
================================================================================
2. Zero-Shot Cross-Domain and Unseen-Object Cascade Prompts
================================================================================
Waymo Open Dataset zero-shot cascade (Section 3.6.3):
  Step 1 (P1): "What are the objects in the front view?"
  Step 2 (P2, triggered if Step 1 detects 'car', 'truck', or 'pedestrian'):
    "What is the moving status of the <detected_class> in the front view?"
  Step 3 (P3):
    "What should the ego vehicle do given the <detected_class> in the front view?"
  Step 4 (B):
    Append (Step 1, Step 2, Step 3) QAs as context to the Stage B query.

DriveLM-CARLA unseen-pedestrian cascade (Section 3.6.4):
  Step 1 (Presence check):
    "Is there a person in the scene?"
  Step 2a (If Step 1 answer is 'Yes'):
    "What should the vehicle do based on the pedestrian that is crossing the road?"
  Step 2b (If Step 1 answer is 'No'):
    "What should the ego vehicle do?"
  Step 3 (B):
    Append the resulting QA pair to the behaviour classification context.

================================================================================
3. GPT Score Evaluation Prompt (gpt-3.5-turbo)
================================================================================
messages = [
    {
        "role": "system",
        "content": "An evaluator who rates my answer based on the correct answer."
    },
    {
        "role": "user",
        "content": (
            "Rate my answer based on the correct answer out of 100, with higher "
            "scores indicating that the answer is closer to the correct answer, "
```

```
            "and you should be accurate to single digits like 62, 78, 41, etc. "
            f"This is the correct answer: {GT}. This is my answer: {Pred}."
        )
    }
]
```

References

[1] Zhiqi Li, Wenhai Wang, Hongyang Li, Enze Xie, Chonghao Sima, Tong Lu, Yu Qiao, and Jifeng Dai. BEVFormer: Learning Bird's-Eye-View Representation from Multi-Camera Images via Spatiotemporal Transformers. In *European Conference on Computer Vision (ECCV)*, pages 1–18, 2022. doi: 10.1007/978-3-031-20077-9_1.

[2] Hongyang Li, Chonghao Sima, Jifeng Dai, Wenhai Wang, Lewei Lu, Huijie Wang, Jia Zeng, Zhiqi Li, Jiazhi Yang, Hanming Deng, Hao Tian, Enze Xie, Jiangwei Xie, Li Chen, Tianyu Li, Yang Li, Yulu Gao, Xiaosong Jia, Si Liu, Jianping Shi, Dahua Lin, and Yu Qiao. Delving Into the Devils of Bird's-Eye-View Perception: A Review, Evaluation and Recipe. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(4):2151–2170, 2024. doi: 10.1109/TPAMI.2023.3333838. Published online in 2023.

[3] Yihan Hu, Jiazhi Yang, Li Chen, Keyu Li, Chonghao Sima, Xizhou Zhu, Siqi Chai, Senyao Du, Tianwei Lin, Wenhai Wang, Lewei Lu, Xiaosong Jia, Qiang Liu, Jifeng Dai, Yu Qiao, and Hongyang Li. Planning-oriented Autonomous Driving. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 17853–17862, 2023.

[4] Chonghao Sima, Katrin Renz, Kashyap Chitta, Li Chen, Hanxue Zhang, Chengen Xie, Jens Beißwenger, Ping Luo, Andreas Geiger, and Hongyang Li. DriveLM: Driving with Graph Visual Question Answering. In *European Conference on Computer Vision (ECCV)*, volume 15110 of *Lecture Notes in Computer Science*, pages 256–274, 2024. doi: 10.1007/978-3-031-72943-0_15.

[5] Qingwen Bu, Jisong Cai, Li Chen, Xiuqi Cui, Yan Ding, Siyuan Feng, Xindong He, Xu Huang, et al. AgiBot World Colosseo: A Large-Scale Manipulation Platform for Scalable and Intelligent Embodied Systems. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025. doi: 10.1109/IROS60139.2025.11247088.

[6] Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, et al. $\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization. In *Proceedings of the 9th Conference on Robot Learning (CoRL)*, 2025. arXiv:2504.16054.

[7] Stéphane Ross and Drew Bagnell. Efficient Reductions for Imitation Learning. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 9 of *Proceedings of Machine Learning Research*, pages 661–668, 2010.

[8] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. In *Proceedings of the*

International Conference on Artificial Intelligence and Statistics (AISTATS), pages 627–635, 2011.

[9] Henry X. Liu and Shuo Feng. Curse of rarity for autonomous vehicles. *Nature Communications*, 15:4808, 2024.

[10] Yulu Gao, Chonghao Sima, Shaoshuai Shi, Shangzhe Di, Si Liu, and Hongyang Li. Sparse Dense Fusion for 3D Object Detection. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 10939–10946, 2023. doi: 10.1109/IROS55552.2023.10341620.

[11] Holger Caesar, Varun Bankiti, Alex H. Lang, Sourabh Vora, Venice Erin Liong, Qiang Xu, Anush Krishnan, Yu Pan, Giancarlo Baldan, and Oscar Beijbom. nuScenes: A Multimodal Dataset for Autonomous Driving. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

[12] Pei Sun, Henrik Kretzschmar, Xerxes Dotiwalla, Aurelien Chouard, Vijaysai Patnaik, Paul Tsui, James Guo, Yin Zhou, Yuning Chai, Benjamin Caine, et al. Scalability in Perception for Autonomous Driving: Waymo Open Dataset. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

[13] Wenwen Tong, Chonghao Sima, Tai Wang, Li Chen, Silei Wu, Hanming Deng, Yi Gu, Lewei Lu, Ping Luo, Dahua Lin, and Hongyang Li. Scene as Occupancy. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 8406–8415, 2023. doi: 10.1109/ICCV51070.2023.00772.

[14] Li Chen, Chonghao Sima, Yang Li, Zehan Zheng, Jiajie Xu, Xiangwei Geng, Hongyang Li, Conghui He, Jianping Shi, Yu Qiao, and Junchi Yan. PersFormer: 3D Lane Detection via Perspective Transformer and the OpenLane Benchmark. In *European Conference on Computer Vision (ECCV)*, volume 13698 of *Lecture Notes in Computer Science*, pages 550–567, 2022. doi: 10.1007/978-3-031-19839-7_32.

[15] Huijie Wang, Tianyu Li, Yang Li, Li Chen, Chonghao Sima, Zhenbo Liu, Bangjun Wang, Peijin Jia, Yuting Wang, Shengyin Jiang, Feng Wen, Hang Xu, Ping Luo, Junchi Yan, Wei Zhang, and Hongyang Li. OpenLane-V2: A Topology Reasoning Benchmark for Unified 3D HD Mapping. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, volume 36, pages 18873–18884, 2023.

[16] Shaoyuan Xie, Lingdong Kong, Yuhao Dong, Chonghao Sima, Wenwei Zhang, Qi Alfred Chen, Ziwei Liu, and Liang Pan. Are VLMs Ready for Autonomous Driving? An Empirical Study from the Reliability, Data, and Metric Perspectives. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 6585–6597, 2025.

[17] Chonghao Sima, Kashyap Chitta, Zhiding Yu, Shiyi Lan, Ping Luo, Andreas Geiger, Hongyang Li, and Jose M. Alvarez. Centaur: Robust End-to-End Autonomous Driving with Test-Time Training. *arXiv preprint arXiv:2503.11650*, 2025.

[18] Checheng Yu, Chonghao Sima, Gangcheng Jiang, Hai Zhang, Haoguang Mai, Hongyang Li, Huijie Wang, Jin Chen, Kaiyang Wu, Li Chen, Lirui Zhao, Modi Shi, Ping Luo, Qingwen Bu, Shijia Peng, Tianyu Li, and Yibo Yuan. χ_0: Resource-Aware Robust Manipulation via Taming Distributional Inconsistencies. *arXiv preprint arXiv:2602.09021*, 2026.

[19] Herbert A. Simon. *The Sciences of the Artificial*. MIT Press, Cambridge, MA, 3rd edition, 1996.

[20] Alan R. Hevner, Salvatore T. March, Jinsoo Park, and Sudha Ram. Design Science in Information Systems Research. *MIS Quarterly*, 28(1):75–105, 2004.

[21] Jiang-Tian Zhai, Ze Feng, Jinhao Du, Yongqiang Mao, Jiang-Jiang Liu, Zichang Tan, Yifu Zhang, Xiaoqing Ye, and Jingdong Wang. Rethinking the Open-Loop Evaluation of End-to-End Autonomous Driving in nuScenes. *arXiv preprint arXiv:2305.10430*, 2023.

[22] Zhiqi Li, Zhiding Yu, Shiyi Lan, Jiahan Li, Jan Kautz, Tong Lu, and Jose M. Alvarez. Is Ego Status All You Need for Open-Loop End-to-End Autonomous Driving? In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[23] Daniel Dauner, Marcel Hallgarten, Tianyu Li, Xinshuo Weng, Zhiyu Huang, Zetong Yang, Hongyang Li, Igor Gilitschenski, Boris Ivanovic, Marco Pavone, Andreas Geiger, and Kashyap Chitta. NAVSIM: Data-Driven Non-Reactive Autonomous Vehicle Simulation and Benchmarking. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[24] Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio Lopez, and Vladlen Koltun. CARLA: An Open Urban Driving Simulator. In *Conference on Robot Learning (CoRL)*, 2017.

[25] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C. Courville, and Marc G. Bellemare. Deep Reinforcement Learning at the Edge of the Statistical Precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 29304–29320, 2021.

[26] Lawrence D. Brown, T. Tony Cai, and Anirban DasGupta. Interval Estimation for a Binomial Proportion. *Statistical Science*, 16(2):101–133, 2001. doi: 10.1214/ss/1009213286.

[27] Hadas Kress-Gazit, Kunimatsu Hashimoto, Naveen Kuppuswamy, Paarth Shah, Phoebe Horgan, Gordon Richardson, Siyuan Feng, and Benjamin Burchfiel. Robot Learning as an Empirical Science: Best Practices for Policy Evaluation. *arXiv preprint arXiv:2409.09491*, 2024.

[28] Yiming Li, Zhiding Yu, Christopher Choy, Chaowei Xiao, Jose M Alvarez, Sanja Fidler, Chen Feng, and Anima Anandkumar. VoxFormer: Sparse Voxel Transformer for Camera-based 3D Semantic Scene Completion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9087–9098, 2023. doi: 10.1109/CVPR52729.2023.00877.

[29] OpenScene Contributors. OpenScene: The Largest Up-to-Date 3D Occupancy Prediction Benchmark in Autonomous Driving. `https://github.com/OpenDriveLab/OpenScene`, 2023.

[30] Yuanhui Huang, Wenzhao Zheng, Yunpeng Zhang, Jie Zhou, and Jiwen Lu. Tri-Perspective View for Vision-Based 3D Semantic Occupancy Prediction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9223–9232, 2023. doi: 10.1109/CVPR52729.2023.00890.

[31] J. Beißwenger. PDM-Lite: A Rule-Based Planner for CARLA Leaderboard 2.0. `https://github.com/OpenDriveLab/DriveLM/blob/DriveLM-CARLA/pdm_lite/docs/report.pdf`, 2024.

[32] Linyan Huang, Zhiqi Li, Chonghao Sima, Wenhai Wang, Jingdong Wang, Yu Qiao, and Hongyang Li. Leveraging Vision-Centric Multi-Modal Expertise for 3D Object Detection. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 38504–38519, 2023.

[33] Kairui Ding, Boyuan Chen, Yuchen Su, Huan-ang Gao, Bu Jin, Chonghao Sima, Wuqiang Zhang, Xiaohui Li, Paul Barsch, Hongyang Li, and Hao Zhao. Hint-AD: Holistically Aligned Interpretability in End-to-End Autonomous Driving. In *Proceedings of the 8th Conference on Robot Learning (CoRL 2024)*, volume 270 of *Proceedings of Machine Learning Research*, pages 3742–3765. PMLR, 2025. Conference held in 2024.

[34] Shadi Hamdan, Chonghao Sima, Zetong Yang, Hongyang Li, and Fatma Güney. ETA: Efficiency Through Thinking Ahead, A Dual Approach to Self-Driving with Large Models. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 26529–26538, 2025.

[35] Chengen Xie, Chonghao Sima, Tianyu Li, Bin Sun, Junjie Wu, Zhihui Hao, and Hongyang Li. FLARE: Learning Future-Aware Latent Representations from Vision-Language Models for Autonomous Driving. *arXiv preprint arXiv:2601.05611*, 2026.

[36] Chonghao Sima and Yexiang Xue. LSH-SMILE: Locality Sensitive Hashing Accelerated Simulation and Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 7484–7496, 2021.

[37] Mingyu Liu, Chonghao Sima, Tianjian Feng, Hanqing Wang, Cong Chen, Hao Chen, and Chunhua Shen. PerturBot: Breaking Shortcut Priors in Vision-Language-Action Models with Perturbative Training. *arXiv preprint arXiv:2610.04616*, 2026.

[38] Li Chen, Chonghao Sima, Kashyap Chitta, Antonio Loquercio, Ping Luo, Yi Ma, and Hongyang Li. Intelligent Robot Manipulation Requires Self-Directed Learning. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 842–853, 2026. Preprint on Authorea, 2025.

[39] Lars Mescheder, Michael Oechsle, Michael Niemeyer, Sebastian Nowozin, and Andreas Geiger. Occupancy Networks: Learning 3D Reconstruction in Function Space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4455–4465, 2019. doi: 10.1109/CVPR.2019.00459.

[40] Whye Kit Fong, Rohit Mohan, Juana Valeria Hurtado, Lubing Zhou, Holger Caesar, Oscar Beijbom, and Abhinav Valada. Panoptic nuScenes: A Large-Scale Benchmark for LiDAR Panoptic Segmentation and Tracking. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2022.

[41] Xizhou Zhu, Weijie Su, Lewei Lu, Bin Li, Xiaogang Wang, and Jifeng Dai. Deformable DETR: Deformable Transformers for End-to-End Object Detection. In *International Conference on Learning Representations (ICLR)*, 2021.

[42] Mobileye. CES 2020: An Hour with Amnon – Autonomous Vehicles Powered by Mobileye. `https://youtu.be/HPWGFzqd7pI`, 2020.

[43] Tesla. Tesla AI Day. `https://www.youtube.com/watch?v=j0z4FweCy4M`, 2021.

[44] Zetong Yang, Li Chen, Yanan Sun, and Hongyang Li. Visual Point Cloud Forecasting Enables Scalable Autonomous Driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[45] Julien Moras, J. Dezert, and Benjamin Pannetier. Grid Occupancy Estimation for Environment Perception Based on Belief Functions and PCR6. In *Signal Processing, Sensor/Information Fusion, and Target Recognition XXIV, Proceedings of SPIE*, volume 9474, April 2015. doi: 10.1117/12.2177653.

[46] Shuran Song, Fisher Yu, Andy Zeng, Angel X. Chang, Manolis Savva, and Thomas Funkhouser. Semantic Scene Completion from a Single Depth Image. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 190–198, 2017. doi: 10.1109/CVPR.2017.28.

[47] Anh-Quan Cao and Raoul de Charette. MonoScene: Monocular 3D Semantic Scene Completion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3981–3991, 2022. doi: 10.1109/CVPR52688.2022.00396.

[48] Ruihang Miao, Weizhou Liu, Mingrui Chen, Zheng Gong, Weixin Xu, Chen Hu, and Shuchang Zhou. OccDepth: A Depth-Aware Method for 3D Semantic Scene Completion. *arXiv preprint arXiv:2302.13540*, 2023.

[49] Tai Wang, Jiangmiao Pang, and Dahua Lin. Monocular 3D Object Detection with Depth from Motion. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 386–403. Springer, 2022.

[50] Andres Milioto, Ignacio Vizzo, Jens Behley, and Cyrill Stachniss. RangeNet++: Fast and Accurate LiDAR Semantic Segmentation. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 4213–4220. IEEE, 2019.

[51] Tai Wang, Xinge Zhu, Jiangmiao Pang, and Dahua Lin. FCOS3D: Fully Convolutional One-Stage Monocular 3D Object Detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, pages 913–922, 2021. doi: 10.1109/ICCVW54120.2021.00107.

[52] Shaoshuai Shi, Chaoxu Guo, Li Jiang, Zhe Wang, Jianping Shi, Xiaogang Wang, and Hongsheng Li. PV-RCNN: Point-Voxel Feature Set Abstraction for 3D Object Detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10529–10538, 2020.

[53] Zhijian Liu, Haotian Tang, Alexander Amini, Xinyu Yang, Huizi Mao, Daniela Rus, and Song Han. BEVFusion: Multi-Task Multi-Sensor Fusion with Unified Bird's-Eye View Representation. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 2774–2781, 2023. doi: 10.1109/ICRA48891.2023.10160968.

[54] Xinge Zhu, Hui Zhou, Tai Wang, Fangzhou Hong, Yuexin Ma, Wei Li, Hongsheng Li, and Dahua Lin. Cylindrical and Asymmetrical 3D Convolution Networks for LiDAR Segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9939–9948, 2021.

[55] Brady Zhou and Philipp Krähenbühl. Cross-view Transformers for Real-time Map-view Semantic Segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 13760–13769, 2022.

[56] Qi Li, Yue Wang, Yilun Wang, and Hang Zhao. HDMapNet: An Online HD Map Construction and Evaluation Framework. In *Proceedings of the International*

Conference on Robotics and Automation (ICRA), pages 4628–4634, 2022. doi: 10.1109/ICRA46639.2022.9812383.

[57] R. I. Hartley and A. Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, ISBN: 0521540518, second edition, 2004.

[58] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2020.

[59] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant Neural Graphics Primitives with a Multiresolution Hash Encoding. *ACM Transactions on Graphics*, 41(4):102:1–102:15, July 2022. doi: 10.1145/3528223.3530127.

[60] Anh-Quan Cao and Raoul de Charette. SceneRF: Self-Supervised Monocular 3D Scene Reconstruction with Radiance Fields. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9353–9364, 2023. doi: 10.1109/ICCV51070.2023.00861. arXiv:2212.02501 (2022).

[61] Matthew Tancik, Vincent Casser, Xinchen Yan, Sabeek Pradhan, Ben Mildenhall, Pratul P. Srinivasan, Jonathan T. Barron, and Henrik Kretzschmar. Block-NeRF: Scalable Large Scene Neural View Synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8248–8258, June 2022.

[62] Xiaofeng Wang, Zheng Zhu, Wenbo Xu, Yunpeng Zhang, Yi Wei, Xu Chi, Yun Ye, Dalong Du, Jiwen Lu, and Xingang Wang. OpenOccupancy: A Large Scale Benchmark for Surrounding Semantic Occupancy Perception. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 17804–17813, 2023. doi: 10.1109/ICCV51070.2023.01636.

[63] Jens Behley, Martin Garbade, Andres Milioto, Jan Quenzel, Sven Behnke, Cyrill Stachniss, and Juergen Gall. SemanticKITTI: A Dataset for Semantic Scene Understanding of LiDAR Sequences. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019. doi: 10.1109/ICCV.2019.00939.

[64] Nathan Silberman, Derek Hoiem, Pushmeet Kohli, and Rob Fergus. Indoor Segmentation and Support Inference from RGBD Images. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2012.

[65] Junjie Huang and Guan Huang. BEVDet4D: Exploit Temporal Cues in Multi-camera 3D Object Detection. *arXiv preprint arXiv:2203.17054*, 2022.

[66] Shengchao Hu, Li Chen, Penghao Wu, Hongyang Li, Junchi Yan, and Dacheng Tao. ST-P3: End-to-End Vision-Based Autonomous Driving via Spatial-Temporal Feature Learning. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 533–549, 2022. doi: 10.1007/978-3-031-19839-7_31.

[67] Tingting Liang, Hongwei Xie, Kaicheng Yu, Zhongyu Xia, Zhiwei Lin, Yongtao Wang, Tao Tang, Bing Wang, and Zhi Tang. BEVFusion: A Simple and Robust LiDAR-Camera Fusion Framework. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 10421–10434, 2022.

[68] Yin Zhou and Oncel Tuzel. VoxelNet: End-to-End Learning for Point Cloud Based 3D Object Detection. In *Proceedings of the IEEE Conference on Computer Vision and*

Pattern Recognition (CVPR), pages 4490–4499, 2018.

[69] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal Loss for Dense Object Detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988, 2017.

[70] Xiaoyu Tian, Tao Jiang, Longfei Yun, Yucheng Mao, Huitong Yang, Yue Wang, Yilun Wang, and Hang Zhao. Occ3D: A Large-Scale 3D Occupancy Prediction Benchmark for Autonomous Driving. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[71] Yi Wei, Linqing Zhao, Wenzhao Zheng, Zheng Zhu, Jie Zhou, and Jiwen Lu. SurroundOcc: Multi-Camera 3D Occupancy Prediction for Autonomous Driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 21672–21683, 2023. doi: 10.1109/ICCV51070.2023.01986.

[72] Ming Fang and Zhiqi Li. occupancy-for-nuscenes. `https://github.com/FANG-MING/occupancy-for-nuscenes`, 2023.

[73] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[74] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A Large-Scale Hierarchical Image Database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 248–255, 2009.

[75] Haisong Liu, Yang Chen, Haiguang Wang, Zetong Yang, Tianyu Li, Jia Zeng, Li Chen, Hongyang Li, and Limin Wang. Fully Sparse 3D Occupancy Prediction. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 54–71, 2024. doi: 10.1007/978-3-031-72698-9_4.

[76] Yinhao Li, Zheng Ge, Guanyi Yu, Jinrong Yang, Zengran Wang, Yukang Shi, Jianjian Sun, and Zeming Li. BEVDepth: Acquisition of Reliable Depth for Multi-view 3D Object Detection. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2023.

[77] Junjie Huang, Guan Huang, Zheng Zhu, Yun Ye, and Dalong Du. BEVDet: High-performance Multi-camera 3D Object Detection in Bird-Eye-View. *arXiv preprint arXiv:2112.11790*, 2021.

[78] Zhiqi Li, Zhiding Yu, David Austin, Mingsheng Fang, Shiyi Lan, Jan Kautz, and Jose M. Alvarez. FB-OCC: 3D Occupancy Prediction Based on Forward-Backward View Transformation. In *CVPR 2023 Workshop on End-to-End Autonomous Driving*, 2023. Winning technical report of the 3D Occupancy Prediction Challenge; arXiv:2307.01492.

[79] Zichen Yu, Changyong Shu, Jiajun Deng, Kangjie Lu, Zongdai Liu, Jiangyong Yu, Dawei Yang, Hui Li, and Yan Chen. FlashOcc: Fast and Memory-Efficient Occupancy Prediction via Channel-to-Height Plugin. *arXiv preprint arXiv:2311.12058*, 2023.

[80] Yuqi Wang, Yuntao Chen, Xingyu Liao, Lue Fan, and Zhaoxiang Zhang. PanoOcc: Unified Occupancy Representation for Camera-based 3D Panoptic Segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 17158–17168, 2024.

[81] Pin Tang, Zhongdao Wang, Guoqing Wang, Jilai Zheng, Xiangxuan Ren, Bailan Feng, and Chao Ma. SparseOcc: Rethinking Sparse Latent Representation for Vision-Based Semantic Occupancy Prediction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15035–15044, 2024. doi: 10.1109/CVPR52733.2024.01424.

[82] Jiabao Wang, Zhaojiang Liu, Qiang Meng, Liujiang Yan, Ke Wang, Jie Yang, Wei Liu, Qibin Hou, and Ming-Ming Cheng. OPUS: Occupancy Prediction Using a Sparse Set. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 119861–119885, 2024.

[83] Yuanhui Huang, Wenzhao Zheng, Yunpeng Zhang, Jie Zhou, and Jiwen Lu. GaussianFormer: Scene as Gaussians for Vision-Based 3D Semantic Occupancy Prediction. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 376–393, 2024. doi: 10.1007/978-3-031-73383-3_22.

[84] Yuanhui Huang, Amonnut Thammatadatrakoon, Wenzhao Zheng, Yunpeng Zhang, Dalong Du, and Jiwen Lu. GaussianFormer-2: Probabilistic Gaussian Superposition for Efficient 3D Occupancy Prediction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 27477–27486, 2025. doi: 10.1109/CVPR52734.2025.02559.

[85] Mingjie Pan, Jiaming Liu, Renrui Zhang, Peixiang Huang, Xiaoqi Li, Hongwei Xie, Bing Wang, Li Liu, and Shanghang Zhang. RenderOcc: Vision-Centric 3D Occupancy Prediction with 2D Rendering Supervision. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 12404–12411, 2024. doi: 10.1109/ICRA57147.2024.10611537.

[86] Yuanhui Huang, Wenzhao Zheng, Borui Zhang, Jie Zhou, and Jiwen Lu. SelfOcc: Self-Supervised Vision-Based 3D Occupancy Prediction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19946–19956, 2024. doi: 10.1109/CVPR52733.2024.01885.

[87] Antonin Vobecky, Oriane Siméoni, David Hurych, Spyros Gidaris, Andrei Bursuc, Patrick Pérez, and Josef Sivic. POP-3D: Open-Vocabulary 3D Occupancy Prediction from Images. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 50545–50557, 2023.

[88] Jilai Zheng, Pin Tang, Zhongdao Wang, Guoqing Wang, Xiangxuan Ren, Bailan Feng, and Chao Ma. VEON: Vocabulary-Enhanced Occupancy Prediction. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 92–108, 2024. doi: 10.1007/978-3-031-72949-2_6.

[89] Junyi Ma, Xieyuanli Chen, Jiawei Huang, Jingyi Xu, Zhen Luo, Jintao Xu, Weihao Gu, Rui Ai, and Hesheng Wang. Cam4DOcc: Benchmark for Camera-Only 4D Occupancy Forecasting in Autonomous Driving Applications. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 21486–21495, 2024. doi: 10.1109/CVPR52733.2024.02030.

[90] Wenzhao Zheng, Weiliang Chen, Yuanhui Huang, Borui Zhang, Yueqi Duan, and Jiwen Lu. OccWorld: Learning a 3D Occupancy World Model for Autonomous Driving. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 55–72, 2024. doi: 10.1007/978-3-031-72624-8_4.

[91] Lening Wang, Wenzhao Zheng, Yilong Ren, Han Jiang, Zhiyong Cui, Haiyang Yu, and Jiwen Lu. OccSora: 4D Occupancy Generation Models as World Simulators for Autonomous Driving. *IEEE Transactions on Image Processing*, 35:4947–4960, 2026. doi: 10.1109/TIP.2026.3687468.

[92] Bohan Li, Jiazhe Guo, Hongsi Liu, Yingshuang Zou, Yikang Ding, Xiwu Chen, Hu Zhu, Feiyang Tan, Chi Zhang, Tiancai Wang, Shuchang Zhou, Li Zhang, Xiaojuan Qi, Hao Zhao, Mu Yang, Wenjun Zeng, and Xin Jin. UniScene: Unified Occupancy-centric Driving Scene Generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11971–11981, 2025. doi: 10.1109/CVPR52734.2025.01118.

[93] Honghui Yang, Sha Zhang, Di Huang, Xiaoyang Wu, Haoyi Zhu, Tong He, Shixiang Tang, Hengshuang Zhao, Qibo Qiu, Binbin Lin, Xiaofei He, and Wanli Ouyang. UniPAD: A Universal Pre-Training Paradigm for Autonomous Driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15238–15250, 2024. doi: 10.1109/CVPR52733.2024.01443.

[94] Xiangchao Yan, Runjian Chen, Bo Zhang, Hancheng Ye, Renqiu Xia, Jiakang Yuan, Hongbin Zhou, Xinyu Cai, Botian Shi, Wenqi Shao, Ping Luo, Yu Qiao, Tao Chen, and Junchi Yan. SPOT: Scalable 3D Pre-Training via Occupancy Prediction for Learning Transferable 3D Representations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 47:9609–9625, 2025. doi: 10.1109/TPAMI.2025.3586961.

[95] Holger Caesar, Juraj Kabzan, Kok Seang Tan, Whye Kit Fong, Eric Wolff, Alex Lang, Luke Fletcher, Oscar Beijbom, and Sammy Omari. nuPlan: A Closed-Loop ML-Based Planning Benchmark for Autonomous Vehicles. In *CVPR 2021 ADP3 Workshop*, 2021. Also available as arXiv:2106.11810.

[96] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature Pyramid Networks for Object Detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2117–2125, 2017.

[97] Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[98] Li Chen, Penghao Wu, Kashyap Chitta, Bernhard Jaeger, Andreas Geiger, and Hongyang Li. End-to-End Autonomous Driving: Challenges and Frontiers. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12):10164–10183, 2024. doi: 10.1109/TPAMI.2024.3435937.

[99] P. S. Chib and P. Singh. Recent Advancements in End-to-End Autonomous Driving Using Deep Learning: A Survey. *IEEE Transactions on Intelligent Vehicles*, 2023.

[100] Shahin Atakishiyev, Mohammad Salameh, Housam Babiker, and Randy Goebel. Explaining Autonomous Driving Actions with Visual Question Answering. In *Proceedings of the IEEE International Conference on Intelligent Transportation Systems (ITSC)*, pages 1207–1214, 2023. doi: 10.1109/ITSC57777.2023.10421901.

[101] Kashyap Chitta, Aditya Prakash, Bernhard Jaeger, Zehao Yu, Katrin Renz, and Andreas Geiger. TransFuser: Imitation with Transformer-Based Sensor Fusion for Autonomous Driving. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(11):12878–12895, 2023. doi: 10.1109/TPAMI.2022.3200245.

[102] D. Marr. *Vision: A Computational Investigation into the Human Representation and Processing of Visual Information*. The MIT Press, 2010.

[103] E. S. Spelke and K. D. Kinzler. Core Knowledge. *Developmental Science*, 2007.

[104] J. Li, D. Li, S. Savarese, and S. Hoi. BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2023.

[105] Renrui Zhang, Jiaming Han, Chris Liu, Aojun Zhou, Pan Lu, Yu Qiao, Hongsheng Li, and Peng Gao. LLaMA-Adapter: Efficient Fine-tuning of Large Language Models with Zero-initialized Attention. In *International Conference on Learning Representations (ICLR)*, 2024.

[106] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual Instruction Tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[107] Wenhai Wang, Zhe Chen, Xiaokang Chen, Jiannan Wu, Xizhou Zhu, Gang Zeng, Ping Luo, Tong Lu, Jie Zhou, Yu Qiao, and Jifeng Dai. VisionLLM: Large Language Model Is Also an Open-Ended Decoder for Vision-Centric Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 61501–61513, 2023.

[108] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. In *Proceedings of the 7th Conference on Robot Learning (CoRL)*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183, 2023.

[109] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, et al. PaLM-E: An Embodied Multimodal Language Model. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2023.

[110] K. Renz, K. Chitta, O. B. Mercea, A. S. Koepke, Z. Akata, and A. Geiger. PlanT: Explainable Planning Transformers via Object-Level Representations. In *Proceedings of the Conference on Robot Learning (CoRL)*, 2022.

[111] Daniel Dauner, Marcel Hallgarten, Andreas Geiger, and Kashyap Chitta. Parting with Misconceptions about Learning-based Vehicle Motion Planning. In *Proceedings of the 7th Conference on Robot Learning (CoRL)*, volume 229 of *Proceedings of Machine Learning Research*, pages 1268–1281, 2023.

[112] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[113] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *International Conference on Learning Representations (ICLR)*, 2023.

[114] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 11809–11822, 2023.

[115] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Ny-

czyk, and Torsten Hoefler. Graph of Thoughts: Solving Elaborate Problems with Large Language Models. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, volume 38, pages 17682–17690, 2024. doi: 10.1609/aaai.v38i16.29720. arXiv:2308.09687 (2023).

[116] Ruomeng Ding, Chaoyun Zhang, Lu Wang, Yong Xu, Minghua Ma, Wei Zhang, Si Qin, Saravan Rajmohan, Qingwei Lin, and Dongmei Zhang. Everything of Thoughts: Defying the Law of Penrose Triangle for Thought Generation. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 1638–1662, 2024. doi: 10.18653/v1/2024.findings-acl.95. arXiv:2311.04254 (2023).

[117] Jinkyu Kim, Anna Rohrbach, Trevor Darrell, John Canny, and Zeynep Akata. Textual Explanations for Self-Driving Vehicles. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 577–593, 2018. doi: 10.1007/978-3-030-01216-8_35.

[118] J. Kim, T. Misu, Y. T. Chen, A. Tawari, and J. Canny. Grounding Human-to-Vehicle Advice for Self-Driving Vehicles. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[119] Enna Sachdeva, Nakul Agarwal, Suhas Chundi, Sean Roelofs, Jiachen Li, Mykel Kochenderfer, Chiho Choi, and Behzad Dariush. Rank2Tell: A Multimodal Driving Dataset for Joint Importance Ranking and Reasoning. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 7498–7507, 2024. doi: 10.1109/WACV57701.2024.00734. arXiv:2309.06597 (2023).

[120] S. Malla, C. Choi, I. Dwivedi, J. H. Choi, and J. Li. DRAMA: Joint Risk Localization and Captioning in Driving. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2023.

[121] Tianwen Qian, Jingjing Chen, Linhai Zhuo, Yang Jiao, and Yu-Gang Jiang. NuScenes-QA: A Multi-Modal Visual Question Answering Benchmark for Autonomous Driving Scenario. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2024.

[122] S. Teng, X. Hu, P. Deng, B. Li, Y. Li, Y. Ai, D. Yang, L. Li, Z. Xuanyuan, F. Zhu, et al. Motion Planning for Autonomous Driving: The State of the Art and Future Perspectives. *IEEE Transactions on Intelligent Vehicles*, 2023.

[123] A. Tampuu, T. Matiisen, M. Semikin, D. Fishman, and N. Muhammad. A Survey of End-to-End Driving: Architectures and Training Methods. *IEEE Transactions on Neural Networks and Learning Systems*, 2020.

[124] S. Suo, S. Regalado, S. Casas, and R. Urtasun. TrafficSim: Learning to Simulate Realistic Multi-Agent Behaviors. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

[125] D. Chen and P. Krähenbühl. Learning from All Vehicles. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[126] S. Akhauri, L. Y. Zheng, and M. C. Lin. Enhanced Transfer Learning for Autonomous Driving with Systematic Accident Simulation. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.

[127] J. Wang, A. Pun, J. Tu, S. Manivasagam, A. Sadat, S. Casas, M. Ren, and R. Urtasun. AdvSim: Generating Safety-Critical Scenarios for Self-Driving Vehicles. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*,

2021.

[128] N. Hanselmann, K. Renz, K. Chitta, A. Bhattacharyya, and A. Geiger. KING: Generating Safety-Critical Driving Scenarios for Robust Imitation via Kinematics Gradients. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2022.

[129] A. Elhafsi, R. Sinha, C. Agia, E. Schmerling, I. A. Nesnas, and M. Pavone. Semantic Anomaly Detection with Large Language Models. *Autonomous Robots*, 2023.

[130] Tsun-Hsuan Wang, Alaa Maalouf, Wei Xiao, Yutong Ban, Alexander Amini, Guy Rosman, Sertac Karaman, and Daniela Rus. Drive Anywhere: Generalizable End-to-End Autonomous Driving with Multi-Modal Foundation Models. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 6687–6694, 2024. doi: 10.1109/ICRA57147.2024.10611590. arXiv:2310.17642 (2023).

[131] Yuxiao Chen, Sander Tonkens, and Marco Pavone. Categorical Traffic Transformer: Interpretable and Diverse Behavior Prediction with Tokenized Latent. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 2423–2430, 2025. doi: 10.1109/ICRA55743.2025.11128234. arXiv:2311.18307 (2023).

[132] Chenbin Pan, Burhaneddin Yaman, Tommaso Nesti, Abhirup Mallik, Alessandro G. Allievi, Senem Velipasalar, and Liu Ren. VLP: Vision Language Planning for Autonomous Driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14760–14769, 2024. doi: 10.1109/CVPR52733.2024.01398.

[133] A. Seff, B. Cera, D. Chen, M. Ng, A. Zhou, N. Nayakanti, K. S. Refaat, R. Al-Rfou, and B. Sapp. MotionLM: Multi-Agent Motion Forecasting as Language Modeling. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[134] Jiageng Mao, Yuxi Qian, Junjie Ye, Hang Zhao, and Yue Wang. GPT-Driver: Learning to Drive with GPT. In *NeurIPS 2023 Workshop on Foundation Models for Decision Making*, 2023. arXiv:2310.01415.

[135] Long Chen, Oleg Sinavski, Jan Hünermann, Alice Karnsund, Andrew James Willmott, Danny Birch, Daniel Maund, and Jamie Shotton. Driving with LLMs: Fusing Object-Level Vector Modality for Explainable Autonomous Driving. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 14093–14100, 2024. doi: 10.1109/ICRA57147.2024.10611018.

[136] B. Jin, X. Liu, Y. Zheng, P. Li, H. Zhao, T. Zhang, Y. Zheng, G. Zhou, and J. Liu. ADAPT: Action-Aware Driving Caption Transformer. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2023.

[137] Zhenhua Xu, Yujia Zhang, Enze Xie, Zhen Zhao, Yong Guo, Kwan-Yee K. Wong, Zhenguo Li, and Hengshuang Zhao. DriveGPT4: Interpretable End-to-End Autonomous Driving via Large Language Model. *IEEE Robotics and Automation Letters*, 9:8186–8193, 2024. doi: 10.1109/LRA.2024.3440097.

[138] Wayve. LINGO-1: Exploring Natural Language for Autonomous Driving. `https://wayve.ai/thinking/lingo-natural-language-autonomous-driving/`, 2023.

[139] Ana-Maria Marcu, Long Chen, Jan Hünermann, Alice Karnsund, Benoit Hanotte, Prajwal Chidananda, Saurabh Nair, Vijay Badrinarayanan, Alex Kendall, Jamie Shotton, Elahe Arani, and Oleg Sinavski. LingoQA: Visual Question Answering for Autonomous

Driving. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 252–269, 2024. doi: 10.1007/978-3-031-72980-5_15.

[140] Erfei Cui, Wenhai Wang, Zhiqi Li, Jiangwei Xie, Haoming Zou, Hanming Deng, Gen Luo, Lewei Lu, Xizhou Zhu, and Jifeng Dai. DriveMLM: Aligning Multi-Modal Large Language Models with Behavioral Planning States for Autonomous Driving. *Visual Intelligence*, 3(1):22, 2025. doi: 10.1007/s44267-025-00095-w. arXiv:2312.09245 (2023).

[141] J. Mao, J. Ye, Y. Qian, M. Pavone, and Y. Wang. A Language Agent for Autonomous Driving. In *Conference on Language Modeling (COLM)*, 2024.

[142] Xiaoyu Tian, Junru Gu, Bailin Li, Yicheng Liu, Yang Wang, Zhiyong Zhao, Kun Zhan, Peng Jia, Xianpeng Lang, and Hang Zhao. DriveVLM: The Convergence of Autonomous Driving and Large Vision-Language Models. In *Proceedings of the Conference on Robot Learning (CoRL)*, volume 270 of *Proceedings of Machine Learning Research*, pages 4698–4726, 2024.

[143] Hao Shao, Yuxuan Hu, Letian Wang, Steven L. Waslander, Yu Liu, and Hongsheng Li. LMDrive: Closed-Loop End-to-End Driving with Large Language Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15120–15130, 2024. doi: 10.1109/CVPR52733.2024.01432.

[144] Jianhao Yuan, Shuyang Sun, Daniel Omeiza, Bo Zhao, Paul Newman, Lars Kunze, and Matthew Gadd. RAG-Driver: Generalisable Driving Explanations with Retrieval-Augmented In-Context Multi-Modal Large Language Model Learning. In *Robotics: Science and Systems (RSS)*, 2024. doi: 10.15607/RSS.2024.XX.075.

[145] Wencheng Han, Dongqian Guo, Cheng-Zhong Xu, and Jianbing Shen. DME-Driver: Integrating Human Decision Logic and 3D Scene Perception in Autonomous Driving. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, volume 39, pages 3347–3355, 2025. doi: 10.1609/aaai.v39i3.32346. arXiv:2401.03641 (2024).

[146] Xingcheng Zhou, Mingyu Liu, Ekim Yurtsever, Bare Luka Zagar, Walter Zimmer, Hu Cao, and Alois C. Knoll. Vision Language Models in Autonomous Driving: A Survey and Outlook. *IEEE Transactions on Intelligent Vehicles*, pages 1–20, 2024. doi: 10.1109/TIV.2024.3402136.

[147] Tushar Choudhary, Vikrant Dewangan, Shivam Chandhok, Shubham Priyadarshan, Anushka Jain, Arun K. Singh, Siddharth Srivastava, Krishna Murthy Jatavallabhula, and K. Madhava Krishna. Talk2BEV: Language-Enhanced Bird's-Eye View Maps for Autonomous Driving. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 16345–16352, 2024. doi: 10.1109/ICRA57147.2024.10611485. arXiv:2310.02251 (2023).

[148] D. Wu, W. Han, T. Wang, X. Dong, X. Zhang, and J. Shen. Referring Multi-Object Tracking. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[149] Dongming Wu, Wencheng Han, Yingfei Liu, Tiancai Wang, Cheng-Zhong Xu, Xiangyu Zhang, and Jianbing Shen. Language Prompt for Autonomous Driving. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, volume 39, pages 8359–8367, 2025. doi: 10.1609/aaai.v39i8.32902.

[150] Xinpeng Ding, Jianhua Han, Hang Xu, Xiaodan Liang, Wei Zhang, and Xiaomeng Li. Holistic Autonomous Driving Understanding by Bird's-Eye-View Injected Multi-Modal Large Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and*

Pattern Recognition (CVPR), pages 13668–13677, 2024. doi: 10.1109/CVPR52733.2024.01297.

[151] H. Touvron, L. Martin, K. R. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288*, 2023.

[152] L. Floridi and M. Chiriatti. GPT-3: Its Nature, Scope, Limits, and Consequences. *Minds and Machines*, 2020.

[153] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa. Large Language Models Are Zero-Shot Reasoners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[154] B. Ichter, A. Brohan, Y. Chebotar, C. Finn, K. Hausman, A. Herzog, D. Ho, J. Ibarz, A. Irpan, et al. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. In *Proceedings of the Conference on Robot Learning (CoRL)*, 2022.

[155] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, et al. Inner Monologue: Embodied Reasoning through Planning with Language Models. In *Proceedings of the Conference on Robot Learning (CoRL)*, 2022.

[156] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng. Code as Policies: Language Model Programs for Embodied Control. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2023.

[157] S. Karamcheti, S. Nair, A. S. Chen, T. Kollar, C. Finn, D. Sadigh, and P. Liang. Language-Driven Representation Learning for Robotics. In *Robotics: Science and Systems (RSS)*, 2023.

[158] K. Rana, J. Haviland, S. Garg, J. Abou-Chakra, I. Reid, and N. Suenderhauf. SayPlan: Grounding Large Language Models Using 3D Scene Graphs for Scalable Robot Task Planning. In *Proceedings of the Conference on Robot Learning (CoRL)*, 2023.

[159] X. Chen, S. Jia, and Y. Xiang. A Review: Knowledge Reasoning over Knowledge Graph. *Expert Systems with Applications*, 2020.

[160] J. Shi, H. Zhang, and J. Li. Explainable and Explicit Visual Reasoning over Scene Graphs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[161] Yunpeng Chen, Marcus Rohrbach, Zhicheng Yan, Shuicheng Yan, Jiashi Feng, and Yannis Kalantidis. Graph-Based Global Reasoning Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 433–442, 2019. doi: 10.1109/CVPR.2019.00052.

[162] C. C. Macadam. Understanding and Modeling the Human Driver. *Vehicle System Dynamics*, 2003.

[163] J. A. Groeger. *Understanding Driving: Applying Cognitive Psychology to a Complex Everyday Task*. Routledge, 2013.

[164] P. Anderson, B. Fernando, M. Johnson, and S. Gould. SPICE: Semantic Propositional Image Caption Evaluation. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2016.

[165] Bernhard Jaeger, Kashyap Chitta, and Andreas Geiger. Hidden Biases of End-to-End Driving Models. In *Proceedings of the IEEE/CVF International Conference on*

Computer Vision (ICCV), pages 8206–8215, 2023. doi: 10.1109/ICCV51070.2023.00757.

[166] Martin Treiber, Ansgar Hennecke, and Dirk Helbing. Congested Traffic States in Empirical Observations and Microscopic Simulations. *Physical Review E*, 62(2):1805–1824, 2000. doi: 10.1103/PhysRevE.62.1805.

[167] Aditya Prakash, Kashyap Chitta, and Andreas Geiger. Multi-Modal Fusion Transformer for End-to-End Autonomous Driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

[168] Kashyap Chitta, Aditya Prakash, and Andreas Geiger. NEAT: Neural Attention Fields for End-to-End Autonomous Driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[169] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language Models Are Unsupervised Multitask Learners. *OpenAI blog*, 2019.

[170] Penghao Wu, Xiaosong Jia, Li Chen, Junchi Yan, Hongyang Li, and Yu Qiao. Trajectory-guided Control Prediction for End-to-end Autonomous Driving: A Simple yet Strong Baseline. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[171] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations (ICLR)*, 2022.

[172] T. Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO: Common Objects in Context. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2014.

[173] Drew A. Hudson and Christopher D. Manning. GQA: A New Dataset for Real-World Visual Reasoning and Compositional Question Answering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6693–6702, 2019. doi: 10.1109/CVPR.2019.00686.

[174] P. Gao, J. Han, R. Zhang, Z. Lin, S. Geng, A. Zhou, W. Zhang, P. Lu, C. He, X. Yue, H. Li, and Y. Qiao. LLaMA-Adapter V2: Parameter-Efficient Visual Instruction Model. *arXiv preprint arXiv:2304.15010*, 2023.

[175] Zhiliang Peng, Wenhui Wang, Li Dong, Yaru Hao, Shaohan Huang, Shuming Ma, Qixiang Ye, and Furu Wei. Grounding Multimodal Large Language Models to the World. In *International Conference on Learning Representations (ICLR)*, 2024.

[176] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.

[177] D. Shi, C. Tao, A. Rao, Z. Yang, C. Yuan, and J. Wang. CrossGET: Cross-Guided Ensemble of Tokens for Accelerating Vision-Language Transformers. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, pages 44960–44990, 2024.

[178] Tanmay Gupta and Aniruddha Kembhavi. Visual Programming: Compositional Visual Reasoning without Training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14953–14962, 2023. doi: 10.1109/CVPR52729.2023.01436.

[179] Dídac Surís, Sachit Menon, and Carl Vondrick. ViperGPT: Visual Inference via Python Execution for Reasoning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 11854–11864, 2023. doi: 10.1109/ICCV51070.2023.01092.

[180] Hao Shao, Shengju Qian, Han Xiao, Guanglu Song, Zhuofan Zong, Letian Wang, Yu Liu, and Hongsheng Li. Visual CoT: Advancing Multi-Modal Language Models with a Comprehensive Dataset and Benchmark for Chain-of-Thought Reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 8612–8642, 2024.

[181] Ming Nie, Renyuan Peng, Chunwei Wang, Xinyue Cai, Jianhua Han, Hang Xu, and Li Zhang. Reason2Drive: Towards Interpretable and Chain-based Reasoning for Autonomous Driving. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 292–308, 2024.

[182] Shihao Wang, Zhiding Yu, Xiaohui Jiang, Shiyi Lan, Min Shi, Nadine Chang, Jan Kautz, Ying Li, and Jose M. Alvarez. OmniDrive: A Holistic Vision-Language Dataset for Autonomous Driving with Counterfactual Reasoning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 22442–22452, 2025.

[183] Ayesha Ishaq, Jean Lahoud, Ketan More, Omkar Thawakar, Ritesh Thawkar, Dinura Dissanayake, Noor Ahsan, Yuhao Li, Fahad Shahbaz Khan, Hisham Cholakkal, Ivan Laptev, Rao Muhammad Anwer, and Salman Khan. DriveLMM-o1: A Step-by-Step Reasoning Dataset and Large Multimodal Model for Driving Scenario Understanding. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 20501–20508, 2025. doi: 10.1109/IROS60139.2025.11247360.

[184] Jyh-Jing Hwang, Runsheng Xu, Hubert Lin, Wei-Chih Hung, Jingwei Ji, Kristy Choi, Di Huang, Tong He, Paul Covington, Benjamin Sapp, Yin Zhou, James Guo, Dragomir Anguelov, and Mingxing Tan. EMMA: End-to-End Multimodal Model for Autonomous Driving. *Transactions on Machine Learning Research (TMLR)*, 2025.

[185] Zewei Zhou, Tianhui Cai, Seth Z. Zhao, Yun Zhang, Zhiyu Huang, Bolei Zhou, and Jiaqi Ma. AutoVLA: A Vision-Language-Action Model for End-to-End Autonomous Driving with Adaptive Reasoning and Reinforcement Fine-Tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[186] NVIDIA, Yan Wang, Wenjie Luo, Junjie Bai, Yulong Cao, Tong Che, Ke Chen, Yuxiao Chen, et al. Alpamayo-R1: Bridging Reasoning and Action Prediction for Generalizable Autonomous Driving in the Long Tail. *arXiv preprint arXiv:2511.00088*, 2025.

[187] Xiaosong Jia, Zhenjie Yang, Qifeng Li, Zhiyuan Zhang, and Junchi Yan. Bench2Drive: Towards Multi-Ability Benchmarking of Closed-Loop End-To-End Autonomous Driving. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[188] Haoyu Fu, Diankun Zhang, Zongchuang Zhao, Jianfeng Cui, Dingkang Liang, Chong Zhang, Dingyuan Zhang, Hongwei Xie, Bing Wang, and Xiang Bai. ORION: A Holistic End-to-End Autonomous Driving Framework by Vision-Language Instructed Action Generation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 24823–24834, 2025. doi: 10.1109/ICCV51701.2025.02302.

[189] Katrin Renz, Long Chen, Elahe Arani, and Oleg Sinavski. SimLingo: Vision-Only Closed-Loop Autonomous Driving with Language-Action Alignment. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11993–12003, 2025. doi: 10.1109/CVPR52734.2025.01120.

[190] Bo Jiang, Shaoyu Chen, Bencheng Liao, Xingyu Zhang, Wei Yin, Qian Zhang, Chang Huang, Wenyu Liu, and Xinggang Wang. Senna: Bridging Large Vision-Language Models and End-to-End Autonomous Driving. *International Journal of Computer Vision*, 134, 2026. doi: 10.1007/s11263-026-03002-y.

[191] Runsheng Xu, Hubert Lin, Wonseok Jeon, Hao Feng, Yuliang Zou, Liting Sun, John Gorman, Kate Tolstaya, Sarah Tang, Brandyn White, Ben Sapp, Mingxing Tan, Jyh-Jing Hwang, and Dragomir Anguelov. WOD-E2E: Waymo Open Dataset for End-to-End Driving in Challenging Long-tail Scenarios. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.

[192] K. Papineni, S. Roukos, T. Ward, and W. J. Zhu. BLEU: A Method for Automatic Evaluation of Machine Translation. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2002.

[193] C. Y. Lin. ROUGE: A Package for Automatic Evaluation of Summaries. In *ACL Workshop*, 2004.

[194] A. Lavie and A. Agarwal. METEOR: An Automatic Metric for MT Evaluation with High Levels of Correlation with Human Judgments. In *ACL Workshop*, 2007.

[195] Ramakrishna Vedantam, C. Lawrence Zitnick, and Devi Parikh. CIDEr: Consensus-Based Image Description Evaluation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4566–4575, 2015. doi: 10.1109/CVPR.2015.7299087.

[196] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, et al. Training Language Models to Follow Instructions with Human Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[197] OpenAI. OpenAI: Introducing ChatGPT. `https://openai.com/blog/chatgpt`, 2022.

[198] Alex M. Lamb, Anirudh Goyal, Ying Zhang, Saizheng Zhang, Aaron C. Courville, and Yoshua Bengio. Professor Forcing: A New Algorithm for Training Recurrent Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

[199] N. B. Toomarian and J. Barhen. Learning a Trajectory Using Adjoint Functions and Teacher Forcing. *Neural Networks*, 1992.

[200] Daehee Park, Jaeseok Jeong, Sung-Hoon Yoon, Jaewoo Jeong, and Kuk-Jin Yoon. T4P: Test-Time Training of Trajectory Prediction via Masked Autoencoder and Actor-specific Token Memory. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. doi: 10.1109/CVPR52733.2024.01427.

[201] Brian Yang, Huangyuan Su, Nikolaos Gkanatsios, Tsung-Wei Ke, Ayush Jain, Jeff Schneider, and Katerina Fragkiadaki. Diffusion-ES: Gradient-Free Planning with Diffusion for Autonomous and Instruction-Guided Driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15342–15353, 2024. doi: 10.1109/CVPR52733.2024.01453.

[202] Wassim G. Najm, John D. Smith, and Mikio Yanagisawa. Pre-Crash Scenario Typology for Crash Avoidance Research. Technical report, United States. Department of Transportation. NHTSA, 2007.

[203] Zhenxin Li, Kailin Li, Shihao Wang, Shiyi Lan, Zhiding Yu, Yishen Ji, Zhiqi Li, Ziyue Zhu, Jan Kautz, Zuxuan Wu, Yu-Gang Jiang, and Jose M. Alvarez. Hydra-MDP: End-to-end Multimodal Planning with Multi-target Hydra-Distillation. *arXiv preprint arXiv:2406.06978*, 2024.

[204] Shai Shalev-Shwartz, Shaked Shammah, and Amnon Shashua. On a Formal Model of Safe and Scalable Self-driving Cars. *arXiv preprint arXiv:1708.06374*, 2017.

[205] Xinshuo Weng, Boris Ivanovic, Yan Wang, Yue Wang, and Marco Pavone. PARA-Drive: Parallelized Architecture for Real-time Autonomous Driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15449–15458, 2024. doi: 10.1109/CVPR52733.2024.01463.

[206] Hongyang Li, Yang Li, Huijie Wang, Jia Zeng, Huilin Xu, Pinlong Cai, Li Chen, Junchi Yan, Feng Xu, Lu Xiong, et al. Open-sourced Data Ecosystem in Autonomous Driving: the Present and Future. *Scientia Sinica Informationis*, 54(6):1283–1318, 2024. doi: 10.1360/SSI-2023-0313.

[207] Matt Vitelli, Yan Chang, Yawei Ye, Ana Ferreira, Maciej Wołczyk, Błażej Osiński, Moritz Niendorf, Hugo Grimmett, Qiangui Huang, Ashesh Jain, and Peter Ondruska. SafetyNet: Safe Planning for Real-World Self-Driving Vehicles Using Machine-Learned Policies. In *Proceedings of the International Conference on Robotics and Automation (ICRA)*, pages 897–904, 2022. doi: 10.1109/ICRA46639.2022.9811576.

[208] Alex Kendall. The Road to Embodied AI. Keynote at the CVPR 2024 Workshop on Autonomous Driving (WAD), `https://youtu.be/AEfq5nFi7s8`, 2024.

[209] Yihan Hu, Kun Li, Pingyuan Liang, Jingyu Qian, Zhening Yang, Haichao Zhang, Wenxin Shao, Zhuangzhuang Ding, Wei Xu, and Qiang Liu. Imitation with Spatial-Temporal Heatmap: 2nd Place Solution for nuPlan Challenge. *arXiv preprint arXiv:2306.15700*, 2023.

[210] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-Time Training with Self-Supervision for Generalization under Distribution Shifts. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*, 2020.

[211] Jian Liang, Ran He, and Tieniu Tan. A Comprehensive Survey on Test-Time Adaptation under Distribution Shifts. *International Journal of Computer Vision*, 133(1): 31–64, 2025. doi: 10.1007/s11263-024-02181-w.

[212] Qin Wang, Olga Fink, Luc Van Gool, and Dengxin Dai. Continual Test-Time Domain Adaptation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. doi: 10.1109/CVPR52688.2022.00706.

[213] Junyu Gao, Xuan Yao, and Changsheng Xu. Fast-Slow Test-Time Adaptation for Online Vision-and-Language Navigation. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.

[214] Abbas Sadat, Sergio Casas, Mengye Ren, Xinyu Wu, Pranaab Dhawan, and Raquel Urtasun. Perceive, Predict, and Plan: Safe Motion Planning Through Interpretable Semantic Representations. In *Proceedings of the European Conference on Computer*

Vision (ECCV), 2020.

[215] Sergio Casas, Abbas Sadat, and Raquel Urtasun. MP3: A Unified Model to Map, Perceive, Predict and Plan. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

[216] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully Test-Time Adaptation by Entropy Minimization. In *International Conference on Learning Representations (ICLR)*, 2021.

[217] Julian Zimmerlin, Jens Beißwenger, Bernhard Jaeger, Andreas Geiger, and Kashyap Chitta. Hidden Biases of End-to-End Driving Datasets. *arXiv preprint arXiv:2412.09602*, 2024.

[218] Bo Jiang, Shaoyu Chen, Hao Gao, Bencheng Liao, Qian Zhang, Wenyu Liu, and Xinggang Wang. VADv2: End-to-End Vectorized Autonomous Driving via Probabilistic Planning. In *International Conference on Learning Representations (ICLR)*, 2026.

[219] Bo Jiang, Shaoyu Chen, Qing Xu, Bencheng Liao, Jiajie Chen, Helong Zhou, Qian Zhang, Wenyu Liu, Chang Huang, and Xinggang Wang. VAD: Vectorized Scene Representation for Efficient Autonomous Driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 8306–8316, 2023. doi: 10.1109/ICCV51070.2023.00766.

[220] Anthony Hu, Gianluca Corrado, Nicolas Griffiths, Zachary Murez, Corina Gurau, Hudson Yeo, Alex Kendall, Roberto Cipolla, and Jamie Shotton. Model-Based Imitation Learning for Urban Driving. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[221] Xiaosong Jia, Penghao Wu, Li Chen, Jiangwei Xie, Conghui He, Junchi Yan, and Hongyang Li. Think Twice before Driving: Towards Scalable Decoders for End-to-End Autonomous Driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[222] Xiaofeng Wang, Zheng Zhu, Guan Huang, Xinze Chen, Jiagang Zhu, and Jiwen Lu. DriveDreamer: Towards Real-World-Driven World Models for Autonomous Driving. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 55–72, 2024. doi: 10.1007/978-3-031-73195-2_4. arXiv:2309.09777 (2023).

[223] Xiaosong Jia, Yulu Gao, Li Chen, Junchi Yan, Patrick Langechuan Liu, and Hongyang Li. DriveAdapter: Breaking the Coupling Barrier of Perception and Planning in End-to-End Autonomous Driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[224] Bencheng Liao, Shaoyu Chen, Haoran Yin, Bo Jiang, Cheng Wang, Sixu Yan, Xinbang Zhang, Xiangyu Li, Ying Zhang, Qian Zhang, and Xinggang Wang. DiffusionDrive: Truncated Diffusion Model for End-to-End Autonomous Driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12037–12047, 2025. doi: 10.1109/CVPR52734.2025.01124.

[225] OpenAI. OpenAI o1 System Card. `https://openai.com/index/openai-o1-system-card/`, 2024.

[226] Gustavo A Vargas Hakim, David Osowiechi, Mehrdad Noori, Milad Cheraghalikhani, Ali Bahri, Ismail Ben Ayed, and Christian Desrosiers. ClusT3: Information Invariant Test-Time Training. In *Proceedings of the IEEE/CVF International Conference on*

Computer Vision (ICCV), 2023.

[227] Robert A. Marsden, Mario Döbler, and Bin Yang. Universal Test-Time Adaptation through Weight Ensembling, Diversity Weighting, and Prior Correction. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 2543–2553, 2024. doi: 10.1109/WACV57701.2024.00254.

[228] Manli Shu, Weili Nie, De-An Huang, Zhiding Yu, Tom Goldstein, Anima Anandkumar, and Chaowei Xiao. Test-Time Prompt Tuning for Zero-Shot Generalization in Vision-Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[229] Hee Suk Yoon, Eunseop Yoon, Joshua Tian Jin Tee, Mark Hasegawa-Johnson, Yingzhen Li, and Chang D Yoo. C-TPT: Calibrated Test-Time Prompt Tuning for Vision-Language Models via Text Feature Dispersion. In *International Conference on Learning Representations (ICLR)*, 2024.

[230] Nicklas Hansen, Rishabh Jangir, Yu Sun, Guillem Alenyà, Pieter Abbeel, Alexei A. Efros, Lerrel Pinto, and Xiaolong Wang. Self-Supervised Policy Adaptation during Deployment. In *International Conference on Learning Representations (ICLR)*, 2021.

[231] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, et al. Learning to (Learn at Test Time): RNNs with Expressive Hidden States. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, pages 57503–57522, 2025.

[232] Yihong Xu, Eloi Zablocki, Yuan Yin, Elias Ramzi, Ellington Kirby, Alexandre Boulch, and Matthieu Cord. Test-Time Trajectory Optimization for Autonomous Driving. *arXiv preprint arXiv:2606.07170*, 2026.

[233] Hao Gao, Shaoyu Chen, Bo Jiang, Bencheng Liao, Yiang Shi, Xiaoyang Guo, Yuechuan Pu, Haoran Yin, Xiangyu Li, Xinbang Zhang, Ying Zhang, Wenyu Liu, Qian Zhang, and Xinggang Wang. RAD: Training an End-to-End Driving Policy via Large-Scale 3DGS-based Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[234] Jiazhi Yang, Shenyuan Gao, Yihang Qiu, Li Chen, Tianyu Li, Bo Dai, Kashyap Chitta, Penghao Wu, Jia Zeng, Ping Luo, Jun Zhang, Andreas Geiger, Yu Qiao, and Hongyang Li. Generalized Predictive Model for Autonomous Driving. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[235] Shenyuan Gao, Jiazhi Yang, Li Chen, Kashyap Chitta, Yihang Qiu, Andreas Geiger, Jun Zhang, and Hongyang Li. Vista: A Generalizable Driving World Model with High Fidelity and Versatile Controllability. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 91560–91596, 2024.

[236] Gabriel Ilharco, Mitchell Wortsman, Ross Wightman, Cade Gordon, Nicholas Carlini, Rohan Taori, Achal Dave, Vaishaal Shankar, Hongseok Namkoong, John Miller, Hannaneh Hajishirzi, Ali Farhadi, and Ludwig Schmidt. OpenCLIP. Zenodo, version 0.1, `https://doi.org/10.5281/zenodo.5143773`, 2021.

[237] Eric Thorn, Shawn C. Kimmel, and Michelle Chaka. A Framework for Automated Driving System Testable Cases and Scenarios. Technical report, United States. Department of Transportation. NHTSA, 2018.

[238] Dennis Park, Rares Ambrus, Vitor Guizilini, Jie Li, and Adrien Gaidon. Is Pseudo-Lidar Needed for Monocular 3D Object Detection? In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[239] Claude E. Shannon. A Mathematical Theory of Communication. *The Bell System Technical Journal*, 27:379–423, 623–656, 1948. doi: 10.1002/j.1538-7305.1948.tb01338.x.

[240] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation. In *International Conference on Learning Representations (ICLR)*, 2023.

[241] Wei Cao, Marcel Hallgarten, Tianyu Li, Daniel Dauner, Xunjiang Gu, Caojun Wang, Yakov Miron, Marco Aiello, Hongyang Li, Igor Gilitschenski, Boris Ivanovic, Marco Pavone, Andreas Geiger, and Kashyap Chitta. Pseudo-Simulation for Autonomous Driving. In *Proceedings of the 9th Conference on Robot Learning (CoRL)*, 2025.

[242] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 6402–6413, 2017.

[243] Xiao Li, H. Eric Tseng, Anouck Girard, and Ilya Kolmanovsky. Autonomous Driving with Perception Uncertainties: Deep-Ensemble Based Adaptive Cruise Control. In *Proceedings of the IEEE Conference on Decision and Control (CDC)*, pages 8186–8192, 2024. doi: 10.1109/CDC56724.2024.10886150.

[244] Marcel Hallgarten, Julian Zapata, Martin Stoll, Katrin Renz, and Andreas Zell. Can Vehicle Motion Planning Generalize to Realistic Long-tail Scenarios? In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5388–5395, 2024. doi: 10.1109/IROS58592.2024.10803052.

[245] Alexander Amini, Wilko Schwarting, Ava Soleimany, and Daniela Rus. Deep Evidential Regression. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[246] Kristofer D. Kusano, John M. Scanlon, Yin-Hsiu Chen, Timothy L. McMurry, Tilia Gode, and Trent Victor. Comparison of Waymo Rider-Only Crash Rates by Crash Type to Human Benchmarks at 56.7 Million Miles. *Traffic Injury Prevention*, 26 (sup1):S8–S20, 2025. doi: 10.1080/15389588.2025.2499887.

[247] Tesla. Full Self-Driving (Supervised). `https://www.tesla.com/fsd`, 2025.

[248] Waymo. Demonstrably safe AI for autonomous driving. `https://waymo.com/blog/2025/12/demonstrably-safe-ai-for-autonomous-driving`, 2025.

[249] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0: A Vision-Language-Action Flow Model for General Robot Control. In *Robotics: Science and Systems (RSS)*, 2025. doi: 10.15607/RSS.2025.XXI.010. arXiv:2410.24164 (2024).

[250] Sunday Team. ACT-1: A robot foundation model trained on zero robot data. `https://www.sunday.ai/journal/no-robot-data`, 2025.

[251] Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, et al. X-VLA: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. In *International Conference on Learning Representations (ICLR)*, 2026.

[252] Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Kevin Black, Ken Conley, Grace Connors, James Darpinian, Karan Dhabalia, et al. $\pi^*_{0.6}$: a VLA That Learns From Experience. In *Robotics: Science and Systems (RSS)*, 2026. doi: 10.15607/RSS.2026.XXII.087. arXiv:2511.14759 (2025), listed under Physical Intelligence.

[253] Hao Luo, Ye Wang, Wanpeng Zhang, Sipeng Zheng, Ziheng Xi, Chaoyi Xu, Haiweng Xu, Haoqi Yuan, Chi Zhang, Yiqing Wang, Yicheng Feng, and Zongqing Lu. Being-H0.5: Scaling human-centric robot learning for cross-embodiment generalization. *arXiv preprint arXiv:2601.12993*, 2026.

[254] Generalist AI Team. GEN-0: Embodied foundation models that scale with physical interaction. `https://generalistai.com/blog/gen-0`, 2025.

[255] Tianxing Chen, Kaixuan Wang, Zhaohui Yang, Yuhao Zhang, Zanxin Chen, Baijun Chen, Wanxi Dong, Ziyuan Liu, Dong Chen, Tianshuo Yang, et al. Benchmarking generalizable bimanual manipulation: RoboTwin dual-arm collaboration challenge at CVPR 2025 MEIS workshop. *arXiv preprint arXiv:2506.23351*, 2025.

[256] Modi Shi, Li Chen, Jin Chen, Yuxiang Lu, Chiming Liu, Guanghui Ren, Ping Luo, Di Huang, Maoqing Yao, and Hongyang Li. Is diversity all you need for scalable robotic manipulation? *IEEE Transactions on Robotics*, 42:1872–1883, 2026. doi: 10.1109/TRO.2026.3686184. arXiv:2507.06219 (2025).

[257] Yunfan Jiang, Ruohan Zhang, Josiah Wong, Chen Wang, Yanjie Ze, Hang Yin, Cem Gokmen, Shuran Song, Jiajun Wu, and Li Fei-Fei. BEHAVIOR Robot Suite: Streamlining real-world whole-body manipulation for everyday household activities. In *Proceedings of the Conference on Robot Learning (CoRL)*, 2025.

[258] Kevin Black, Manuel Y. Galliker, and Sergey Levine. Real-Time Execution of Action Chunking Flow Policies. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[259] Jiaming Tang, Yufei Sun, Yilong Zhao, Shang Yang, Yujun Lin, Zhuoyang Zhang, James Hou, Yao Lu, Zhijian Liu, and Song Han. VLASH: Real-time VLAs via future-state-aware asynchronous inference. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2026. To appear.

[260] Zheyuan Hu, Robyn Wu, Naveen Enock, Jasmine Li, Riya Kadakia, Zackory Erickson, and Aviral Kumar. RaC: Robot learning for long-horizon tasks by scaling recovery and correction. *IEEE Transactions on Robotics*, 2026. doi: 10.1109/TRO.2026.3706552. arXiv:2509.07953 (2025).

[261] Mingjie Pan, Siyuan Feng, Qinglin Zhang, Xinchen Li, Jianheng Song, Chendi Qu, Yi Wang, Chuankang Li, Ziyu Xiong, Zhi Chen, et al. SOP: A scalable online post-training system for vision-language-action models. *arXiv preprint arXiv:2601.03044*, 2026.

[262] Open X-Embodiment Collaboration, Abby O'Neill, Abdul Rehman, Abhinav Gupta, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, et al. Open X-Embodiment: Robotic Learning Datasets and RT-X Models. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2024. doi: 10.1109/ICRA57147.2024.10611477.

[263] Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang

Chen, Kirsty Ellis, et al. DROID: A Large-Scale In-The-Wild Robot Manipulation Dataset. In *Robotics: Science and Systems (RSS)*, 2024.

[264] Ekin D Cubuk, Barret Zoph, Dandelion Mane, Vijay Vasudevan, and Quoc V Le. AutoAugment: Learning augmentation strategies from data. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 113–123, 2019.

[265] Ekin D Cubuk, Barret Zoph, Jonathon Shlens, and Quoc V Le. RandAugment: Practical automated data augmentation with a reduced search space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2020.

[266] Michael Laskin, Kimin Lee, Adam Stooke, Lerrel Pinto, Pieter Abbeel, and Aravind Srinivas. Reinforcement learning with augmented data. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[267] Xue Bin Peng, Aviral Kumar, Grace Zhang, and Sergey Levine. Advantage-Weighted Regression: Simple and Scalable Off-Policy Reinforcement Learning. *arXiv preprint arXiv:1910.00177*, 2019.

[268] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. In *Robotics: Science and Systems (RSS)*, 2023. doi: 10.15607/RSS.2023.XIX.016.

[269] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. In *Robotics: Science and Systems (RSS)*, 2023. doi: 10.15607/RSS.2023.XIX.026.

[270] Cheng Chi, Zhenjia Xu, Chuer Pan, Eric Cousineau, Benjamin Burchfiel, Siyuan Feng, Russ Tedrake, and Shuran Song. Universal Manipulation Interface: In-the-wild robot teaching without in-the-wild robots. In *Robotics: Science and Systems (RSS)*, 2024.

[271] Jia Zeng, Qingwen Bu, Bangjun Wang, Wenke Xia, Li Chen, Hao Dong, Haoming Song, Dong Wang, Di Hu, Ping Luo, et al. Learning manipulation by predicting interaction. In *Robotics: Science and Systems (RSS)*, 2024.

[272] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. RT-1: Robotics Transformer for Real-World Control at Scale. In *Robotics: Science and Systems (RSS)*, 2023. doi: 10.15607/RSS.2023.XIX.025.

[273] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P. Foster, Pannag R. Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An Open-Source Vision-Language-Action Model. In *Proceedings of the 8th Conference on Robot Learning (CoRL)*, 2024.

[274] Qingwen Bu, Yanting Yang, Jisong Cai, Shenyuan Gao, Guanghui Ren, Maoqing Yao, Ping Luo, and Hongyang Li. UniVLA: Learning to act anywhere with task-centric latent actions. In *Robotics: Science and Systems (RSS)*, 2025.

[275] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An Open-Source Generalist Robot Policy. In *Robotics: Science and Systems (RSS)*, 2024. doi: 10.15607/RSS.2024.XX.090.

[276] Homer Rich Walke, Kevin Black, Tony Z. Zhao, Quan Vuong, Chongyi Zheng, Philippe Hansen-Estruch, Andre Wang He, Vivek Myers, Moo Jin Kim, Max Du, Abraham Lee, Kuan Fang, Chelsea Finn, and Sergey Levine. BridgeData V2: A Dataset for Robot Learning at Scale. In *Proceedings of the 7th Conference on Robot Learning (CoRL)*, volume 229 of *Proceedings of Machine Learning Research*, pages 1723–1736, 2023.

[277] Tao Jiang, Tianyuan Yuan, Yicheng Liu, Chenhao Lu, Jianning Cui, Xiao Liu, Shuiqi Cheng, Jiyang Gao, Huazhe Xu, and Hang Zhao. Galaxea open-world dataset and G0 dual-system VLA model. *arXiv preprint arXiv:2509.00576*, 2025.

[278] Michael Kelly, Chelsea Sidrane, Katherine Driggs-Campbell, and Mykel J. Kochenderfer. HG-DAgger: Interactive Imitation Learning with Human Experts. In *Proceedings of the International Conference on Robotics and Automation (ICRA)*, 2019. doi: 10.1109/ICRA.2019.8793698.

[279] Tianhe Yu et al. Scaling robot learning with semantically imagined experience. In *Robotics: Science and Systems (RSS)*, 2023.

[280] Denis Yarats, Ilya Kostrikov, and Rob Fergus. Image augmentation is all you need: Regularizing deep reinforcement learning from pixels. In *International Conference on Learning Representations (ICLR)*, 2021.

[281] Yunfei Li, Xiao Ma, Jiafeng Xu, Yu Cui, Zhongren Cui, Zhigang Han, Liqun Huang, Tao Kong, Yuxiao Liu, Hao Niu, et al. GR-RL: Going dexterous and precise for long-horizon robotic manipulation. *arXiv preprint arXiv:2512.01801*, 2025.

[282] Howard Chen, Noam Razin, Karthik Narasimhan, and Danqi Chen. Retaining by doing: The role of on-policy data in mitigating forgetting. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2026.

[283] Fahim Tajwar, Anikait Singh, Archit Sharma, Rafael Rafailov, Jeff Schneider, Tengyang Xie, Stefano Ermon, Chelsea Finn, and Aviral Kumar. Preference fine-tuning of LLMs should leverage suboptimal, on-policy data. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.

[284] Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model Soups: Averaging Weights of Multiple Fine-Tuned Models Improves Accuracy without Increasing Inference Time. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, 2022.

[285] Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-Merging: Resolving Interference When Merging Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[286] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing Models with Task Arithmetic. In *International Conference on Learning Representations (ICLR)*, 2023.

[287] Enneng Yang, Li Shen, Guibing Guo, Xingwei Wang, Xiaochun Cao, Jie Zhang, and Dacheng Tao. Model merging in LLMs, MLLMs, and beyond: Methods, theories, applications, and opportunities. *ACM Computing Surveys*, 2024.

[288] Takuya Akiba, Makoto Shing, Yujin Tang, Qi Sun, and David Ha. Evolutionary Optimization of Model Merging Recipes. *Nature Machine Intelligence*, 7(2):195–204, 2025. doi: 10.1038/s42256-024-00975-8.

[289] Giwon Lee, Wooseong Jeong, Daehee Park, Jaewoo Jeong, and Kuk-Jin Yoon. Interaction-merged motion planning: Effectively leveraging diverse motion datasets for robust planning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025.

[290] Lirui Wang, Kaiqing Zhang, Allan Zhou, Max Simchowitz, and Russ Tedrake. Robot Fleet Learning via Policy Merging. In *International Conference on Learning Representations (ICLR)*, 2024.

[291] Yajat Yadav, Zhiyuan Zhou, Andrew Wagenmaker, Karl Pertsch, and Sergey Levine. Robust finetuning of vision-language-action robot policies via parameter merging. In *International Conference on Learning Representations (ICLR)*, 2026.

[292] Jürgen Schmidhuber. Reinforcement learning upside down: Don't predict rewards–just map them to actions. *arXiv preprint arXiv:1912.02875*, 2019.

[293] Qinqing Zheng, Amy Zhang, and Aditya Grover. Online decision transformer. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2022.

[294] Scott Emmons, Benjamin Eysenbach, Ilya Kostrikov, and Sergey Levine. RvS: What is essential for offline RL via supervised learning? In *International Conference on Learning Representations (ICLR)*, 2022.

[295] Jakub Grudzien Kuba, Pieter Abbeel, and Sergey Levine. Advantage-conditioned diffusion: Offline RL via generalization. OpenReview (submitted to ICLR 2024), `https://openreview.net/forum?id=QDrGOALevs`, 2023.

[296] Yueh-Hua Wu, Xiaolong Wang, and Masashi Hamaya. Elastic decision transformer. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[297] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 1889–1897. PMLR, 2015.

[298] Sergey Levine. Reinforcement learning and control as probabilistic inference: Tutorial and review. *arXiv preprint arXiv:1805.00909*, 2018.

[299] Takayuki Osa, Joni Pajarinen, Gerhard Neumann, J Andrew Bagnell, Pieter Abbeel, and Jan Peters. An algorithmic perspective on imitation learning. *Foundations and Trends in Robotics*, 2018.

[300] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.

[301] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 2022.

[302] Shalini Maiti, Amar Budhiraja, Bhavul Gauri, Gaurav Chaurasia, Anton Protopopov, Alexis Audran-Reiss, Michael Slater, Despoina Magka, Tatiana Shavrina, Roberta Raileanu, et al. Souper-model: How simple arithmetic unlocks state-of-the-art LLM performance. *arXiv preprint arXiv:2511.13254*, 2025.

[303] Chilam Cheang, Sijin Chen, Zhongren Cui, Yingdong Hu, Liqun Huang, Tao Kong, Hang Li, Yifeng Li, Yuxiao Liu, Xiao Ma, et al. GR-3 technical report. *arXiv preprint arXiv:2507.15493*, 2025.

[304] Junjie Wen, Yichen Zhu, Jinming Li, Zhibin Tang, Chaomin Shen, and Feifei Feng. DexVLA: Vision-language model with plug-in diffusion expert for general robot control. In *Proceedings of the Conference on Robot Learning (CoRL)*, 2025.

[305] Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are super mario: Absorbing abilities from homologous models as a free lunch. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.

[306] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, et al. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.

[307] Kohei Sendai, Maxime Alvarez, Tatsuya Matsushima, Yutaka Matsuo, and Yusuke Iwasawa. Leave no observation behind: Real-time correction for VLA action chunks. *arXiv preprint arXiv:2509.23224*, 2025.

[308] Zengjue Chen, Runliang Niu, He Kong, Qi Wang, Qianli Xing, and Zipei Fan. TGRPO: Fine-tuning vision-language-action model via trajectory-wise group relative policy optimization. *arXiv preprint arXiv:2506.08440*, 2025.

[309] Haozhan Li, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Zhaohui Yang, Kaiyan Zhang, Xuekai Zhu, Yuchen Zhang, Tianxing Chen, Ganqu Cui, et al. SimpleVLA-RL: Scaling VLA training via reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2026.

[310] Guanxing Lu, Wenkai Guo, Chubin Zhang, Yuheng Zhou, Haonan Jiang, Zifeng Gao, Yansong Tang, and Ziwei Wang. VLA-RL: Towards masterful and general robotic manipulation with scalable reinforcement learning. *arXiv preprint arXiv:2505.18719*, 2025.

[311] Jianlan Luo, Zheyuan Hu, Charles Xu, You Liang Tan, Jacob Berg, Archit Sharma, Stefan Schaal, Chelsea Finn, Abhishek Gupta, and Sergey Levine. SERL: A software suite for sample-efficient robotic reinforcement learning. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2024.

[312] Wenli Xiao, Haotian Lin, Andy Peng, Haoru Xue, Tairan He, Zhengyi Luo, Yuqi Xie, Fengyuan Hu, Linxi Fan, Guanya Shi, and Yuke Zhu. Self-improving vision-language-action models with data generation via residual RL. In *International Conference on Learning Representations (ICLR)*, 2026.

[313] Jianlan Luo, Charles Xu, Jeffrey Wu, and Sergey Levine. Precise and Dexterous Robotic Manipulation via Human-in-the-Loop Reinforcement Learning. *Science Robotics*, 10(105):eads5033, 2025. doi: 10.1126/scirobotics.ads5033.

[314] Kun Lei, Huanyu Li, Dongjie Yu, Zhenyu Wei, Lingxiao Guo, Zhennan Jiang, Ziyu Wang, Shiyu Liang, and Huazhe Xu. Performant Robotic Manipulation with Real-World Reinforcement Learning. *Science Robotics*, 11(116):eaed6267, 2026. doi: 10.1126/scirobotics.aed6267. Preprint titled "RL-100: Performant Robotic Manipulation with Real-World Reinforcement Learning", arXiv:2510.14830 (2025).

[315] Aviral Kumar, Xue Bin Peng, and Sergey Levine. Reward-conditioned policies. *arXiv preprint arXiv:1912.13465*, 2019.

[316] Kevin Frans, Seohong Park, Pieter Abbeel, and Sergey Levine. Diffusion guidance is a controllable policy improvement operator. *arXiv preprint arXiv:2505.23458*, 2025.

[317] Seyed Kamyar Seyed Ghasemipour, Ayzaan Wahid, Jonathan Tompson, Pannag Sanketi, and Igor Mordatch. Self-improving embodied foundation models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pages 124887–124922, 2025.

[318] Shaopeng Zhai, Qi Zhang, Tianyi Zhang, Fuxian Huang, Haoran Zhang, Ming Zhou, Shengzhe Zhang, Litao Liu, Sixu Lin, and Jiangmiao Pang. A vision-language-action-critic model for robotic real-world reinforcement learning. *arXiv preprint arXiv:2509.15937*, 2025.

[319] Jiahui Zhang, Yusen Luo, Abrar Anwar, Sumedh Anand Sontakke, Joseph J Lim, Jesse Thomason, Erdem Biyik, and Jesse Zhang. ReWiND: Language-guided rewards teach robot policies without new demonstrations. In *Proceedings of the Conference on Robot Learning (CoRL)*, 2025.

[320] Yecheng Jason Ma, Joey Hejna, Chuyuan Fu, Dhruv Shah, Jacky Liang, Zhuo Xu, Sean Kirmani, Peng Xu, Danny Driess, Ted Xiao, et al. Vision language models are in-context value learners. In *International Conference on Learning Representations (ICLR)*, 2025.

[321] Richard S. Sutton and Andrew G. Barto. *Reinforcement learning: An introduction*. MIT Press, Cambridge, MA, 1998.

[322] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[323] Andrew Wagenmaker, Yunchu Zhang, Mitsuhiko Nakamoto, Seohong Park, Waleed Yagoub, Anusha Nagabandi, Abhishek Gupta, and Sergey Levine. Steering your diffusion policy with latent space reinforcement learning. In *Proceedings of the Conference on Robot Learning (CoRL)*, 2025.

[324] Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. CALVIN: A Benchmark for Language-Conditioned Policy Learning for Long-Horizon Robot Manipulation Tasks. *IEEE Robotics and Automation Letters*, 7(3):7327–7334, 2022. doi: 10.1109/LRA.2022.3180108.

[325] Yao Mu, Tianxing Chen, Zanxin Chen, Shijia Peng, Zhiqian Lan, Zeyu Gao, Zhixuan Liang, Qiaojun Yu, Yude Zou, Mingkun Xu, et al. RoboTwin: Dual-arm robot benchmark with generative digital twins. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[326] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[327] Haoyi Jiang, Liu Liu, Tianheng Cheng, Xinjie Wang, Tianwei Lin, Zhizhong Su, Wenyu Liu, and Xinggang Wang. GaussTR: Foundation Model-Aligned Gaussian Transformer for Self-Supervised 3D Spatial Understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11960–11970, 2025. doi: 10.1109/CVPR52734.2025.01117.